Data Visualization with ggplot2

Aravind Hebbali

Table of contents

Preface 3
Software information . . . 4

About the Author 5

1 Quick Tour 6
1.1 Introduction . . . 6
1.2 Libraries, Code & Data . . . 6
1.3 Scatter Plot . . . 6
1.4 Bar Plot . . . 11
1.5 Box Plot . . . 13
1.6 Line Chart . . . 15
1.7 Histogram . . . 17
1.8 Exercises . . . 18
Where to go next . . . 18

2 Geoms 19
2.1 Introduction . . . 19
2.2 Libraries, Code & Data . . . 19
2.2.1 Data . . . 19
2.2.2 Data Dictionary . . . 20
2.3 Point . . . 20
2.4 Regression Line . . . 21
2.5 Bar . . . 23
2.5.1 Frequency . . . 23
2.5.2 Weight . . . 24
2.5.3 Stacked Bar Plot . . . 25
2.5.4 Horizontal Bar Plot . . . 26
2.6 Columns . . . 27
2.7 Boxplot . . . 28
2.8 Histogram . . . 29
2.9 Line . . . 31
2.9.1 Data . . . 31
2.9.2 Horizontal Line . . . 33
2.9.3 Vertical Line . . . 33
2.10 Label . . . 34
2.11 Text . . . 35
2.12 Exercises . . . 36

Where to go next . . . 36

3 Aesthetics **37**
3.1 Introduction . . . 37
3.2 Libraries, Code & Data . . . 37
3.2.1 Data . . . 38
3.2.2 Data Dictionary . . . 38
3.3 Color . . . 38
3.3.1 Point . . . 39
3.4 Line Chart . . . 51
3.4.1 Data . . . 51
3.5 Bar Plots . . . 60
3.6 Histograms . . . 61
3.7 Box Plots . . . 62
3.8 Exercises . . . 63
Where to go next . . . 63

4 Labels **64**
4.1 Introduction . . . 64
4.2 Basic Plot . . . 64
4.3 Title & Subtitle . . . 65
4.3.1 ggtitle() . . . 65
4.4 Axis Labels . . . 66
4.5 Labs . . . 67
4.6 Axis Range . . . 68
4.6.1 X Axis . . . 68
4.6.2 Y Axis . . . 69
4.6.3 Expand Limits . . . 70
4.6.4 Remove Axis Labels . . . 71
4.7 Format Title & Axis Labels . . . 72
4.7.1 Color . . . 72
4.7.2 Font Family . . . 73
4.7.3 Font Face . . . 74
4.7.4 Font Size . . . 75
4.8 Horizontal Alignment . . . 76
4.9 Putting it all together . . . 77
4.9.1 Title . . . 77
4.9.2 X axis label . . . 78
4.9.3 Y axis label . . . 79
4.10 Exercises . . . 80
Where to go next . . . 80

5 Text Annotations **81**
5.1 How text annotations work in ggplot2 . . . 81
5.2 How to annotate plots with text in ggplot2 . . . 81
5.2.1 How to add text labels to a ggplot2 plot . . . 82

5.2.2 How to change text annotation color in ggplot2 . . . 82
5.2.3 How to change text annotation size in ggplot2 . . . 83
5.2.4 How to change text annotation font face in ggplot2 . . . 84
5.2.5 How to rotate text annotations in ggplot2 . . . 85
5.3 How to combine text annotation options in ggplot2 . . . 86
5.4 Exercises . . . 87
Where to go next . . . 87

6 Scatter Plots **88**
6.1 Introduction . . . 88
6.2 Basic Plot . . . 88
6.3 Jitter . . . 89
6.4 Aesthetics . . . 91
6.4.1 Specify Values . . . 91
6.4.2 Map Variables . . . 94
6.5 Regression Line . . . 99
6.5.1 Intercept & Slope . . . 101
6.5.2 Add Line . . . 101
6.6 Exercises . . . 102
Where to go next . . . 102

7 Line Graphs **103**
7.1 Introduction . . . 103
7.2 Case Study . . . 103
7.2.1 Data . . . 103
7.3 Line Chart . . . 104
7.3.1 Line Color . . . 104
7.3.2 Line Type . . . 105
7.3.3 Line Size . . . 107
7.4 Multiple Lines . . . 107
7.4.1 Modify Data . . . 107
7.5 Exercises . . . 111
Where to go next . . . 111

8 Bar Plots **112**
8.1 Introduction . . . 112
8.2 Data . . . 112
8.2.1 Data Dictionary . . . 113
8.3 Basic Plot . . . 113
8.4 Bar Color . . . 114
8.5 Stacked Bar Plot . . . 115
8.6 Grouped Bar Plot . . . 116
8.7 Proportional Bar Plot . . . 117
8.8 Horizontal Bar Plot . . . 118
8.9 Bar Line . . . 119
8.10 Exercises . . . 121

Where to go next . . . 121

9 Box Plots **122**
9.1 Introduction . . . 122
9.2 Structure . . . 122
9.3 Data . . . 123
9.4 Single Plot . . . 123
9.5 Compare Distributions . . . 124
9.5.1 Data . . . 124
9.6 Horizontal Box Plot . . . 125
9.6.1 Notch . . . 126
9.6.2 Jitter . . . 127
9.7 Outliers . . . 128
9.8 Aesthetics . . . 131
9.8.1 Specify Values . . . 131
9.9 Map Variables . . . 135
9.10 Exercises . . . 136
Where to go next . . . 136

10 Histograms **137**
10.1 Introduction . . . 137
10.2 Data . . . 138
10.2.1 Data Dictionary . . . 138
10.3 Plot . . . 138
10.3.1 Specify Bins . . . 139
10.4 Aesthetics . . . 140
10.5 Putting it all together . . . 142
10.6 Bin Width . . . 143
10.7 Line Type . . . 144
10.8 Line Size . . . 145
10.9 Map Variables . . . 146
10.10Exercises . . . 147
Where to go next . . . 147

11 Modify Axis **148**
11.1 How to modify a continuous axis in ggplot2 . . . 148
11.1.1 How to combine continuous axis options in ggplot2 . . . 153
11.1.2 How to modify the y-axis in ggplot2 . . . 153
11.2 How to modify a discrete axis in ggplot2 . . . 154
11.2.1 How to change axis labels in ggplot2 . . . 154
11.2.2 How to change axis tick labels in ggplot2 . . . 155
11.2.3 How to change axis breaks in ggplot2 . . . 156
11.2.4 How to move the axis position in ggplot2 . . . 157
11.3 How to combine discrete axis options in ggplot2 . . . 157
11.4 Exercises . . . 158
Where to go next . . . 158

12 Modify Legend **159**
12.1 How to customize legends with guides in ggplot2 . . . 159
How to change the guide title in ggplot2 . . . 159
How to change guide labels in ggplot2 . . . 162
How to change the legend color bar in ggplot2 . . . 169
12.2 Exercises . . . 177
Where to go next . . . 177

13 Legend: Color and Fill **178**
13.1 How to change legend colors in ggplot2 . . . 178
How to create a plot with a color legend in ggplot2 . . . 178
How to change legend color values in ggplot2 . . . 179
How to change the legend title in ggplot2 . . . 180
How to change legend limits in ggplot2 . . . 181
How to change legend labels in ggplot2 . . . 182
How to reorder or hide legend items in ggplot2 . . . 183
How to combine color legend options in ggplot2 . . . 184
13.2 How to change legend fill colors in ggplot2 . . . 185
How to create a plot with a fill legend in ggplot2 . . . 185
How to change the fill legend title in ggplot2 . . . 186
How to change legend fill values in ggplot2 . . . 187
How to change fill legend labels in ggplot2 . . . 188
How to change fill legend limits in ggplot2 . . . 189
How to reorder or hide fill legend items in ggplot2 . . . 190
How to combine fill legend options in ggplot2 . . . 191
13.3 Exercises . . . 192
Where to go next . . . 192

14 Legend: Shape Size and Alpha **193**
14.1 How to change legend point shapes in ggplot2 . . . 193
How to create a plot with a shape legend in ggplot2 . . . 193
How to change the shape legend title in ggplot2 . . . 194
How to change legend shape values in ggplot2 . . . 196
How to change shape legend labels in ggplot2 . . . 196
How to change shape legend limits in ggplot2 . . . 197
How to reorder or hide shape legend items in ggplot2 . . . 198
How to combine shape legend options in ggplot2 . . . 199
14.2 How to change legend point sizes in ggplot2 . . . 200
How to create a plot with a size legend in ggplot2 . . . 200
How to change the size legend title in ggplot2 . . . 201
How to change the legend size range in ggplot2 . . . 202
How to change size legend limits in ggplot2 . . . 203
How to reorder or hide size legend items in ggplot2 . . . 204
How to change size legend labels in ggplot2 . . . 205
How to combine size legend options in ggplot2 . . . 206

14.3 How to change legend transparency in ggplot2 . . . 207
How to create a plot with an alpha legend in ggplot2 . . . 207
How to change the transparency legend title in ggplot2 . . . 208
How to reorder or hide alpha legend items in ggplot2 . . . 209
How to change transparency legend limits in ggplot2 . . . 210
How to change the legend transparency range in ggplot2 . . . 211
How to change transparency legend labels in ggplot2 . . . 212
How to combine transparency legend options in ggplot2 . . . 213
14.4 Exercises . . . 214
Where to go next . . . 214

15 Faceting **215**
15.1 Introduction . . . 215
15.2 Arrange facets in a grid . . . 215
15.2.1 Stack facets vertically . . . 215
15.2.2 Arrange facets horizontally . . . 216
15.2.3 Facet rows and columns together . . . 217
15.2.4 Use free scales in a facet grid . . . 219
15.2.5 Move facet labels with switch . . . 219
15.3 Wrap facets into panels . . . 220
15.3.1 Set facet rows . . . 221
15.3.2 Set facet columns . . . 222
15.3.3 Use free scales in wrapped facets . . . 223
15.3.4 Set facet rows and columns . . . 224
15.4 Exercises . . . 226
Where to go next . . . 226

16 Themes **227**
16.1 Introduction . . . 227
16.2 Basic Plot . . . 227
16.3 Style plot axes . . . 228
16.3.1 Style axis text . . . 228
16.3.2 Style axis ticks . . . 229
16.3.3 Style axis lines . . . 230
16.4 Style the legend . . . 231
16.4.1 Style the legend background . . . 232
16.4.2 Style legend text . . . 233
16.4.3 Style the legend title . . . 234
16.4.4 Move the legend position . . . 235
16.5 Apply a complete theme . . . 236
16.5.1 Classic Dark on Light . . . 236
16.5.2 Default Gray . . . 237
16.5.3 Light . . . 238
16.5.4 Minimal . . . 239
16.5.5 Dark . . . 240
16.5.6 Classic . . . 240

16.5.7 Void (Empty) . . . 241
16.6 Exercises . . . 242
Where to go next . . . 242

17 Position & Stats Tuning **243**
17.1 Position Dodge . . . 243
17.2 Position Jitter . . . 244
17.3 Position Stack . . . 246
17.4 Stat Summary . . . 246
17.5 After Stat . . . 248
Where to go next . . . 248

18 Scales, Formatting & Color Accessibility **249**
18.1 Percent Labels . . . 249
18.2 Dollar Labels . . . 250
18.3 Comma Labels . . . 251
18.4 Viridis Palettes . . . 252
18.5 Contrast Check . . . 254
Where to go next . . . 254

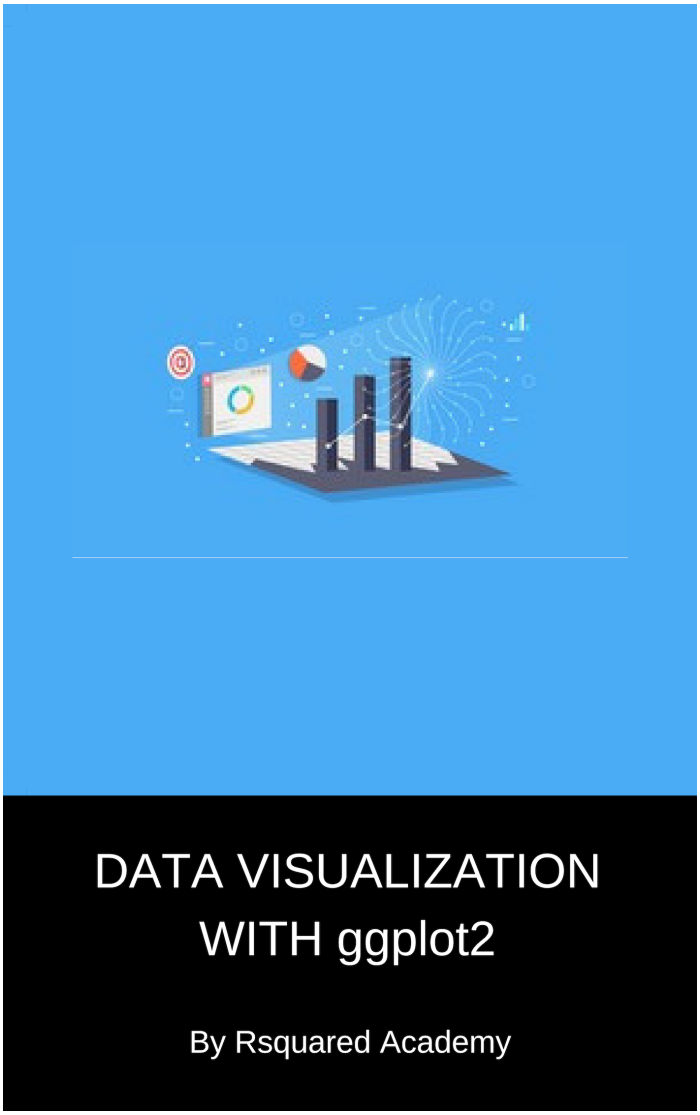
19 Data Reshaping for ggplot2 **255**
19.1 Wide Data . . . 255
19.2 Long Data . . . 256
19.3 Plotting Long Data . . . 256
19.4 Modern Pipe . . . 257
Where to go next . . . 258

20 Label Polishing & Wrapping **259**
20.1 The Overflow Problem . . . 259
20.2 Wrapping with stringr . . . 260
20.3 Wrapping Inside Scales . . . 261
20.4 Wrapping Long Titles . . . 262
Where to go next . . . 263

21 Finale: Publication Polish with patchwork, ggrepel & ggtext **264**
21.1 Case 1: Multi-Plot Layout . . . 264
21.2 Case 2: Collision-Free Labels . . . 266
21.3 Case 3: Rich Text Titles . . . 266
Where to go next . . . 267

22 Export & Communicate **268**
22.1 Saving with ggsave . . . 268
22.2 Vector Formats . . . 268
22.3 Storytelling Checklist . . . 269
22.4 Before and After . . . 269
Where to go next . . . 270

Preface



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Software information

The R session information when compiling this book is shown below:

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.5 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;  LAPACK versio

locale:
 [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
 [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
 [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
[10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C

time zone: UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.5.3  fastmap_1.2.0   cli_3.6.6       tools_4.5.3
 [5] htmltools_0.5.9 otel_0.2.0      rmarkdown_2.32  knitr_1.52
 [9] jsonlite_2.0.0  xfun_0.61       digest_0.6.39   rlang_1.3.0
[13] evaluate_1.0.5
```

We do not add prompts (`>` and `+`) to R source code in this book, and we comment out the text output with two hashes `##` by default, as you can see from the R session information above. This is for your convenience when you want to copy and run the code (the text output will be ignored since it is commented out). Package names are in bold text (e.g., **rmarkdown**), and function names are followed by parentheses (e.g., `bookdown::render_book()`). The double-colon operator `::` means accessing an object from a package.

About the Author

Aravind Hebbali is the founder of Rsquared Academy. He earned his Masters in Economics from Madras School of Economics. As an active R user, he has authored several R packages such as

- olsrr
- rfm
- descriptr
- blorr
- xplorerr

In 2015, he founded Rsquared Academy, a free and open source education initiative with focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on GitHub.

1 Quick Tour

1.1 Introduction

> **Why this chart**
>
> A 5-minute on-ramp: one ggplot() plus one geom_*() draws your first chart.

> **Common pitfall**
>
> Skipping labs() early produces cryptic default titles; label axes from the start.

In this chapter, we will quickly build a set of plots that are routinely used to explore data using `ggplot()` combined with a `geom_*()` layer. Every plot follows the same recipe: map variables to aesthetics in `aes()` and add the geometry that draws them.

> **Note**
>
> `qplot()` was deprecated in ggplot2 3.4.0. It still appears in older tutorials, but all examples in this book use `ggplot() + geom_*()`, which is the supported syntax going forward.

1.2 Libraries, Code & Data

We will use the following libraries in this chapter:

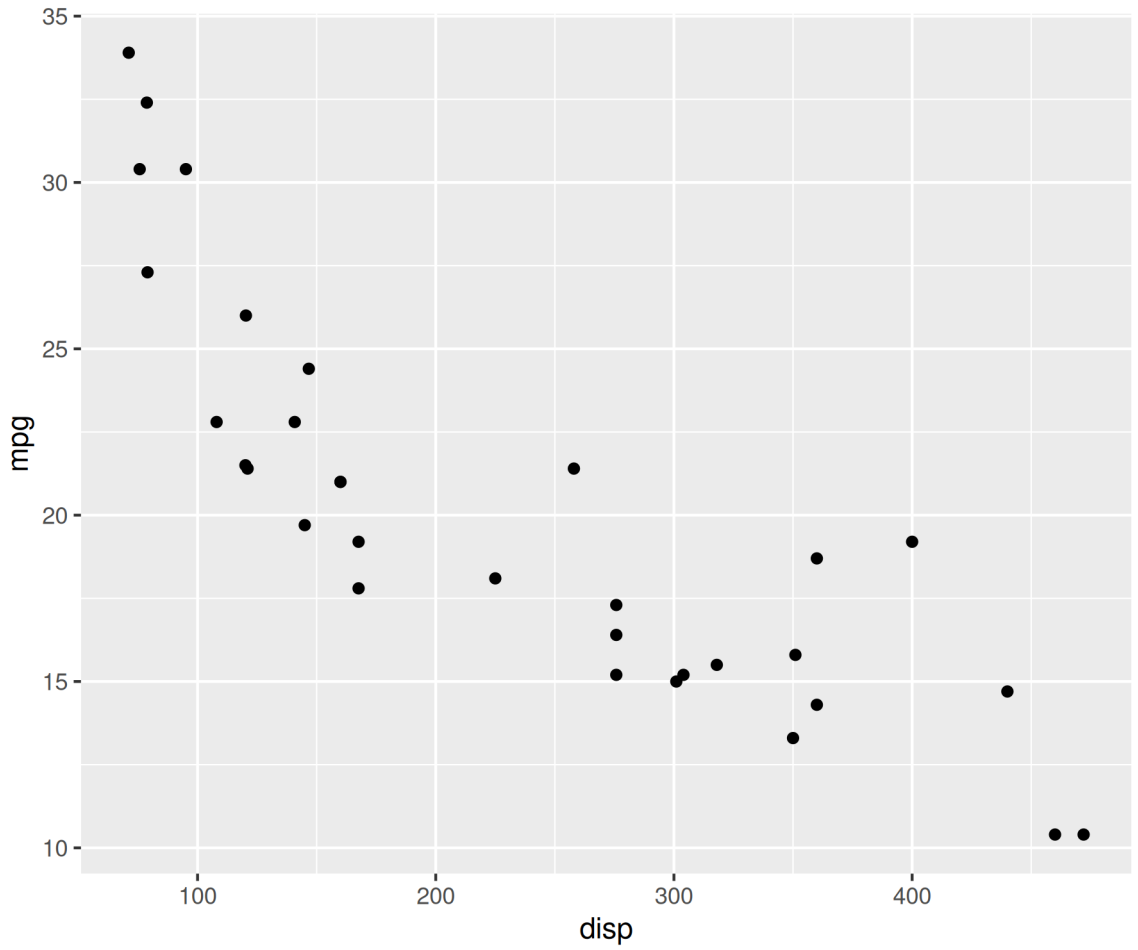
- readr
- ggplot2

All the data sets used in this chapter can be found here and code can be downloaded from here.

1.3 Scatter Plot

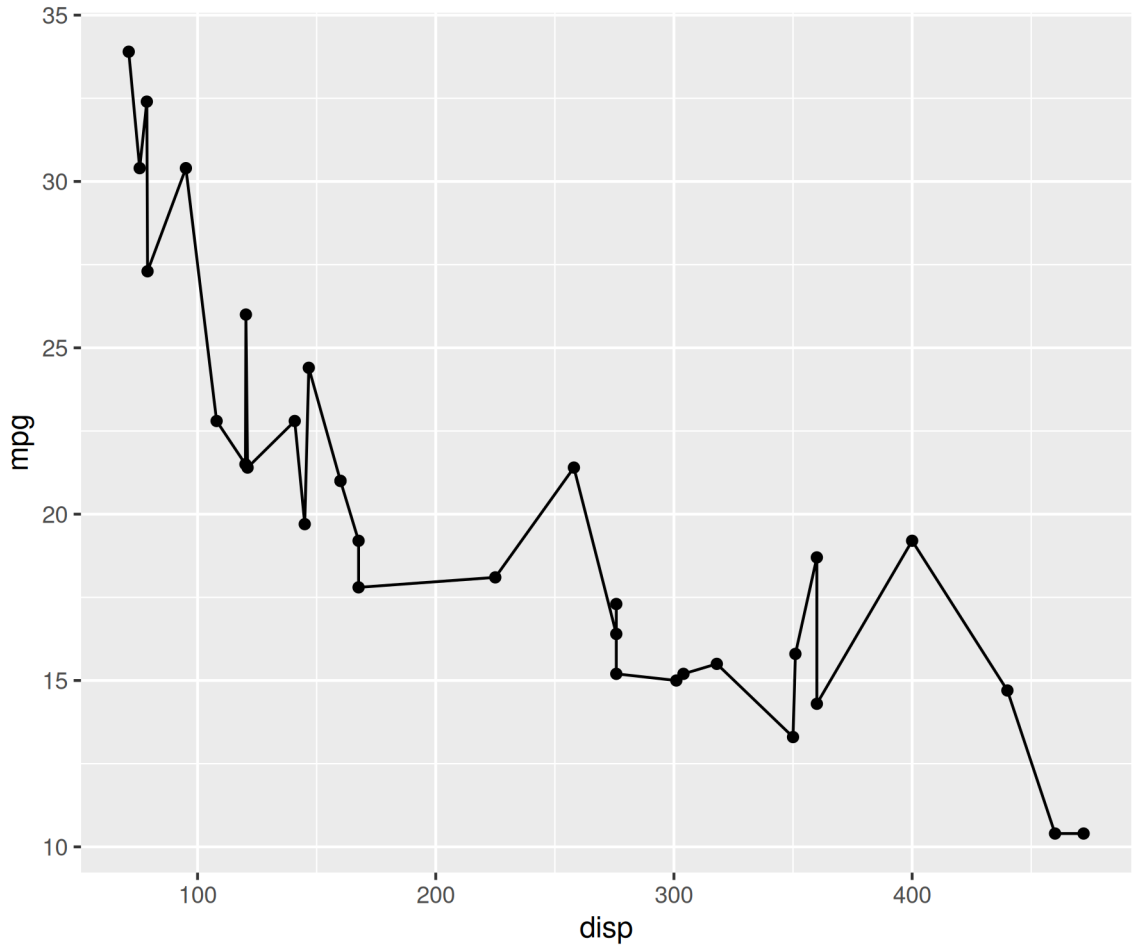
Scatter plots are used to examine the relationship between two continuous variables. The relationship can be examined across the levels of a categorical variable as well. Let us begin by creating scatter plots. Map the X and Y variables inside `aes()` and add `geom_point()`.

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point()
```



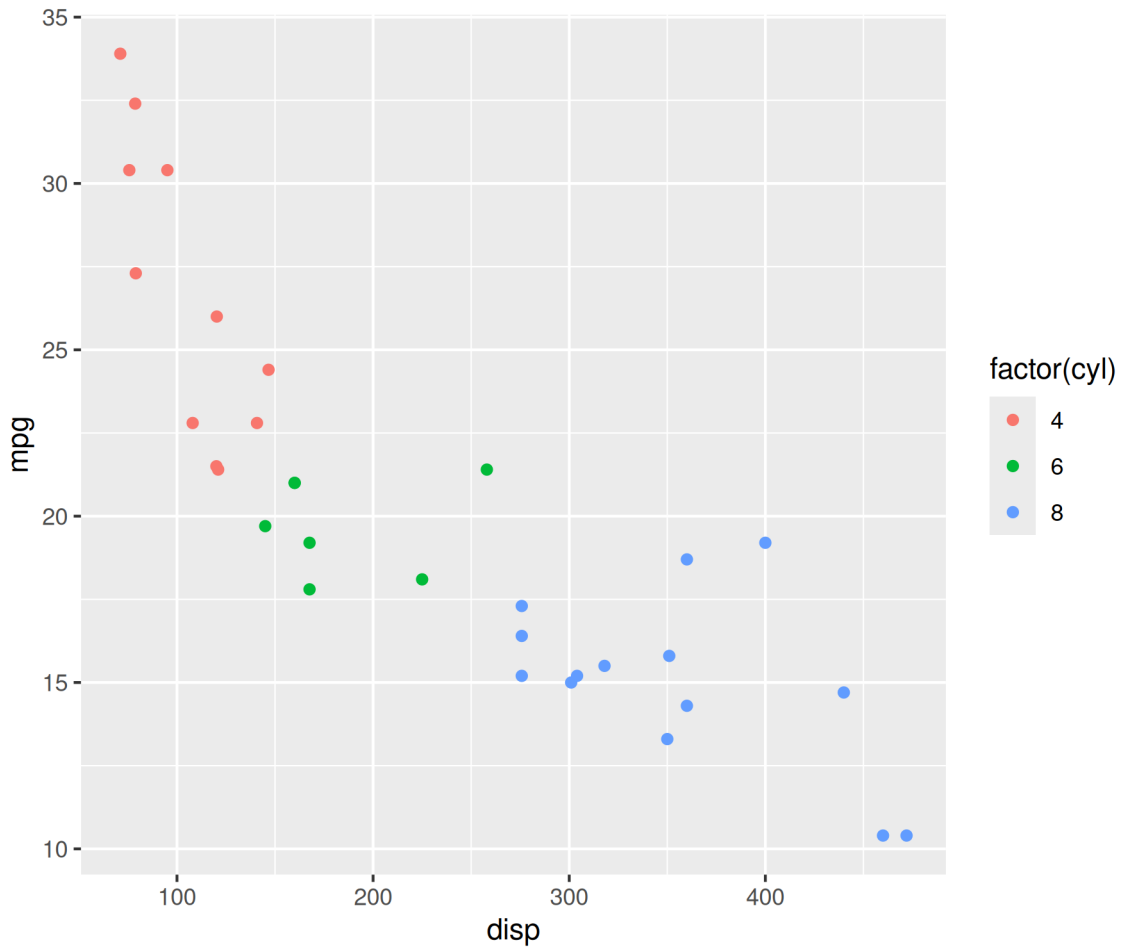
If you want the relationship between the two variables to be represented by both points and a line, add both layers.

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  geom_line()
```



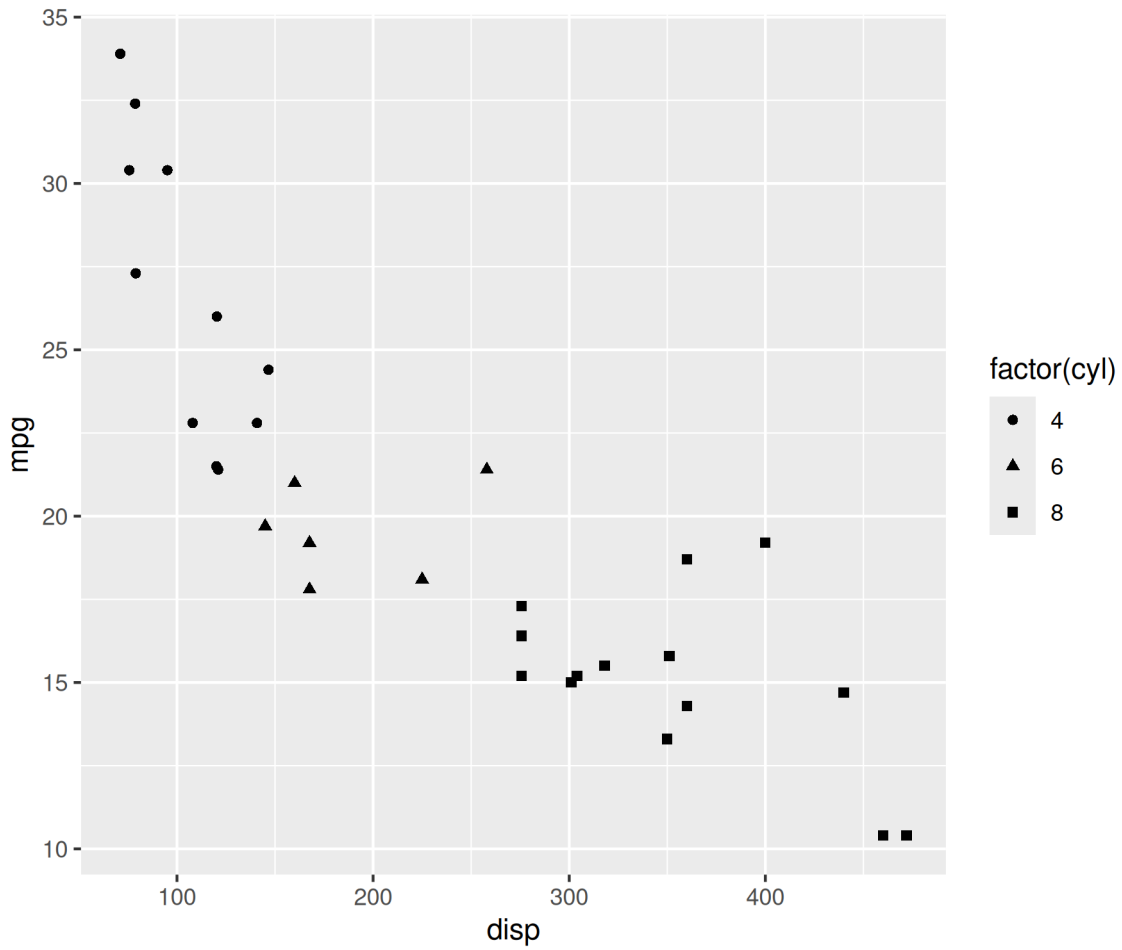
The color of the points can be mapped to a categorical variable, in our case `cyl`, using the `color` aesthetic. Ensure that the variable is categorical using `factor()`.

```
ggplot(mtcars, aes(disp, mpg, color = factor(cyl))) +
  geom_point()
```



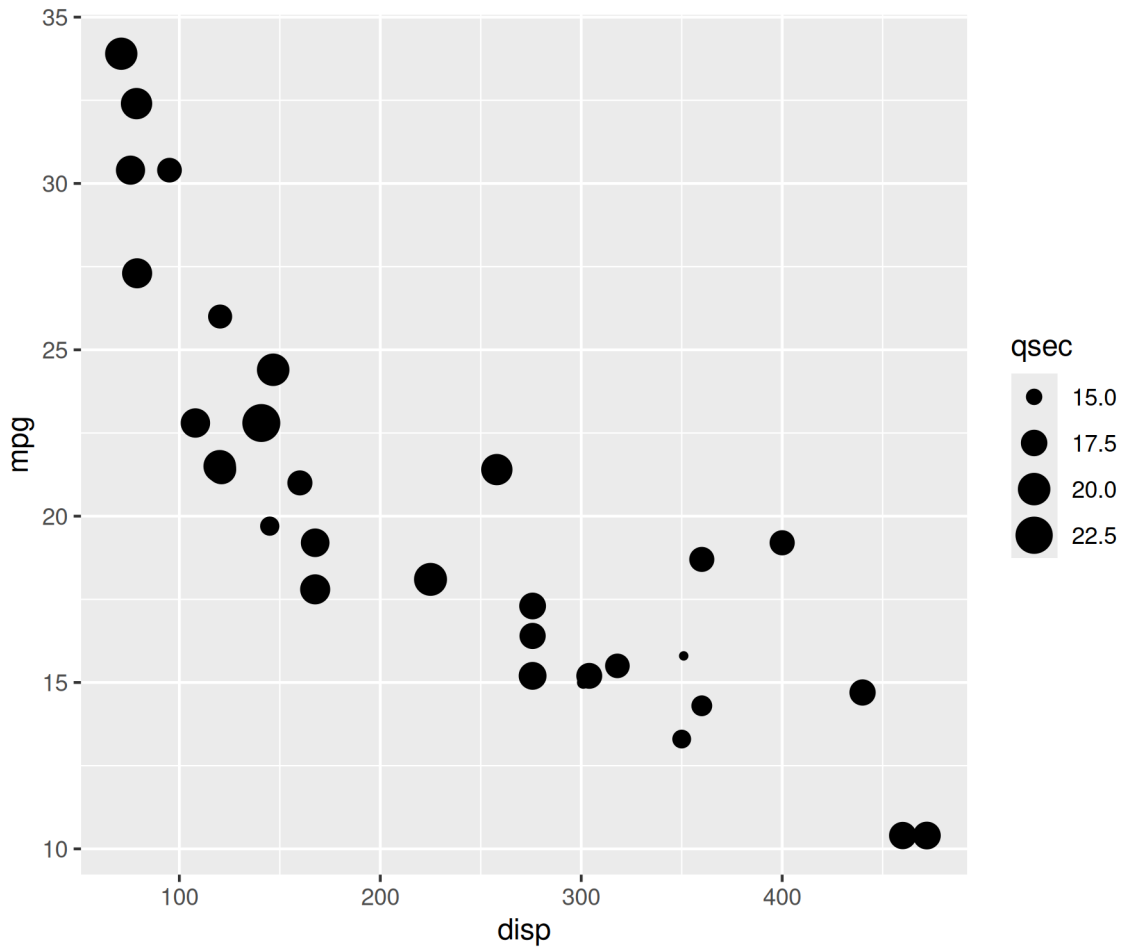
The shape and size of the points can also be mapped to variables using the `shape` and `size` aesthetics as shown in the below examples.

```
ggplot(mtcars, aes(disp, mpg, shape = factor(cyl))) +
  geom_point()
```



Ensure that size is mapped to a continuous variable.

```
ggplot(mtcars, aes(disp, mpg, size = qsec)) +
  geom_point()
```

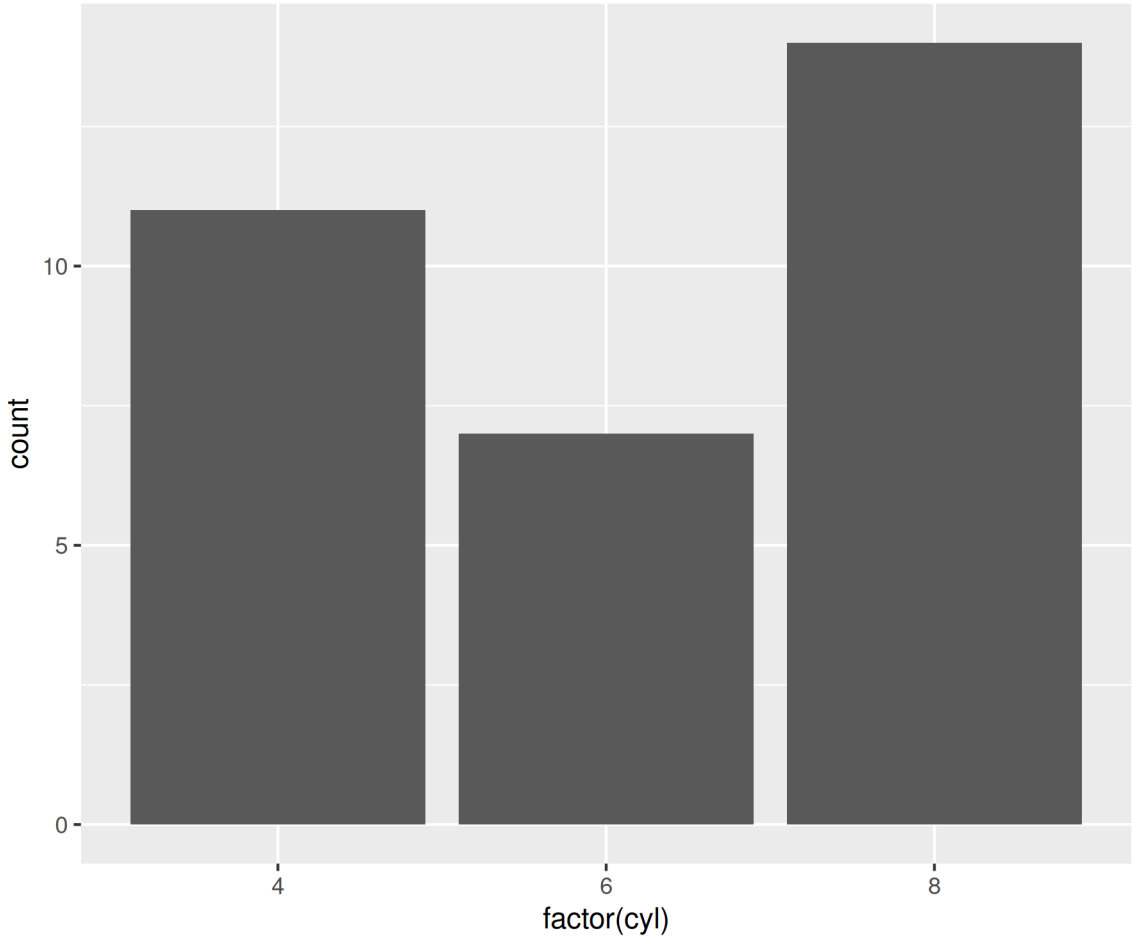


1.4 Bar Plot

A bar plot represents data in rectangular bars. The length of the bars are proportional to the values they represent. Bar plots can be either horizontal or vertical. The X axis of the plot represents the levels or the categories and the Y axis represents the frequency/count of the variable.

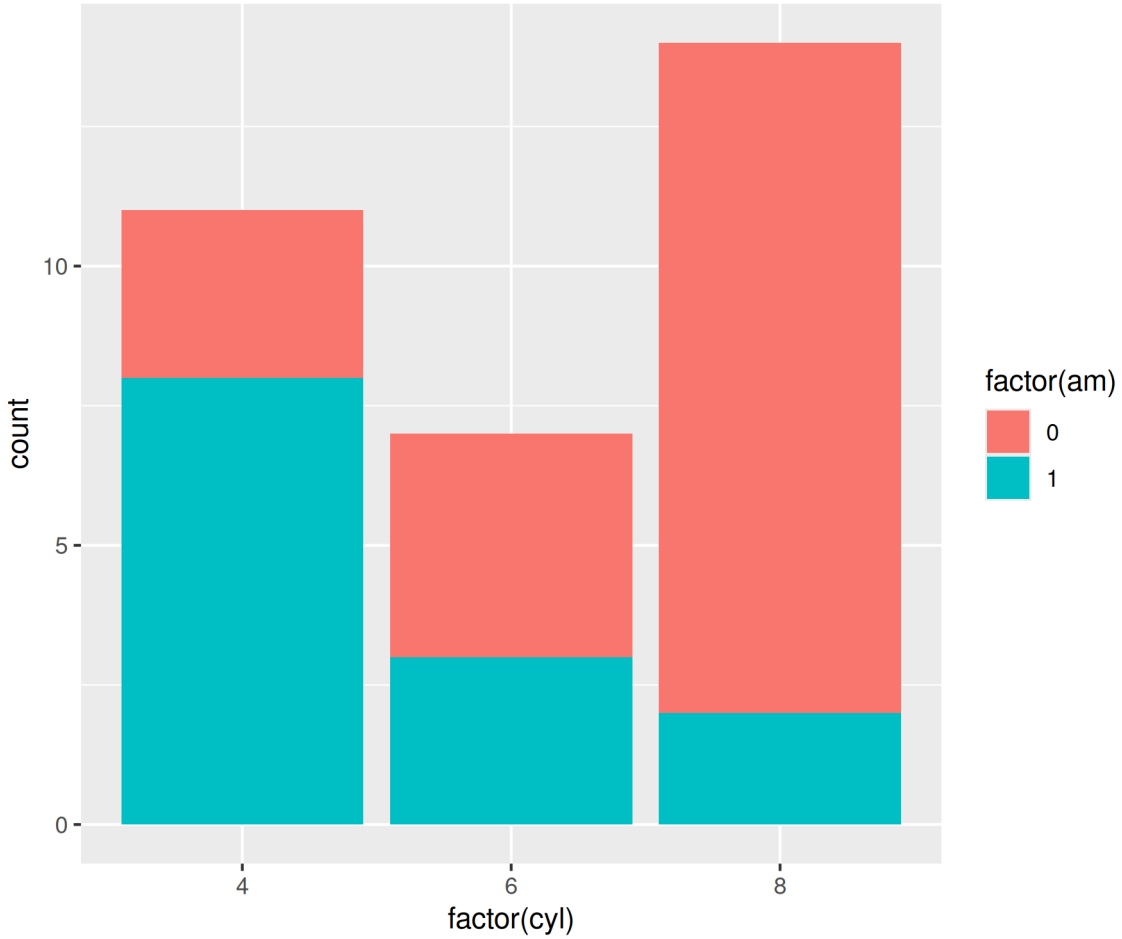
To create a bar plot, map a categorical variable to `x`. You can convert a variable to type `factor` (R equivalent of categorical) using the `factor()` function, and add `geom_bar()`.

```
ggplot(mtcars, aes(factor(cyl))) +
  geom_bar()
```



You can create a stacked bar plot by mapping another categorical variable to `fill`.

```
ggplot(mtcars, aes(factor(cyl), fill = factor(am))) +
  geom_bar()
```

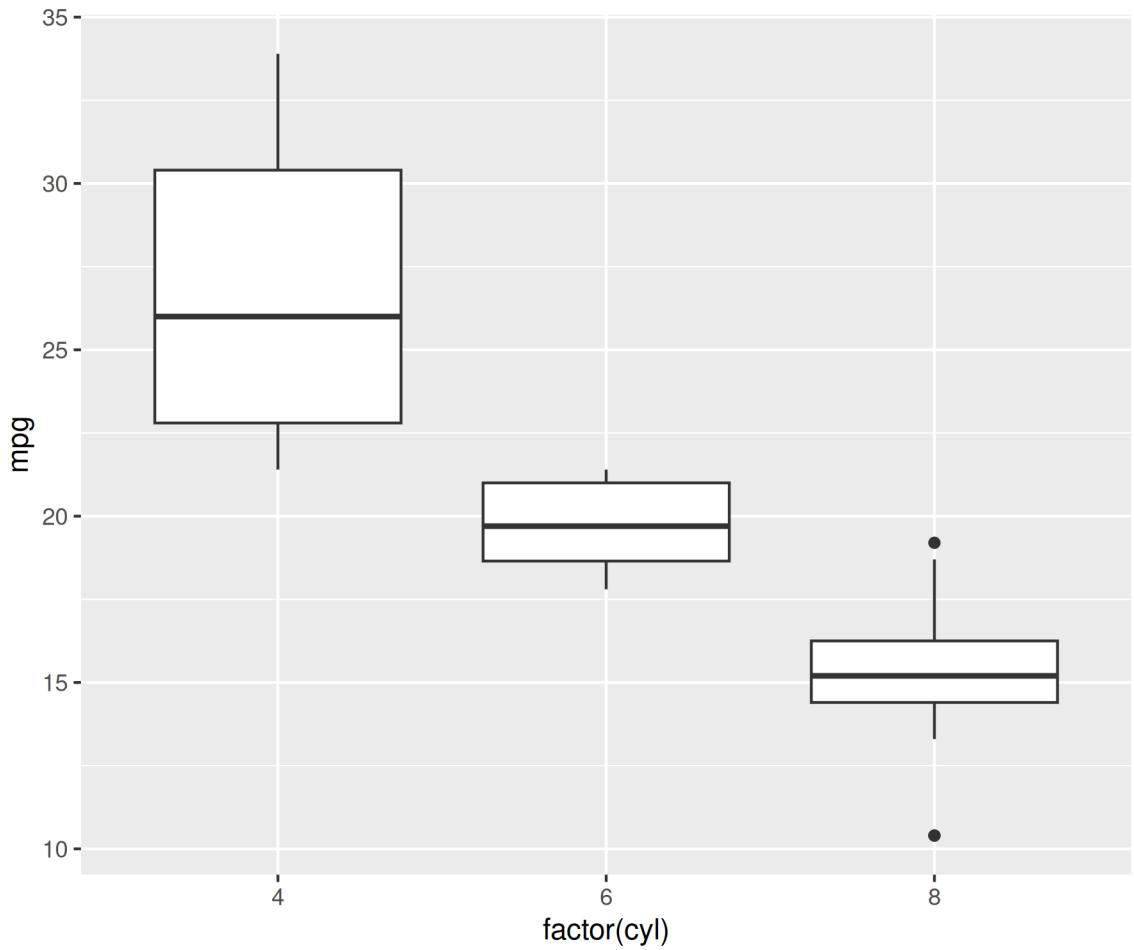


1.5 Box Plot

The box plot is a standardized way of displaying the distribution of data based on the five number summary: minimum, first quartile, median, third quartile, and maximum. Box plots are useful for detecting outliers and for comparing distributions. It shows the shape, central tendancy and variability of the data.

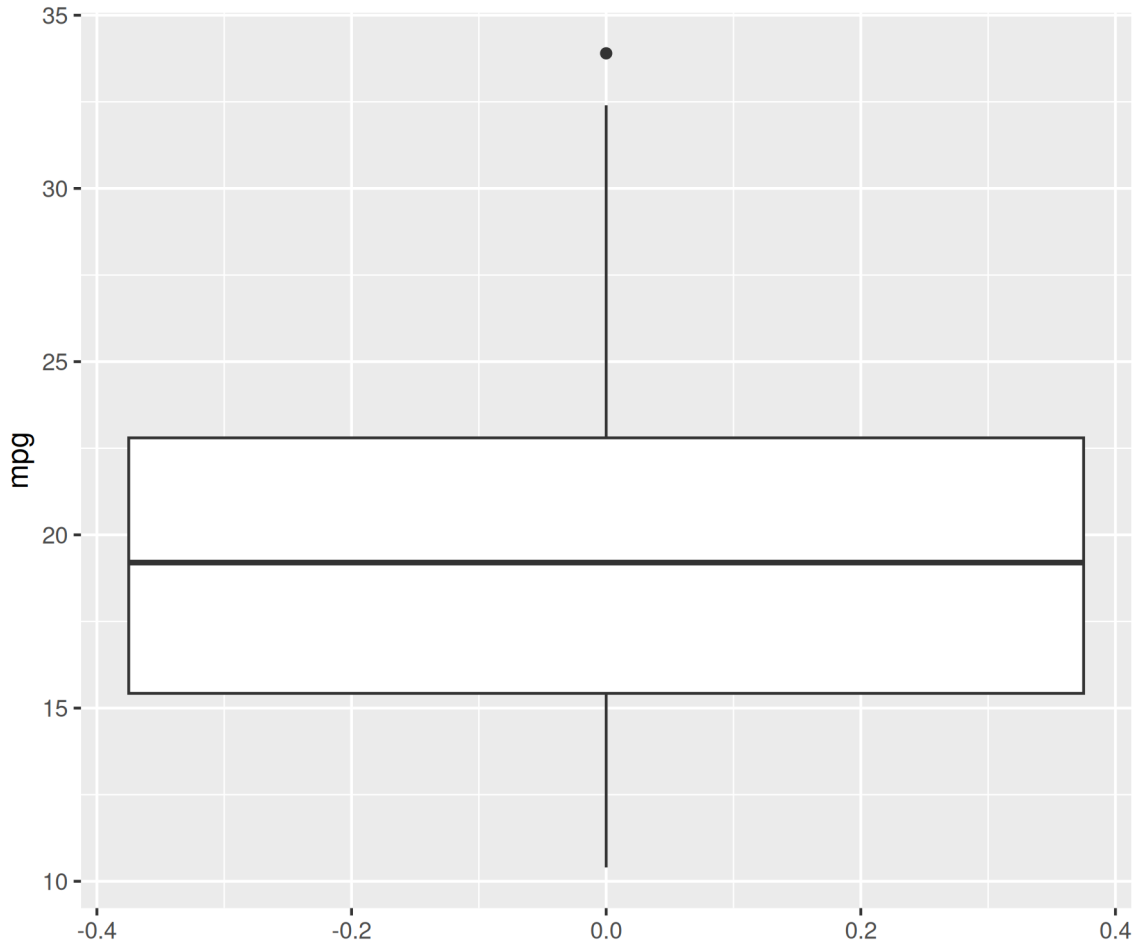
Box plots are created with `geom_boxplot()`. Map a categorical variable to `x` and a continuous variable to `y`.

```
ggplot(mtcars, aes(factor(cyl), mpg)) +
  geom_boxplot()
```



To plot the distribution of a single continuous variable without any grouping, map only `y`. Modern ggplot2 accepts a single-variable boxplot natively.

```
ggplot(mtcars, aes(y = mpg)) +
  geom_boxplot()
```



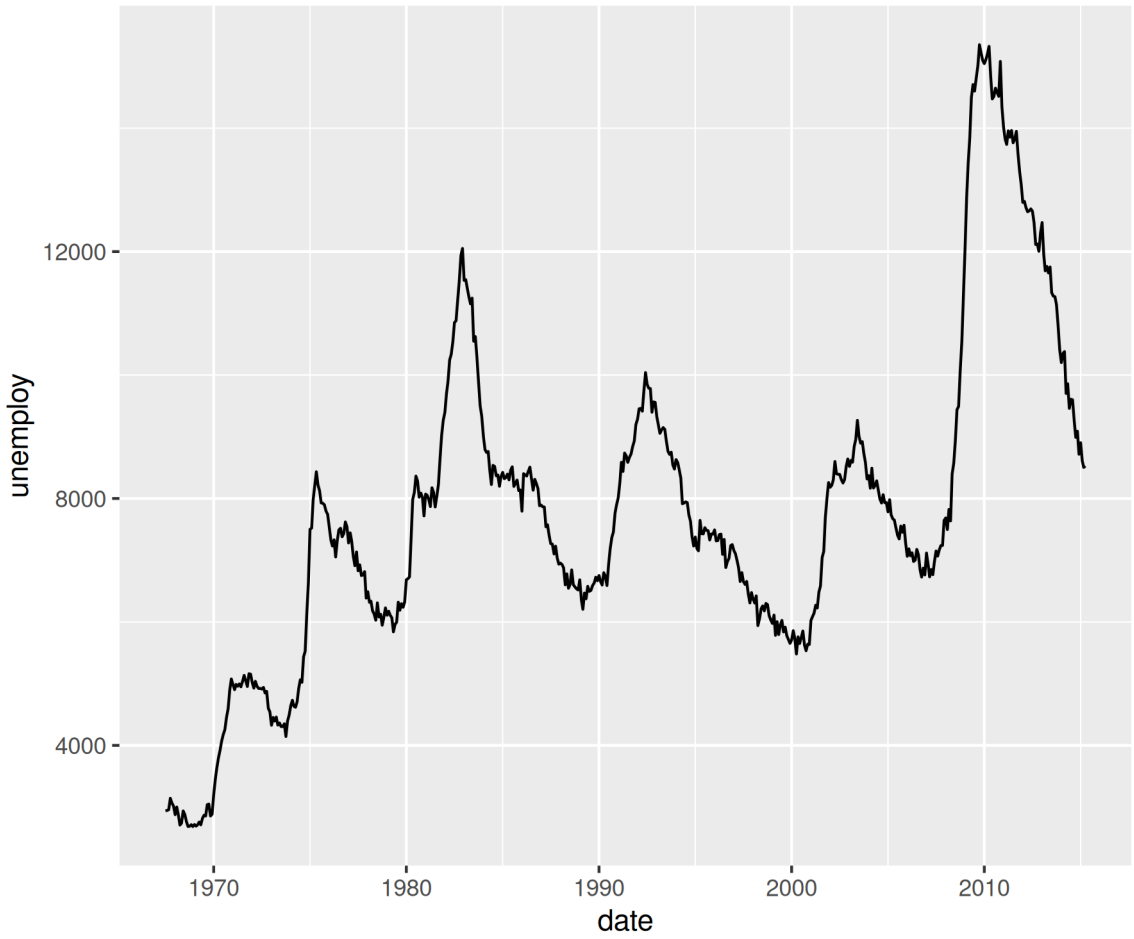
Tip

Older code (including earlier editions of this book) used `aes(x = factor(1))` as a dummy grouping variable for single boxplots. That workaround is no longer needed but still renders, so you may encounter it in the wild.

1.6 Line Chart

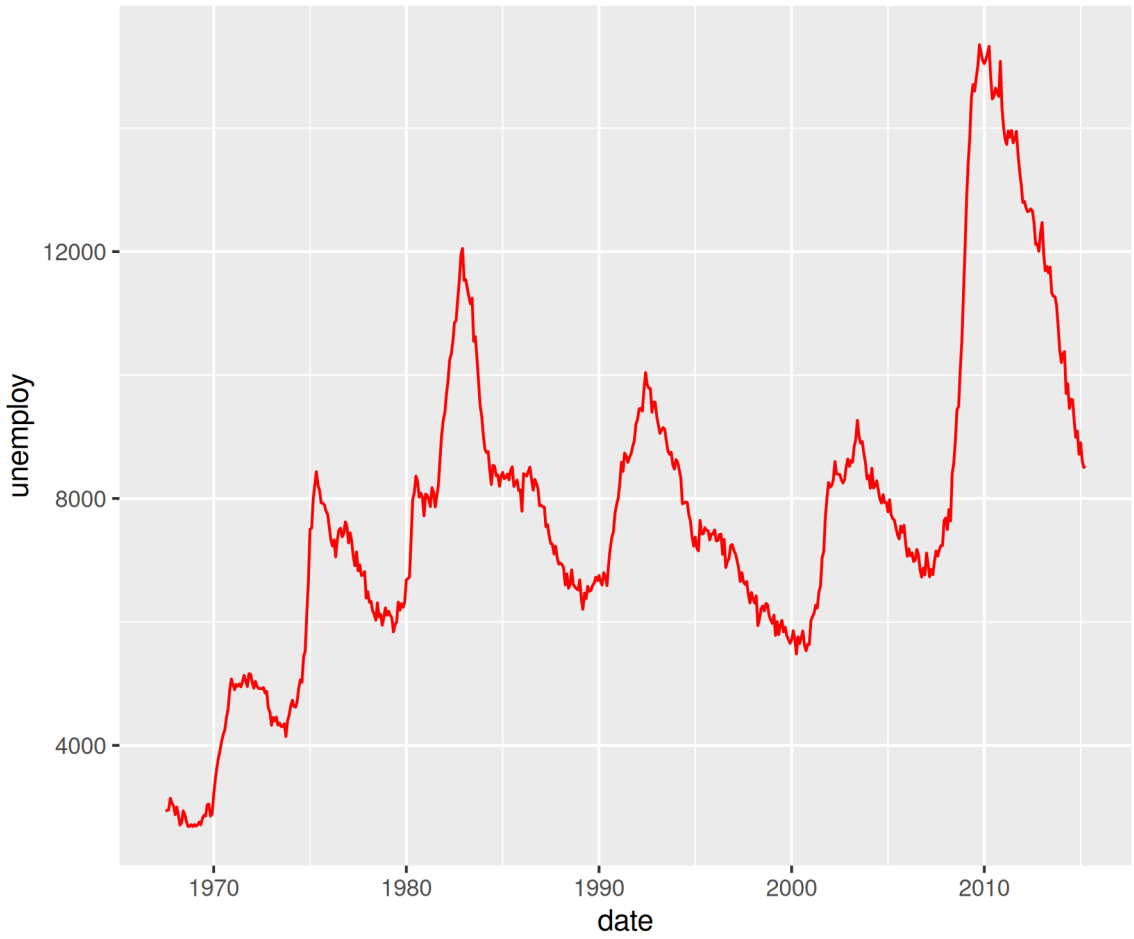
Line charts are used to examing trends across time. Map the time variable to `x`, the measurement to `y`, and add `geom_line()`.

```
ggplot(economics, aes(date, unemploy)) +
  geom_line()
```



The appearance of the line can be modified by setting the `color` outside `aes()` as shown below.

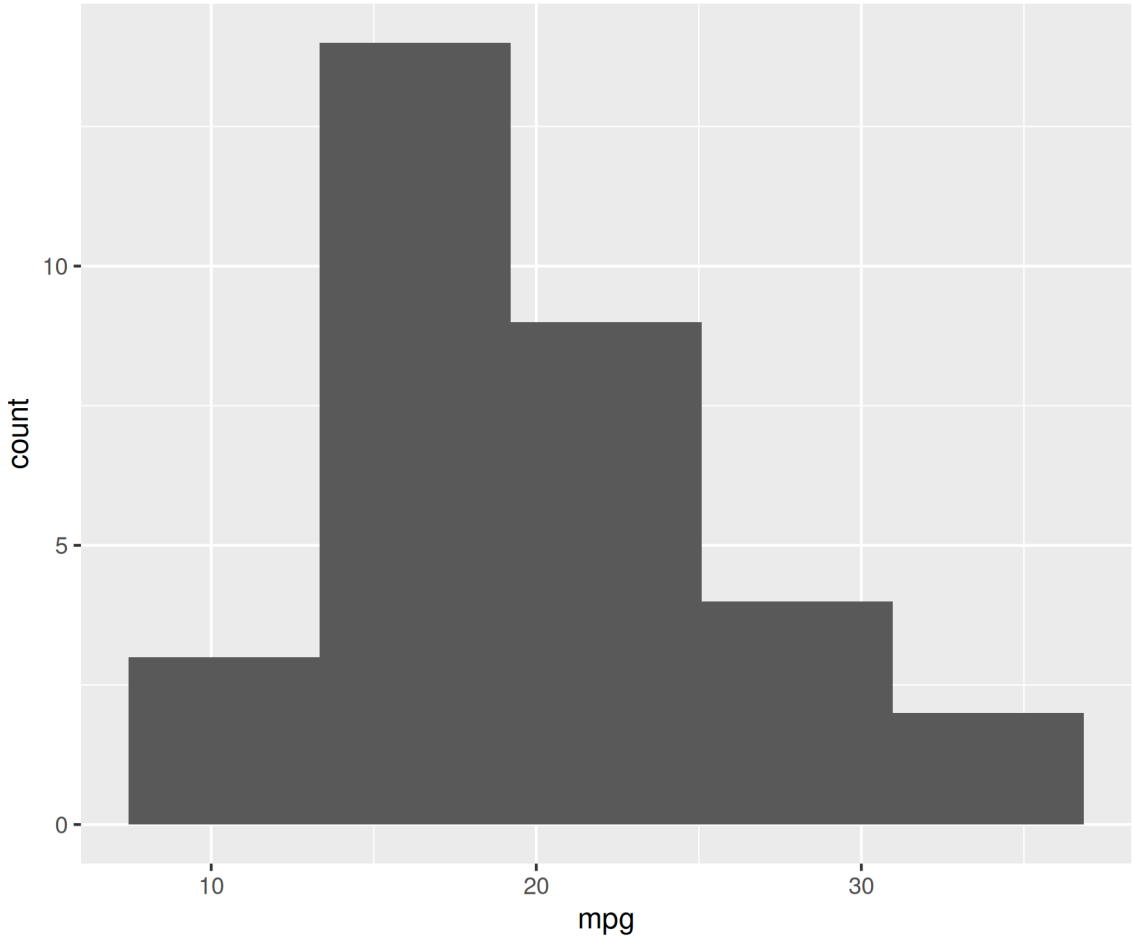
```
ggplot(economics, aes(date, unemploy)) +
  geom_line(color = 'red')
```



1.7 Histogram

A histogram is a plot that can be used to examine the shape and spread of continuous data. It looks very similar to a bar graph and can be used to detect outliers and skewness in data. Map the continuous variable to `x` and add `geom_histogram()`, controlling detail with `bins`.

```
ggplot(mtcars, aes(mpg)) +
  geom_histogram(bins = 5)
```



1.8 Exercises

1. Draw a `ggplot(mtcars)` scatter of `disp` vs `mpg` with `geom_point()`.
2. Add color by cylinders and a `labs()` title.
3. Explain in one sentence why `qplot()` is legacy code.

Worked solutions in `solutions/quicktour.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

2 Geoms

2.1 Introduction

> **Why this chart**
>
> Geoms are the marks readers see: points, lines, bars, smooths. Pick the geom that matches your data type.

> **Common pitfall**
>
> Layering geoms with different data without matching aesthetics silently misaligns marks.

In this chapter, we will create some of the most routinely used plots to explore data using the `geom_*` functions.

2.2 Libraries, Code & Data

We will use the following libraries in this chapter:

- readr
- ggplot2
- tibble
- dplyr

All the data sets used in this chapter can be found here and code can be downloaded from here.

2.2.1 Data

```
ecom <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master
ecom
```

```
# A tibble: 1,000 x 11
      id referrer device bouncers n_visit n_pages duration country      purchase
   <dbl> <chr>    <chr>  <lgl>      <dbl>   <dbl>    <dbl> <chr>        <lgl>
 1     1 google   laptop TRUE          10       1      693 Czech Repub~ FALSE
 2     2 yahoo    tablet TRUE           9       1      459 Yemen        FALSE
 3     3 direct   laptop TRUE           0       1      996 Brazil       FALSE
 4     4 bing     tablet FALSE          3      18      468 China        TRUE
 5     5 yahoo    mobile TRUE           9       1      955 Poland       FALSE
 6     6 yahoo    laptop FALSE          5       5      135 South Africa FALSE
 7     7 yahoo    mobile TRUE          10       1       75 Bangladesh   FALSE
 8     8 direct   mobile TRUE          10       1      908 Indonesia    FALSE
 9     9 bing     mobile FALSE          3      19      209 Netherlands  FALSE
10    10 google   mobile TRUE           6       1      208 Czech Repub~ FALSE
# i 990 more rows
# i 2 more variables: order_items <dbl>, order_value <dbl>
```

2.2.2 Data Dictionary

- id: row id
- referrer: referrer website/search engine
- device: device used to visit the website
- bouncers: whether the visit bounced (single page viewed)
- n_visit: number of visits
- n_pages: number of pages visited
- duration: time spent on the website (in seconds)
- country: country of origin
- purchase: whether visitor purchased
- order_items: number of items ordered
- order_value: order value of visitor (in dollars)

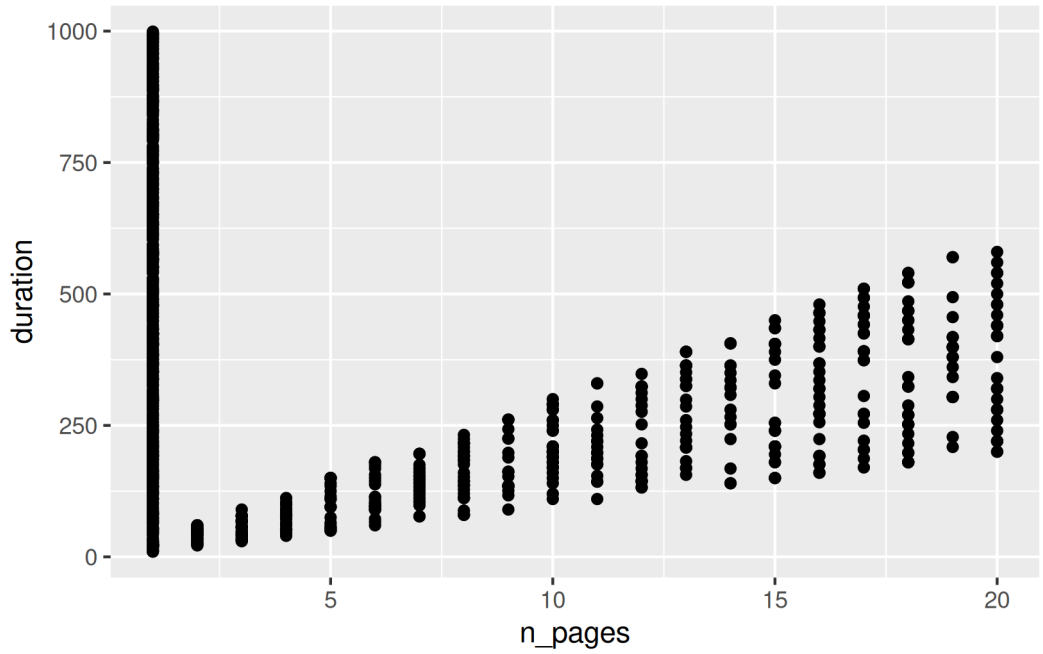
2.3 Point

A scatter plot displays the relationship between two continuous variables. In ggplot2, we can build a scatter plot using `geom_point()`. Scatter plots can show you visually

- the strength of the relationship between the variables
- the direction of the relationship between the variables
- and whether outliers exist

The variables representing the X and Y axis can be specified either in `ggplot()` or in `geom_point()`. We will learn to modify the appearance of the points in a different post.

```
ggplot(ecom, aes(x = n_pages, y = duration)) +
  geom_point()
```



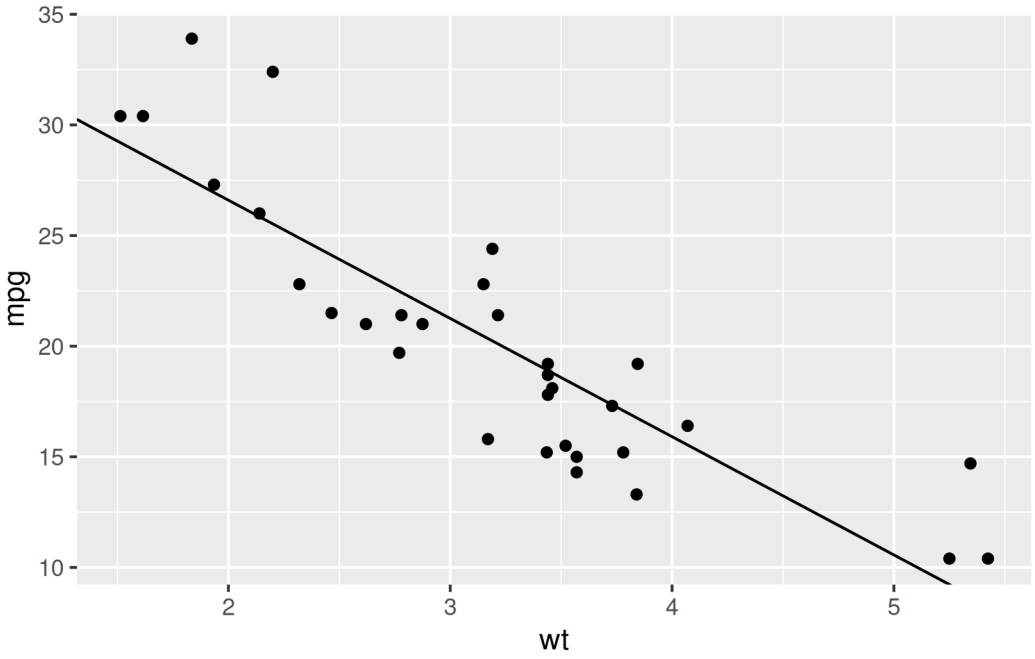
2.4 Regression Line

A regression line can be fit using either:

- `geom_abline()`
- `geom_smooth()`

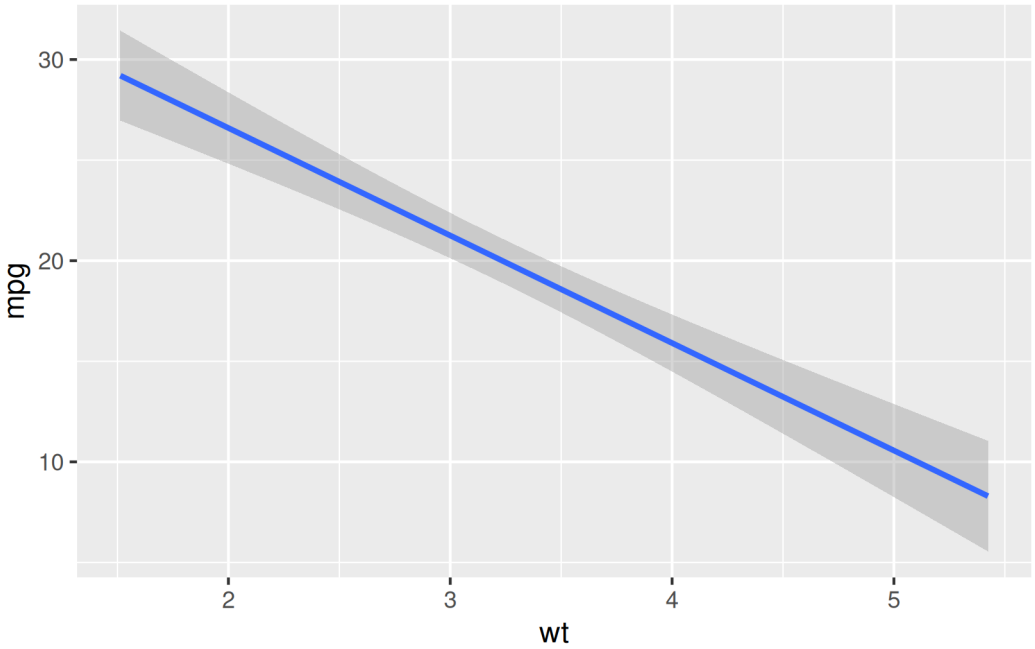
If you are using `geom_abline()`, you need to specify the intercept and slope as shown in the below example:

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_abline(intercept = 37.285, slope = -5.344)
```



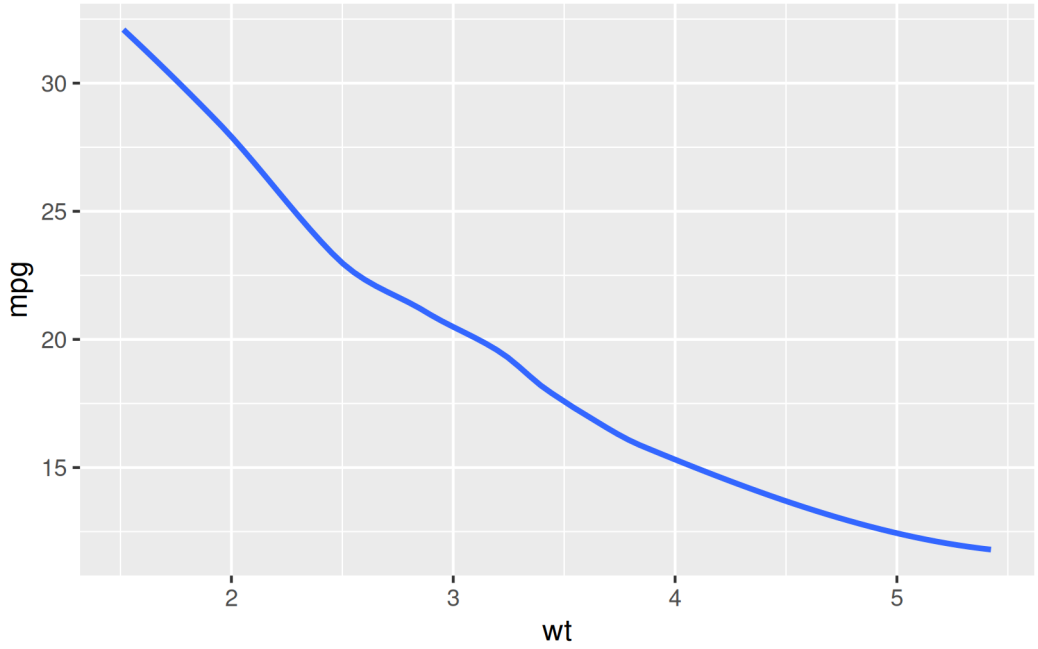
If you are using `geom_smooth()`, you need to specify the method of fitting the line, which can be `lm` or `loess`. You also need to indicate whether the confidence interval must be displayed using the `se` argument.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_smooth(method = 'lm', formula = y ~ x, se = TRUE)
```



Here we use the `'loess'` method to fit the regression line.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_smooth(method = 'loess', formula = y ~ x, se = FALSE)
```



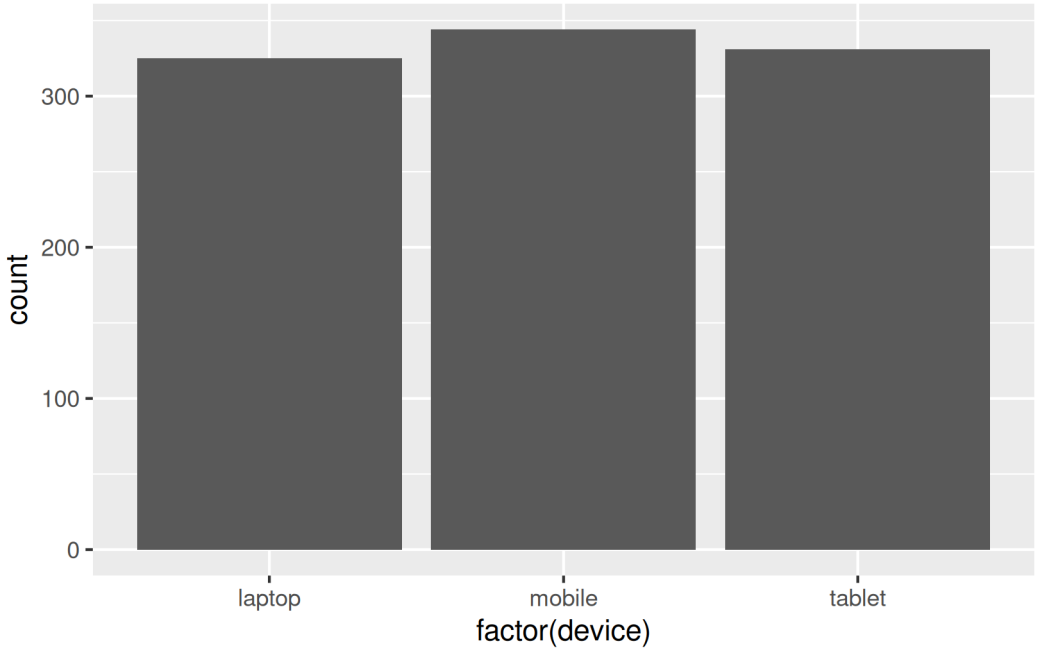
2.5 Bar

Bar plots present grouped data with rectangular bars. The bars may represent the frequency of the groups or values. Bar plots can be:

- horizontal
- vertical
- grouped
- stacked
- proportional

2.5.1 Frequency

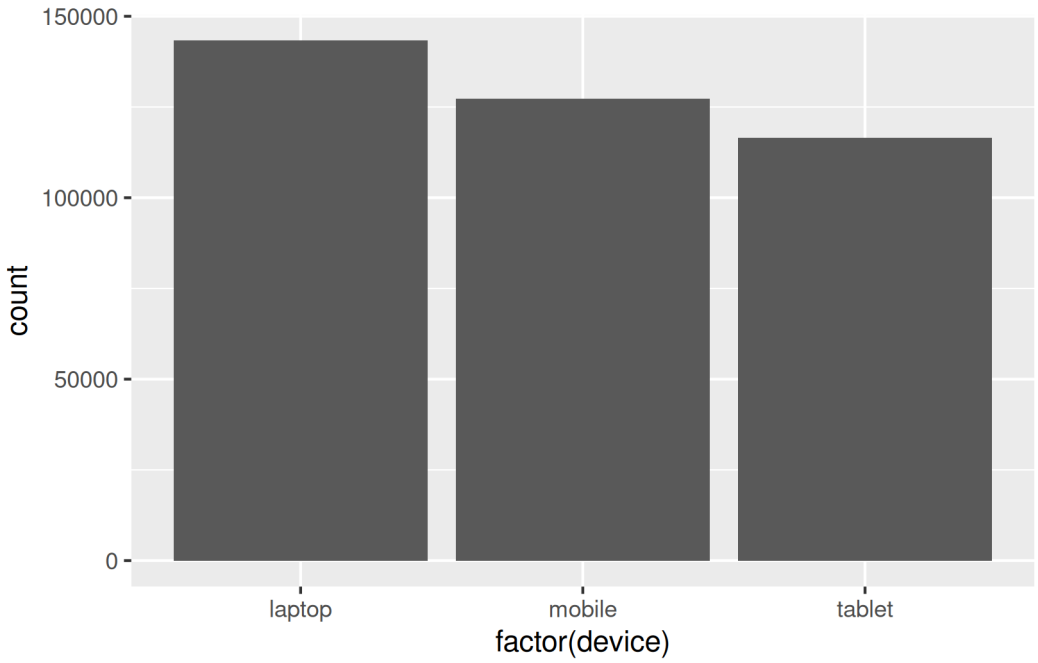
```
ggplot(ecom, aes(x = factor(device))) +
  geom_bar()
```



2.5.2 Weight

If the bars should represent a continuous variable, use the `weight` argument within `aes()`. In the below example, the bars do not represent the count of devices, instead, they represent the total order value for each device type.

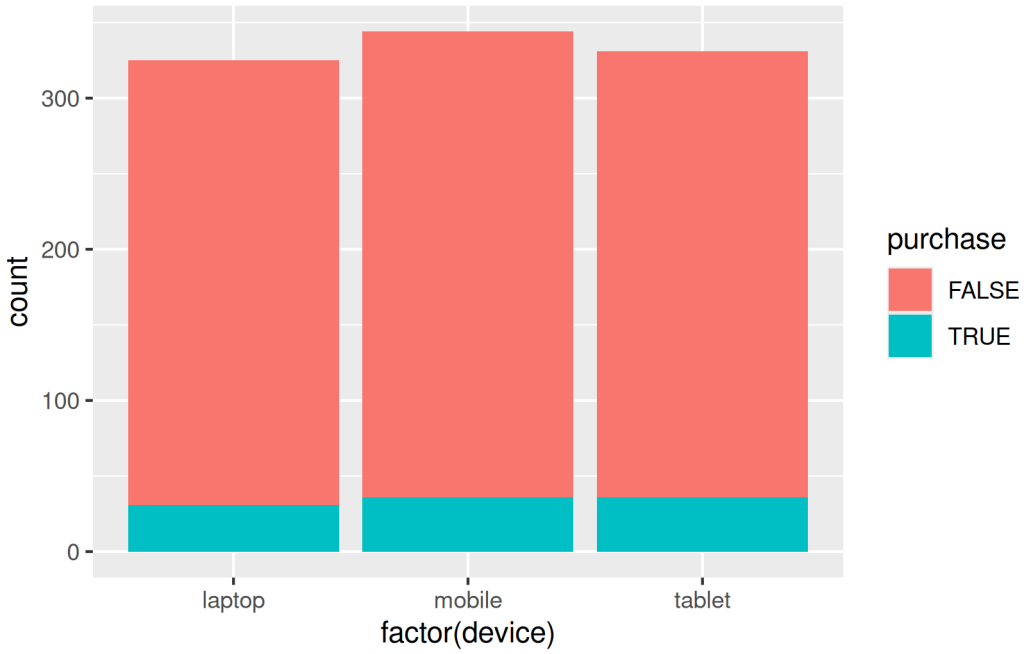
```
ggplot(ecom, aes(x = factor(device))) +
  geom_bar(aes(weight = order_value))
```



2.5.3 Stacked Bar Plot

To create a stacked bar plot, the `fill` argument must be mapped to a categorical variable.

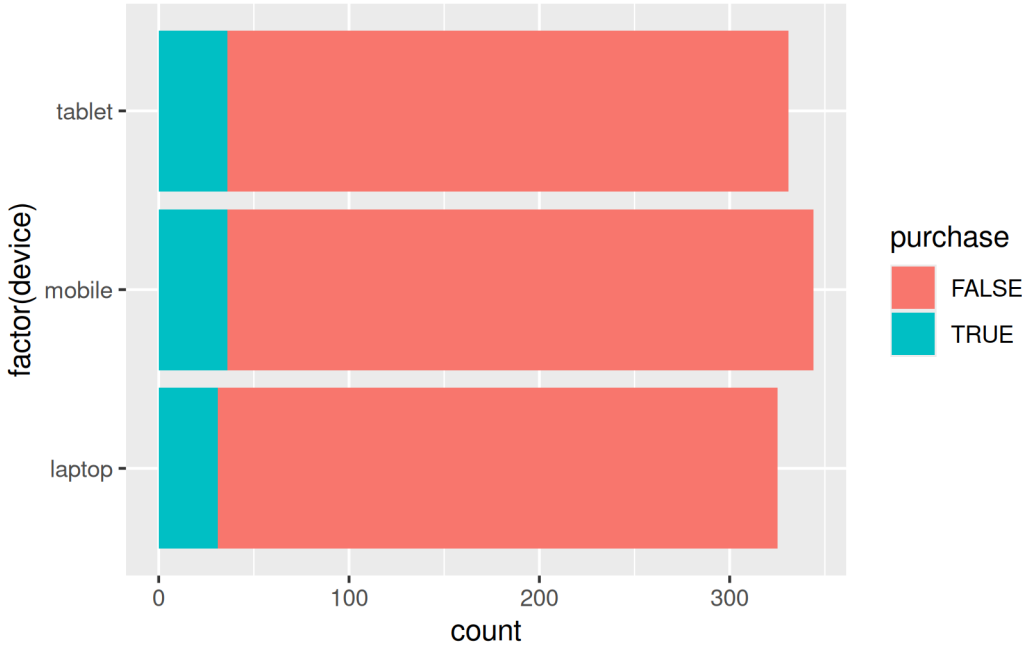
```
ggplot(ecom, aes(x = factor(device))) +
  geom_bar(aes(fill = purchase))
```



2.5.4 Horizontal Bar Plot

A horizontal bar plot can be created by flipping the coordinate axes using the `coord_flip()` function.

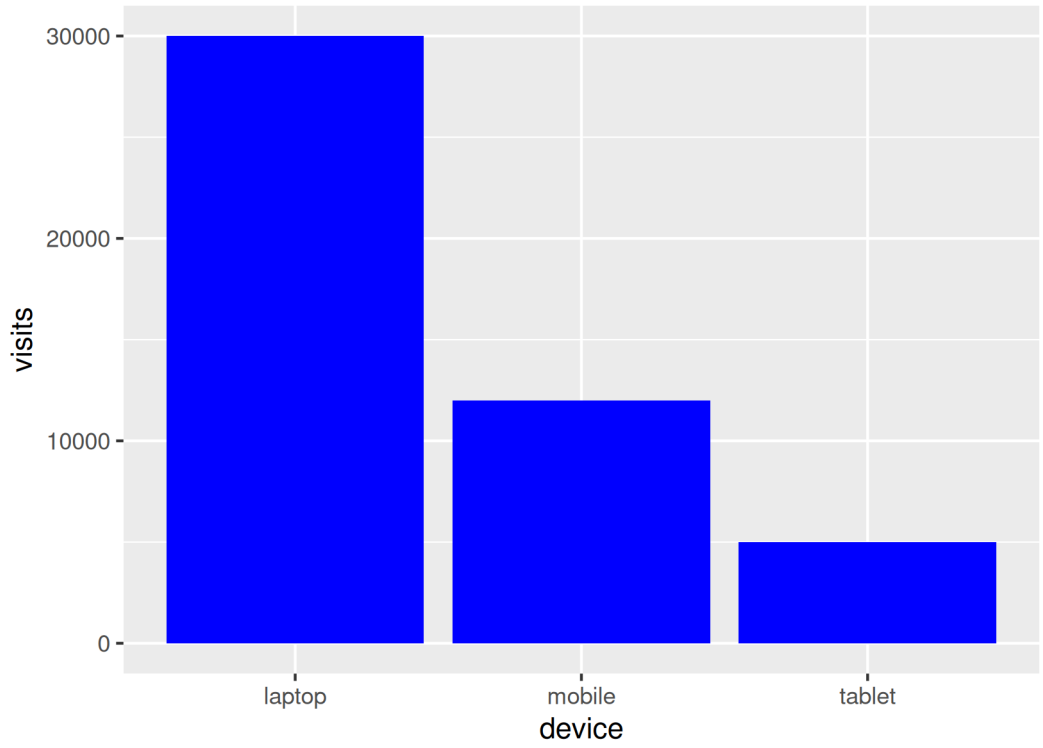
```
ggplot(ecom, aes(x = factor(device))) +
  geom_bar(aes(fill = purchase)) +
  coord_flip()
```



2.6 Columns

If the data has already been summarized, you can use `geom_col()` instead of `geom_bar()`. In the below example, we have the total visits for each device type. The data has already been summarized and as such we cannot use `geom_bar()`.

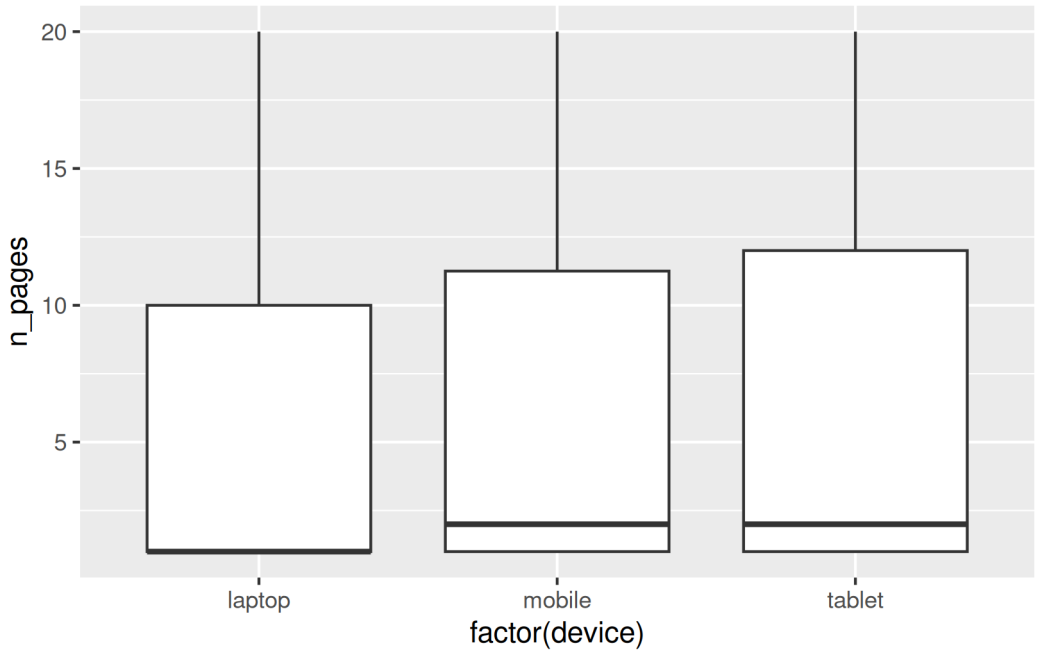
```
device <- c('laptop', 'mobile', 'tablet')
visits <- c(30000, 12000, 5000)
traffic <- tibble::tibble(device, visits)
ggplot(traffic, aes(x = device, y = visits)) +
  geom_col(fill = 'blue')
```



2.7 Boxplot

The box plot is a standardized way of displaying the distribution of data based on the five number summary: minimum, first quartile, median, third quartile, and maximum. Box plots are useful for detecting outliers and for comparing distributions. It shows the shape, central tendancy and variability of the data. Use `geom_boxplot()` to create a box plot.

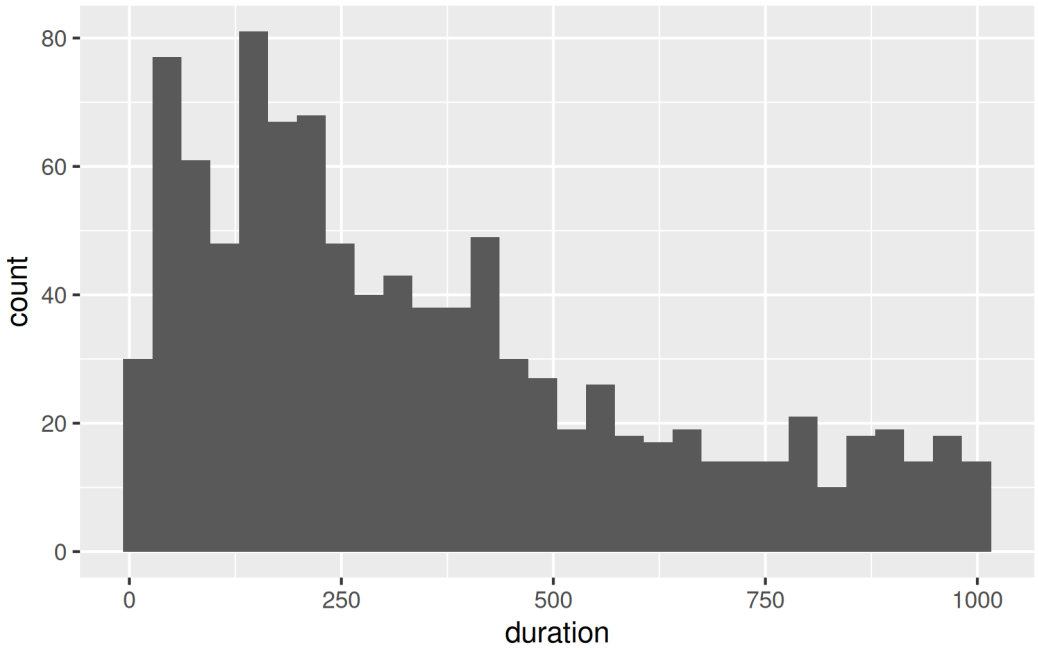
```
ggplot(ecom, aes(x = factor(device), y = n_pages)) +
  geom_boxplot()
```



2.8 Histogram

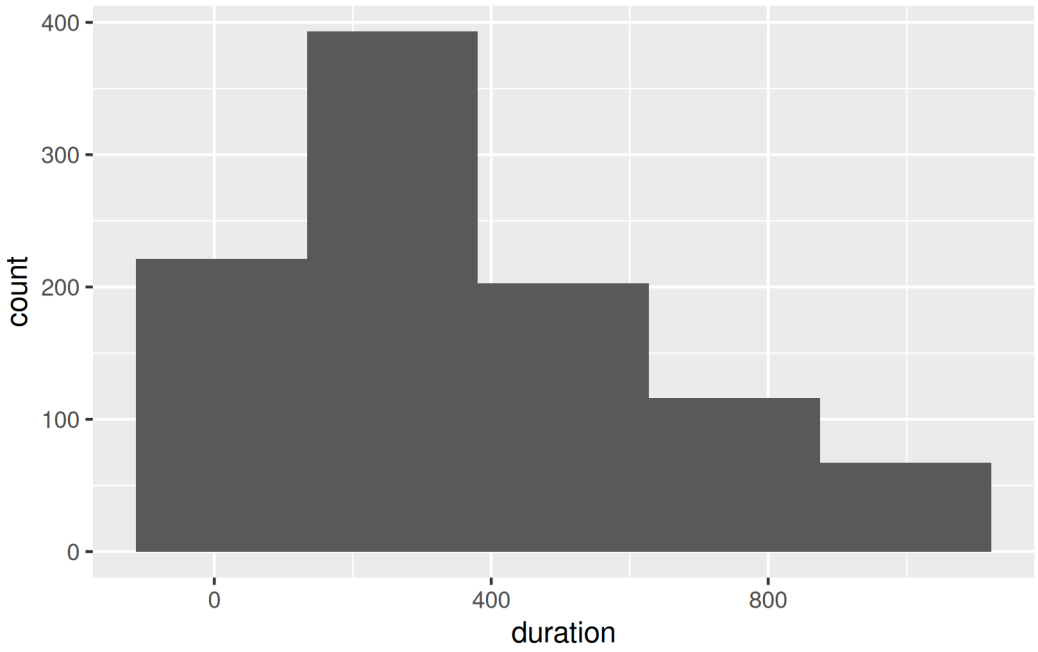
A histogram is a plot that can be used to examine the shape and spread of continuous data. It looks very similar to a bar graph and can be used to detect outliers and skewness in data. Use `geom_histogram()` to create a histogram.

```
ggplot(ecom, aes(x = duration)) +
  geom_histogram()
```



You can control the number of bins using the `bins` argument.

```
ggplot(ecom, aes(x = duration)) +
  geom_histogram(bins = 5)
```



2.9 Line

Line charts are used to examine trends over time. We will use a different data set for exploring line plots.

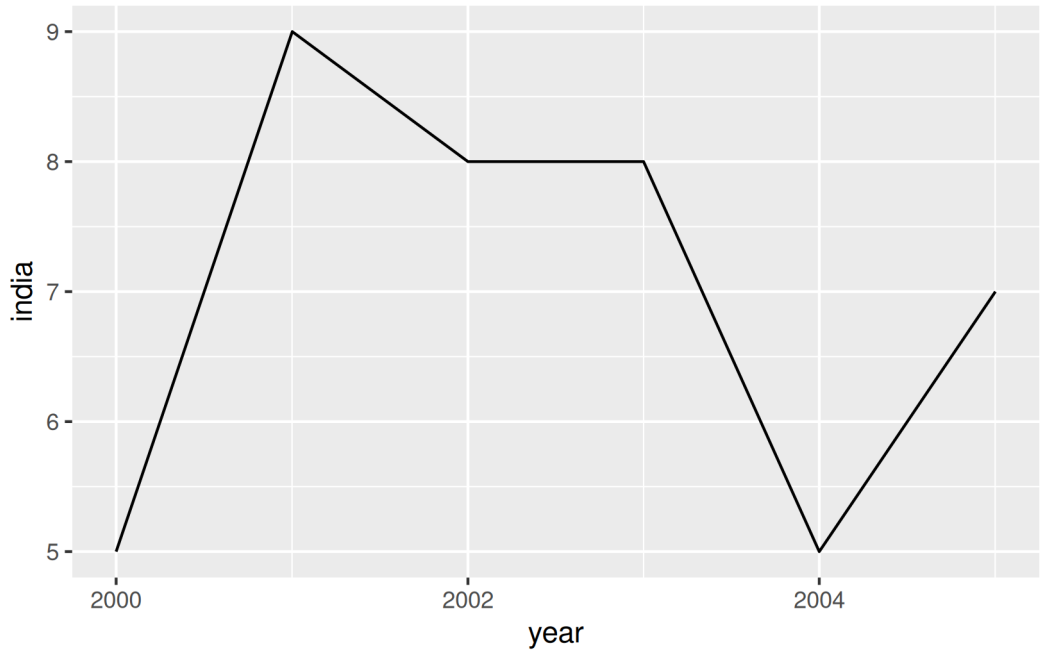
2.9.1 Data

```
gdp <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/
gdp
```

```
# A tibble: 6 x 6
   ...1     X year       growth india china
  <dbl> <dbl> <date>      <dbl> <dbl> <dbl>
1     1     1 2000-01-01      6     5     8
2     2     2 2001-01-01      9     9     5
3     3     3 2002-01-01      8     8     6
4     4     4 2003-01-01      9     8     8
5     5     5 2004-01-01      9     5     9
6     6     6 2005-01-01      8     7     8
```

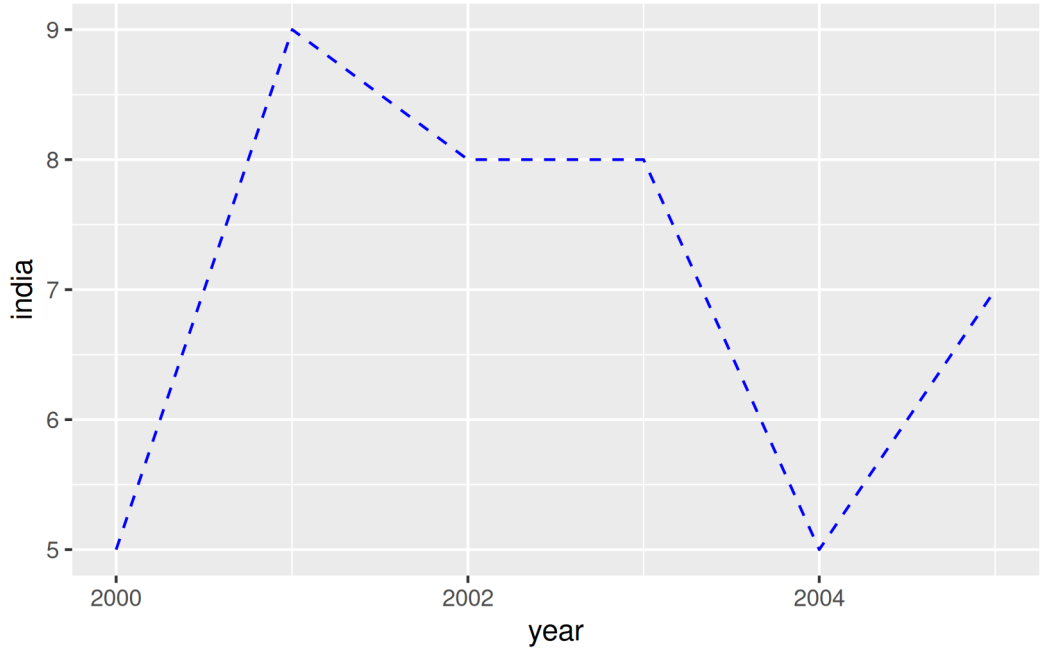
Use `geom_line()` to create a line chart. In the below plot, we chart the GDP of India, the fastest growing economy in emerging markets, across years.

```
ggplot(gdp, aes(year, india)) +
  geom_line()
```



The color and line type can be modified using the `color` and `linetype` arguments. We will explore the different line types in an upcoming post.

```
ggplot(gdp, aes(year, india)) +
  geom_line(color = 'blue', linetype = 'dashed')
```



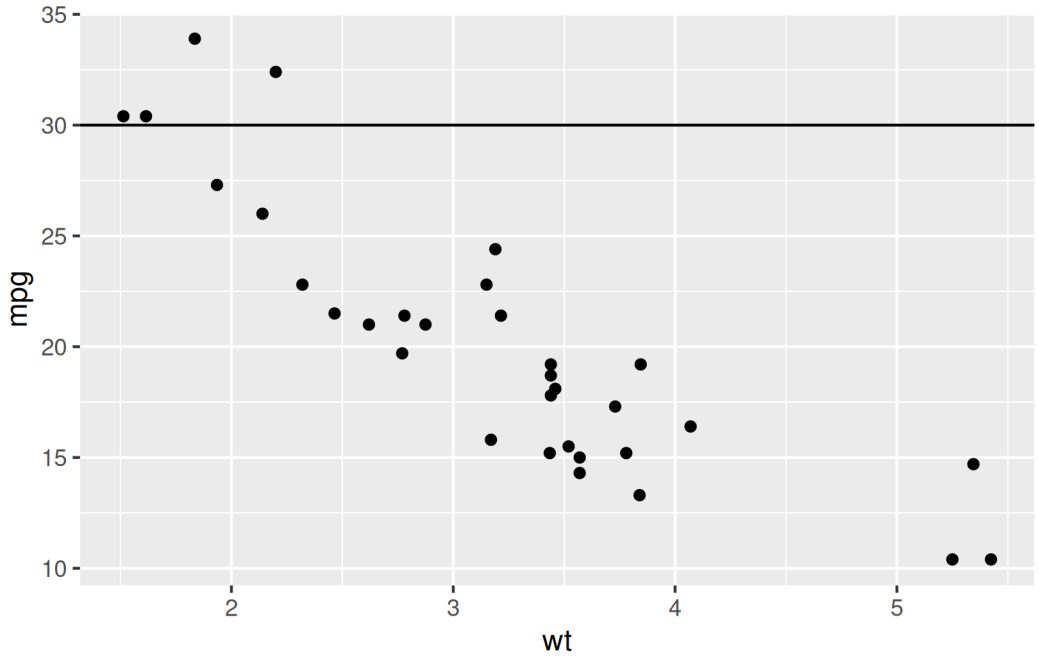
Add horizontal or vertical lines using

- `geom_hline()`
- `geom_vline()`

2.9.2 Horizontal Line

To add a horizontal line, the Y axis intercept must be supplied using the `yintercept` argument.

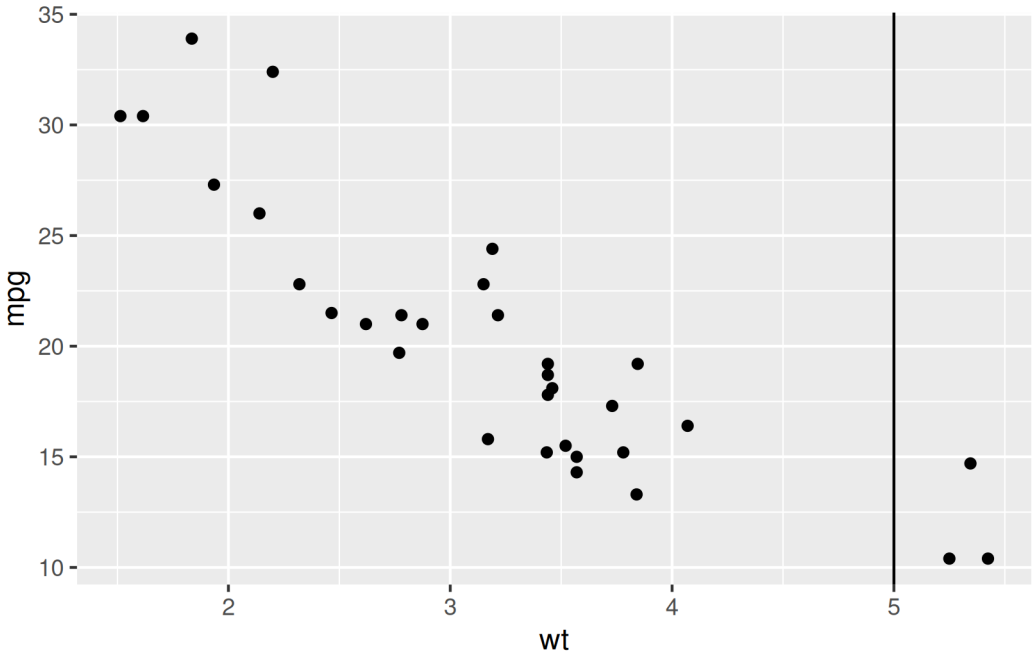
```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_hline(yintercept = 30)
```



2.9.3 Vertical Line

For the vertical line, the X axis intercept must be supplied using the `xintercept` argument.

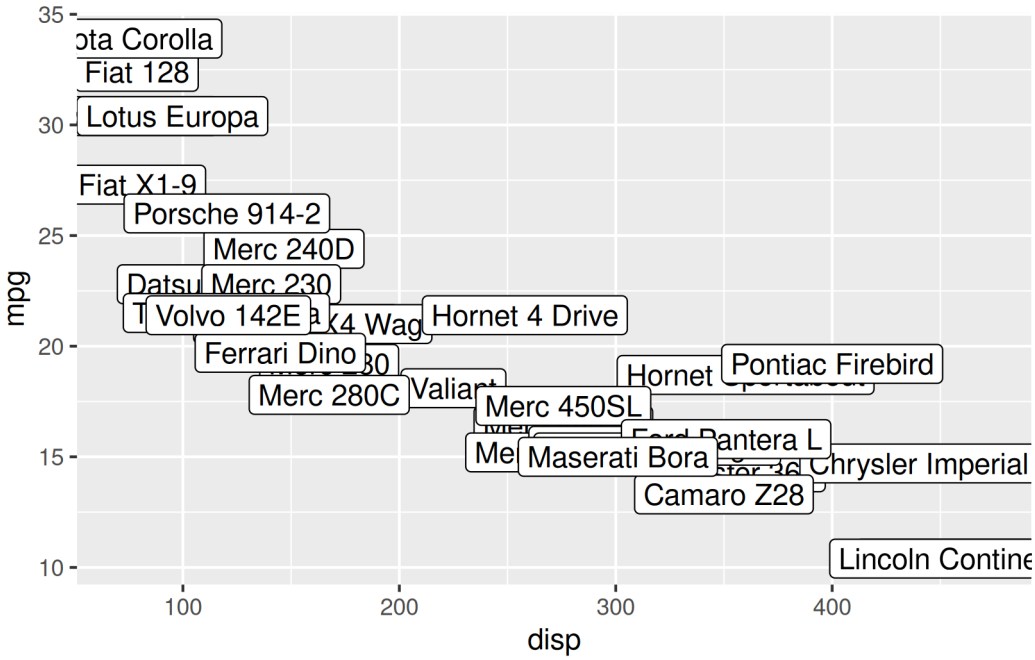
```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_vline(xintercept = 5)
```



2.10 Label

You can label the points using `geom_label()`.

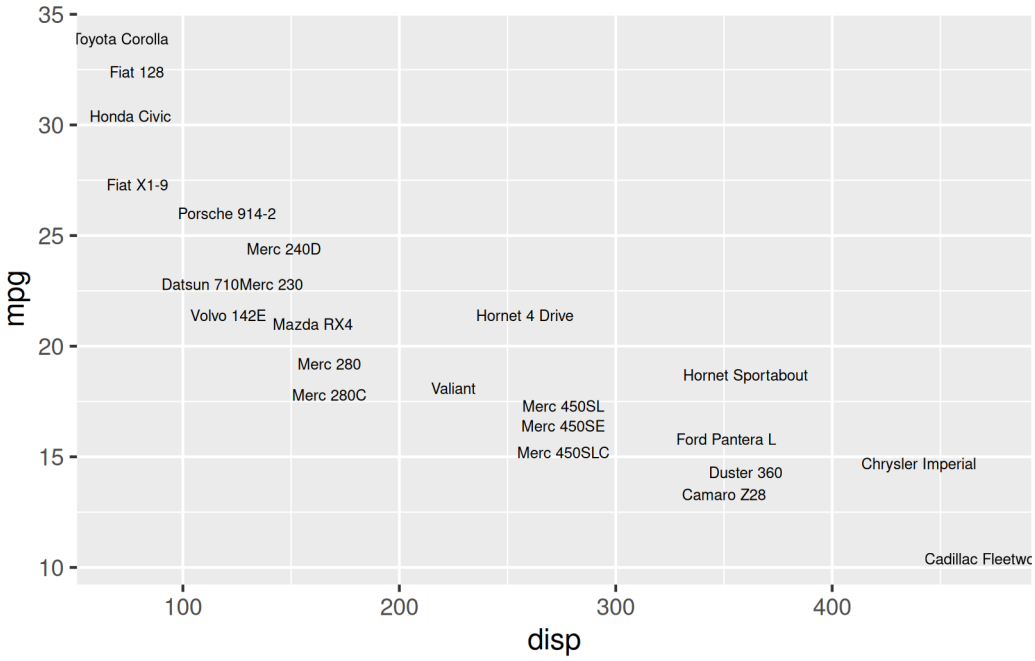
```
ggplot(mtcars, aes(disp, mpg, label = rownames(mtcars))) +
  geom_label()
```



2.11 Text

`geom_text()` offers another way to add text to the plots. We will learn to modify the appearance and location of the text in another post.

```
ggplot(mtcars, aes(disp, mpg, label = rownames(mtcars))) +
  geom_text(check_overlap = TRUE, size = 2)
```



2.12 Exercises

1. Draw `disp` vs `mpg` points plus a `geom_smooth()` trend (`formula = y ~ x`).
2. Draw a histogram of `mpg` with 10 bins.
3. Pick the right geom for median mileage per cylinder class and justify it.

Worked solutions in `solutions/geoms.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

3 Aesthetics

3.1 Introduction

> **Why this chart**
>
> Aesthetics map variables to position, color, size and shape so patterns pop out.

> **Common pitfall**
>
> Putting a constant inside aes() (e.g. aes(color='red')) creates a bogus legend; constants belong outside aes().

In this chapter, we will focus on the aesthetics i.e. color, shape, size, alpha, line type, line width etc. We can map these to variables or specify values for them. If we want to map the above to variables, we have to specify them within the `aes()` function. We will look at both methods in the following sections.

Explore aesthetics such as

- color
- shape
- size
- fill
- alpha
- width

3.2 Libraries, Code & Data

We will use the following libraries in this chapter:

- readr
- ggplot2
- dplyr
- tidyr

All the data sets used in this chapter can be found here and code can be downloaded from here.

3.2.1 Data

```
ecom <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master
ecom
```

```
# A tibble: 1,000 x 11
      id referrer device bouncers n_visit n_pages duration country      purchase
   <dbl> <chr>    <chr>  <lgl>      <dbl>   <dbl>    <dbl> <chr>        <lgl>
 1     1 google   laptop TRUE          10       1      693 Czech Repub~ FALSE
 2     2 yahoo    tablet TRUE           9       1      459 Yemen        FALSE
 3     3 direct   laptop TRUE           0       1      996 Brazil       FALSE
 4     4 bing     tablet FALSE          3      18      468 China        TRUE
 5     5 yahoo    mobile TRUE           9       1      955 Poland       FALSE
 6     6 yahoo    laptop FALSE          5       5      135 South Africa FALSE
 7     7 yahoo    mobile TRUE          10       1       75 Bangladesh   FALSE
 8     8 direct   mobile TRUE          10       1      908 Indonesia    FALSE
 9     9 bing     mobile FALSE          3      19      209 Netherlands  FALSE
10    10 google   mobile TRUE           6       1      208 Czech Repub~ FALSE
# i 990 more rows
# i 2 more variables: order_items <dbl>, order_value <dbl>
```

3.2.2 Data Dictionary

- id: row id
- referrer: referrer website/search engine
- device: device used to visit the website
- bouncers: whether the visit bounced (single page viewed)
- n_visit: number of visits
- n_pages: number of pages visited
- duration: time spent on the website (in seconds)
- country: country of origin
- purchase: whether visitor purchased
- order_items: number of items ordered
- order_value: order value of visitor (in dollars)

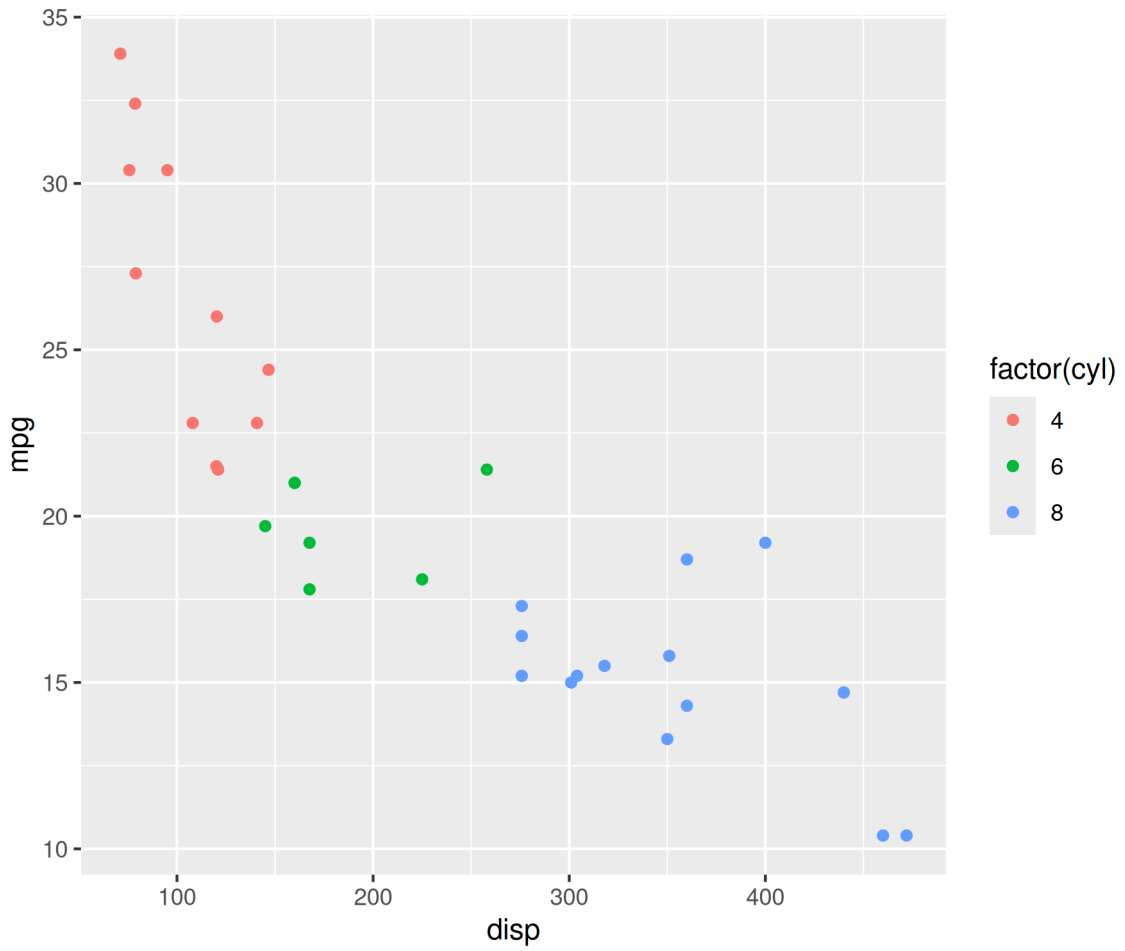
3.3 Color

In ggplot2, when we mention `color` or `colour`, it usually refers to the color of the geoms. The `fill` argument is used to specify the color of the shapes in certain cases. In this first section, we will see how we can specify the color for the different geoms we learnt in the previous chapter.

3.3.1 Point

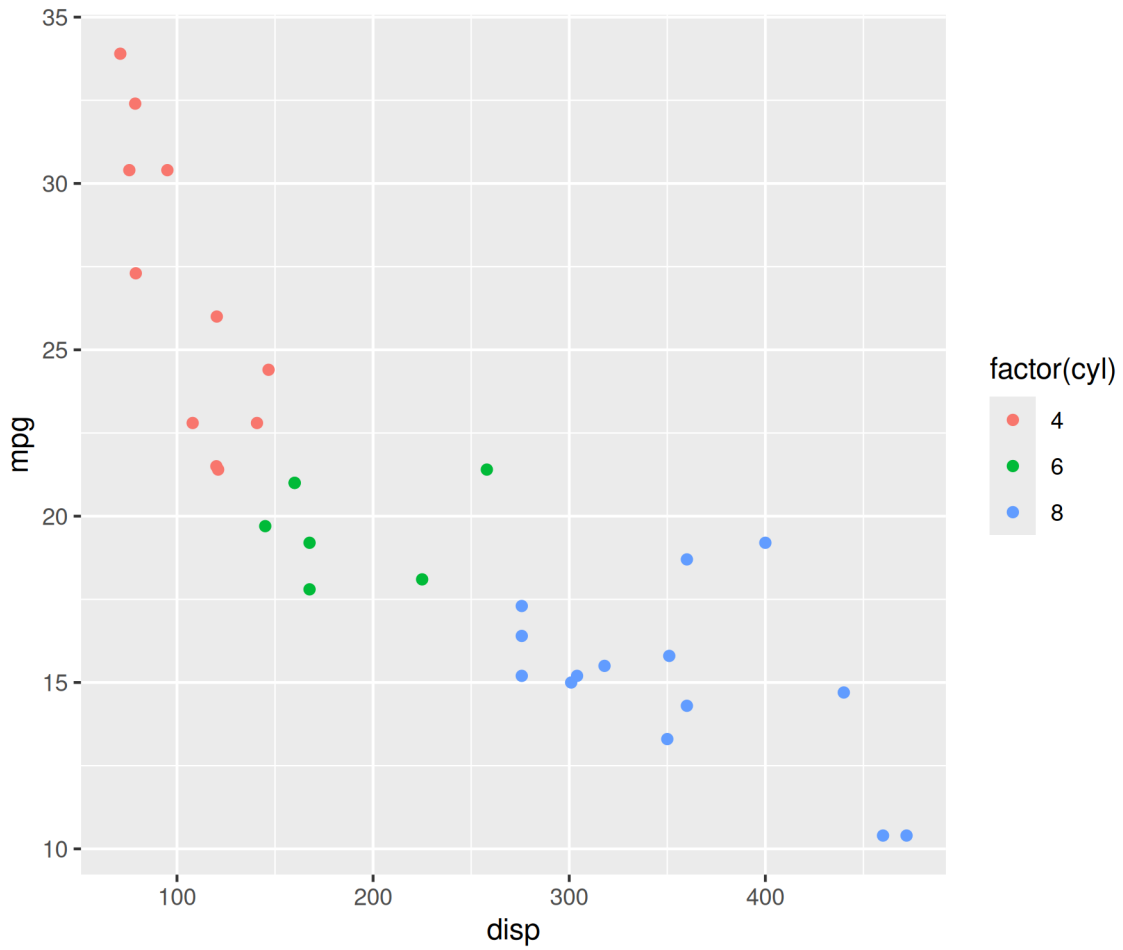
For points, the `color` argument specifies the color of the point for certain shapes and border for others. The `fill` argument is used to specify the background for some shapes and will not work with other shapes. Let us look at an example:

```
ggplot(mtcars, aes(x = disp, y = mpg, color = factor(cyl))) +
  geom_point()
```



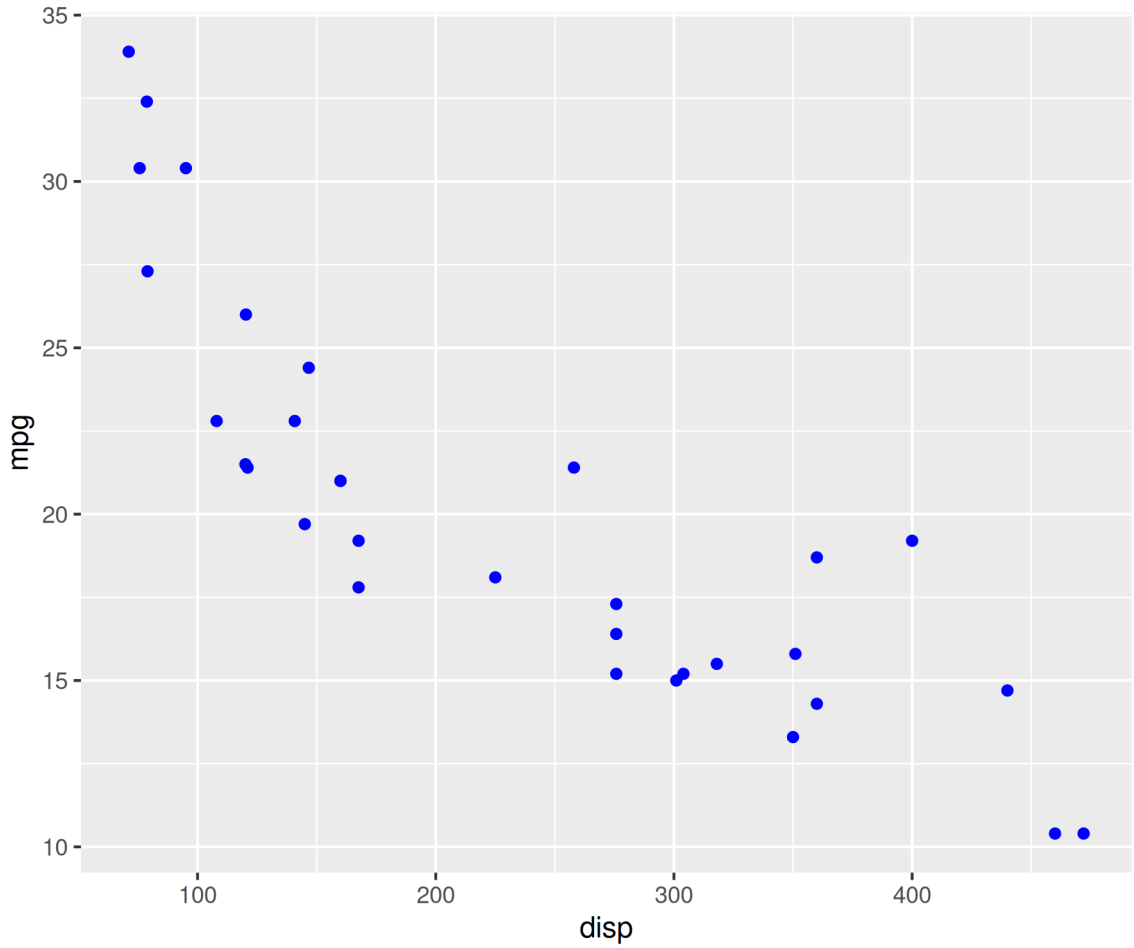
We can map the variable to color in the `geom_point()` function as well since it inherits the data from the `ggplot()` function.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(aes(color = factor(cyl)))
```



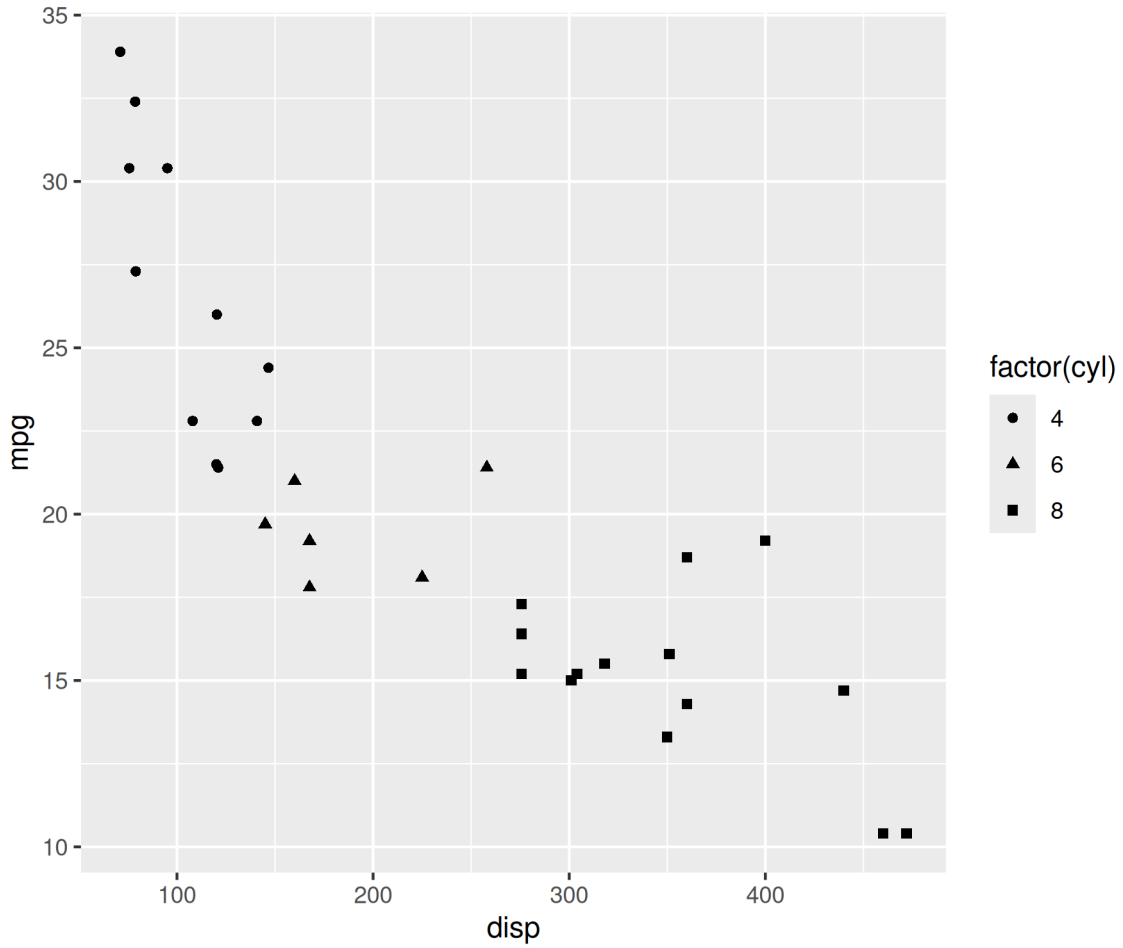
If you do not want to map a variable to color, you can specify it separately using the `color` argument but in this case it should be outside the `aes()` function.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(color = 'blue')
```



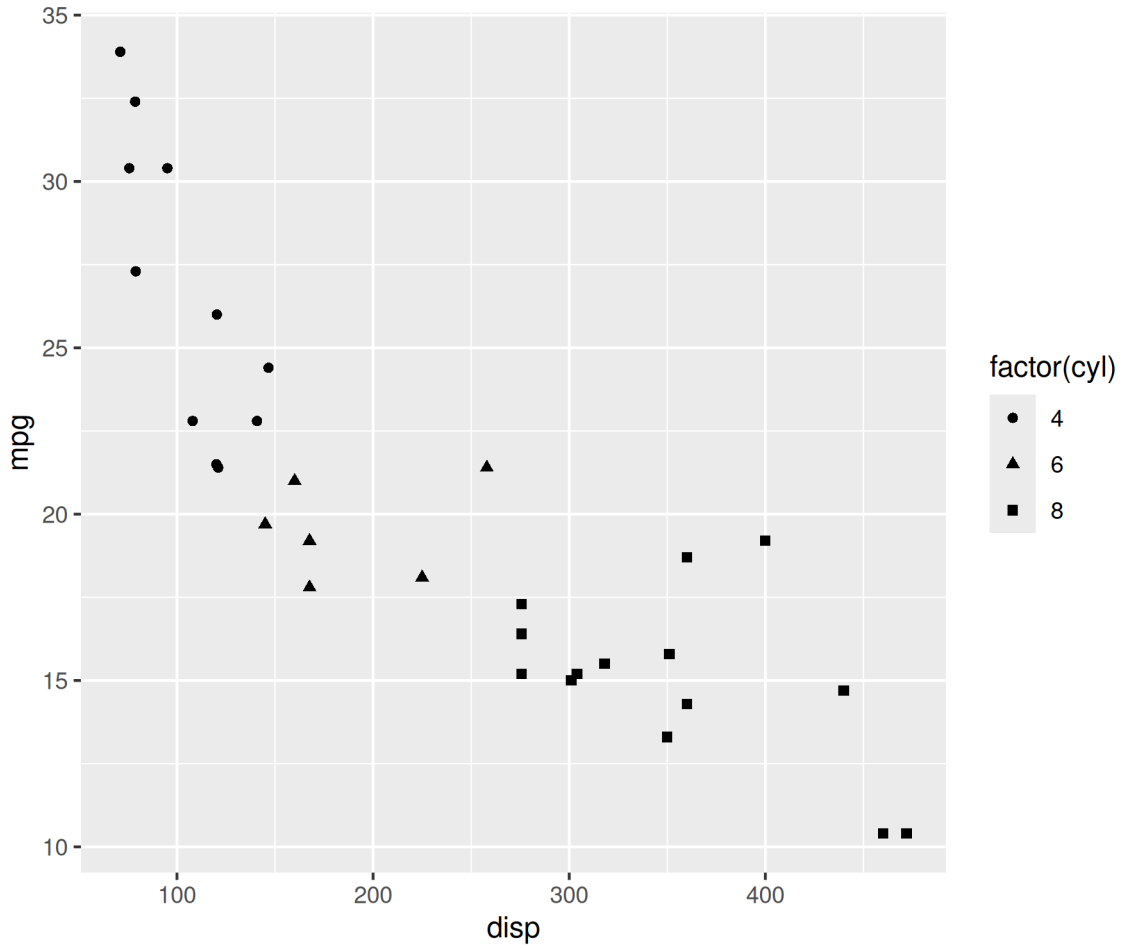
Now we will change the shape of the points to understand the difference between `color` and `fill` arguments. It can be again mapped to variables or values. Let us map shape to variables.

```
ggplot(mtcars, aes(x = disp, y = mpg, shape = factor(cyl))) +
  geom_point()
```



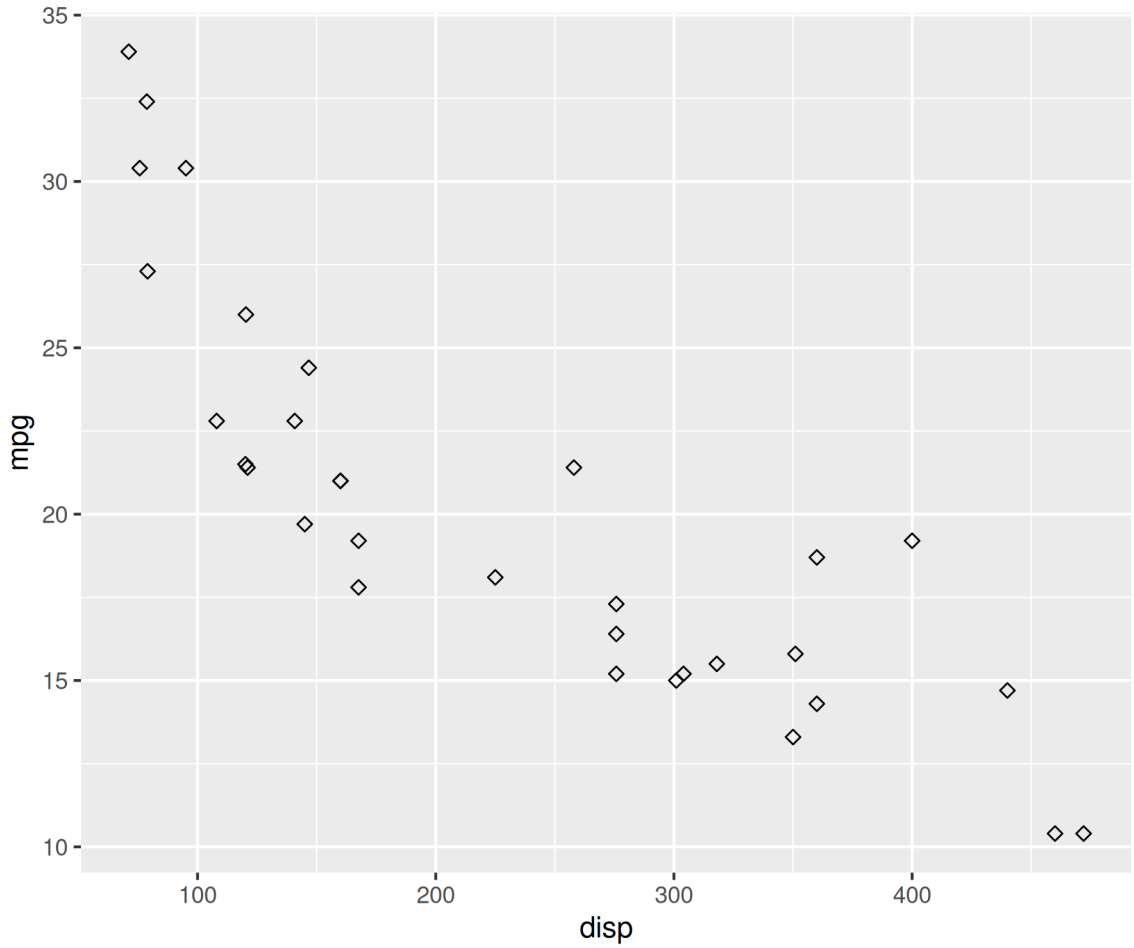
Let us map shape to `cyl` in the `geom_point()` function. Remember, when you are mapping an aesthetic to a variable, it must be inside `aes()`.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(aes(shape = factor(cyl)))
```



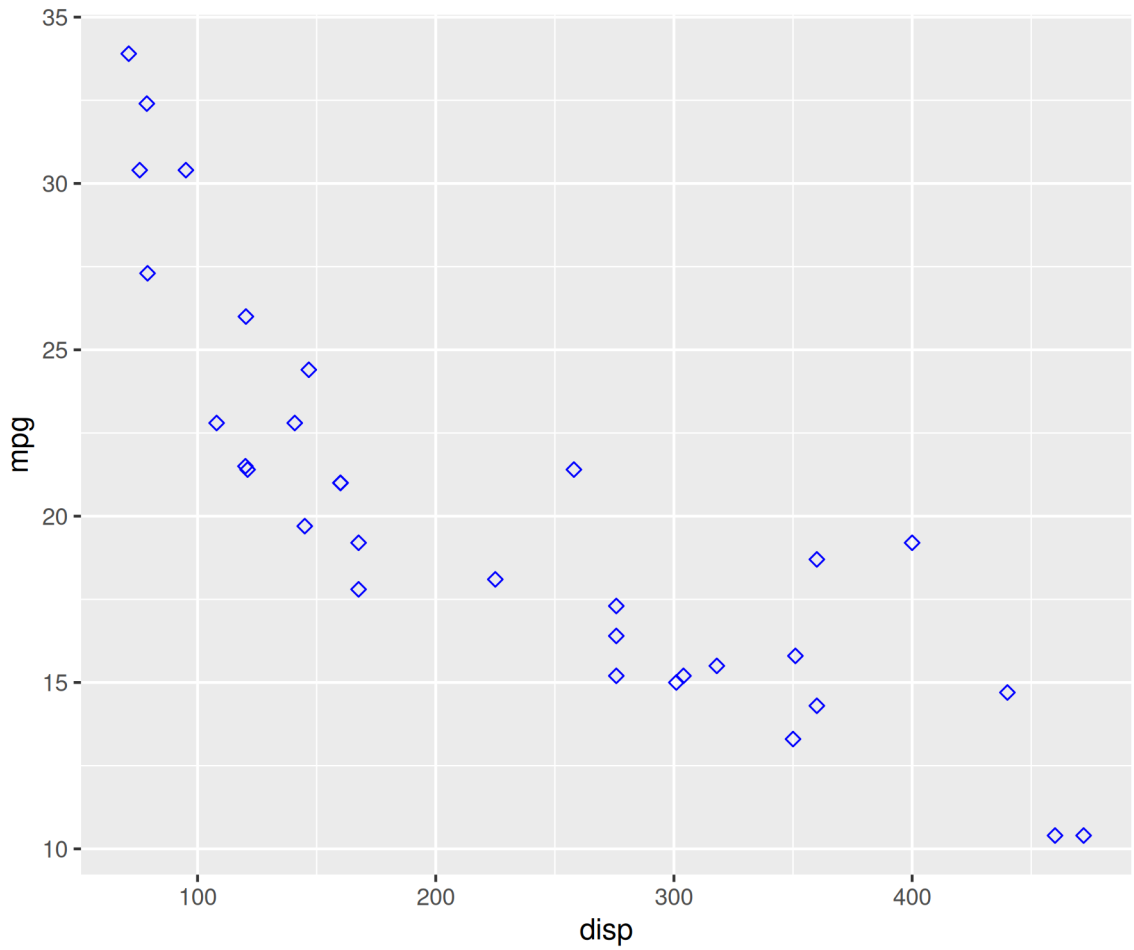
Instead of mapping shape to a variable, let us specify a value for shape. In this case, `shape` is not wrapped inside `aes()` as we are not mapping it to a variable.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(shape = 5)
```



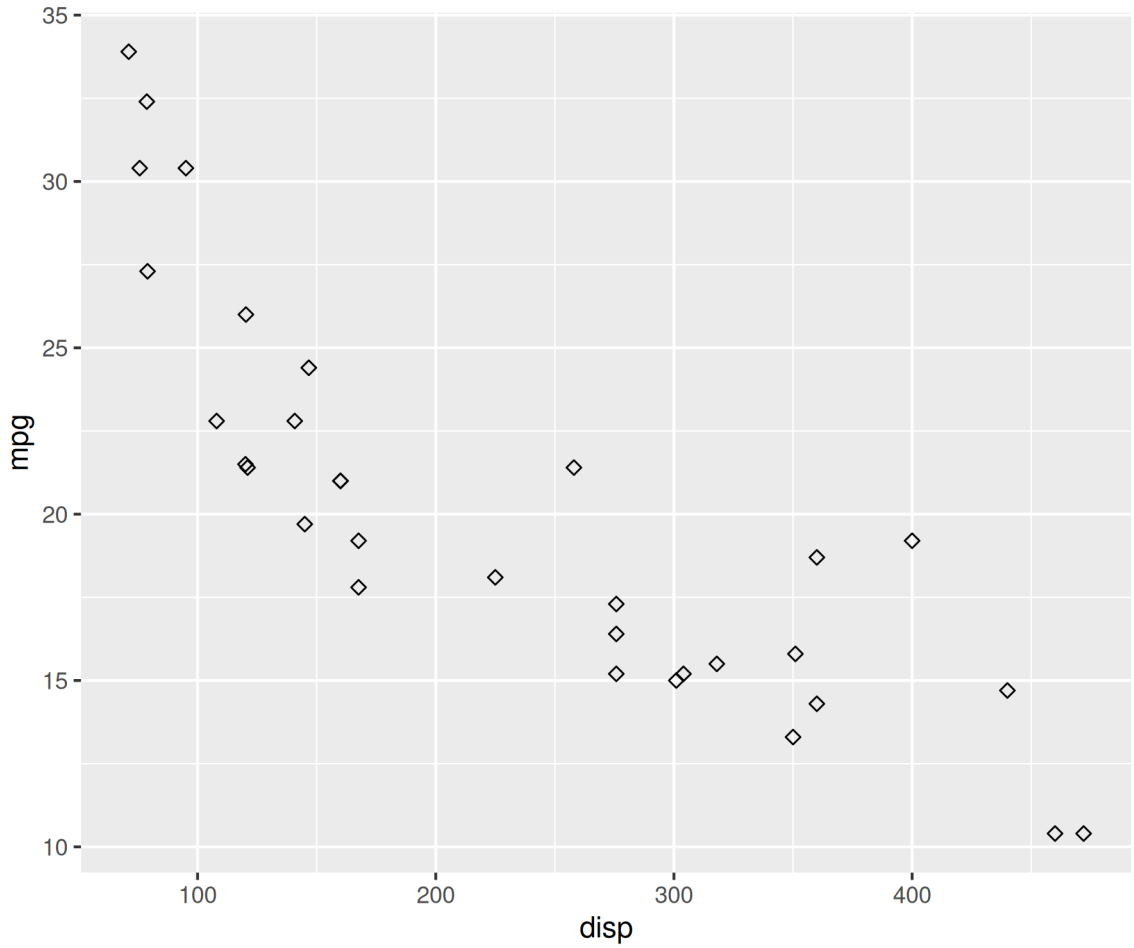
Let us specify a color for the point using `color` argument.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(shape = 5, color = 'blue')
```



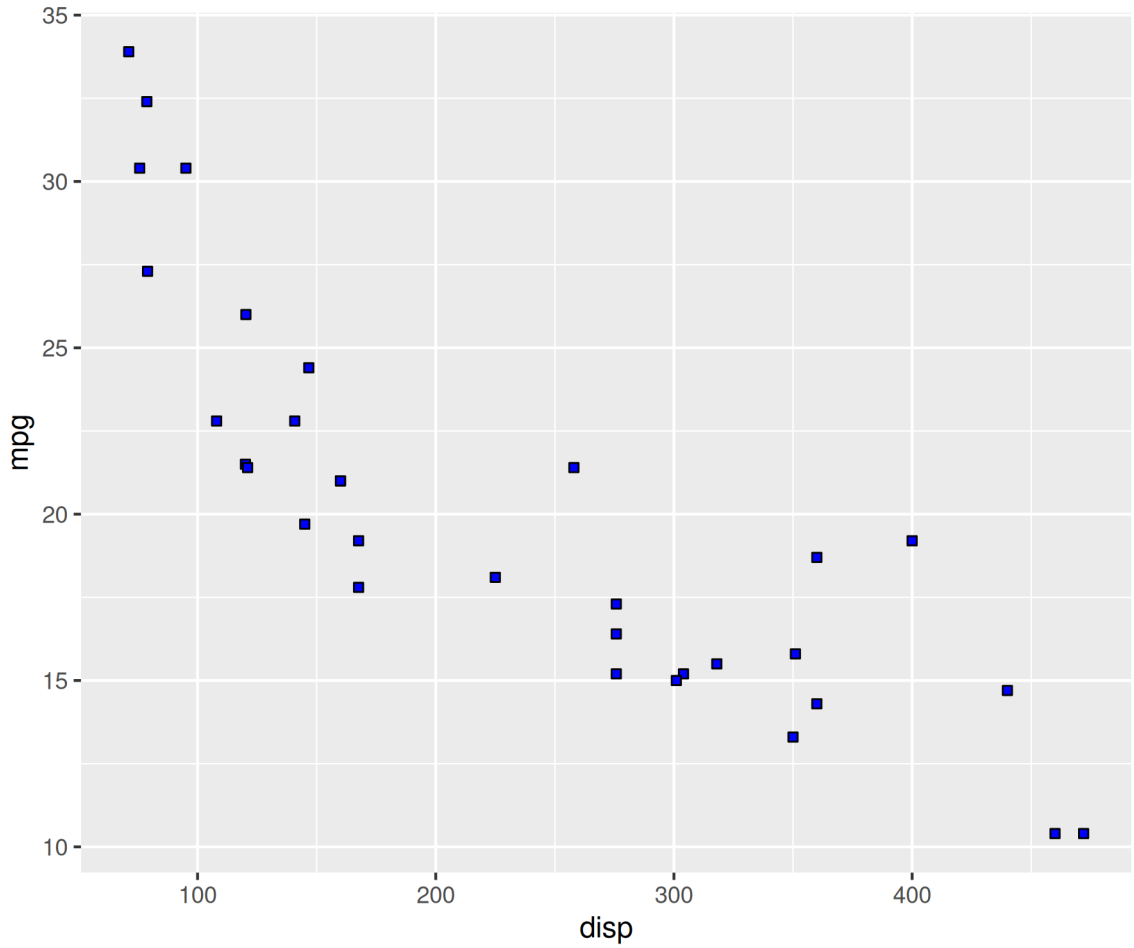
Background color cannot be added for all shapes. In the below example, we try to modify the background color using the `fill` argument but it does not work.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(shape = 5, fill = 'blue')
```



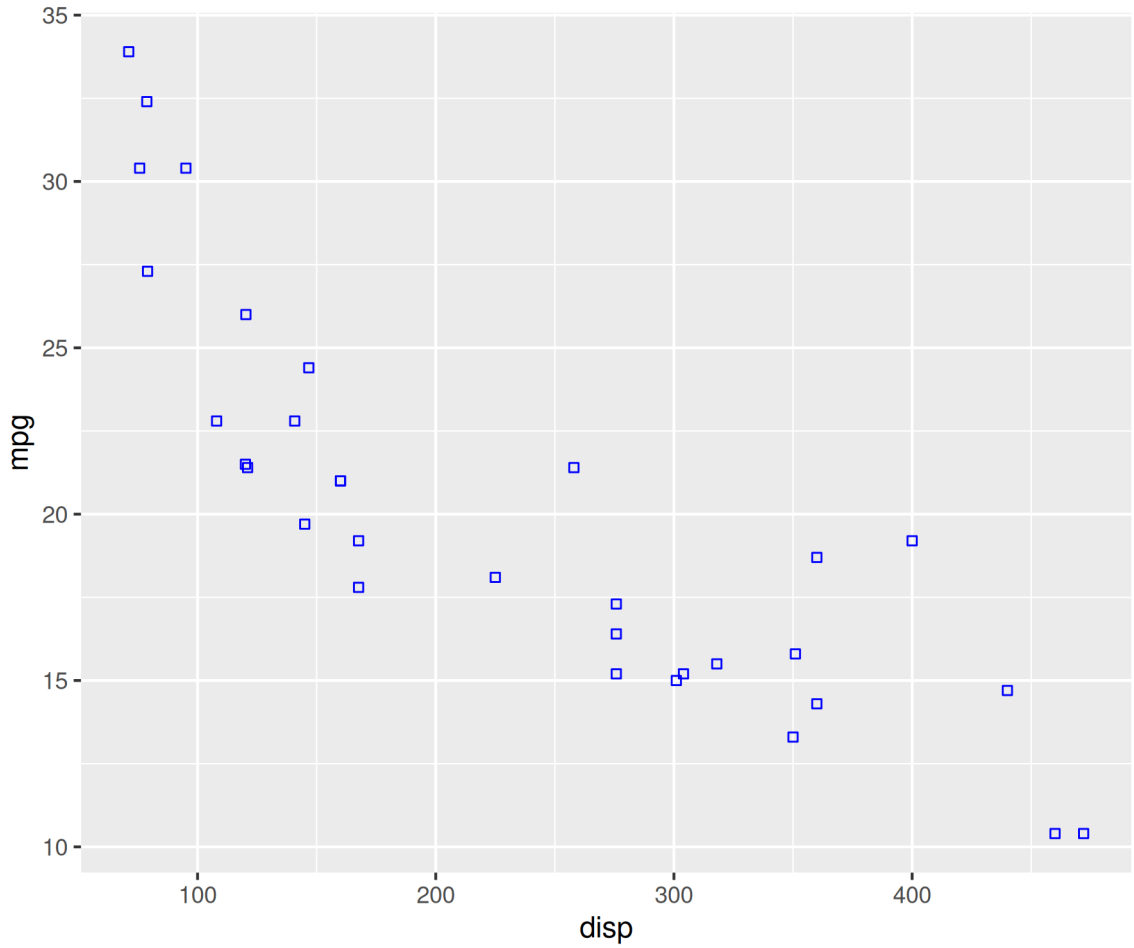
Since the shape number is now greater than 21, `fill` argument will add background color in the below case.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(shape = 22, fill = 'blue')
```



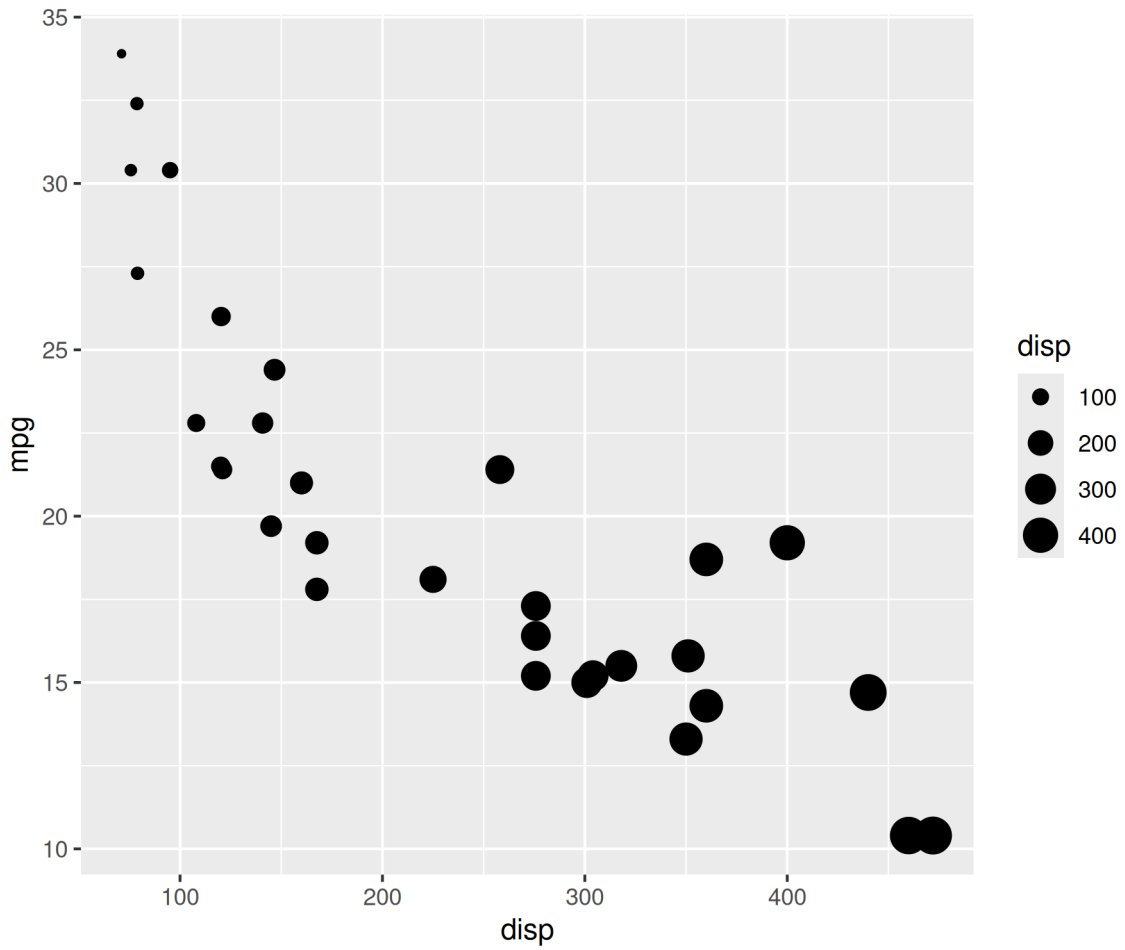
In shapes greater than number 21, `color` argument will modify the border of the shape.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(shape = 22, color = 'blue')
```



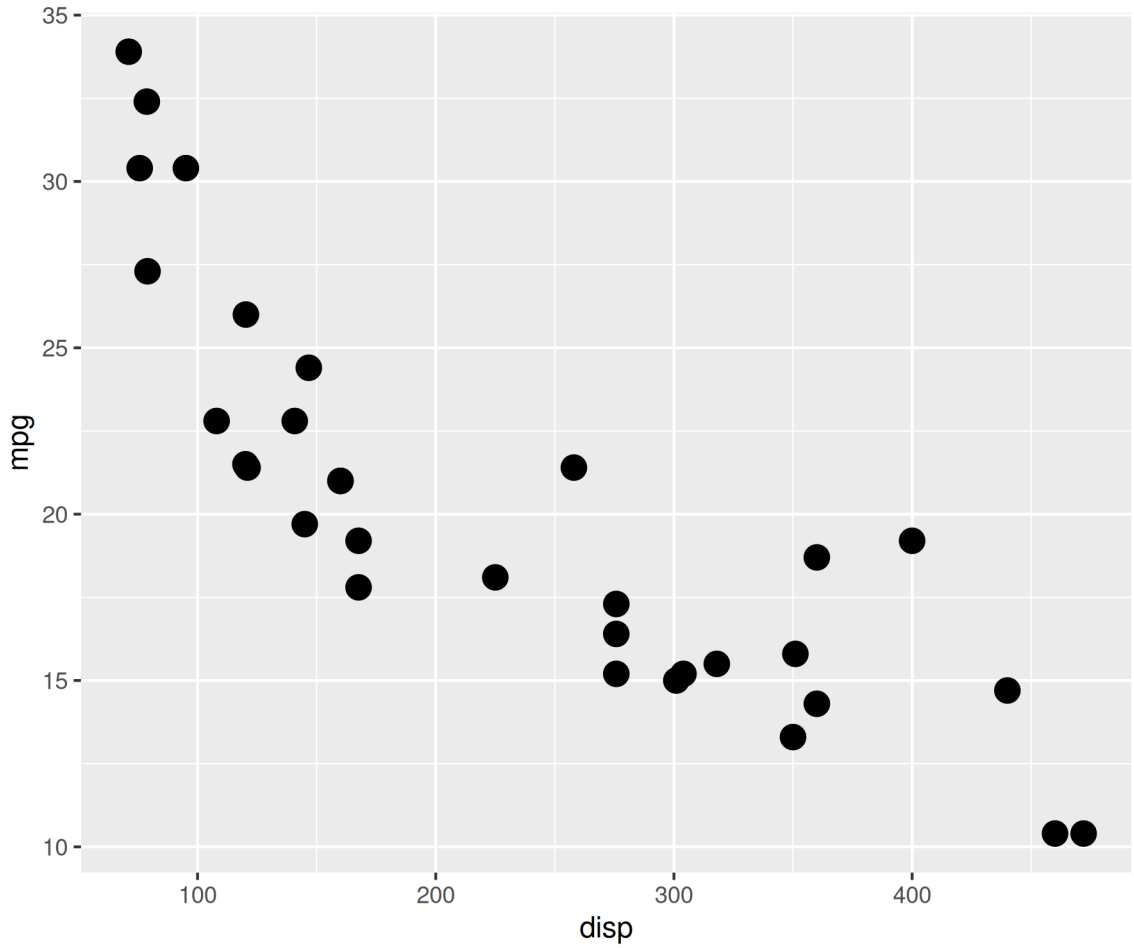
Let us map size of points to a variable. It is advised to map size only to continuous variables and not categorical variables.

```
ggplot(mtcars, aes(x = disp, y = mpg, size = disp)) +
  geom_point()
```



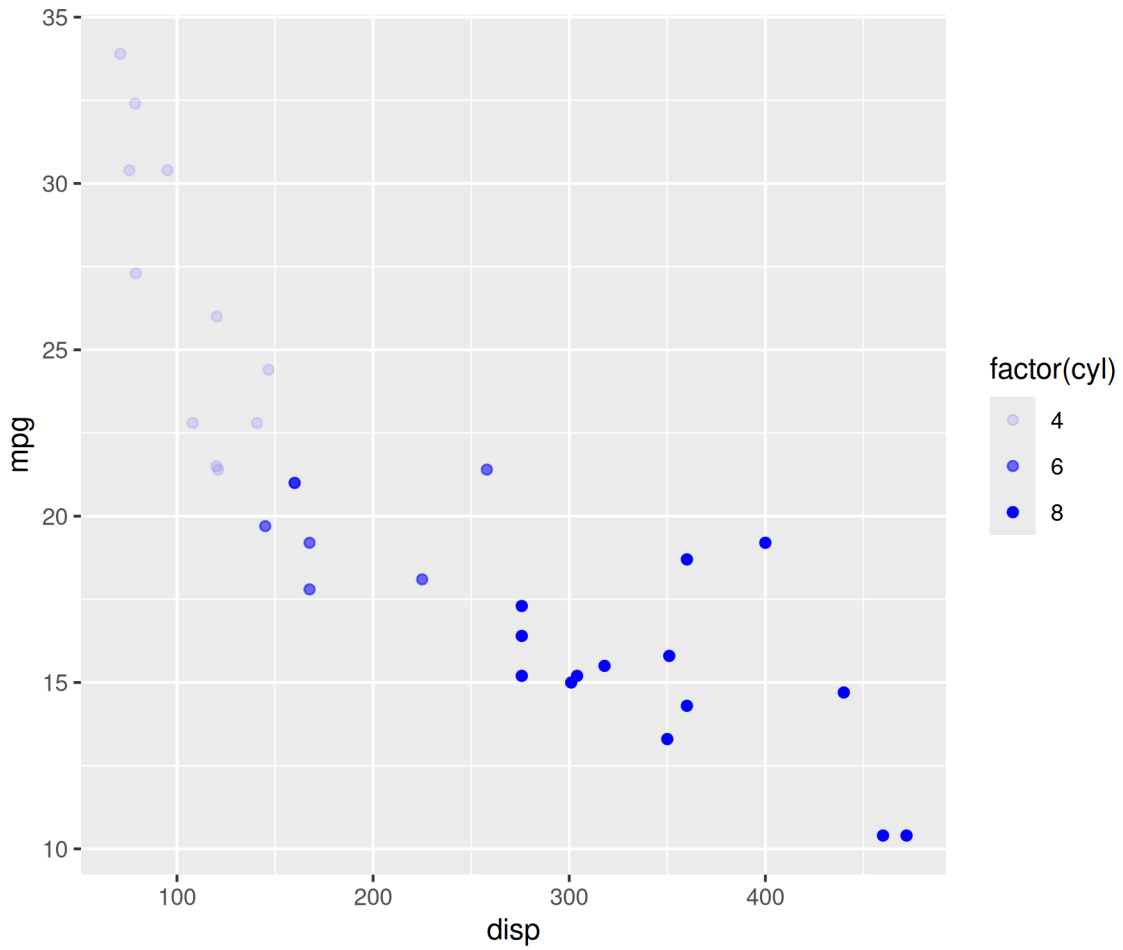
If you map size to categorical variables, ggplot2 will throw a warning.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(size = 4)
```



To modify the opacity of the color, use the `alpha` argument.

```
ggplot(mtcars, aes(x = disp, y = mpg)) +
  geom_point(aes(alpha = factor(cyl)), color = 'blue')
```



3.4 Line Chart

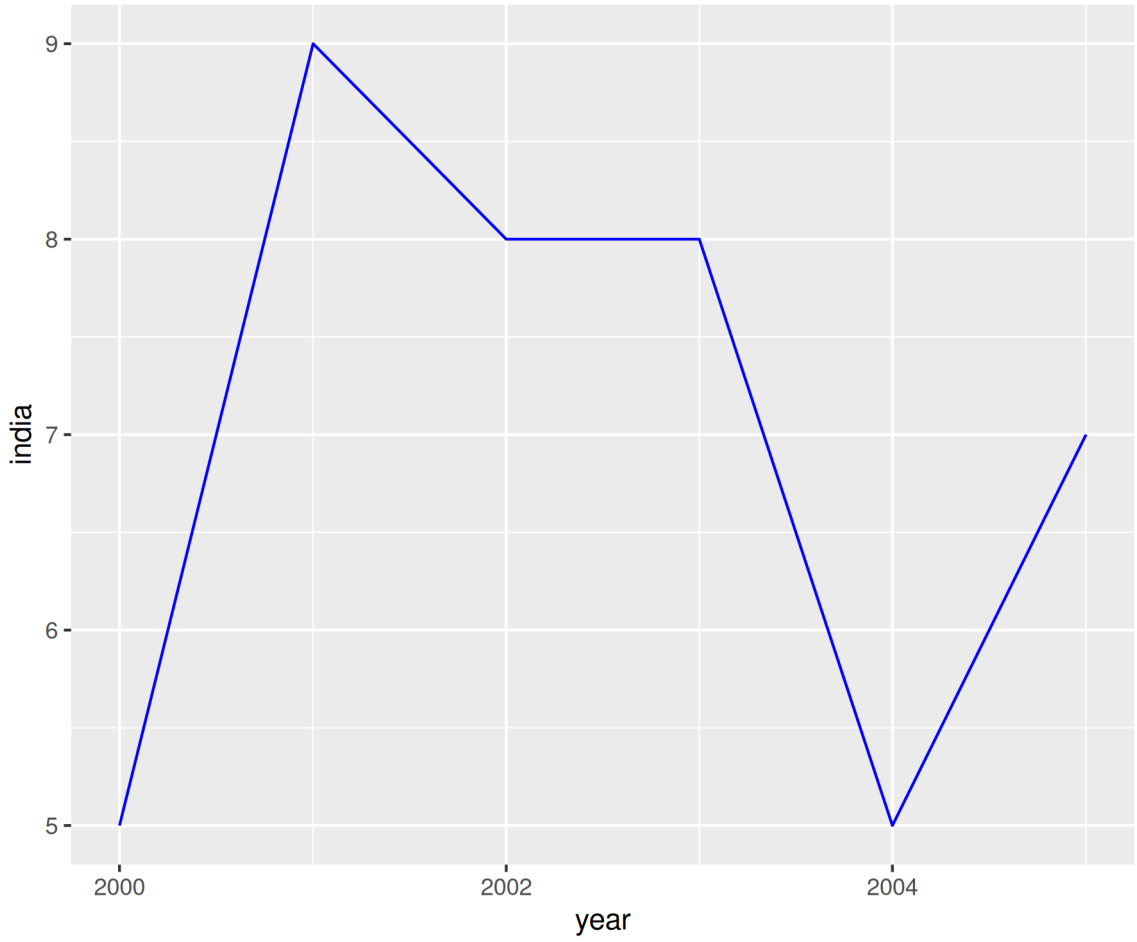
So far we have focussed on `geom_point()` to learn how to map aesthetics to variables. To explore line type and line width, we will use `geom_line()`. In the previous chapter, we used `geom_line()` to build line charts. Now we will modify the appearance of the line. In the section below, we will specify values for color, line type and width. In the next section, we will map the same to variables in the data. We will use a new data set. You can download it from here. It contains GDP (Gross Domestic Product) growth data for the BRICS (Brazil, Russia, India, China, South Africa) for the years 2000 to 2005.

3.4.1 Data

```
gdp <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/
```

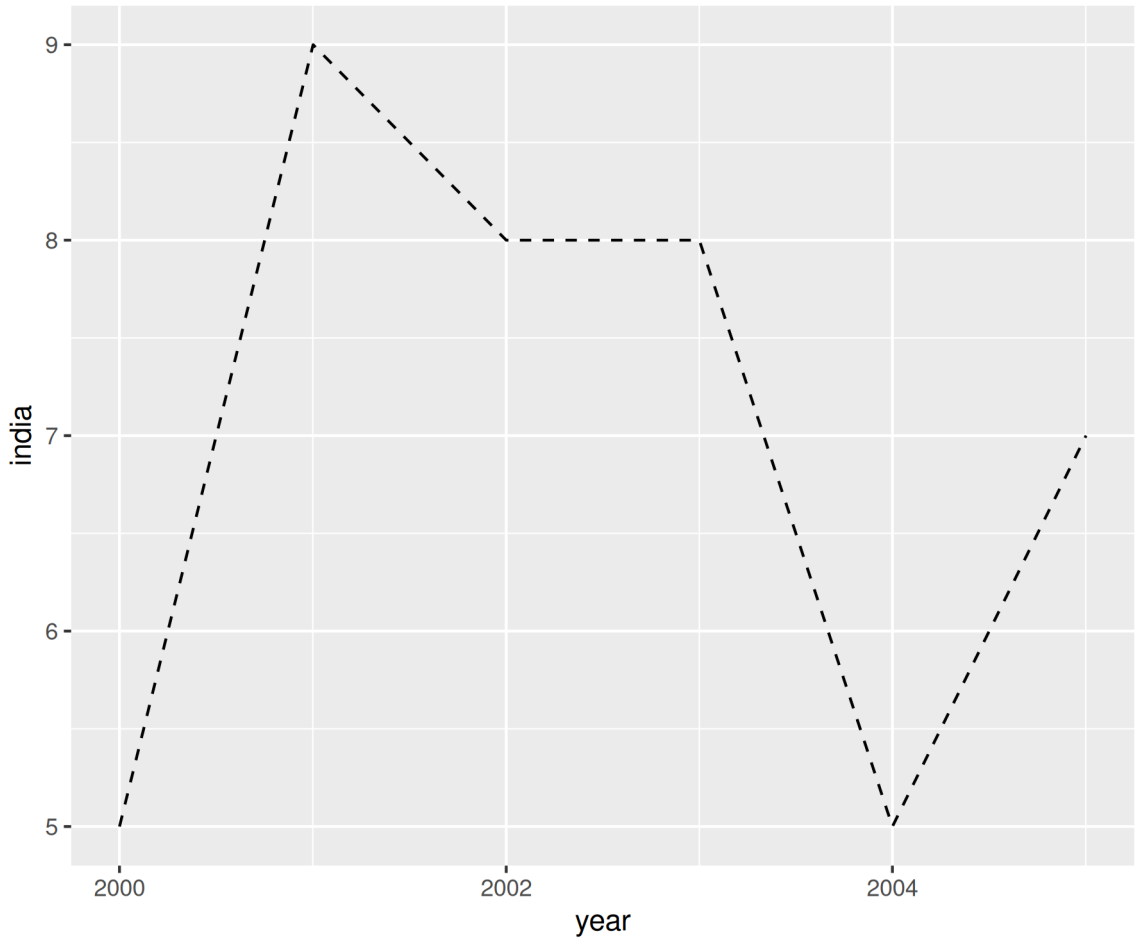
A line chart can be created using `geom_line()`. In the below example, we examine the GDP trend of India and modify the color of the line to `'blue'`.

```
ggplot(gdp, aes(year, india)) +
  geom_line(color = 'blue')
```



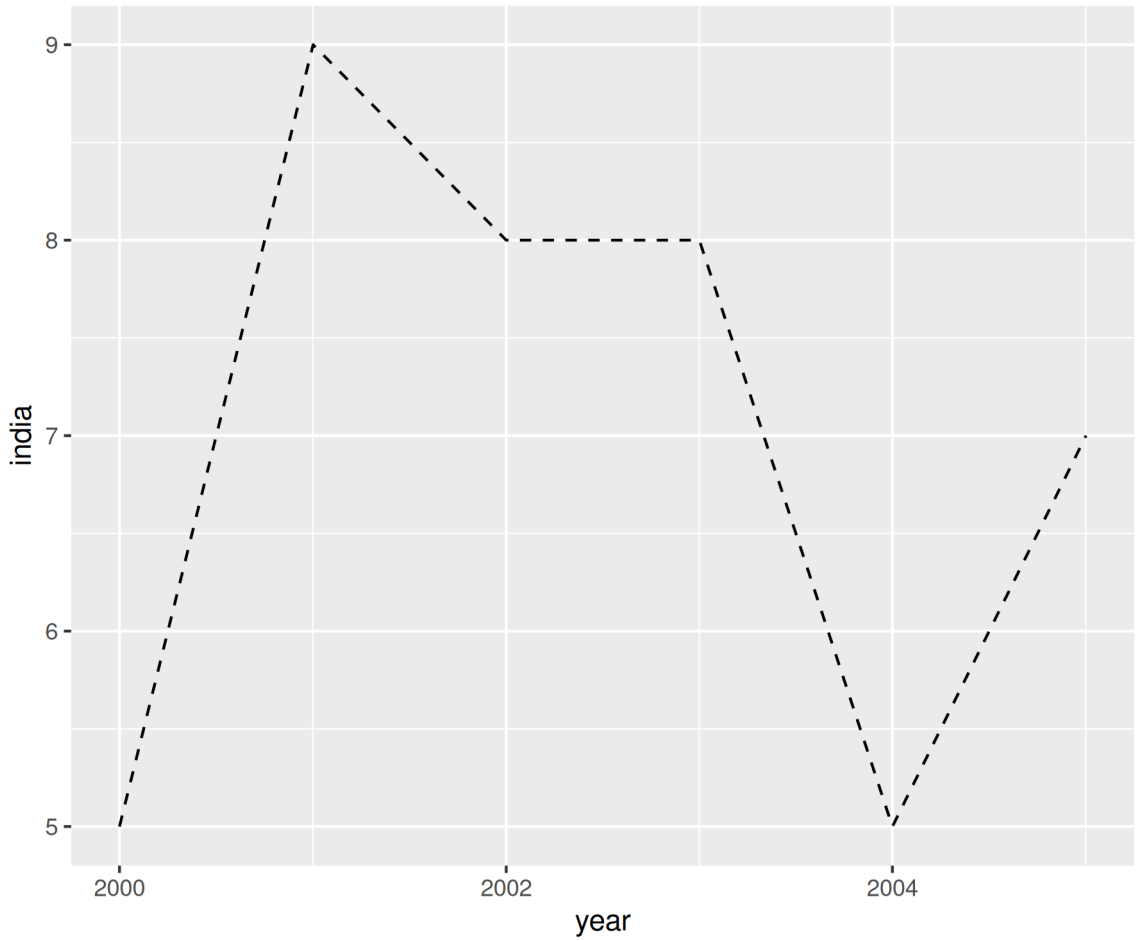
To modify the line type, use the `linetype` argument. It can take values between 1 and 5.

```
ggplot(gdp, aes(year, india)) +
  geom_line(linetype = 2)
```



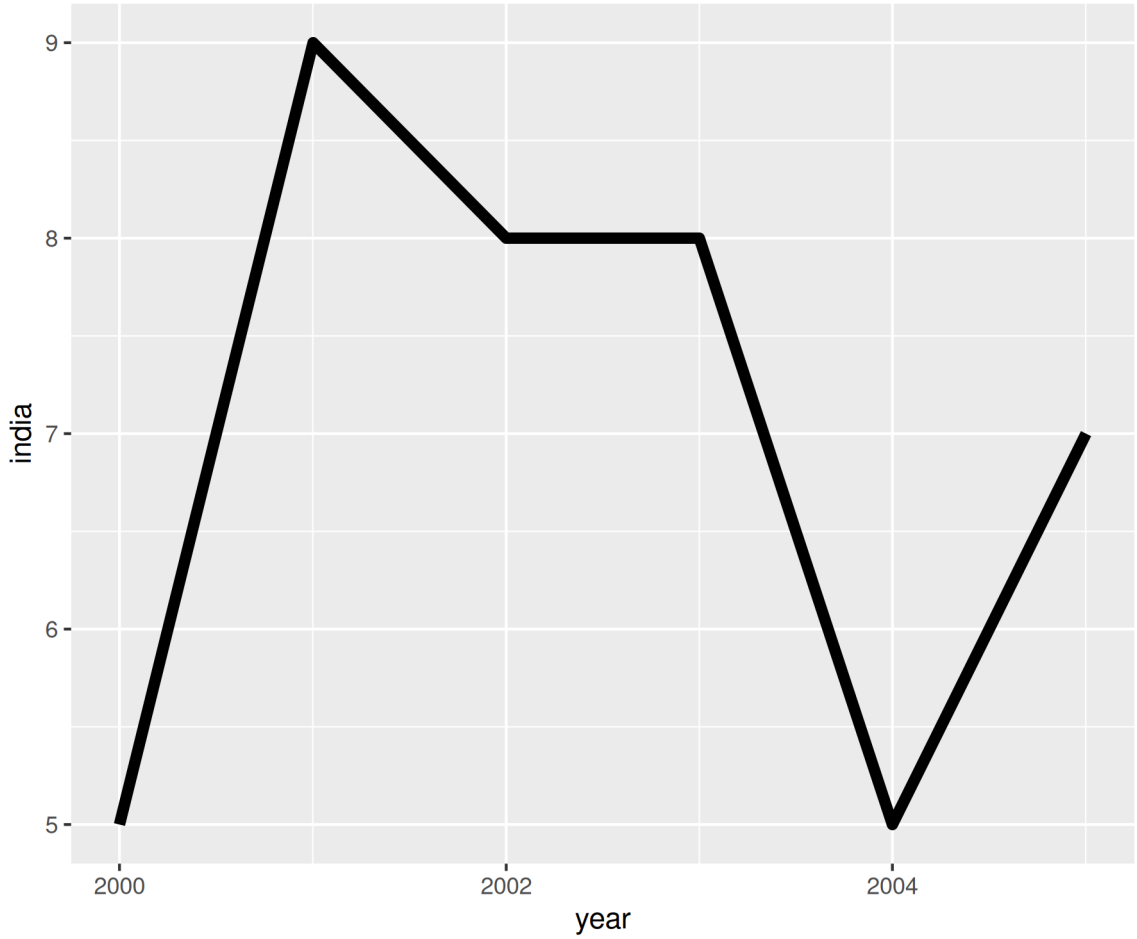
The line type can also be mentioned in the following way:

```
ggplot(gdp, aes(year, india)) +
  geom_line(linetype = 'dashed')
```



The width of the line can be modified using the `linewidth` argument.

```
ggplot(gdp, aes(year, india)) +
  geom_line(linewidth = 2)
```



Now let us map the aesthetics to the variables. The data used in the above example cannot be used as we need a variable with country names. We will use `pivot_longer()` function from the `tidyr` package to reshape the data.

```
gdp2 <-
  gdp |>
  select(year, growth, india, china) |>
  tidyr::pivot_longer(cols = -year, names_to = "country", values_to = "gdp")

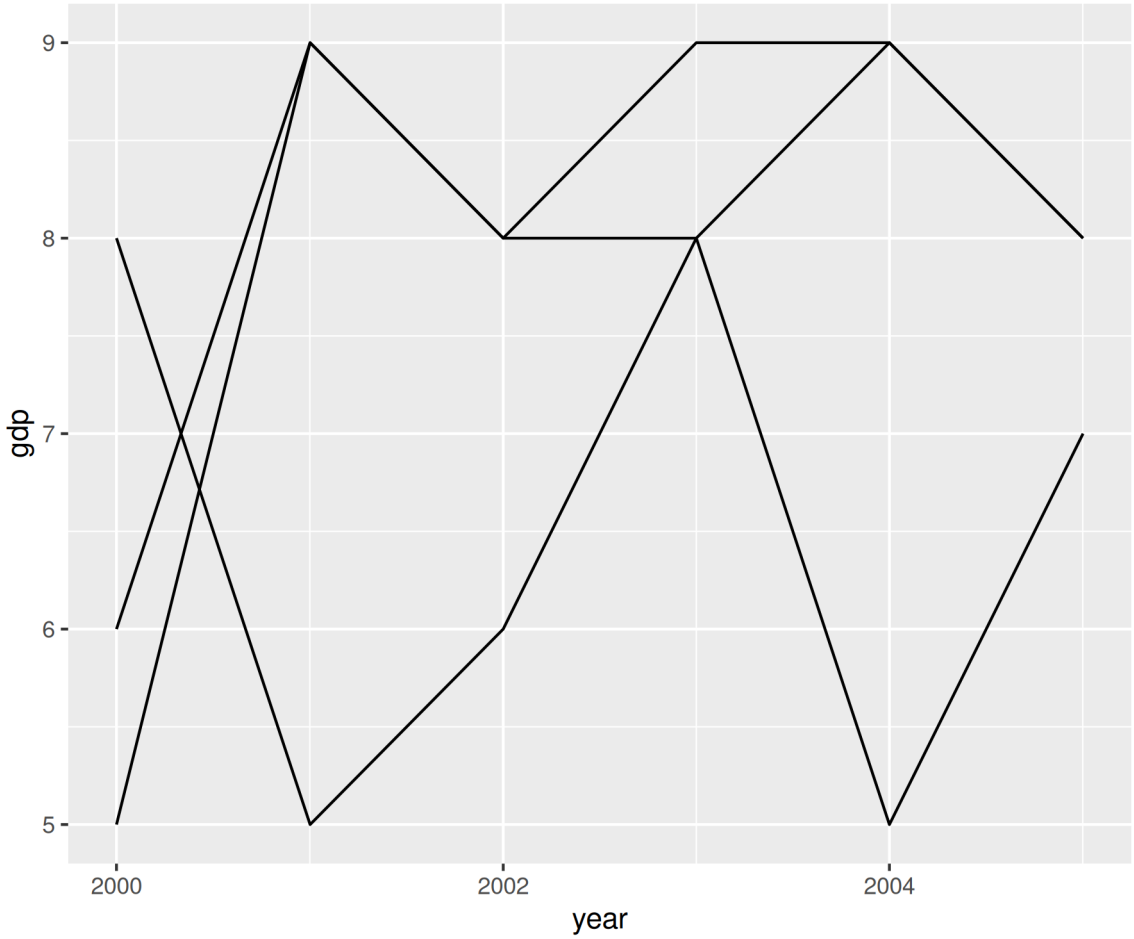
gdp2
```

```
# A tibble: 18 x 3
   year       country   gdp
   <date>     <chr>   <dbl>
 1 2000-01-01 growth      6
 2 2000-01-01 india       5
 3 2000-01-01 china       8
```

```
 4 2001-01-01 growth      9
 5 2001-01-01 india       9
 6 2001-01-01 china       5
 7 2002-01-01 growth      8
 8 2002-01-01 india       8
 9 2002-01-01 china       6
10 2003-01-01 growth      9
11 2003-01-01 india       8
12 2003-01-01 china       8
13 2004-01-01 growth      9
14 2004-01-01 india       5
15 2004-01-01 china       9
16 2005-01-01 growth      8
17 2005-01-01 india       7
18 2005-01-01 china       8
```

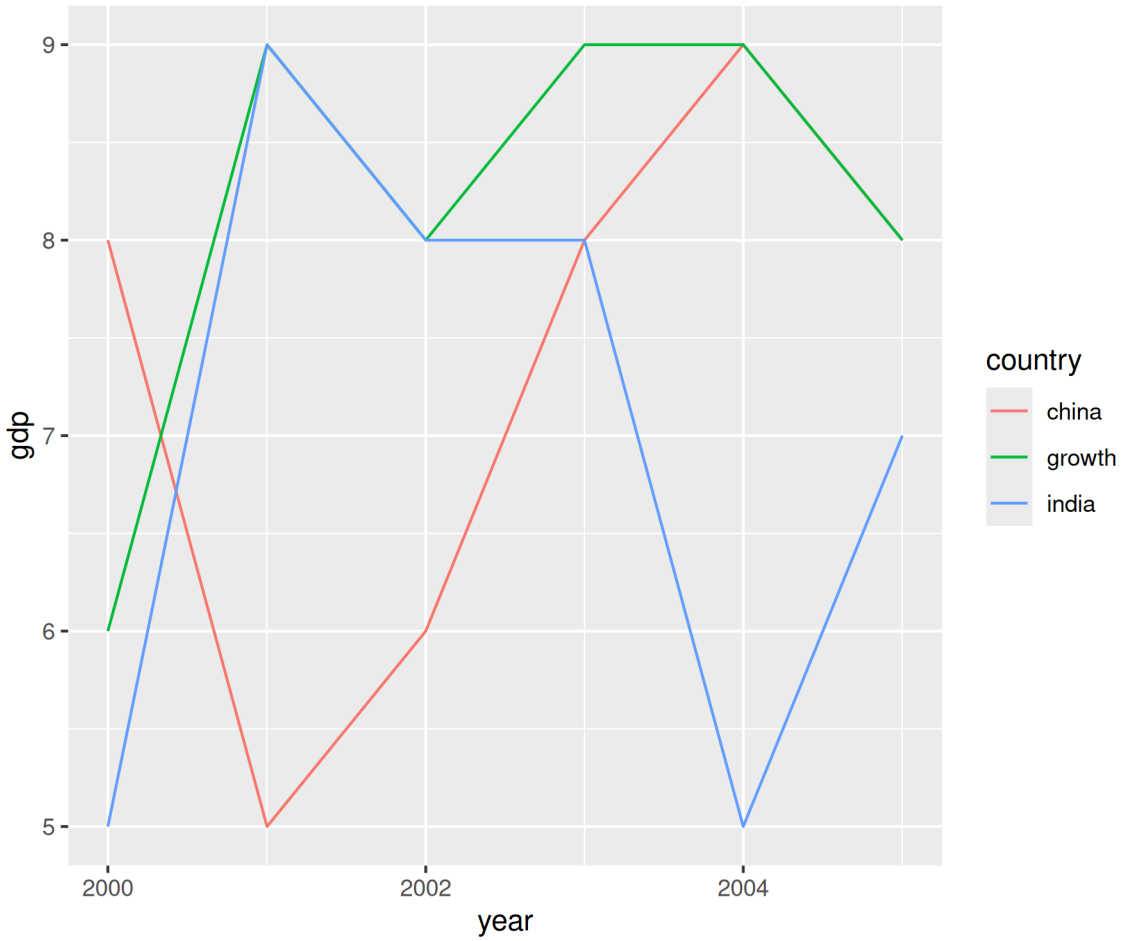
To map the aesthetics to a variable, we must use the `group` argument. In the below example, we map the aesthetics to `country`. But we cannot distinguish between the lines as their color, width and line type are the same. We have easily plotted the GDP trend of all countries using the `group` argument. Now, let us ensure that we can distinguish and identidy them using different aesthetics.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line()
```



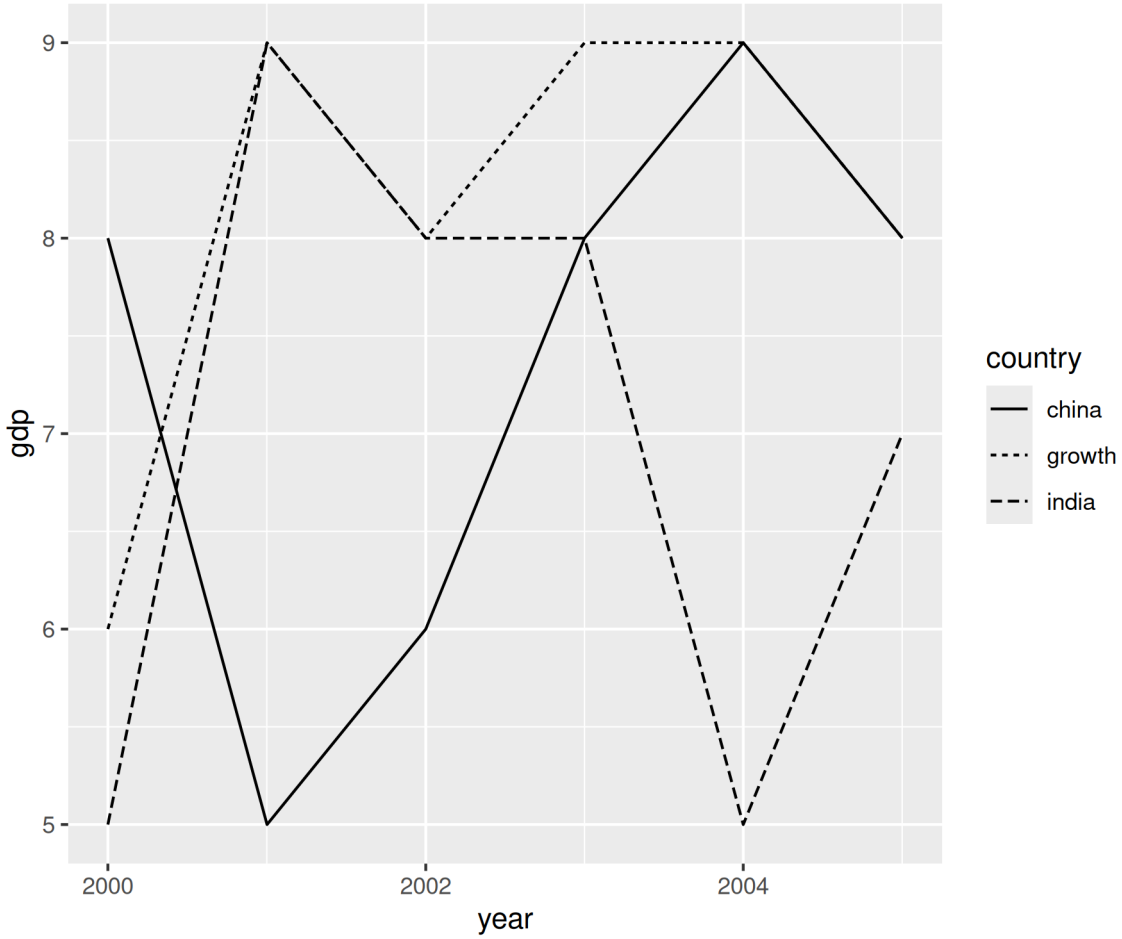
Let us begin by ensuring that the lines have different color using the `color` argument within `aes()` and assigning it the variable `country`.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(color = country))
```



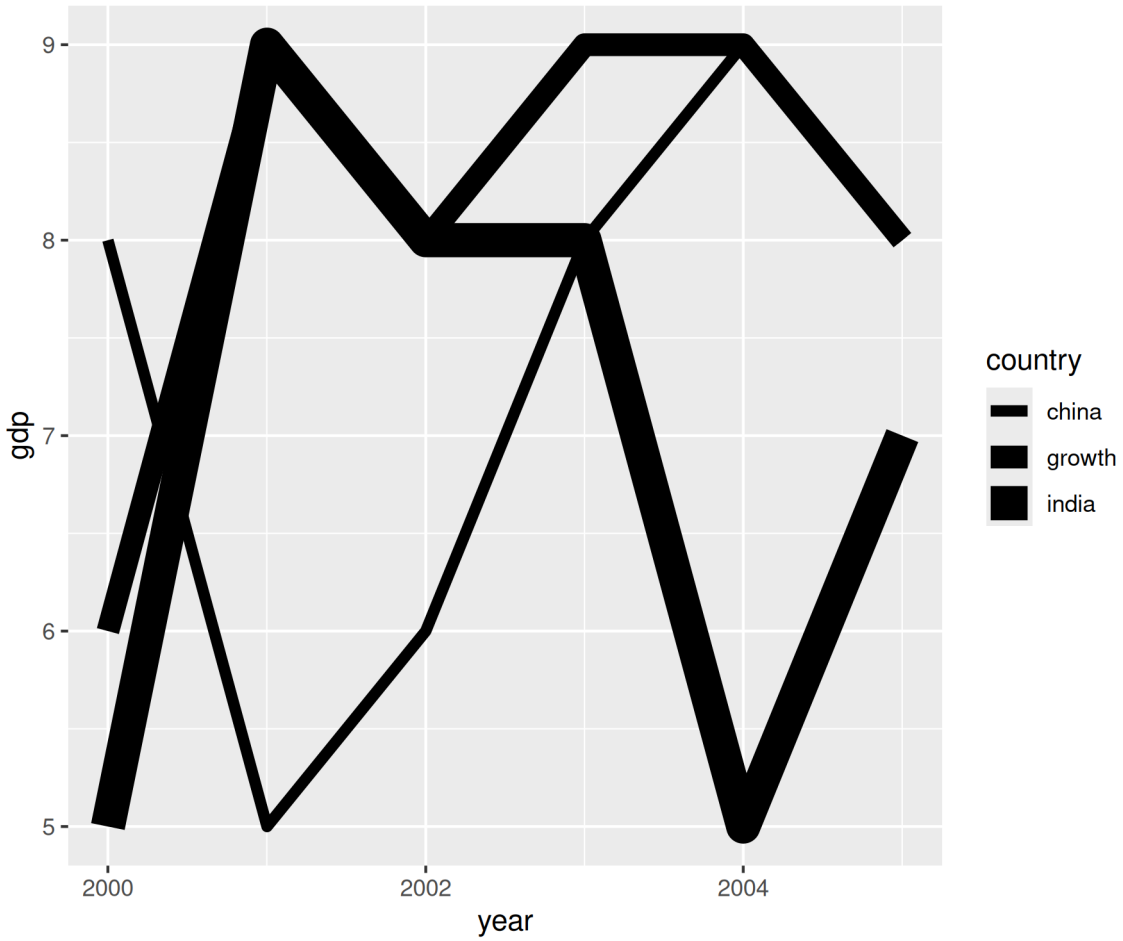
Instead of color, now we modify the line type using the `linetype` argument.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(linetype = country))
```



In the below instance, we assign different width to the lines using the `linewidth` argument.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(linewidth = country))
```

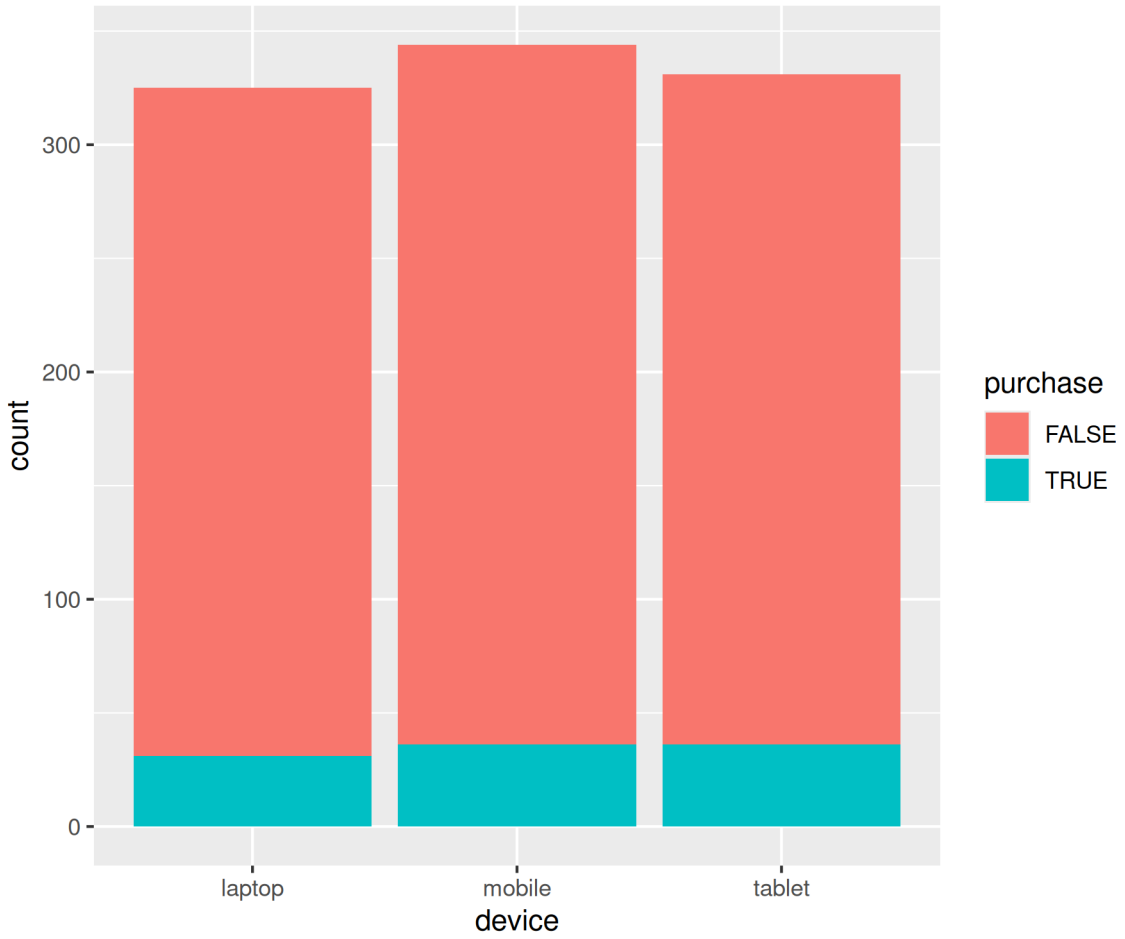


Before we wrap up, let us quickly see how we can map aesthetics to variables for different plots.

3.5 Bar Plots

Here we create a stacked bar plot by mapping `fill` to `purchase`.

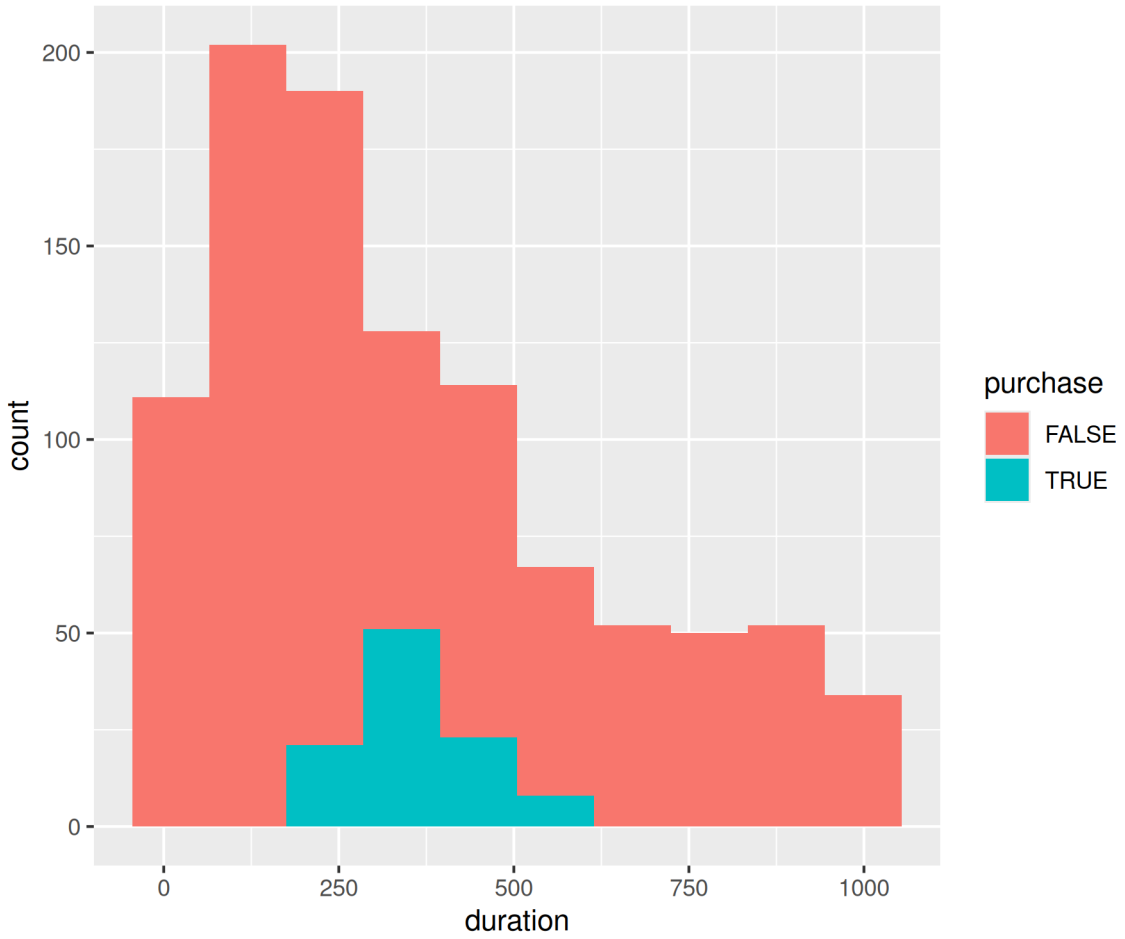
```
ggplot(ecom, aes(device, fill = purchase)) +
  geom_bar()
```



3.6 Histograms

Instead of a bar chart, we create a histogram and again map `fill` to `purchase`.

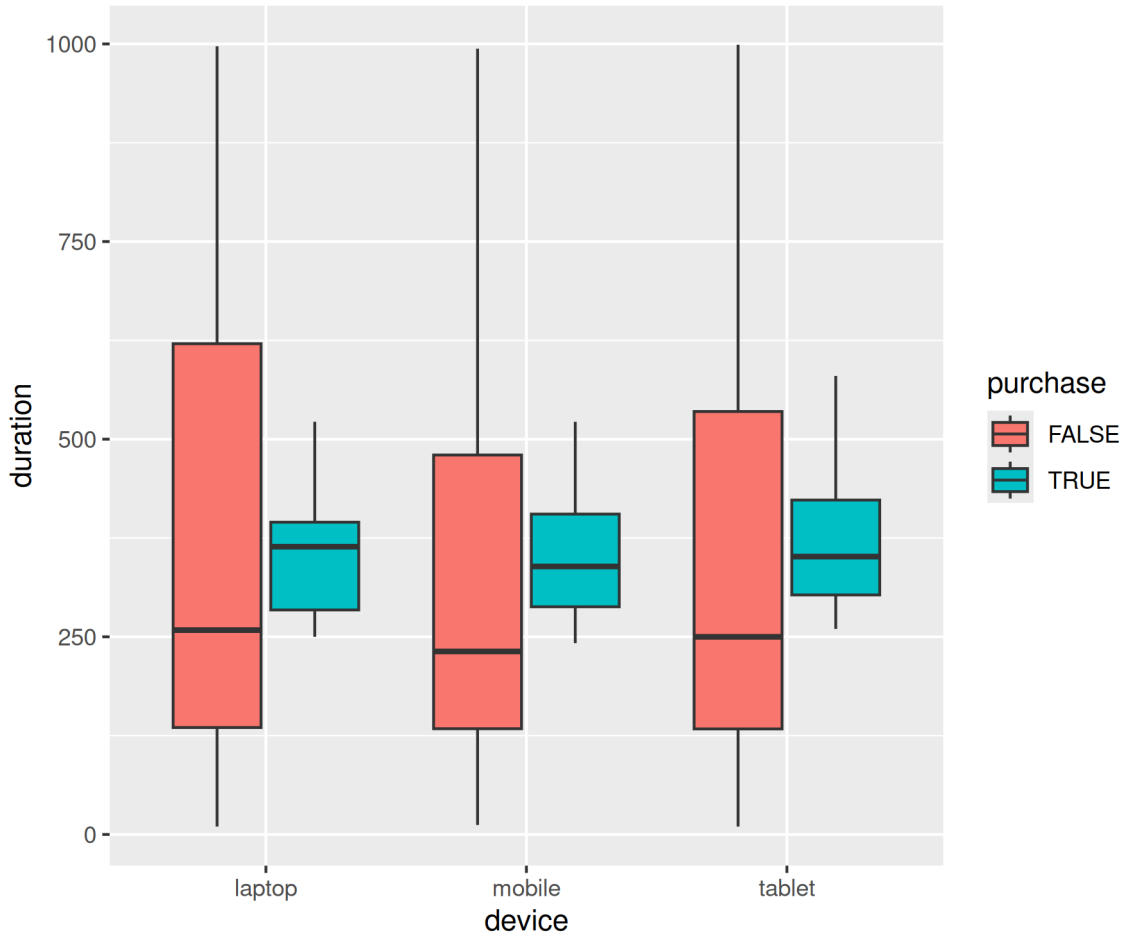
```
ggplot(ecom) +
  geom_histogram(aes(duration, fill = purchase), bins = 10)
```



3.7 Box Plots

We repeat the same exercise below, but replace the bar plot with a box plot.

```
ggplot(ecom) +
  geom_boxplot(aes(device, duration, fill = purchase))
```



In all the above cases, you can observe that when we are mapping aesthetics such as color, fill, shape, size or linetype to variables, they are all wrapped inside `aes()`.

3.8 Exercises

1. Map `color` to cylinders and set a constant point `size = 3` outside `aes()`.
2. Map `linewidth` (not `size`) to a line geom and explain the difference.
3. Reshape a two-column wide table with `pivot_longer()` and map the key to color.

Worked solutions in `solutions/aesthetics.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

4 Labels

4.1 Introduction

> **Why this chart**
>
> Axis labels and titles turn a plot into a message; they are the cheapest readability win.

> **Common pitfall**
>
> Tiny or default labels (e.g. snake_case column names) force readers to guess the message.

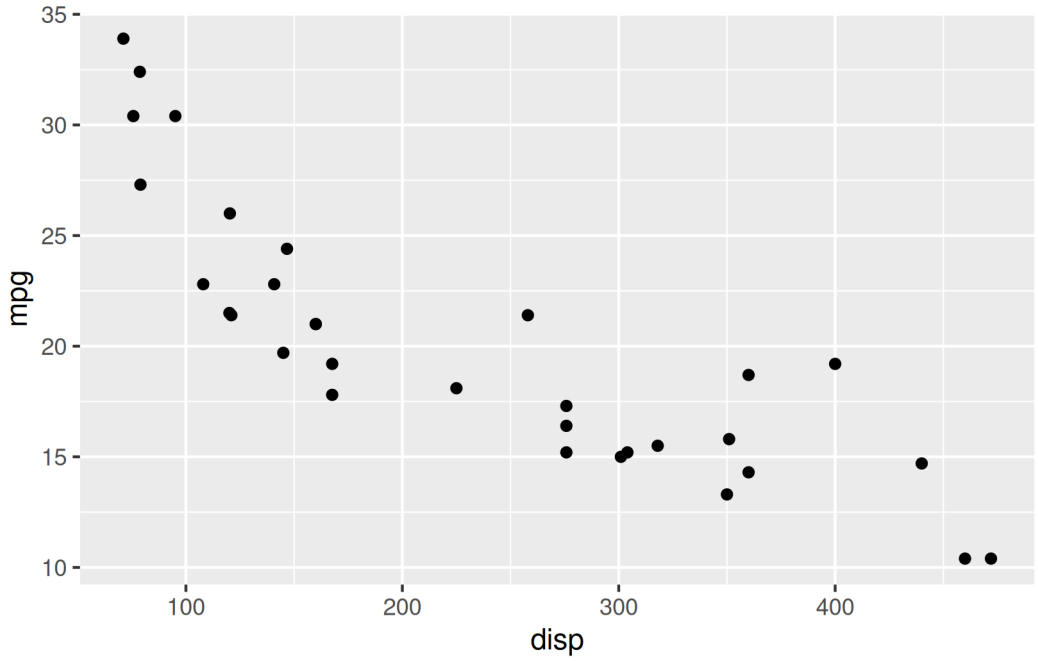
In this chapter, we will learn to:

- add title and subtitle to the plot
- modify axis labels
- modify axis range
- remove axis
- format axis

4.2 Basic Plot

Let us start with a simple scatter plot. We will continue to use the mtcars data set and examine the relationship between displacement and miles per gallon using `geom_point()`.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg))
```



4.3 Title & Subtitle

There are two ways to add title to a plot:

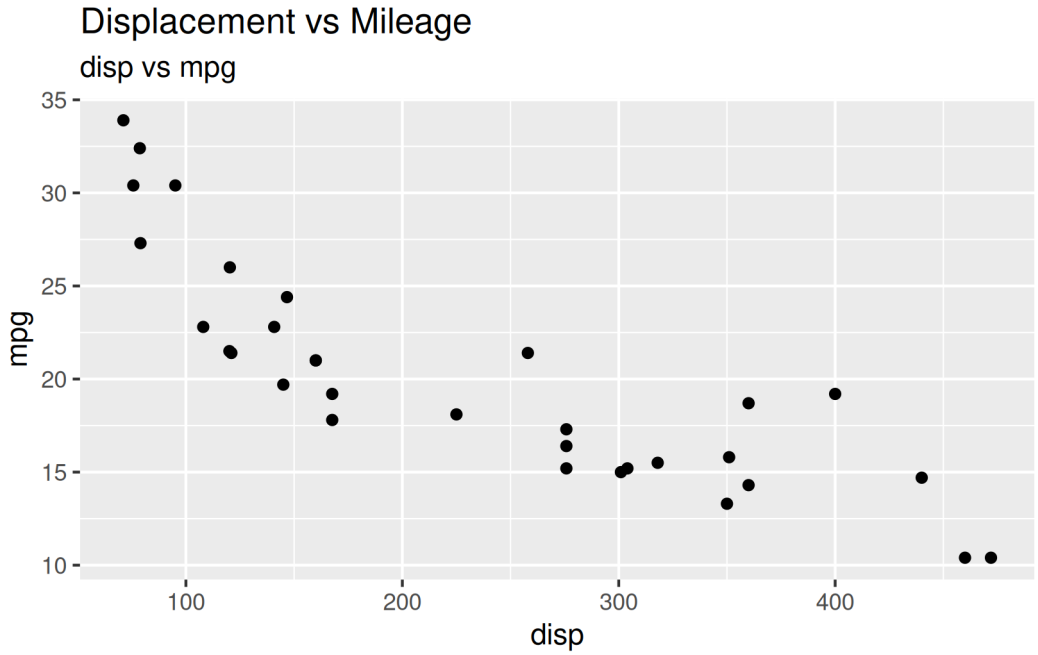
- `ggtitle()`
- `labs()`

4.3.1 ggtitle()

Let us explore the `ggtitle()` function first. It takes two arguments:

- label: title of the plot
- subtitle: subtitle of the plot

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  ggtitle(label = 'Displacement vs Mileage', subtitle = 'disp vs mpg')
```

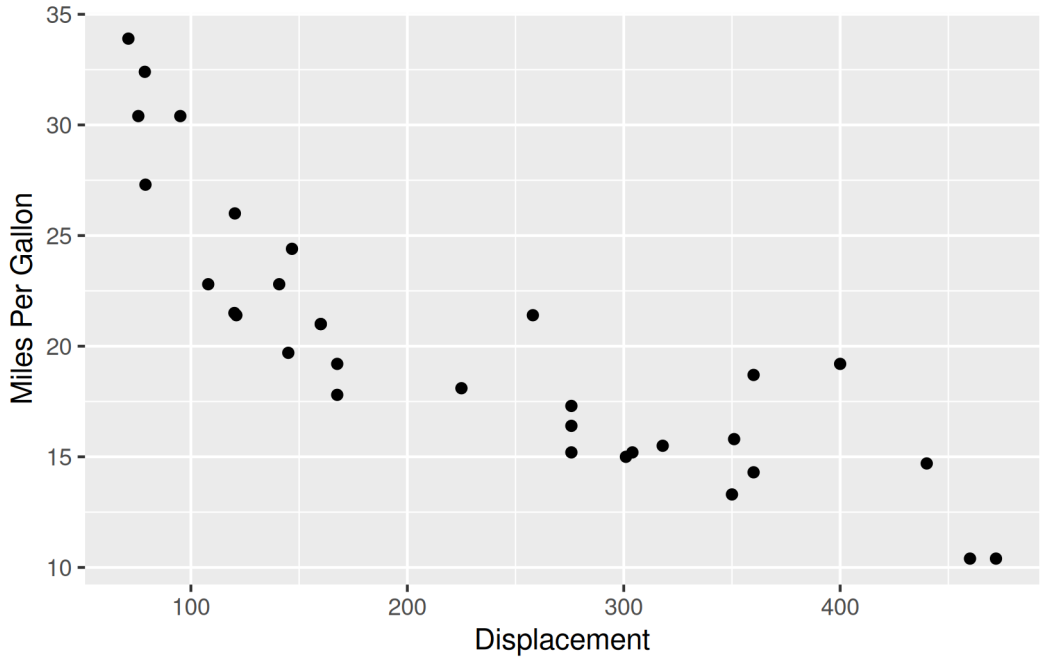


4.4 Axis Labels

You can add labels to the axis using:

- `xlab()`
- `ylab()`
- `labs()`

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  xlab('Displacement') + ylab('Miles Per Gallon')
```

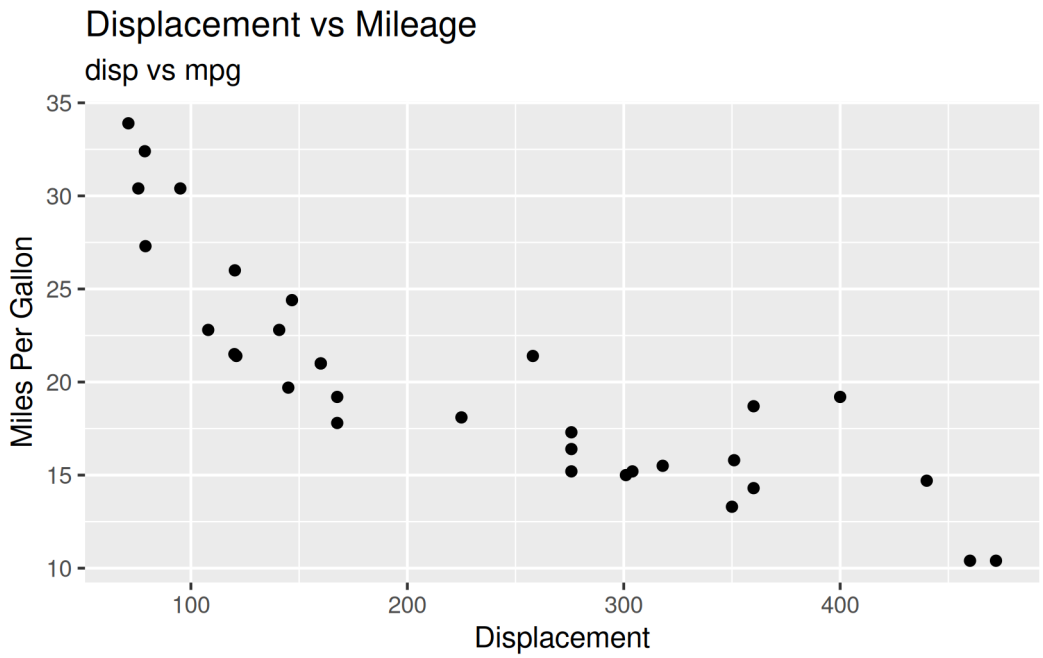


4.5 Labs

The `labs()` function can be used to add the following to a plot:

- title
- subtitle
- X axis label
- Y axis label

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  labs(title = 'Displacement vs Mileage', subtitle = 'disp vs mpg',
    x = 'Displacement', y = 'Miles Per Gallon')
```



4.6 Axis Range

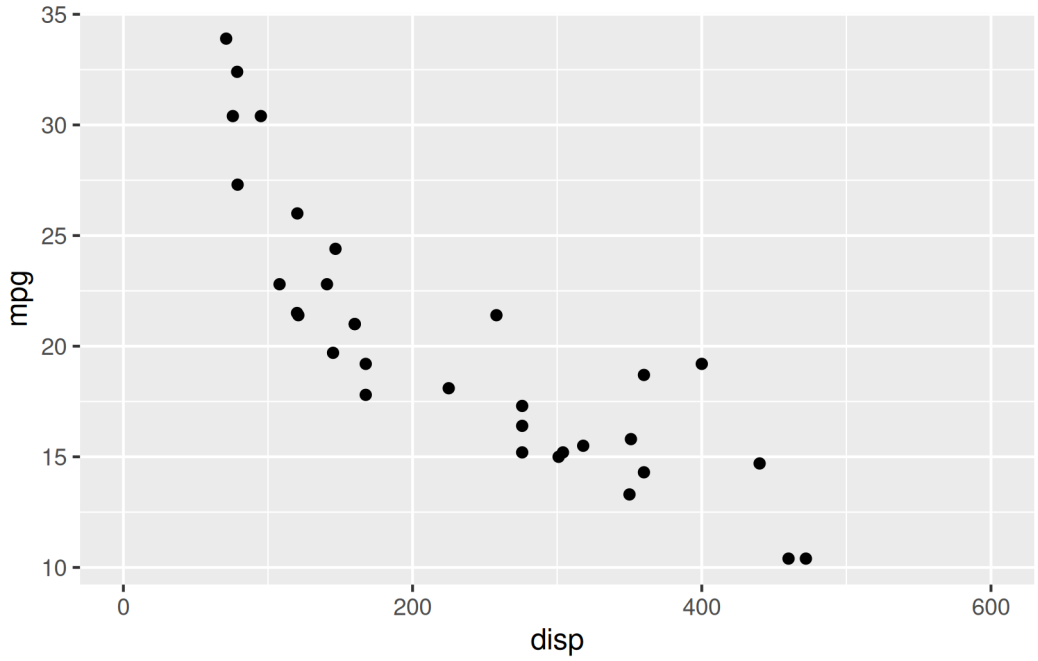
In certain scenarios, you may want to modify the range of the axis. In ggplot2, we can achieve this using:

- `xlim()`
- `ylim()`
- `expand_limits()`
- `xlim()` and `ylim()` take a numeric vector of length 2 as input
- `expand_limits()` takes two numeric vectors (each of length 2), one for each axis
- in all of the above functions, the first element represents the lower limit and the second element represents the upper limit

4.6.1 X Axis

In the below example, we limit the range of the X axis between `0` and `600` using `xlim`.

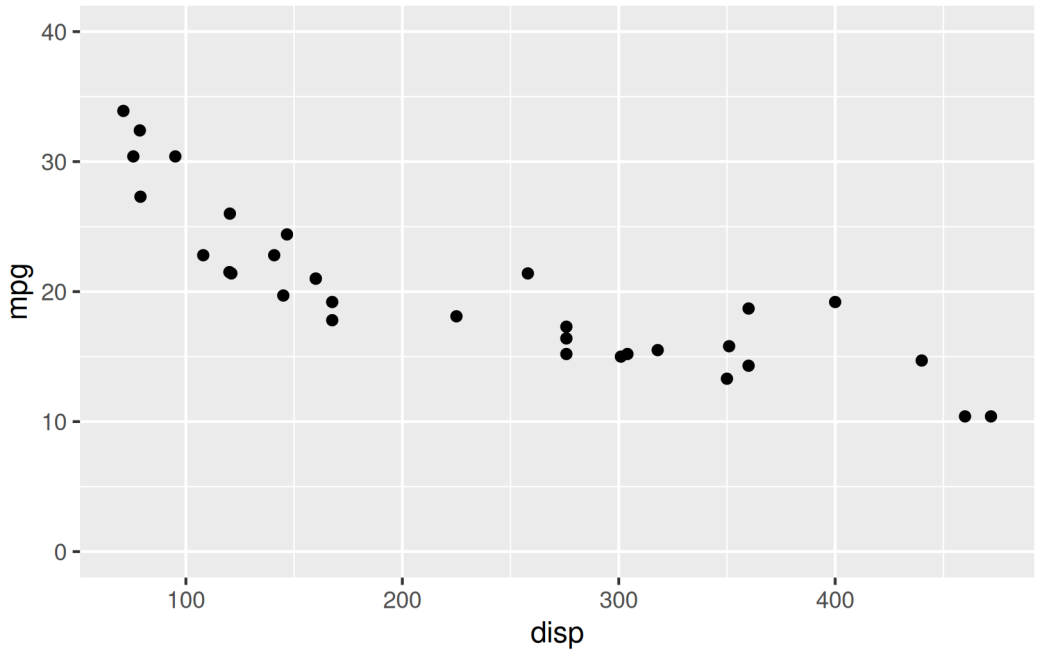
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  xlim(c(0, 600))
```



4.6.2 Y Axis

Let us limit the range of the Y axis between `0` and `40`.

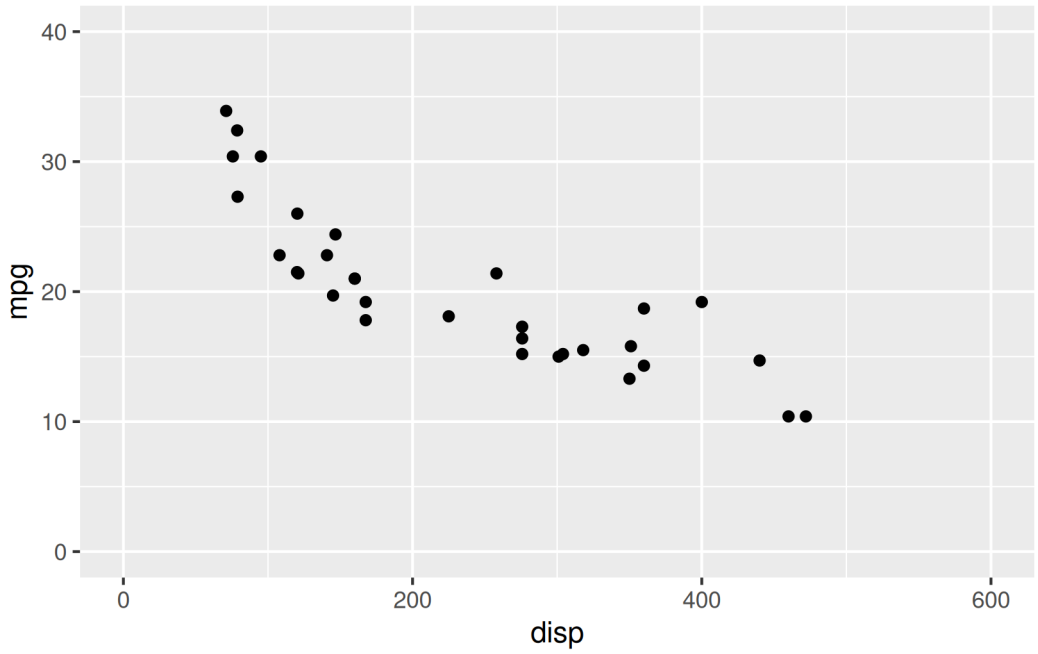
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  ylim(c(0, 40))
```



4.6.3 Expand Limits

Let us use `expand_limits()` to limit the range of both the X and Y axis. The first input is the range for the X axis and the second input for the Y axis. In both the cases, we use a numeric vector of length 2 to specify the lower and upper limit.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  expand_limits(x = c(0, 600), y = c(0, 40))
```



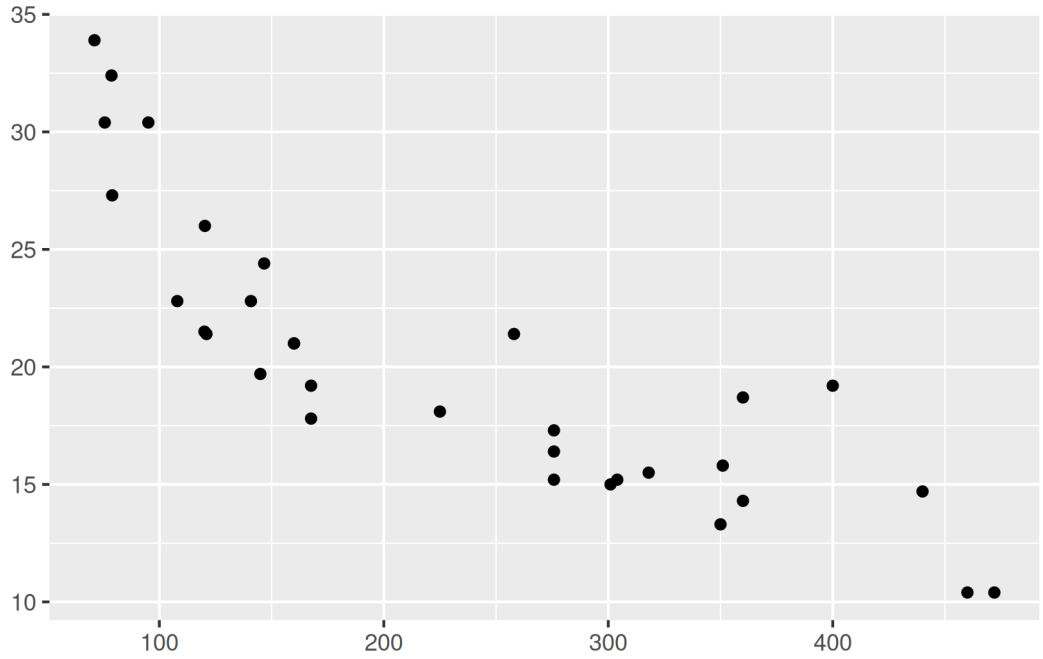
4.6.4 Remove Axis Labels

If you want to remove the axis labels all together, use the `theme()` function. It allows us to modify every aspect of the theme of the plot. Within `theme()`, set the following to `element_blank()`.

- `axis.title.x`
- `axis.title.y`

`element_blank()` will remove the title of the X and Y axis.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme(axis.title.x = element_blank(), axis.title.y = element_blank())
```



4.7 Format Title & Axis Labels

To format the title or the axis labels, we have to modify the theme of the plot using the `theme()` function. We can modify:

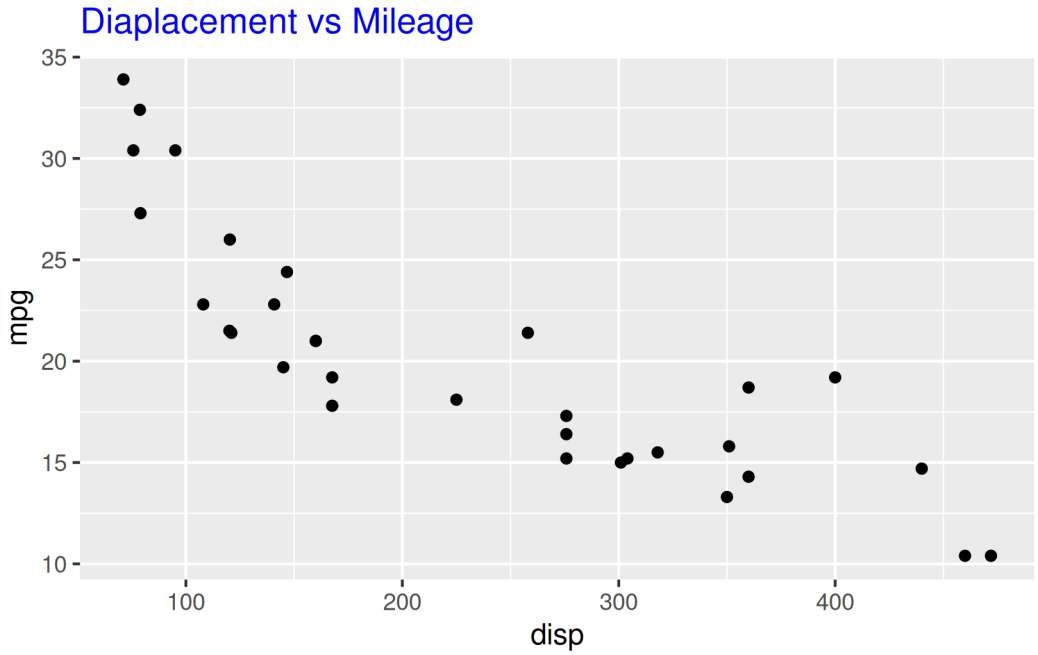
- color
- font family
- font face
- font size
- horizontal alignment
- and angle

In addition to `theme()`, we will also use `element_text()`. It should be used whenever you want to modify the appearance of any text element of your plot.

4.7.1 Color

In the below example, we use the `color` argument within `element_text()` to modify the color of the title of the plot to `'blue'`.

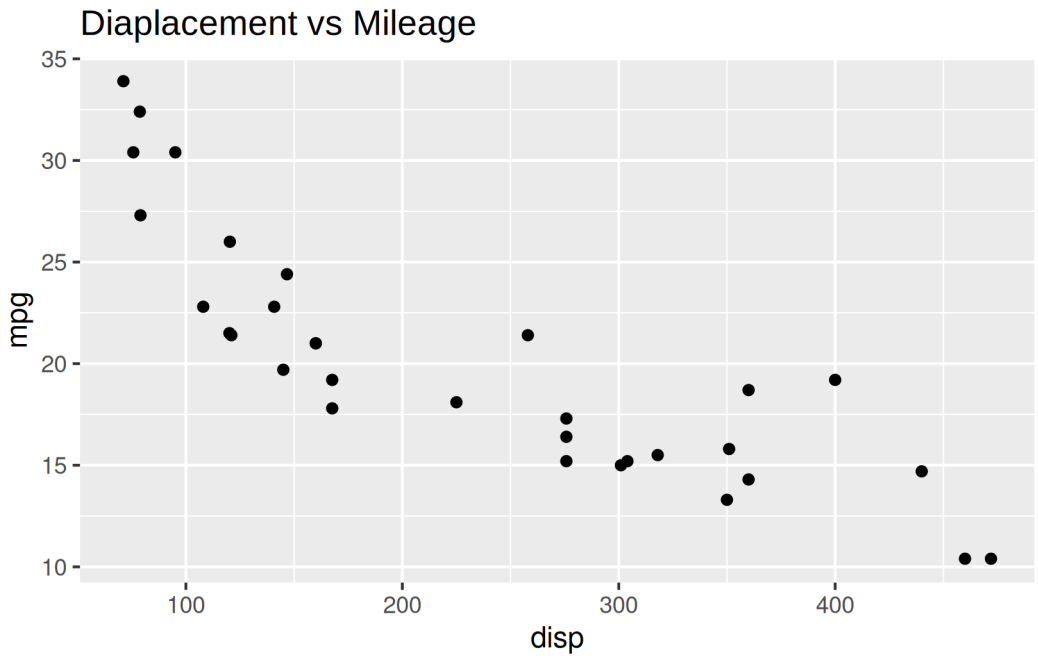
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(color = 'blue'))
```



4.7.2 Font Family

Let us change the font family of the plot title to `'Arial'` by using the `family` argument.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(family = 'Arial'))
```



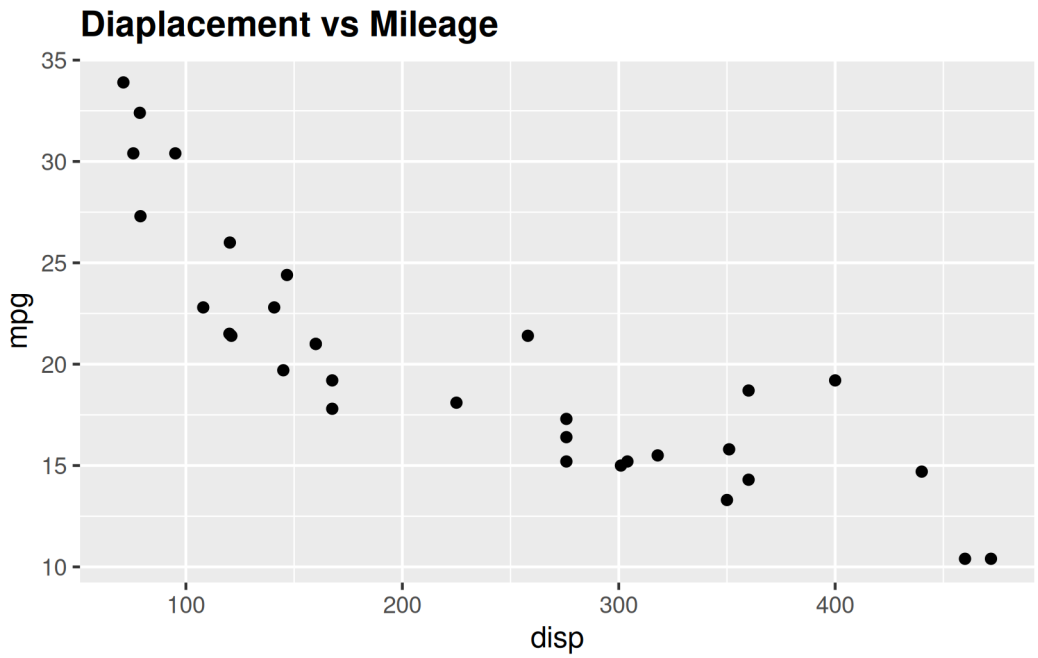
4.7.3 Font Face

The font face can be any of the following:

- `plain`
- `bold`
- `italic`
- `bold.italic`

The `face` argument can be used to modify the font face of the title of the plot.

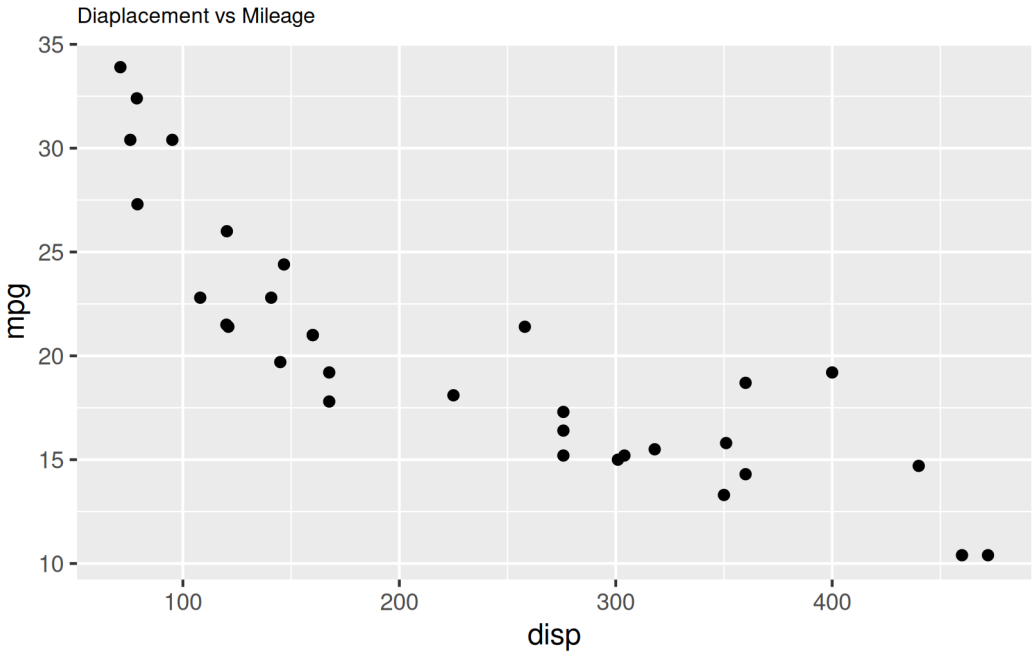
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(face = 'bold'))
```



4.7.4 Font Size

The size of the title of the plot can be modified using the `size` argument.

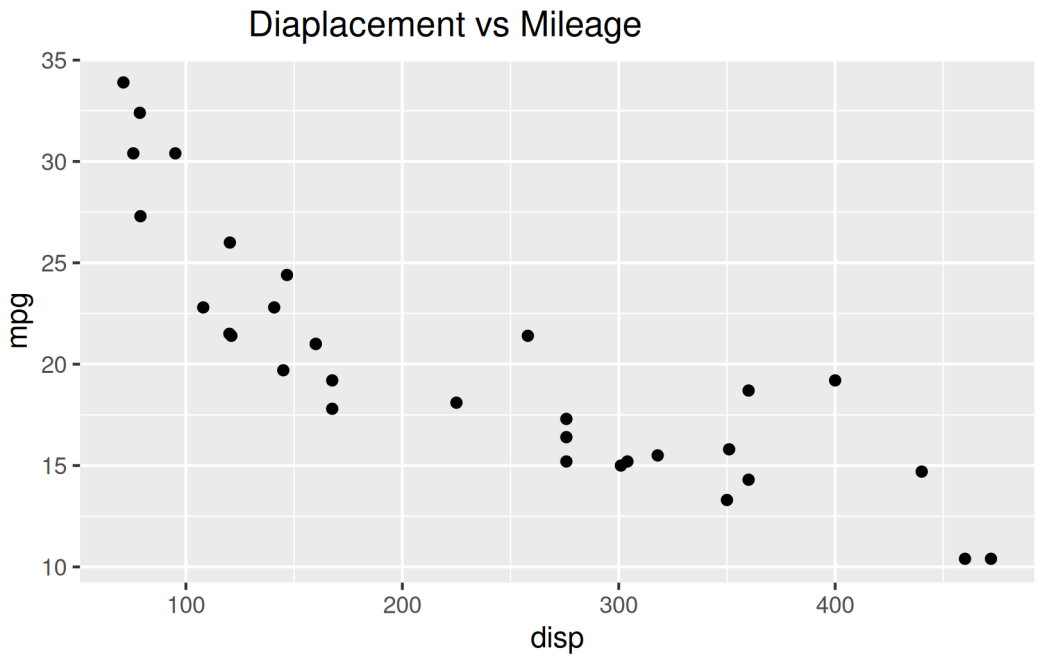
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(size = 8))
```



4.8 Horizontal Alignment

To modify the horizontal alignment of the title, use the `hjust` argument. It can take values between `0` and `1`. If the value is closer to `0`, the text will be left-aligned and viceversa.

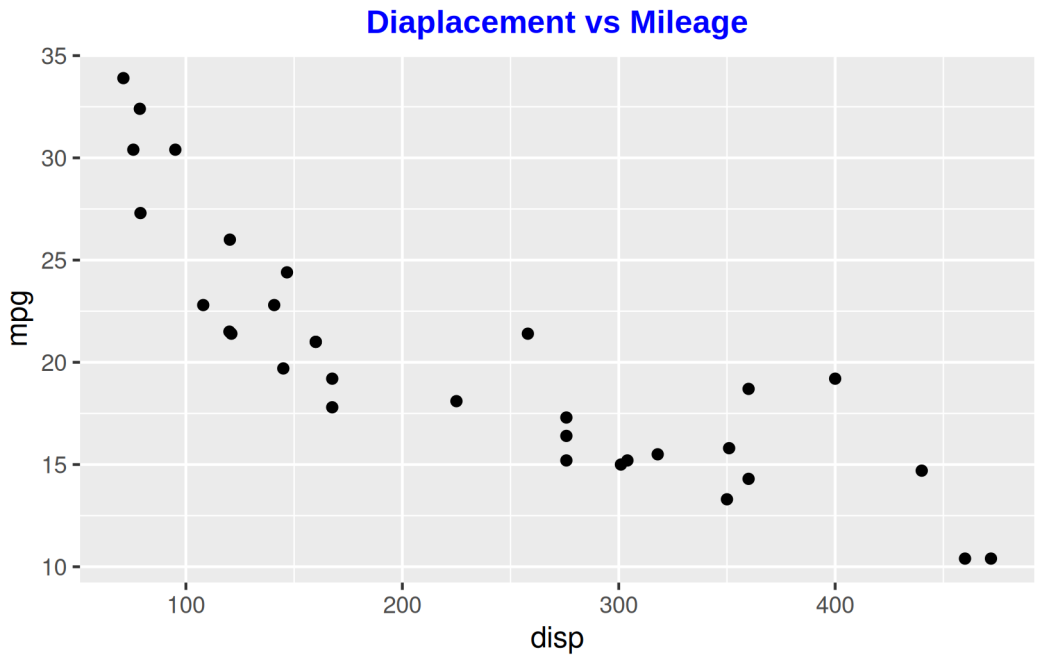
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(hjust = 0.3))
```



4.9 Putting it all together

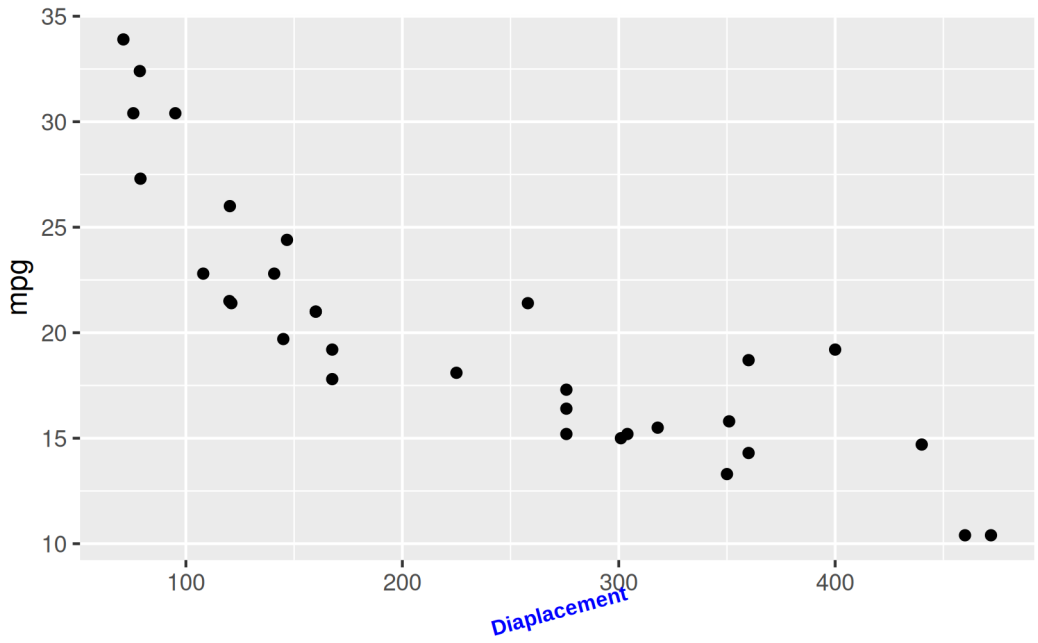
4.9.1 Title

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ggtitle('Diaplacement vs Mileage') +
  theme(plot.title = element_text(color = 'blue', family = 'Arial',
    face = 'bold', size = 12, hjust = 0.5))
```



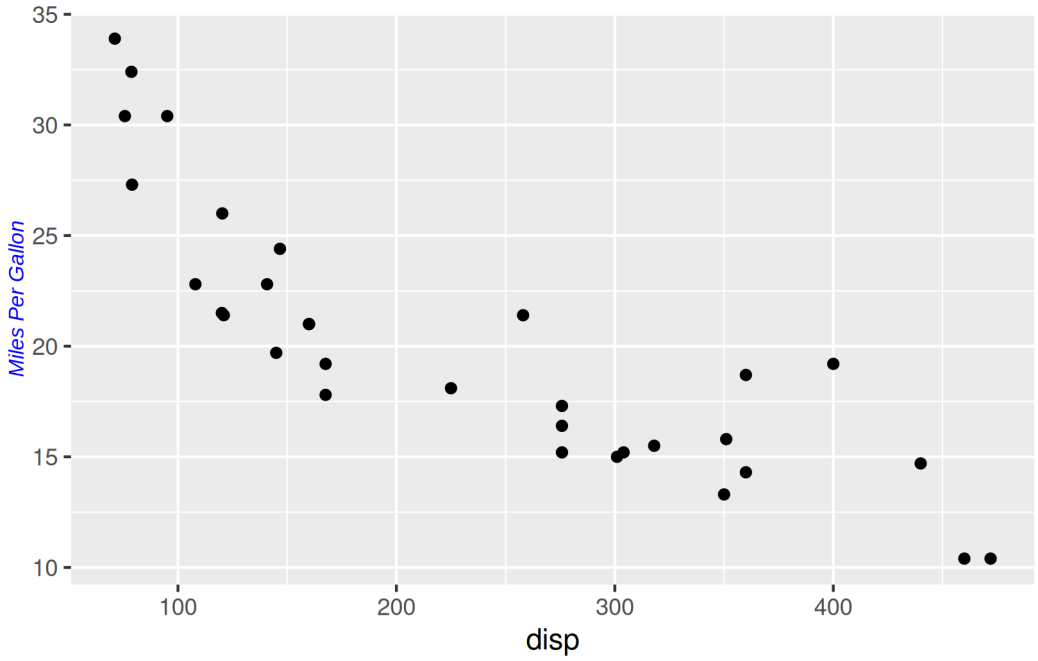
4.9.2 X axis label

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + xlab('Diaplacement') +
  theme(axis.title.x = element_text(color = 'blue', family = 'Arial',
    face = 'bold', size = 8, hjust = 0.5, angle = 15))
```



4.9.3 Y axis label

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) + ylab('Miles Per Gallon') +
  theme(axis.title.y = element_text(color = 'blue', family = 'Arial',
    face = 'italic', size = 8, vjust = 0.3, angle = 90))
```



4.10 Exercises

1. Add `x`, `y` and `title` labels to a `disp` vs `mpg` scatter plot.
2. Rename a legend title with the `name` argument of a scale function.
3. Rewrite a default-titled plot so the title states the conclusion.

Worked solutions in `solutions/labels.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

5 Text Annotations

5.1 How text annotations work in ggplot2

> **Why this chart**
>
> Direct labels and annotations point at the story instead of making readers scan a legend.

> **Common pitfall**
>
> Over-annotating every point creates clutter; annotate only the points that change the conclusion.

In this chapter, we will learn to add text to the plots.

- add custom text
- modify color
- modify size
- modify fontface
- modify angle

5.2 How to annotate plots with text in ggplot2

We will use the `annotate()` function to add custom text to the plots. You can use the `annotate()` function to add rectangles/segments/pointrange as well but our focus will be on adding text. Let us start with a simple scatter plot.

`annotate()` takes the following arguments:

- `geom` : specify text
- `x` : x axis location
- `y` : y axis location
- `label` : custom text
- `color` : color of text
- `size` : size of text
- `fontface` : fontface of text
- `angle` : angle of text

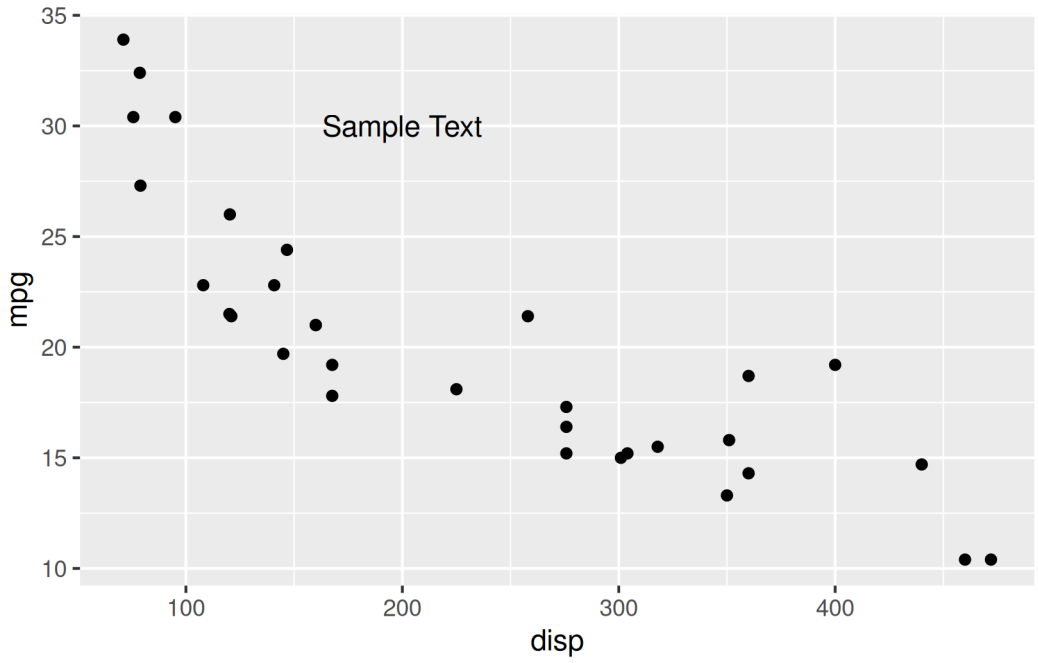
5.2.1 How to add text labels to a ggplot2 plot

Let us begin by adding text to a scatter plot. We will use the mtcars data set and continue to examine the relationship between displacement and miles per gallon. To add the text, we have to indicate that we are using `annotate()` for adding text, and we do this by ensuring that the first input is the word `'text'`. Now, ggplot2 knows that it should add a text to the plot but it still needs other information such as:

- where should the text appear on the plot i.e. location of the text
- and the text itself

We will provide the location by specifying points on the X and Y axis which are also the second and third inputs to `annotate()` and the final input is the text itself, which in our example is `'Sample Text'`.

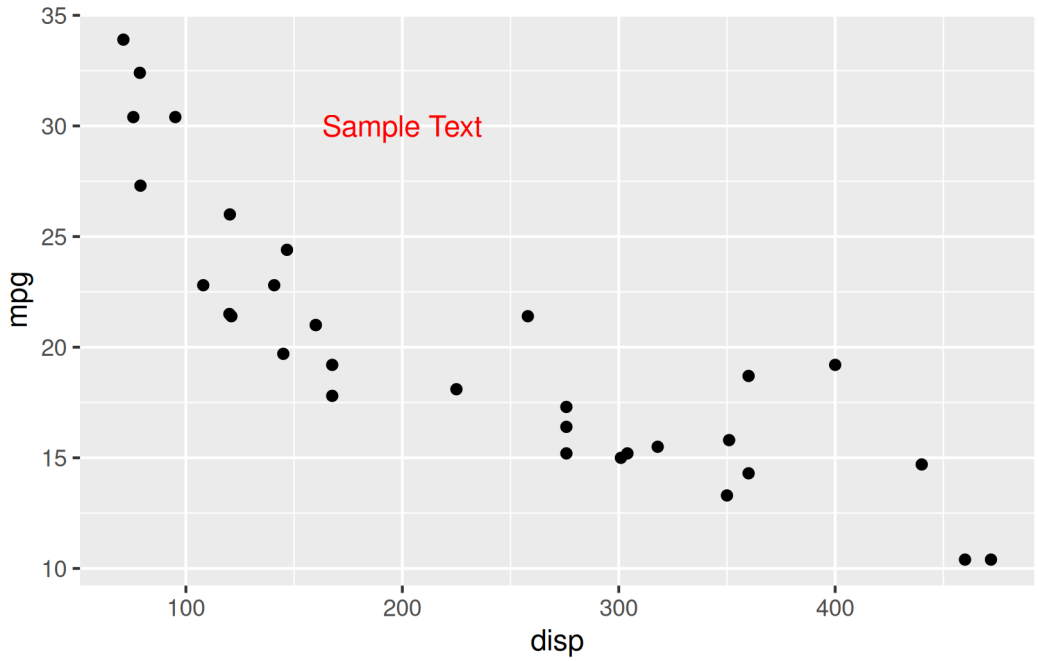
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text')
```



5.2.2 How to change text annotation color in ggplot2

Now that we know how to add text, let us look at modifying the appearance of the text. To change the color, use the `color` argument. In the below example, we modify the color to `'red'`.

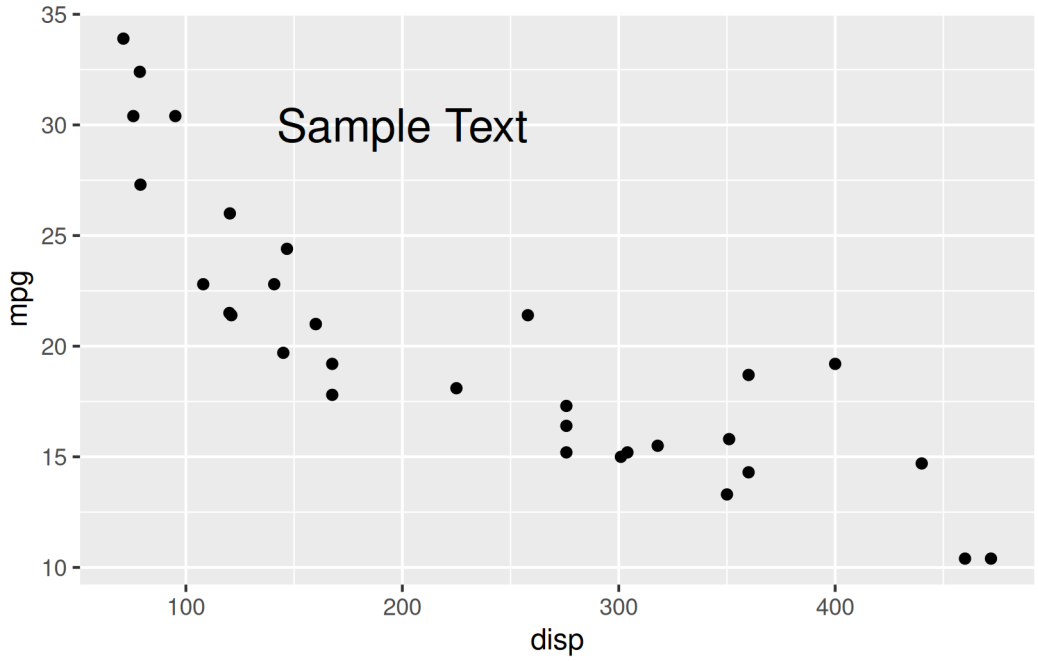
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text', color = 'red')
```



5.2.3 How to change text annotation size in ggplot2

The `size` argument can be used to modify the size of the text.

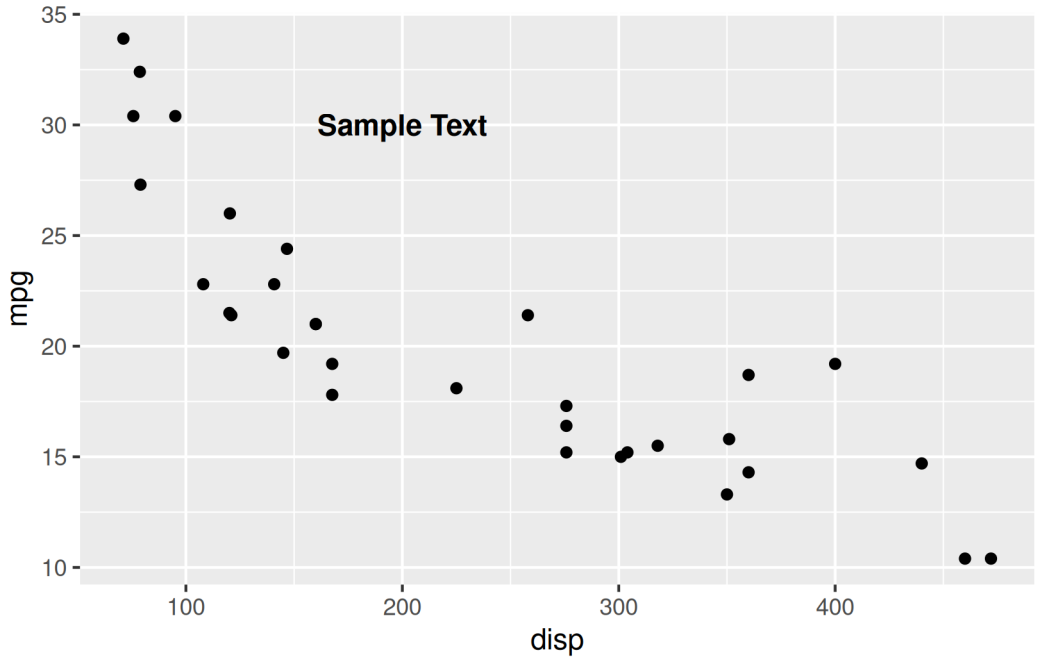
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text', size = 6)
```



5.2.4 How to change text annotation font face in ggplot2

To choose a font of your liking, use the `font` argument and supply it a valid value.

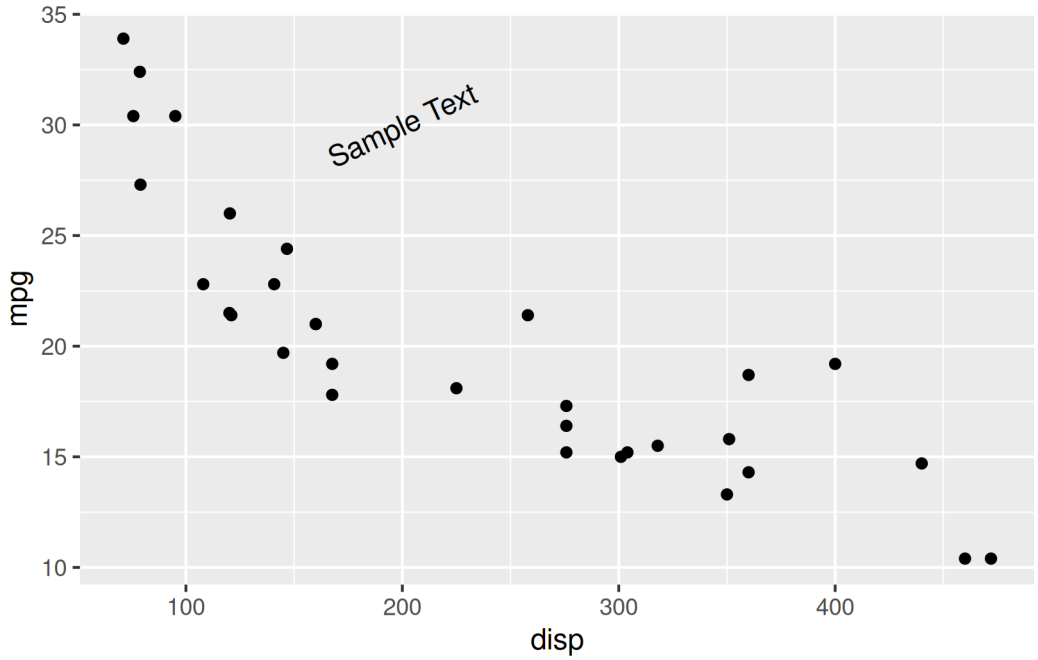
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text', fontface = 'bold')
```



5.2.5 How to rotate text annotations in ggplot2

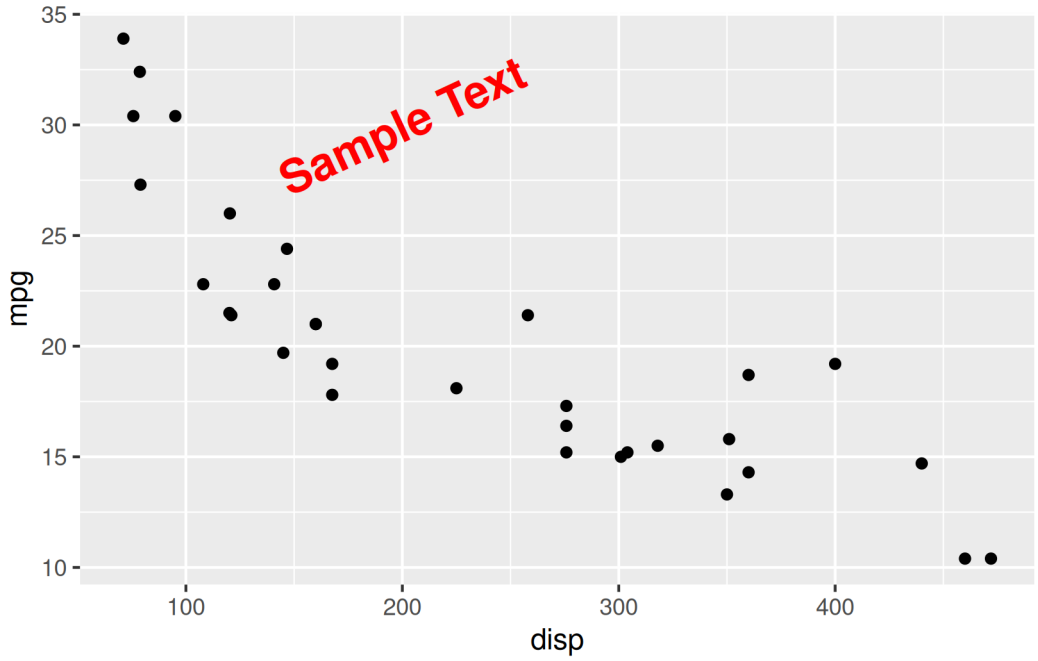
The angle of the text can also be modified using the `angle` argument. In the below example, we modify the angle of the text to 25.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text', angle = 25)
```



5.3 How to combine text annotation options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  annotate('text', x = 200, y = 30, label = 'Sample Text',
           color = 'red', size = 6, fontface = 'bold', angle = 25)
```



5.4 Exercises

1. Annotate the highest-mpg car with `annotate("text", ...)`.
2. Label only cars with `mpg > 30` using `geom_text()`.
3. Decide which two points deserve labels and justify the restraint.

Worked solutions in `solutions/annotations.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

6 Scatter Plots

6.1 Introduction

> **Why this chart**
>
> Scatter plots reveal the relationship between two continuous variables: direction, strength and outliers.

> **Common pitfall**
>
> Overplotting hides density; use alpha transparency or jitter when points pile up.

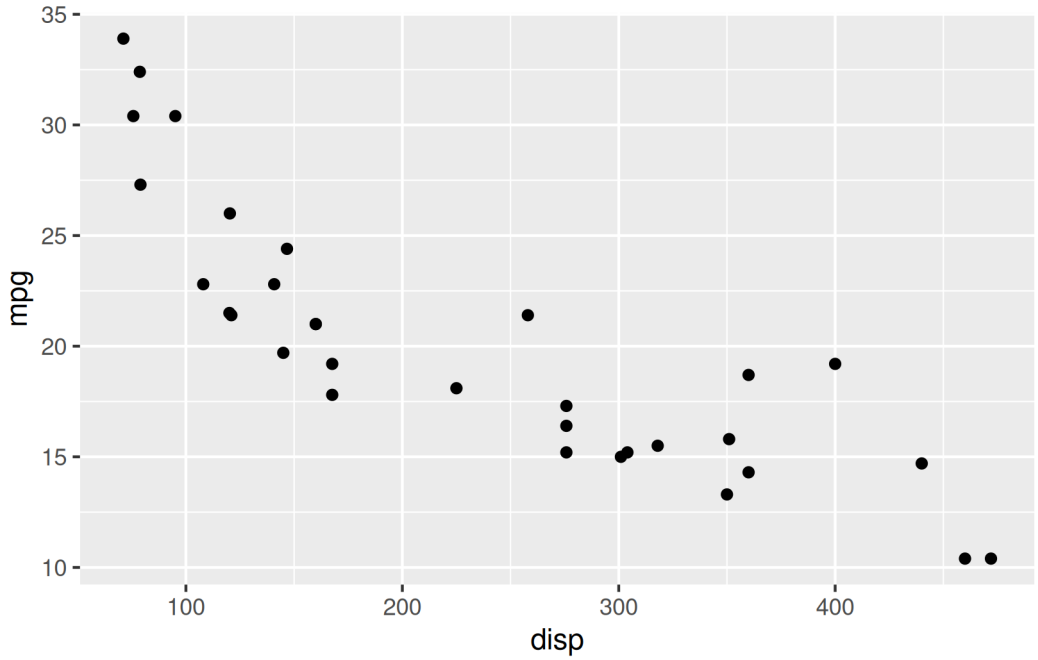
In this chapter, we will:

- build scatter plots
- modify point
  - color
  - fill
  - alpha
  - shape
  - size
- fit regression line

6.2 Basic Plot

As we did in the previous chapter, let us begin by creating a scatter plot using `geom_point()` to examine the relationship between displacement and miles per gallon using the mtcars data.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg))
```



Try it in your browser

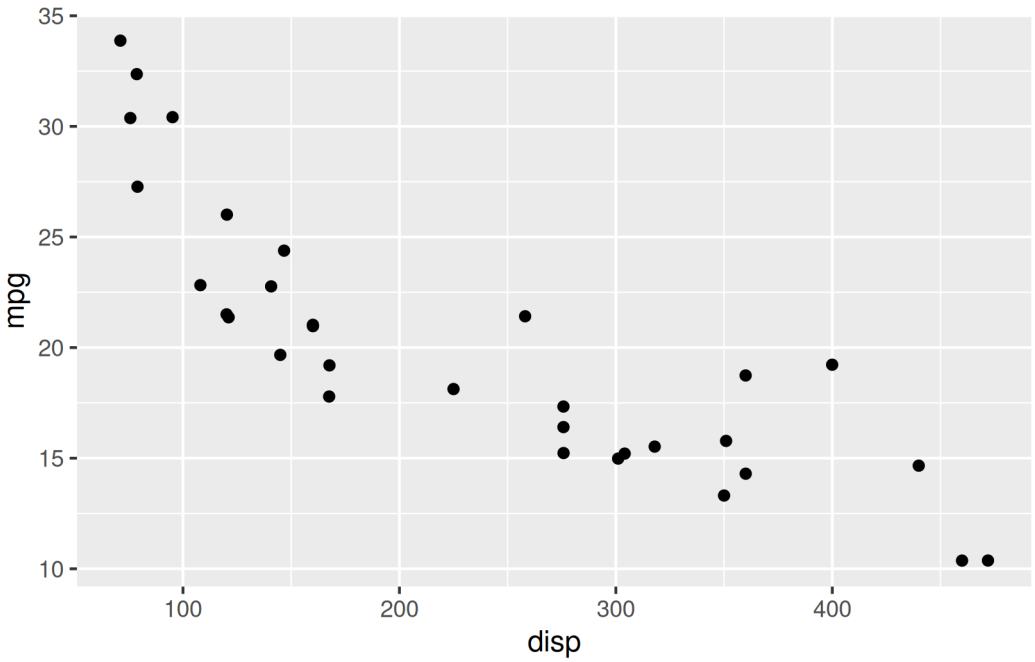
Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime (~30 MB) downloads once, on this page only.

```
library(ggplot2)
ggplot(mtcars, aes(disp, mpg)) +
  geom_point(color = "blue")
```

6.3 Jitter

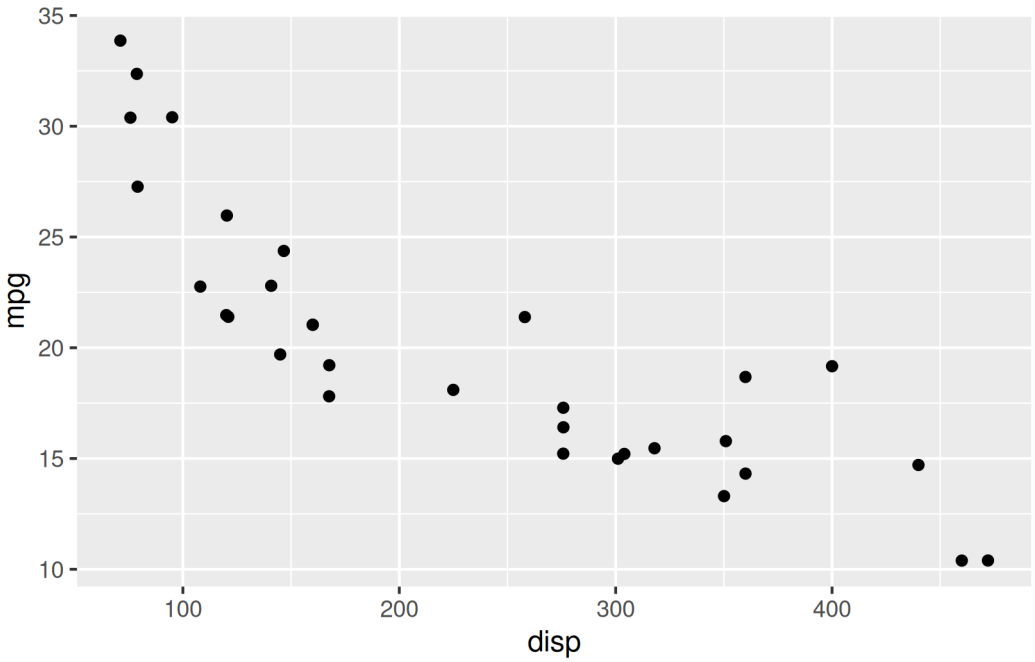
If you want to avoid over plotting, use the `position` argument and supply it the value `'jitter'`. It adds random noise to a plot and makes it easier to read.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg), position = 'jitter')
```



Another way to avoid over plotting is to use `geom_jitter()`.

```
ggplot(mtcars) +
  geom_jitter(aes(disp, mpg))
```



6.4 Aesthetics

Now let us modify the appearance of the points. There are two ways:

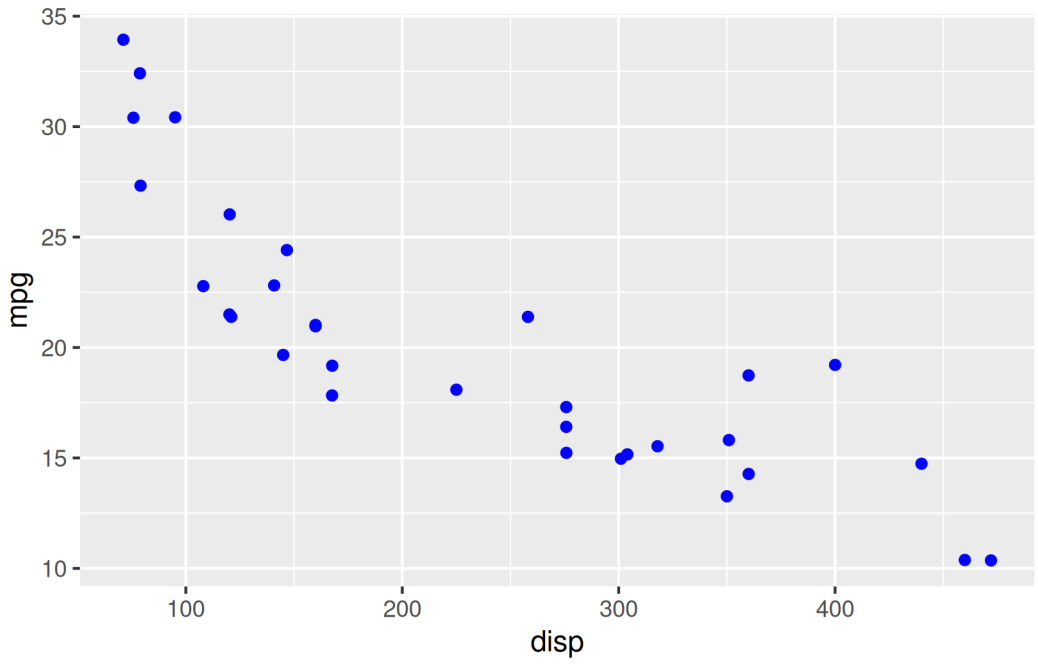
- specify values
- map them to variables using `aes()`

6.4.1 Specify Values

6.4.1.1 Color

To modify the color of the points, you can use the `color` argument and supply it a valid color name. In the below example, we change the color of the points to `'blue'`. Keep in mind that the `color` argument should be outside `aes()`.

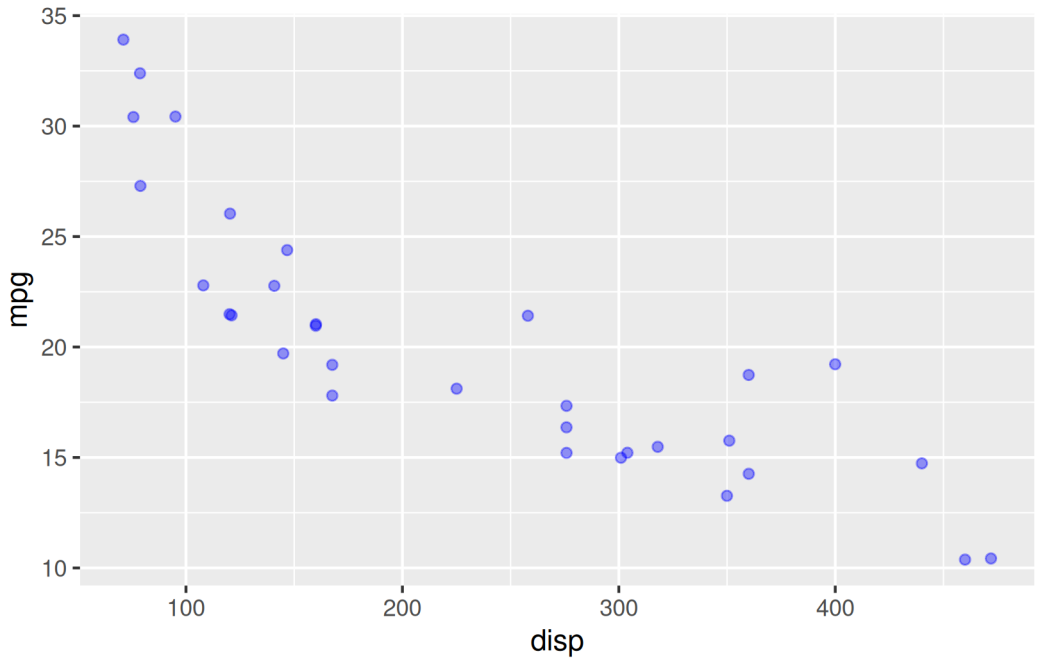
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg), color = 'blue', position = 'jitter')
```



6.4.1.2 Alpha

The transparency of the color can be modified using the `alpha` argument. It takes values between `0` and `1`.

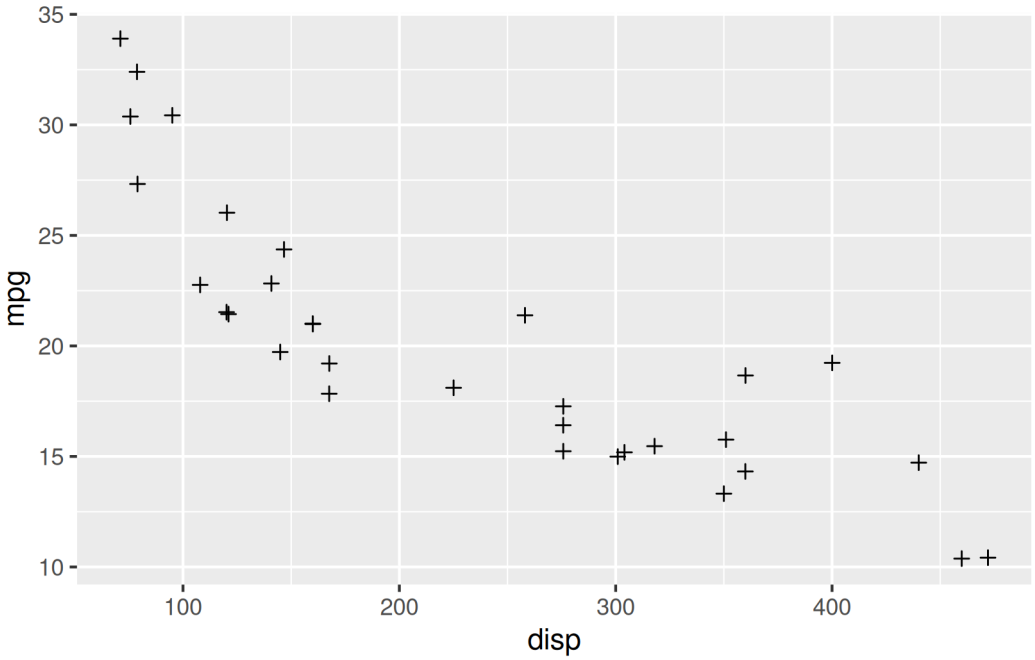
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg), color = 'blue', alpha = 0.4, position = 'jitter')
```



6.4.1.3 Shape

The shape of the points can be modified using the `shape` argument. It takes values between 0 and 25.

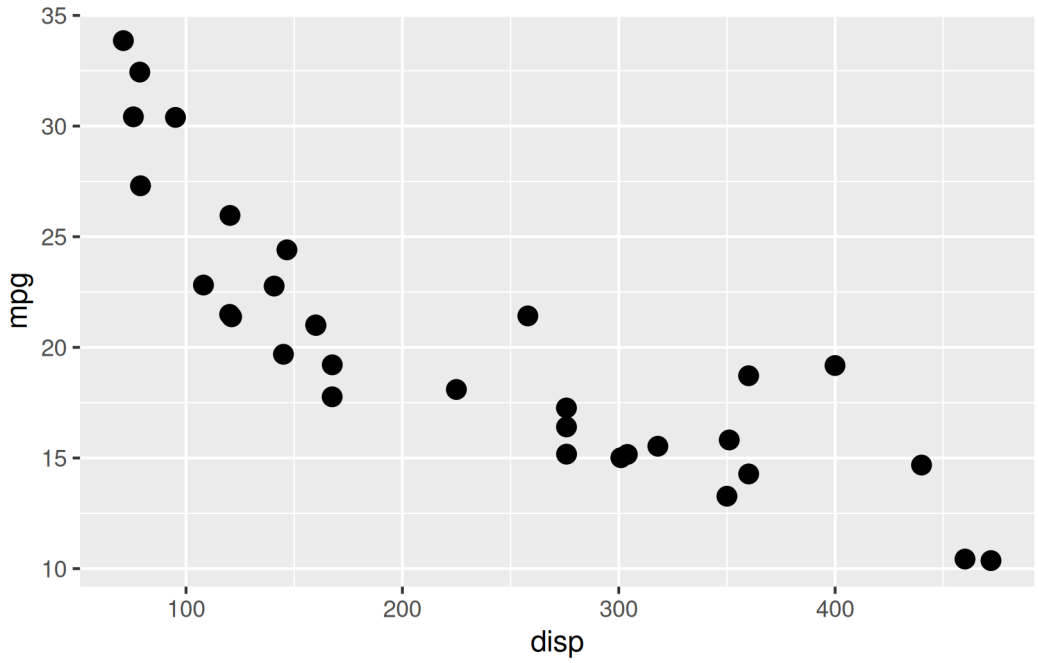
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg), shape = 3, position = 'jitter')
```



6.4.1.4 Size

The size of the points can be modified using the `size` argument. It can take any value greater than `0`.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg), size = 3, position = 'jitter')
```



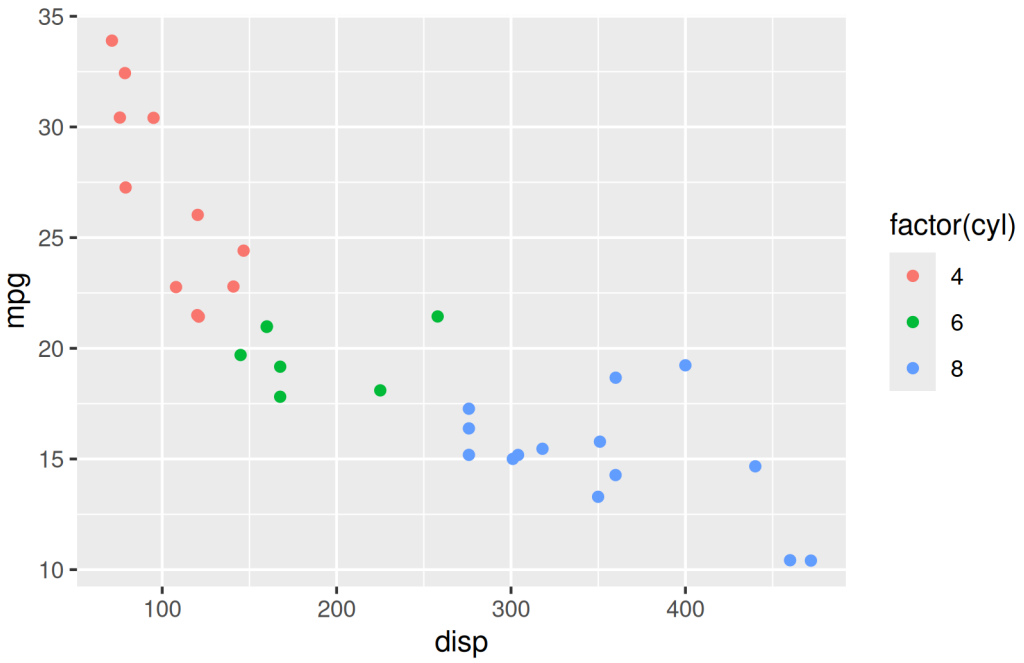
6.4.2 Map Variables

So far, we have specified values for color, shape, size etc. Now, let us map them to variables using `aes()`.

6.4.2.1 Color

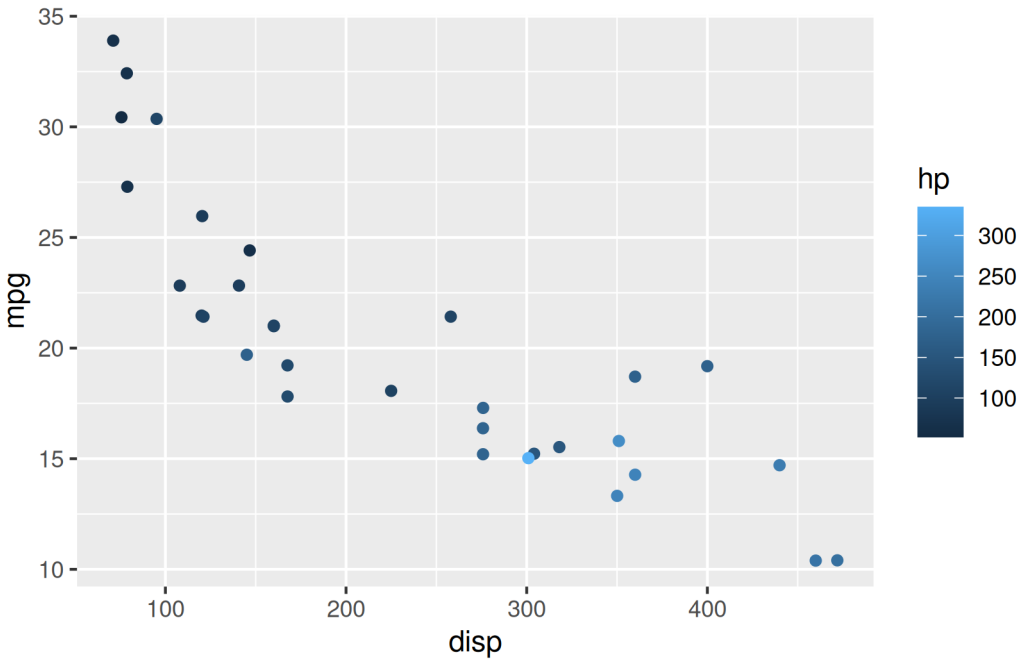
You can modify the color of the points by mapping them to a variable using `aes()`. It allows you to examine the relationship between two continuous variables at different levels of a categorical variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl)),
             position = 'jitter')
```



The color can be mapped to a conitnuous variable as well and in this case you will be able to examine the relationship betweem two continuous variable for a range of value of a third variable.

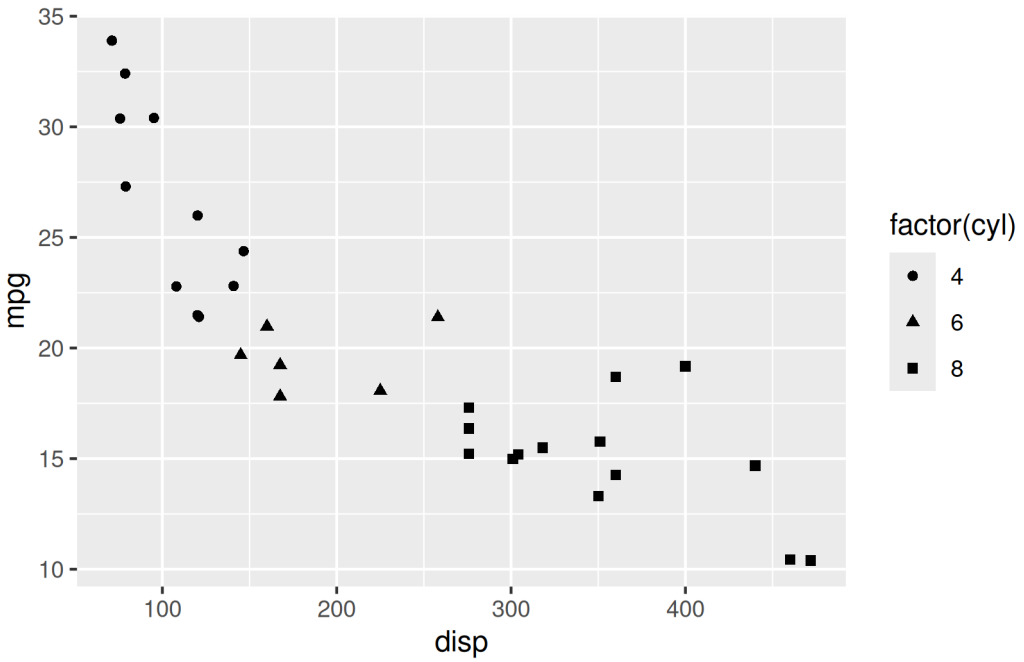
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp),
             position = 'jitter')
```



6.4.2.2 Shape

Shape can be mapped to categorical variables. In the below example, we use `factor()` to convert `cyl` to categorical data before mapping shape to it. ggplot2 will throw an error if you map shape to a continuous variable.

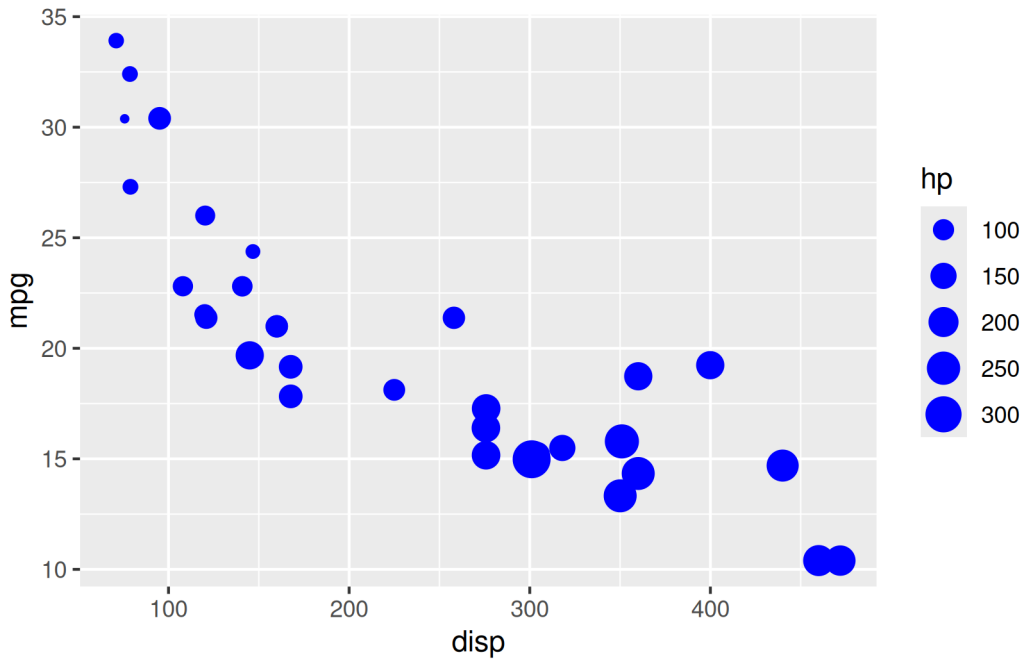
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl)), position = 'jitter')
```



6.4.2.3 Size

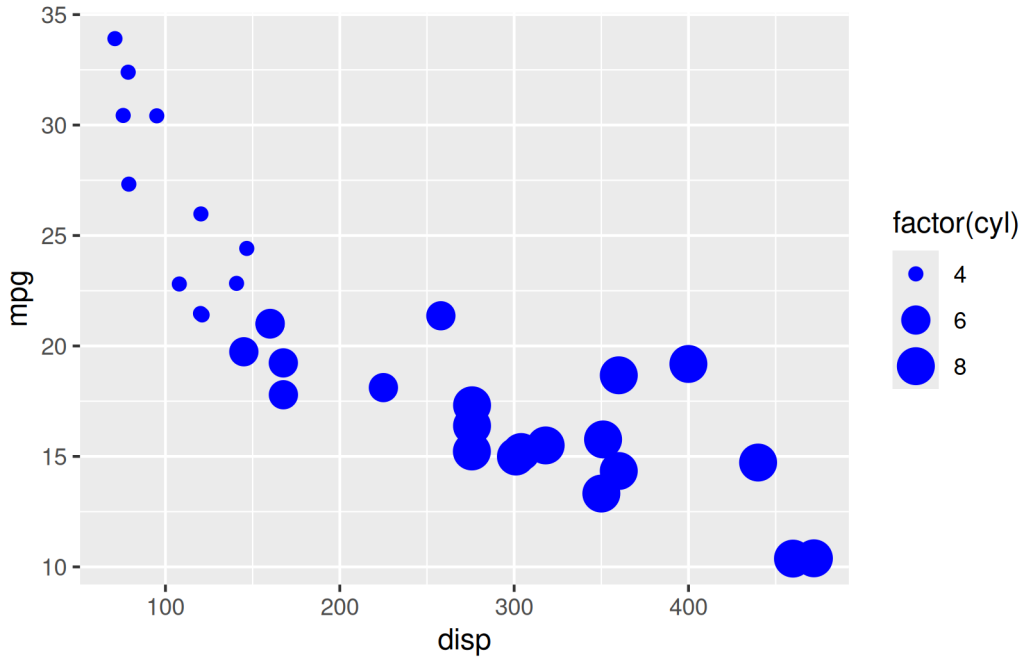
Size must be always mapped to continuous variables. In the below example, we have mapped size to `hp` variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp), color = 'blue', position = 'jitter')
```



If you map size to categorical data as shown in the below example, ggplot2 will throw a warning.

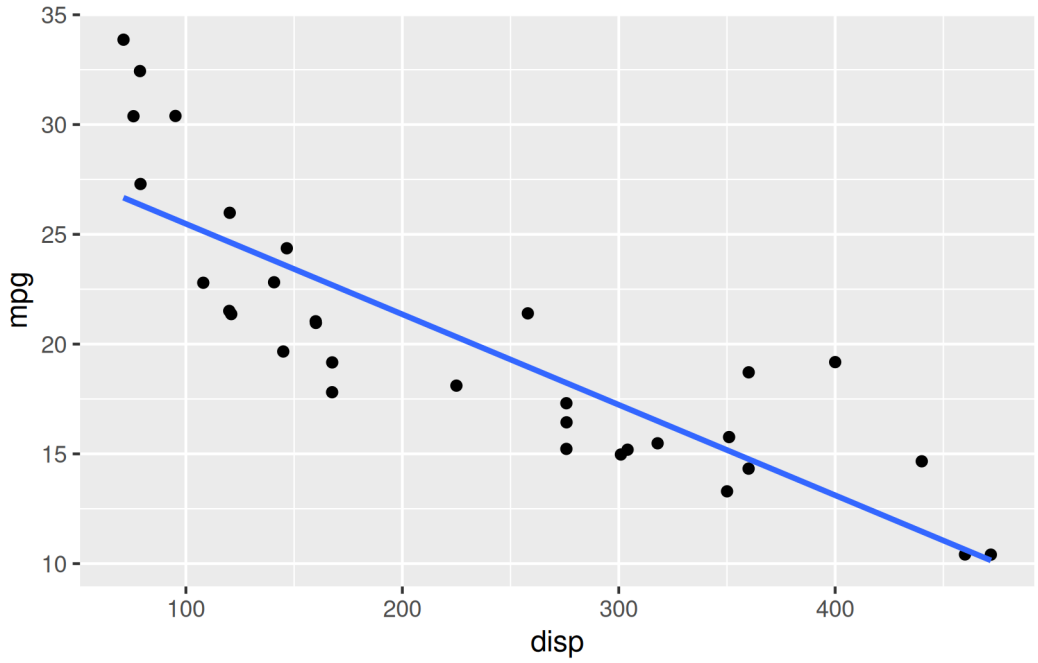
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = factor(cyl)), color = 'blue', position = 'jitter')
```



6.5 Regression Line

`geom_smooth()` allows us to fit a regression line to the plot. By default it will use least squares method to fit the line but you can also use the loess method. In the below example, we fit a regression line using the least squares technique by supplying the value `'lm'` to the `method` argument.

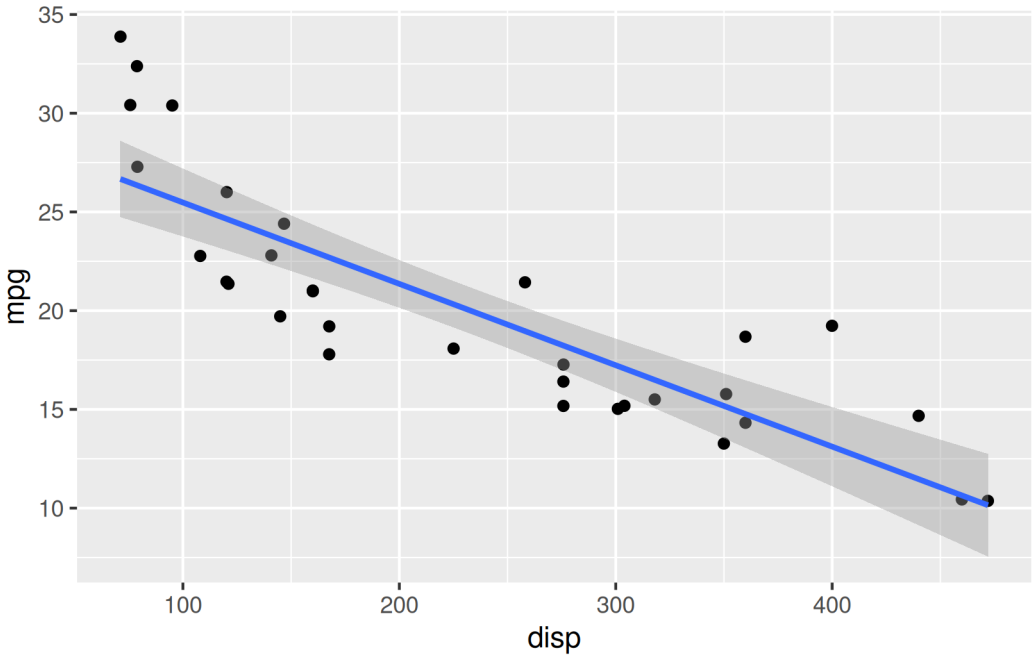
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point(position = 'jitter') +
  geom_smooth(method = 'lm', formula = y ~ x, se = FALSE)
```



The `se` argument will add a confidence interval around the regression line, if set to `TRUE`.

6.5.0.1 Conf. Interval

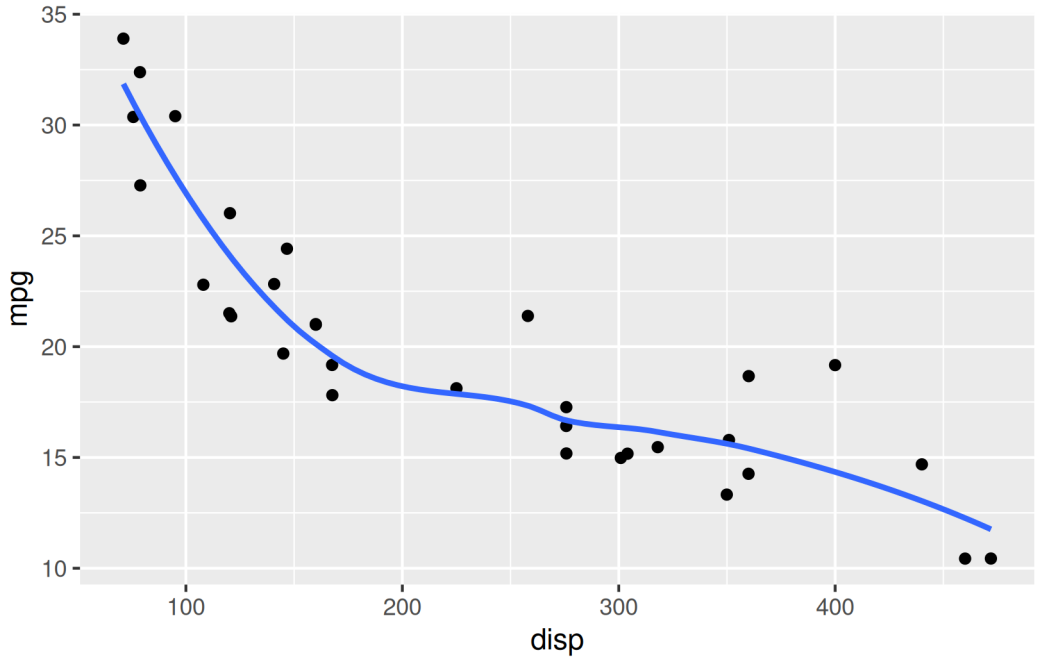
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point(position = 'jitter') +
  geom_smooth(method = 'lm', formula = y ~ x, se = TRUE)
```



6.5.0.2 Loess Method

In the below example, we use the loess method instead of the default least squares method to fit the regression line.

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point(position = 'jitter') +
  geom_smooth(method = 'loess', formula = y ~ x, se = FALSE)
```



6.5.1 Intercept & Slope

If you know the intercept and the slope of the line, you can use `geom_abline()`. Let us regress `mpg` over `disp` and then use the result to add the line.

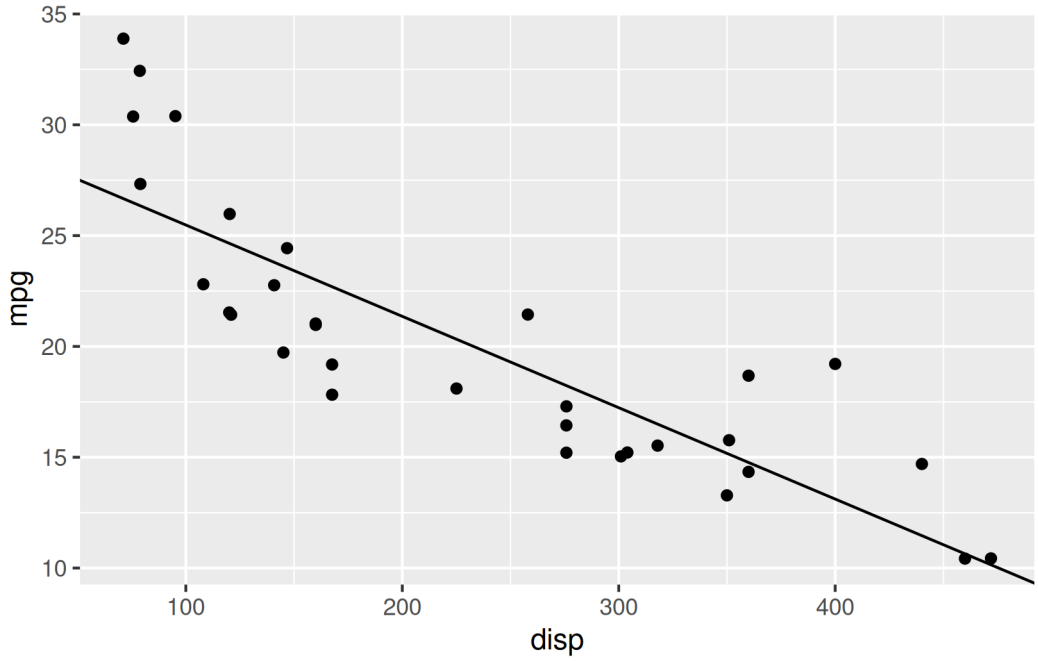
```
lm(mpg ~ disp, data = mtcars)
```

```
Call:
lm(formula = mpg ~ disp, data = mtcars)

Coefficients:
(Intercept)         disp
   29.59985     -0.04122
```

6.5.2 Add Line

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point(position = 'jitter') +
  geom_abline(slope = -0.04122, intercept = 29.59985)
```



6.6 Exercises

1. Color the `disp` vs `mpg` points by cylinders with a plain `Cylinders` title.
2. Add a trend line with `geom_smooth()`: explicit `formula = y ~ x`, no confidence band.
3. Rewrite the plot with a message-first title and `theme_minimal()`.

Worked solutions in `solutions/scatter.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

7 Line Graphs

7.1 Introduction

> **Why this chart**
>
> Line graphs show change over an ordered axis, usually time; slope is the message.

> **Common pitfall**
>
> Unsorted or gappy x values draw zigzag lines; sort by x and handle missing periods explicitly.

In this chapter, we will learn to:

- build
  - simple line chart
  - grouped line chart
- map aesthetics to variables
- modify line
  - color
  - type
  - size

7.2 Case Study

We will use a data set related to GDP growth rate. You can download it from here. It contains GDP (Gross Domestic Product) growth data for the BRICS (Brazil, Russia, India, China, South Africa) for the years 2000 to 2005.

7.2.1 Data

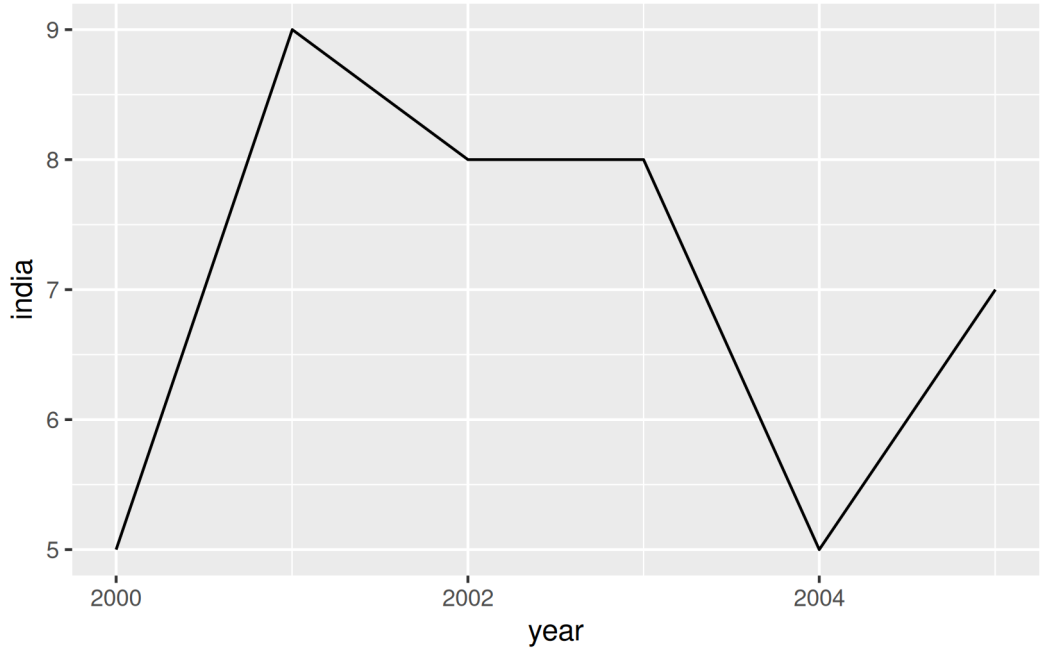
```
gdp <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/
gdp
```

```
# A tibble: 6 x 6
   ...1     X year       growth india china
  <dbl> <dbl> <date>       <dbl> <dbl> <dbl>
1     1     1 2000-01-01       6     5     8
2     2     2 2001-01-01       9     9     5
3     3     3 2002-01-01       8     8     6
4     4     4 2003-01-01       9     8     8
5     5     5 2004-01-01       9     5     9
6     6     6 2005-01-01       8     7     8
```

7.3 Line Chart

To create a line chart, use `geom_line()`. In the below example, we examine the GDP growth rate trend of India for the years 2000 to 2005.

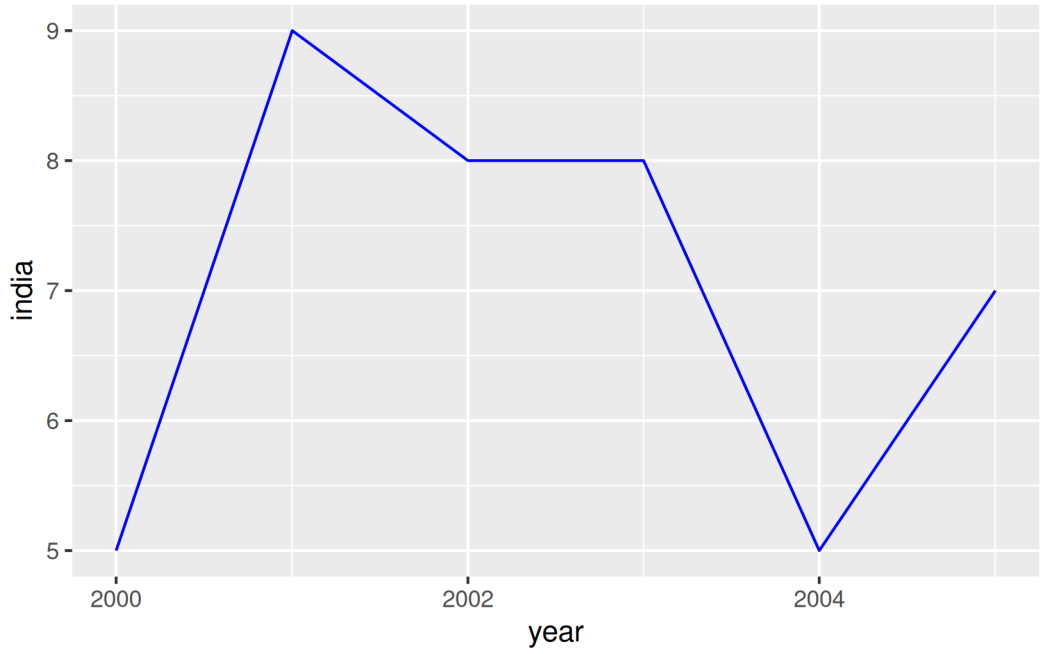
```
ggplot(gdp, aes(year, india)) +
  geom_line()
```



7.3.1 Line Color

To modify the color of the line, use the `color` argument and supply it a valid color name. In the below example, we modify the color of the line to `'blue'`. Remember that the `color` argument should be outside `aes()`.

```
ggplot(gdp, aes(year, india)) +
  geom_line(color = 'blue')
```



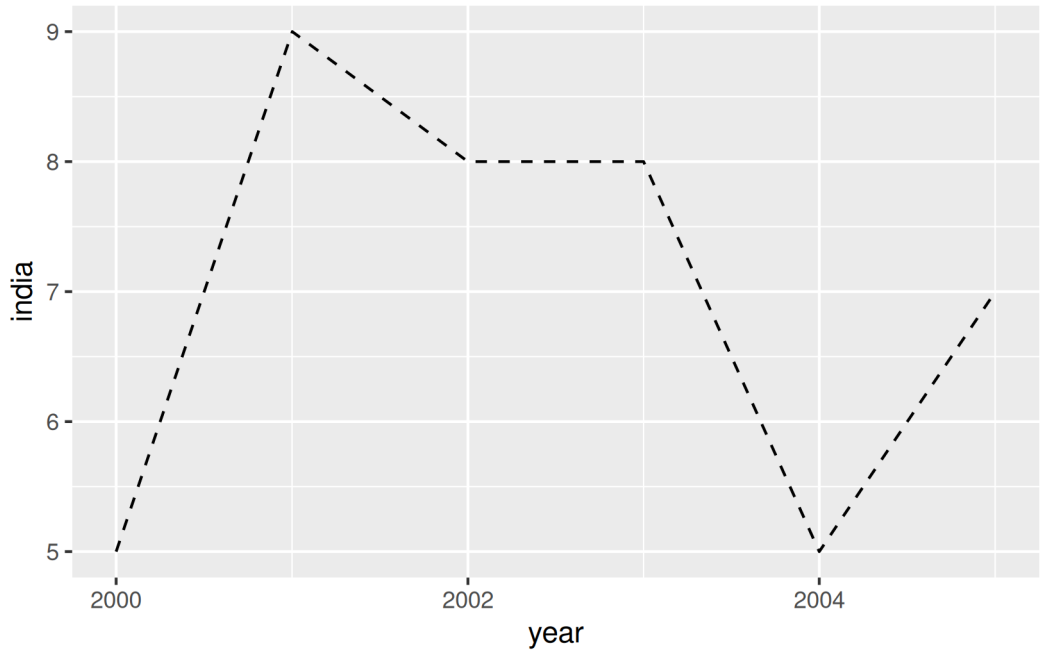
7.3.2 Line Type

The line type can be modified using the `linetype` argument. It can take 7 different values. You can specify the line type either using numbers or words as shown below:

- 0 : blank
- 1 : solid
- 2 : dashed
- 3 : dotted
- 4 : dotdash
- 5 : longdash
- 6 : twodash

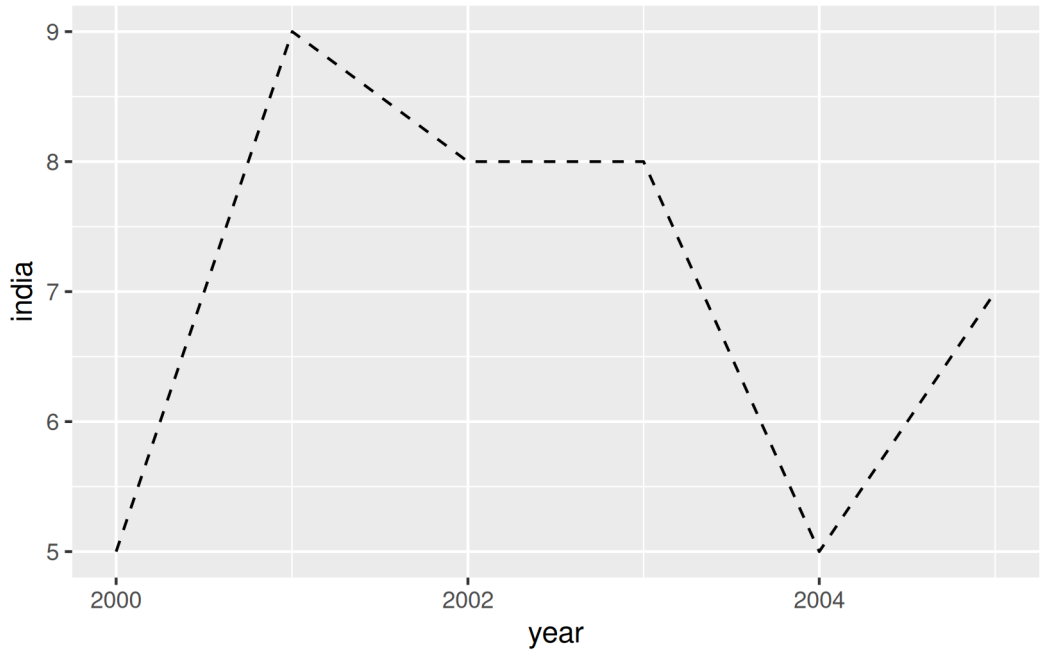
Let us modify the line type to dashed style by supplying the value `2` to the `linetype` argument.

```
ggplot(gdp, aes(year, india)) +
  geom_line(linetype = 2)
```



The above example can be recreated by supplying the value `'dashed'` instead of `2`.

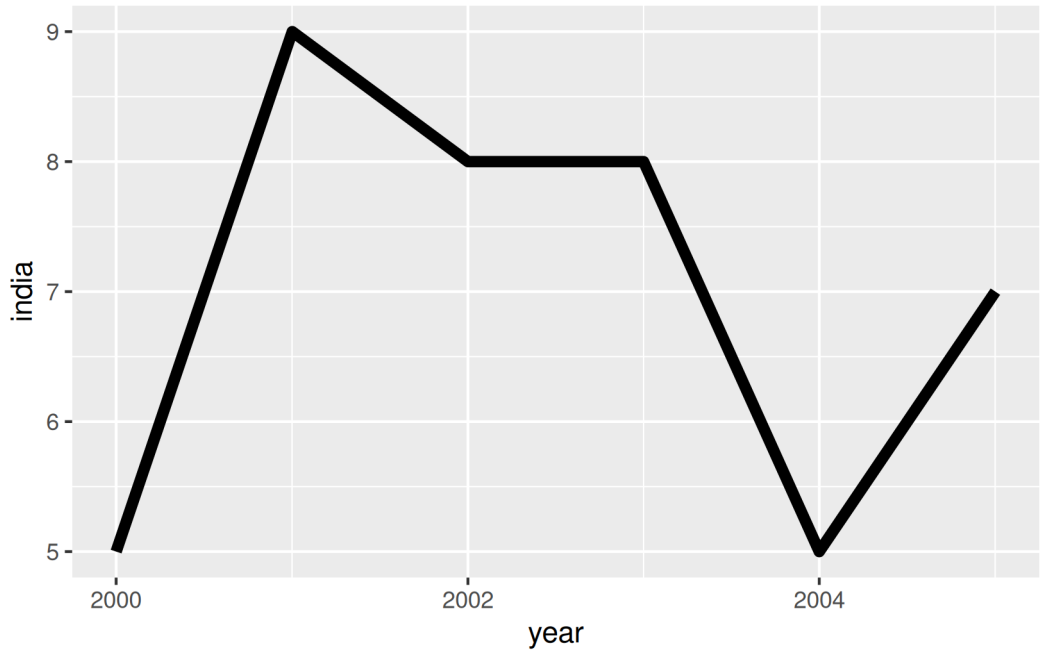
```
ggplot(gdp, aes(year, india)) +
  geom_line(linetype = 'dashed')
```



7.3.3 Line Size

The width of the line can be modified using the `linewidth` argument. It can take any value above 0 as input.

```
ggplot(gdp, aes(year, india)) +
  geom_line(linewidth = 2)
```



7.4 Multiple Lines

7.4.1 Modify Data

Now let us map the aesthetics to the variables. The data used in the above example cannot be used as we need a variable with country names. We will use `pivot_longer()` function from the `tidyr` package to reshape the data.

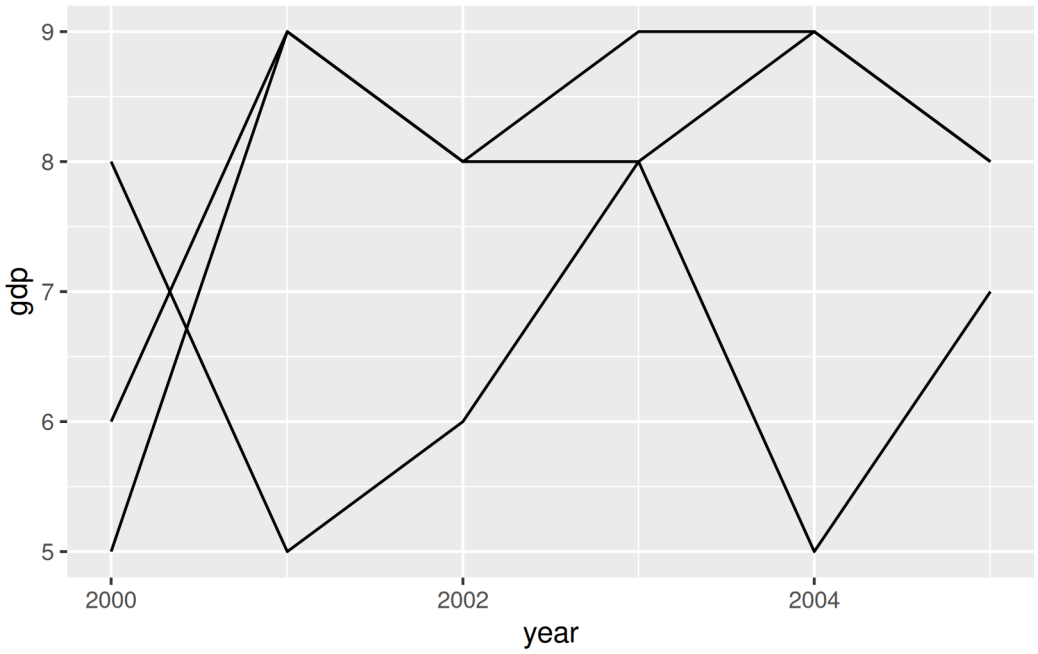
```
gdp2 <- gdp |>
  select(year, growth, india, china) |>
  tidyr::pivot_longer(cols = -year, names_to = "country", values_to = "gdp")

gdp2
```

```
# A tibble: 18 x 3
   year       country   gdp
   <date>     <chr>   <dbl>
 1 2000-01-01 growth      6
 2 2000-01-01 india       5
 3 2000-01-01 china       8
 4 2001-01-01 growth      9
 5 2001-01-01 india       9
 6 2001-01-01 china       5
 7 2002-01-01 growth      8
 8 2002-01-01 india       8
 9 2002-01-01 china       6
10 2003-01-01 growth      9
11 2003-01-01 india       8
12 2003-01-01 china       8
13 2004-01-01 growth      9
14 2004-01-01 india       5
15 2004-01-01 china       9
16 2005-01-01 growth      8
17 2005-01-01 india       7
18 2005-01-01 china       8
```

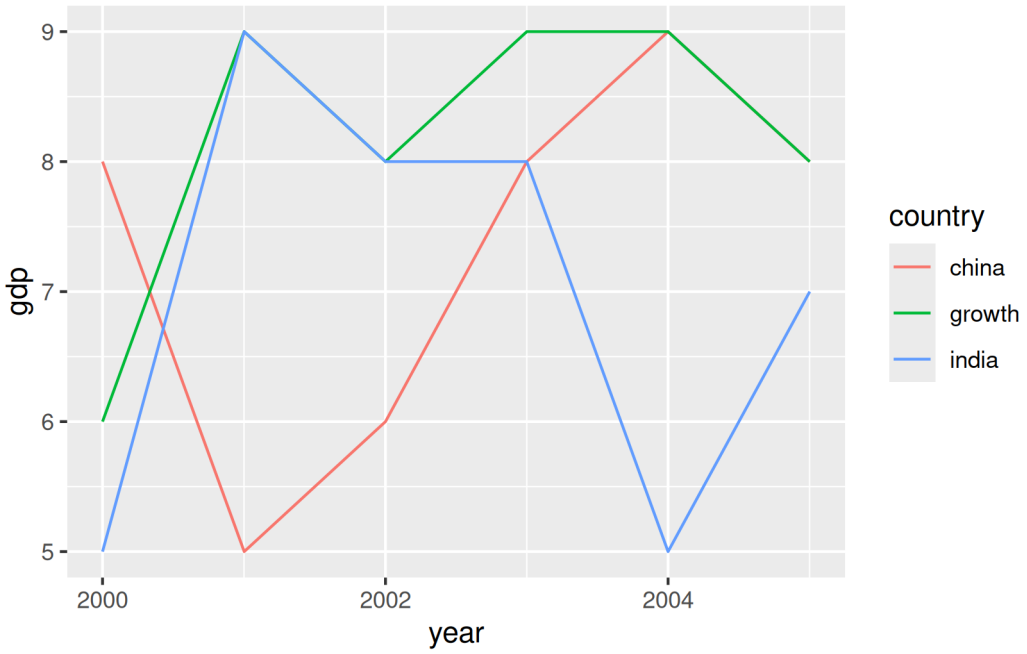
In the original data, to plot GDP trend of multiple countries we will have to use `geom_line()` multiple times. But in the reshaped data, we have the country names as one of the variables and this can be used along with the `group` argument to plot data of multiple countries with a single line of code as shown below. By mapping country to the `group` argument, we have plotted data of all countries.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line()
```



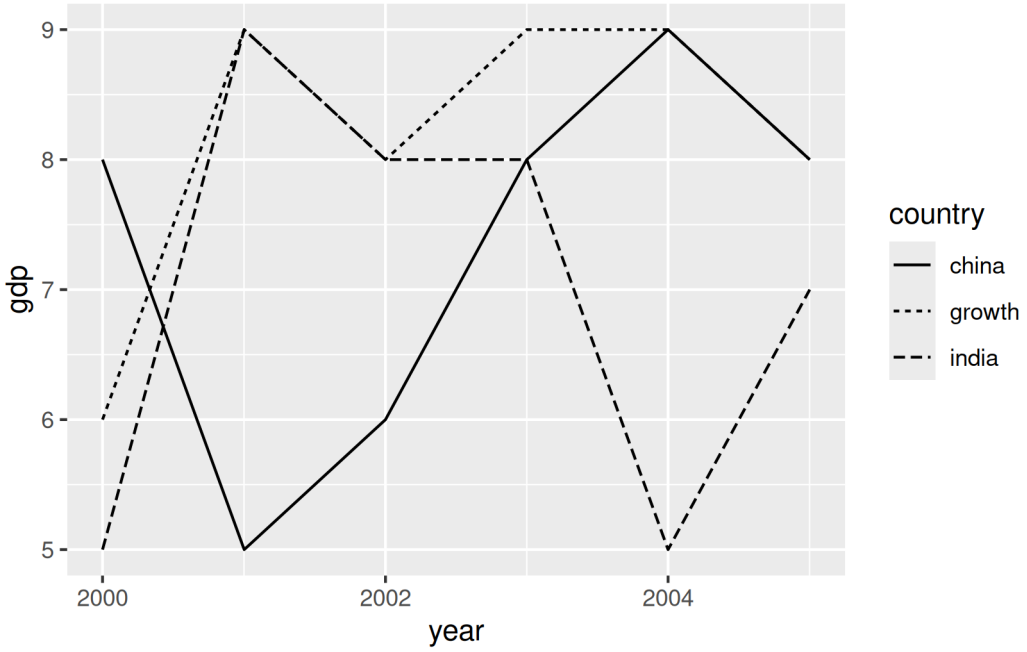
In the above plot, we cannot distinguish between the lines and there is no way to identify which line represents which country. To make it easier to identify the trend of different countries, let us map the `color` argument to the variable country as shown below. Now, each country will be represented by line of different color.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(color = country))
```



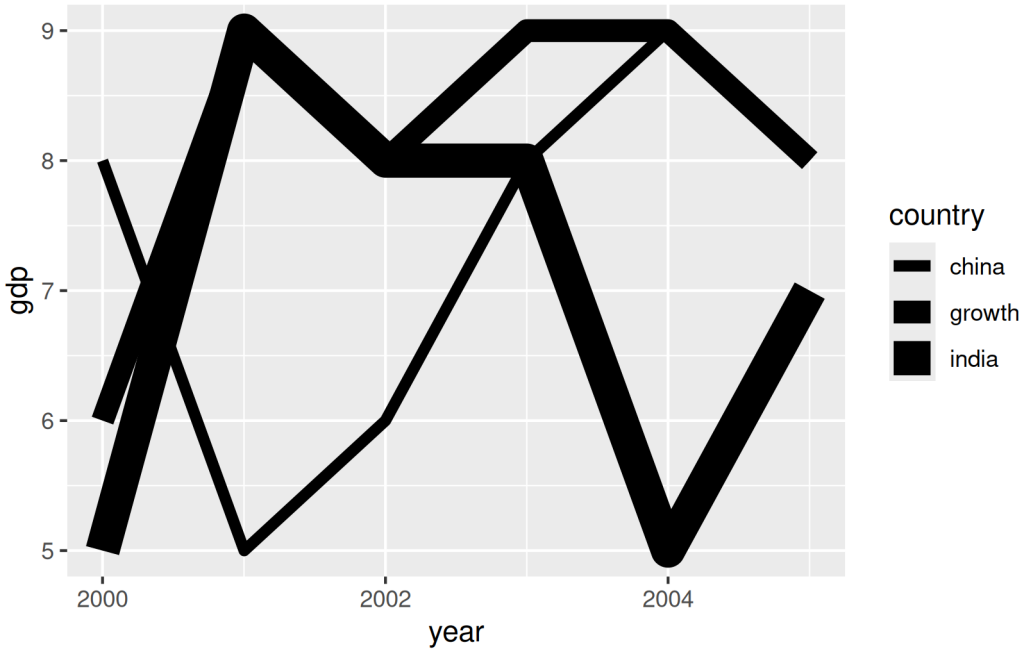
We can map `linetype` argument to country as well. In this case, each country will be represented by a different line type.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(linetype = country))
```



We can map the width of the line to the variable country as well. But in this case, the plot does not look either elegant or intuitive.

```
ggplot(gdp2, aes(year, gdp, group = country)) +
  geom_line(aes(linewidth = country))
```



Remember that in all the above cases, we mapped the arguments to a variable inside `aes()`.

7.5 Exercises

1. Draw the `unemploy` series from `economics` over `date` with `linewidth = 1`.
2. Reshape `unemploy` + `uempmed` to long format and draw both series distinguished by color.
3. Rewrite the plot with a message-first title and units on the y axis.

Worked solutions in `solutions/line.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

8 Bar Plots

8.1 Introduction

> **Why this chart**
>
> Bars encode amounts with length, making comparisons across categories instant.

> **Common pitfall**
>
> Truncating the value axis exaggerates differences; start bars at zero unless you say otherwise.

In this chapter, we will learn to:

- build
  - simple bar plot
  - stacked bar plot
  - grouped bar plot
  - proportional bar plot
- map aesthetics to variables
- specify values for
  - bar color
  - bar line color
  - bar line type
  - bar line size

8.2 Data

```
ecom <- read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/ecom.c
  col_types = list(col_factor(levels = c('Desktop', 'Mobile', 'Tablet')),
  col_logical(), col_logical(),
  col_factor(levels = c('Affiliates', 'Direct', 'Display', 'Organic', 'Paid', 'Referral',
ecom
```

```
# A tibble: 5,000 x 4
   device  bouncers purchase referrer
   <fct>   <lgl>    <lgl>    <fct>
 1 Desktop FALSE    FALSE    Affiliates
 2 Mobile  FALSE    FALSE    Affiliates
 3 Desktop TRUE     FALSE    Organic
 4 Desktop FALSE    FALSE    Organic
 5 Mobile  TRUE     FALSE    Direct
 6 Desktop TRUE     FALSE    Direct
 7 Desktop FALSE    FALSE    Referral
 8 Tablet  TRUE     FALSE    Organic
 9 Mobile  TRUE     FALSE    Social
10 Desktop TRUE     FALSE    Organic
# i 4,990 more rows
```

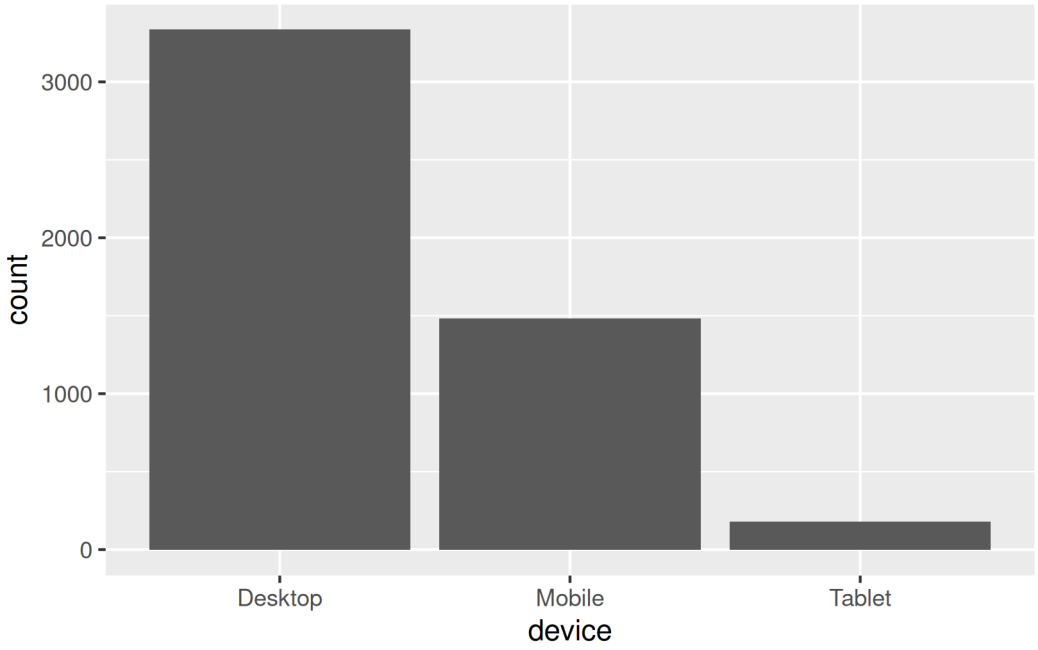
8.2.1 Data Dictionary

- device: device used to visit the website
- bouncers: whether visit was a bouncer (exit website from landing page)
- purchase: whether visitor purchased
- referrer: referrer website/search engine

8.3 Basic Plot

We can create a bar plot using `geom_bar()`. It takes a single input, a categorical variable. In the below example, we plot the number of visits for each device type.

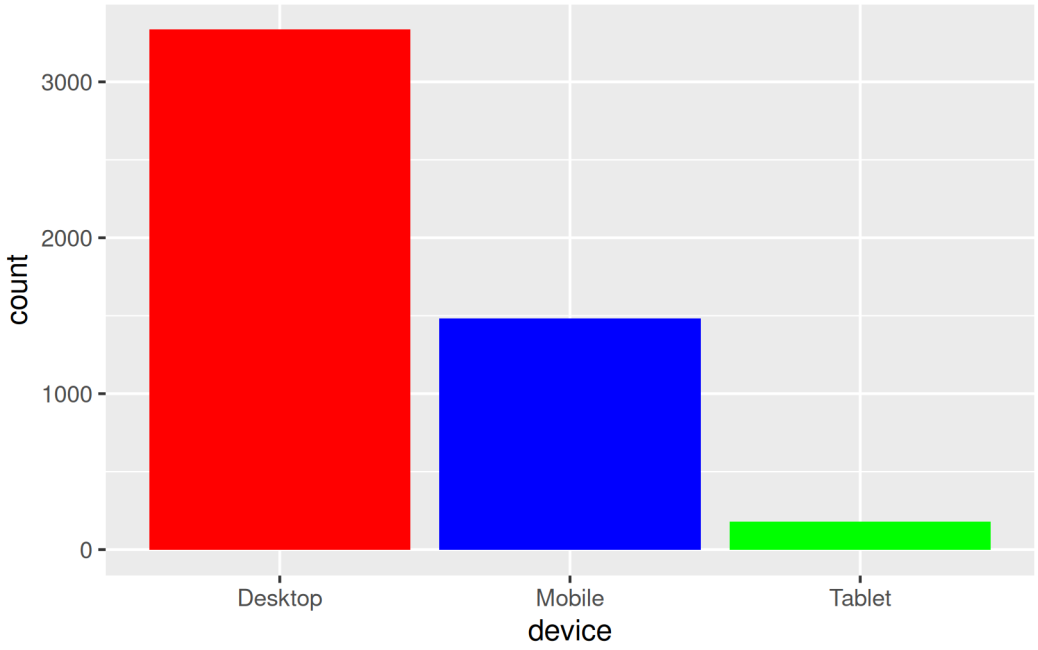
```
ggplot(ecom) +
  geom_bar(aes(device))
```



8.4 Bar Color

The color of the bars can be modified using the `fill` argument. In the below example, we assign different colors to the 3 bars in the plot. If you use the `color` argument, it will modify the color of the bar line and not the background color of the bars. We will look at that later in the chapter.

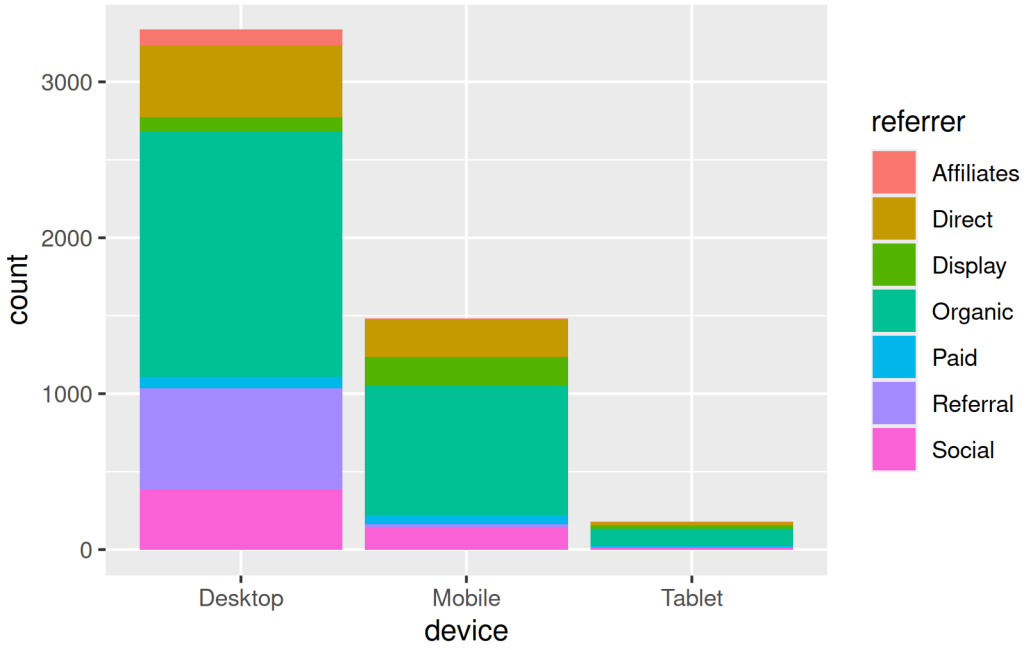
```
ggplot(ecom) +
  geom_bar(aes(device), fill = c('red', 'blue', 'green'))
```



8.5 Stacked Bar Plot

If you want to look at distribution of one categorical variable across the levels of another categorical variable, you can create a stacked bar plot. In ggplot2, a stacked bar plot is created by mapping the `fill` argument to the second categorical variable. In the below example, we have mapped `fill` to `referrer` variable.

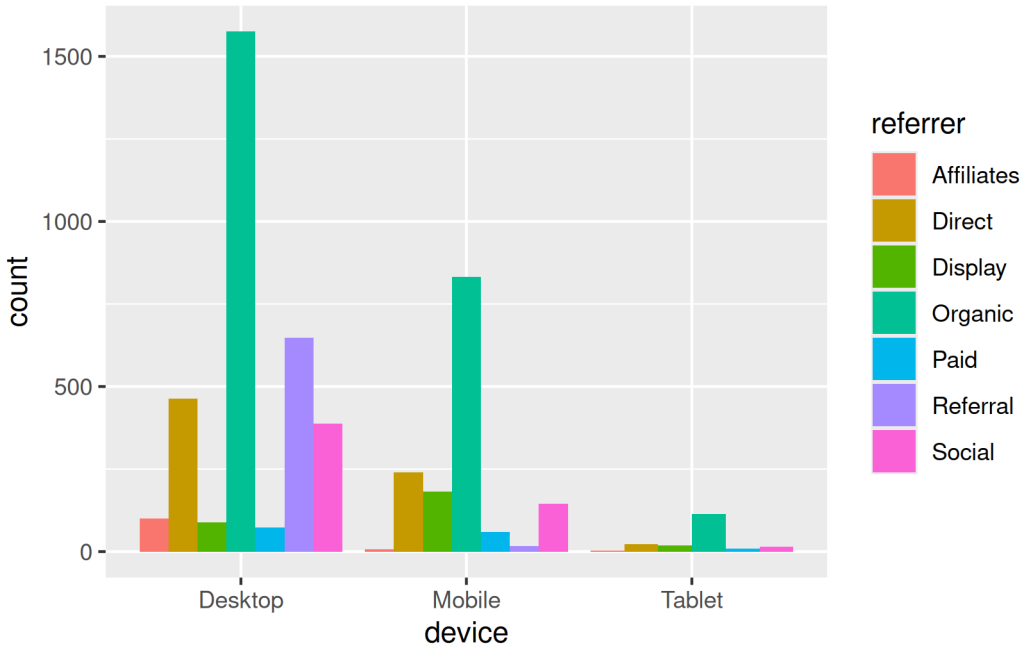
```
ggplot(ecom) +
  geom_bar(aes(device, fill = referrer))
```



8.6 Grouped Bar Plot

Grouped bar plots are a variation of stacked bar plots. Instead of being stacked on top of one another, the bars are placed next to one another and grouped by levels. In the below example, we create a grouped bar plot and you can observe that the bars are placed next to one another instead of being stacked as was shown in the previous example. To create a grouped bar plot, use the `position` argument and set it to `'dodge'`.

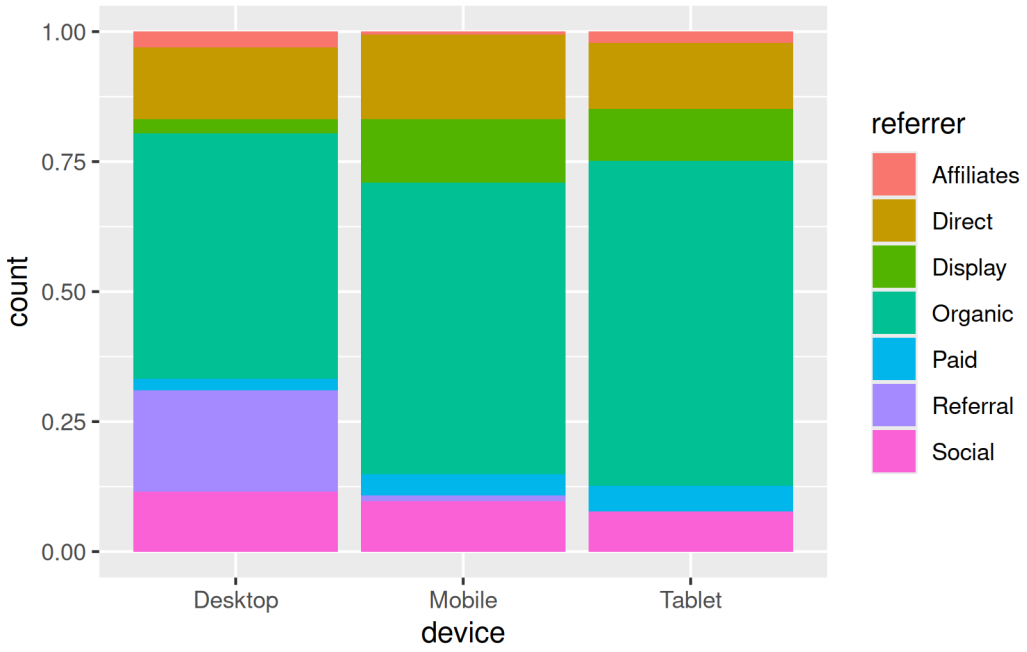
```
ggplot(ecom) +
  geom_bar(aes(device, fill = referrer), position = 'dodge')
```



8.7 Proportional Bar Plot

In a proportional bar plot, the height of all the bars is proportional or same. To create a proportional bar plot, use the `position` argument and set it to `'fill'`.

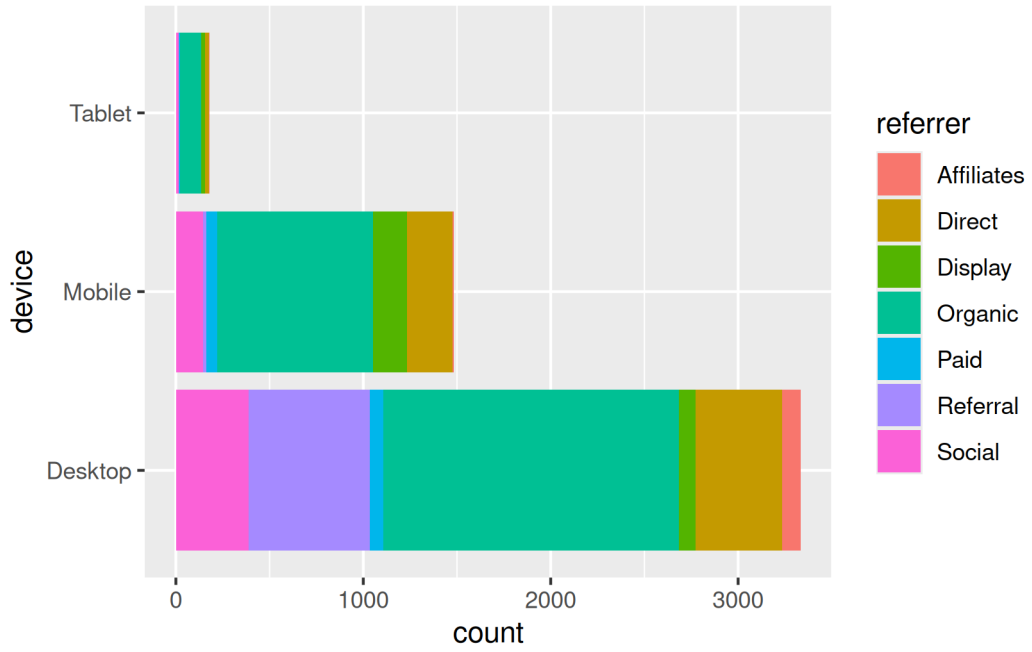
```
ggplot(ecom) +
  geom_bar(aes(device, fill = referrer), position = 'fill')
```



8.8 Horizontal Bar Plot

A horizontal bar plot can be created by flipping the coordinate axes of a regular plot. To flip the axes, use `coord_flip()` as shown below.

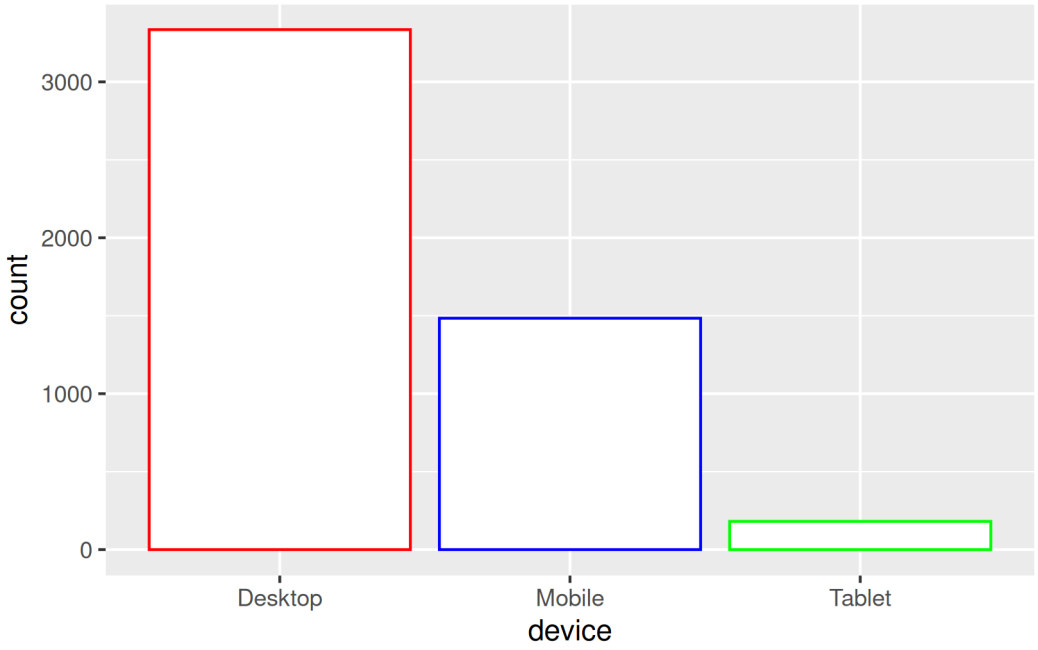
```
ggplot(ecom) +
  geom_bar(aes(device, fill = referrer)) +
  coord_flip()
```



8.9 Bar Line

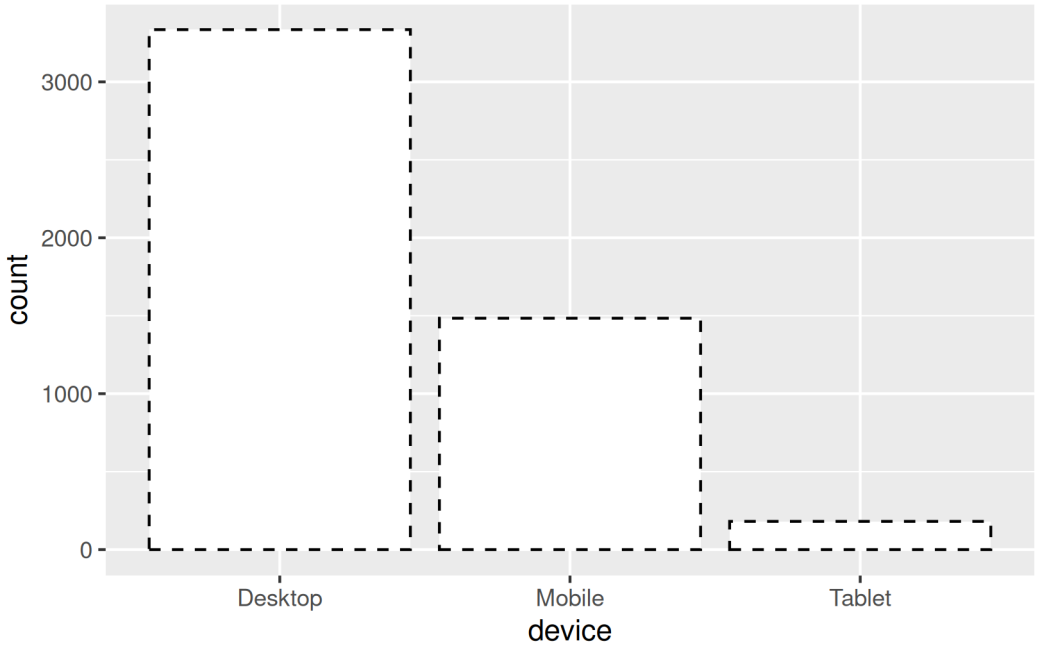
The color of the bar line can be modified using the `color` argument. The color can be specified either using its name or hex code.

```
ggplot(ecom) +
  geom_bar(aes(device), fill = 'white', color = c('red', 'blue', 'green'))
```



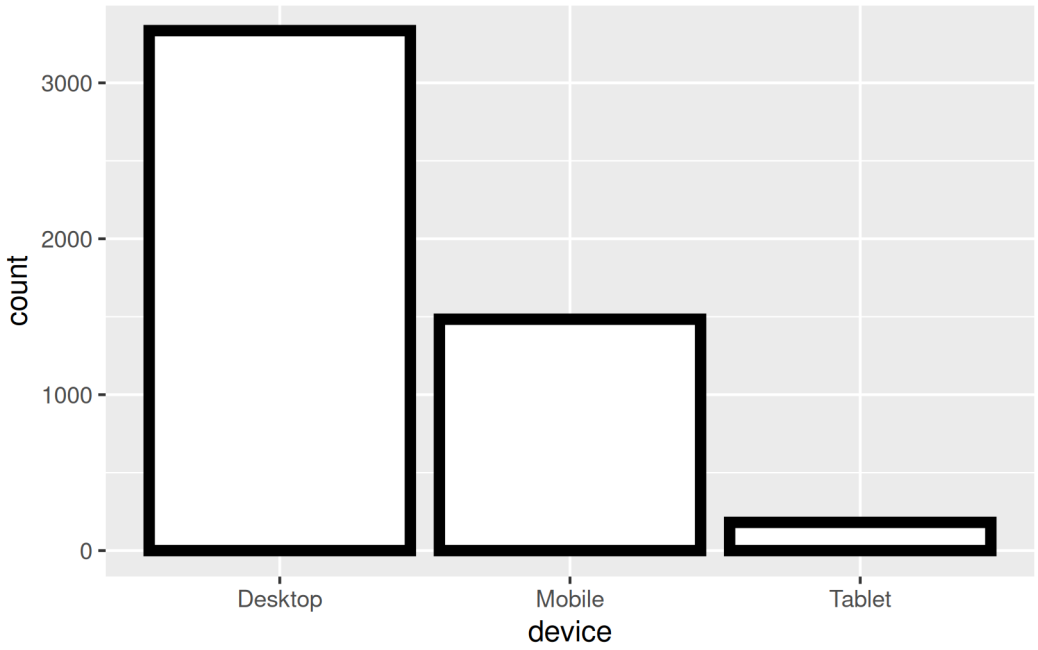
To modify the line type of the bar line, use the `linetype` argument. It can take values between 0 and 6.

```
ggplot(ecom) +
  geom_bar(aes(device), fill = 'white',  color = 'black', linetype = 2)
```



The width of the bar line can be modified using the `linewidth` argument. It can take any value greater than `0`.

```
ggplot(ecom) +
  geom_bar(aes(device), fill = 'white', color = 'black', linewidth = 2)
```



8.10 Exercises

1. Draw counts of cars per cylinder class from `mtcars`.
2. Group by gears with dodged bars, gap tuned via `position_dodge(width = 0.8)`.
3. Rewrite the plot with a message-first title; keep bars starting at zero.

Worked solutions in `solutions/bar.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

9 Box Plots

9.1 Introduction

> **Why this chart**
>
> Box plots compress a distribution into median, quartiles and outliers for fast group comparison.

> **Common pitfall**
>
> Hiding sample size misleads; a slim box from n=5 looks as confident as one from n=5000.

In this chapter, we will learn to:

- build box plots
- modify box
  - color
  - fill
  - alpha
  - line size
  - line type
- modify outlier
  - color
  - shape
  - size
  - alpha

The box plot is a standardized way of displaying the distribution of data. It is useful for detecting outliers and for comparing distributions and shows the shape, central tendancy and variability of the data.

9.2 Structure

- the body of the boxplot consists of a “box” (hence, the name), which goes from the first quartile (Q1) to the third quartile (Q3)

- within the box, a vertical line is drawn at the Q2, the median of the data set
- two horizontal lines, called whiskers, extend from the front and back of the box
- the front whisker goes from Q1 to the smallest non-outlier in the data set, and the back whisker goes from Q3 to the largest non-outlier
- if the data set includes one or more outliers, they are plotted separately as points on the chart

9.3 Data

We are going to use two different data sets in this chapter. Both the data sets have the same data but are in different formats.

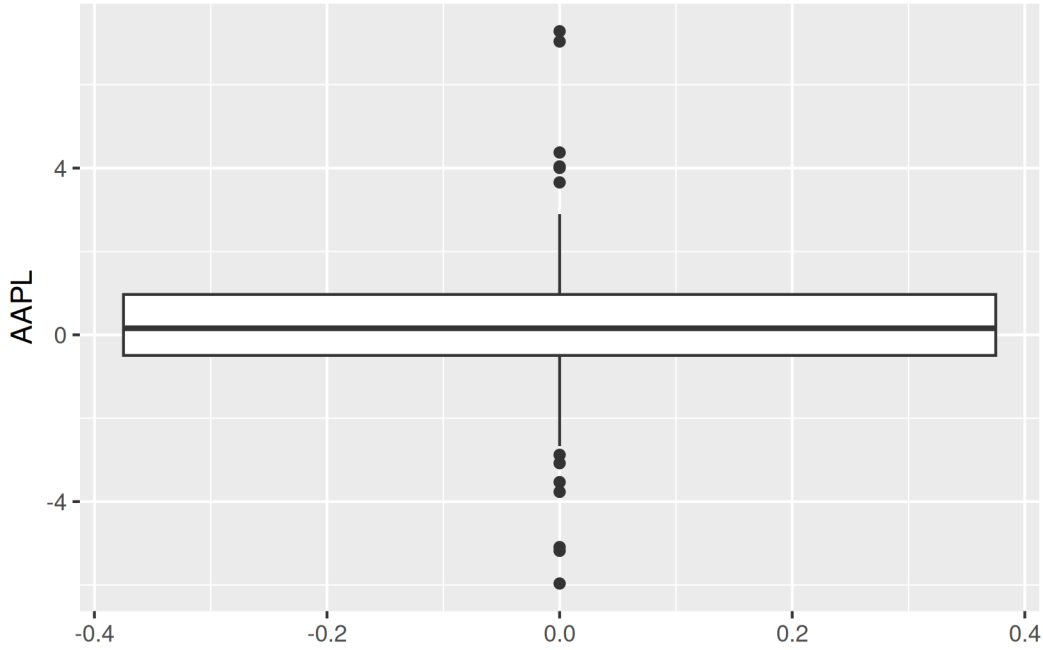
```
daily_returns <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datase
daily_returns
```

```
# A tibble: 250 x 5
      AAPL   AMZN      FB    GOOG    MSFT
     <dbl>  <dbl>   <dbl>   <dbl>   <dbl>
 1  1.38    24.2   2.12    22.4    1.12
 2  2.83     3.25 -0.860    5.99   0.767
 3 -0.0394   9.91  1.45     6.75   0.973
 4  0.108    3.76 -0.770  -10.7   -0.285
 5  1.64    19.8   4.75     8.66   0.501
 6  0.0689   5.33 -0.300   -0.930  0.256
 7 -0.561   -5.21 -0.630   -7.28  -0.708
 8  0.551    0.25 -0.460    0.690  0.128
 9 -0.217  -13.6   0.0300   6.56   0.0786
10 -0.108   -4.25  0.460    2.60   0.472
# i 240 more rows
```

9.4 Single Plot

If you are not comparing the distribution of continuous data, you can create box plot for a single variable. Map the variable to `y` and add `geom_boxplot()`. In the below example, we examine the distribution of stock returns of Apple.

```
ggplot(daily_returns) +
  geom_boxplot(aes(y = AAPL))
```



Tip

Older code (including earlier editions of this book) used `aes(x = factor(1))` as a dummy grouping variable for single boxplots. That workaround is no longer needed but still renders, so you may encounter it in the wild.

9.5 Compare Distributions

9.5.1 Data

For the rest of the chapter, we will use the below data set. Instead of 5 columns, we have two columns. One for the stock names and another for returns.

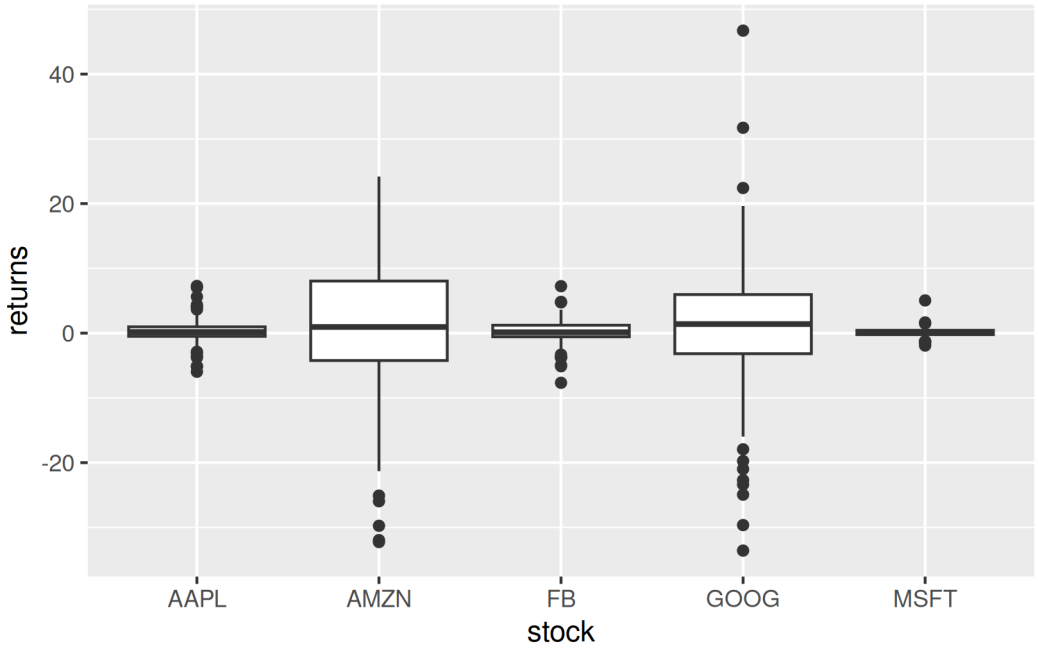
```
tidy_returns <-
  read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master/tidy_tickers
  col_types = list(col_factor(levels = c('AAPL', 'AMZN', 'FB', 'GOOG', 'MSFT')), col_doubl
tidy_returns
```

```
# A tibble: 1,254 x 2
   stock returns
   <fct>   <dbl>
 1 AAPL   1.38
 2 AAPL   2.83
```

```
 3 AAPL  -0.0394
 4 AAPL   0.108
 5 AAPL   1.64
 6 AAPL   0.0689
 7 AAPL  -0.561
 8 AAPL   0.551
 9 AAPL  -0.217
10 AAPL  -0.108
# i 1,244 more rows
```

With the above data, let us create a box plot where we compate the distribution of stock returns of different companies. We map X axis to the column with stock names and Y axis to the column with stock returns. Note that, the column names are wrapped inside `aes()`.

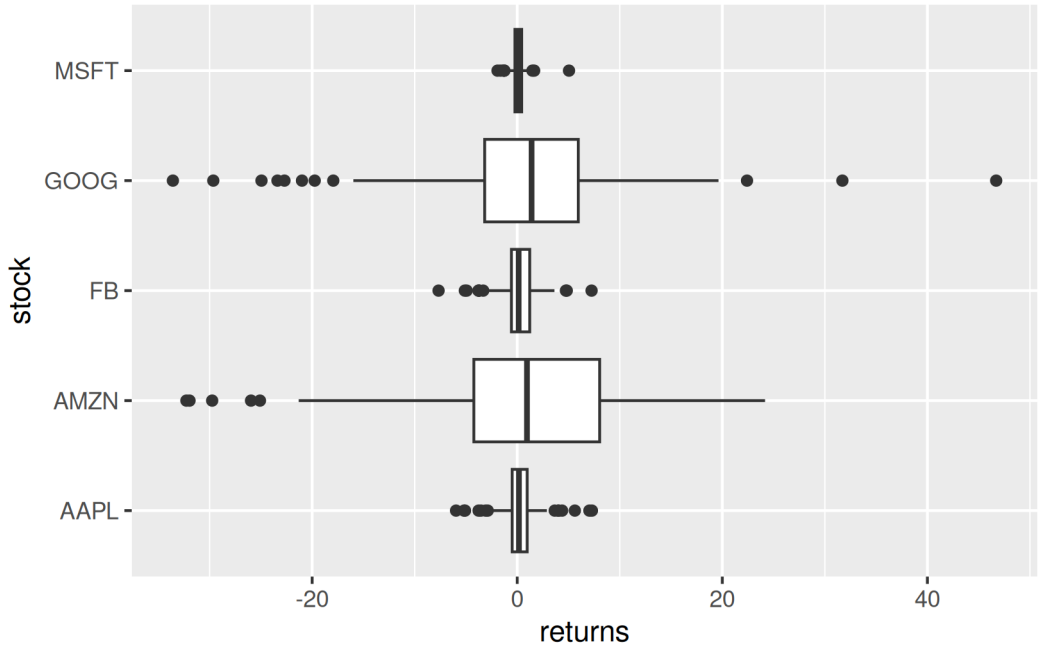
```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns))
```



To create a horizontal bar plot, we can use `coord_flip()` which will flip the coordinate axes.

9.6 Horizontal Box Plot

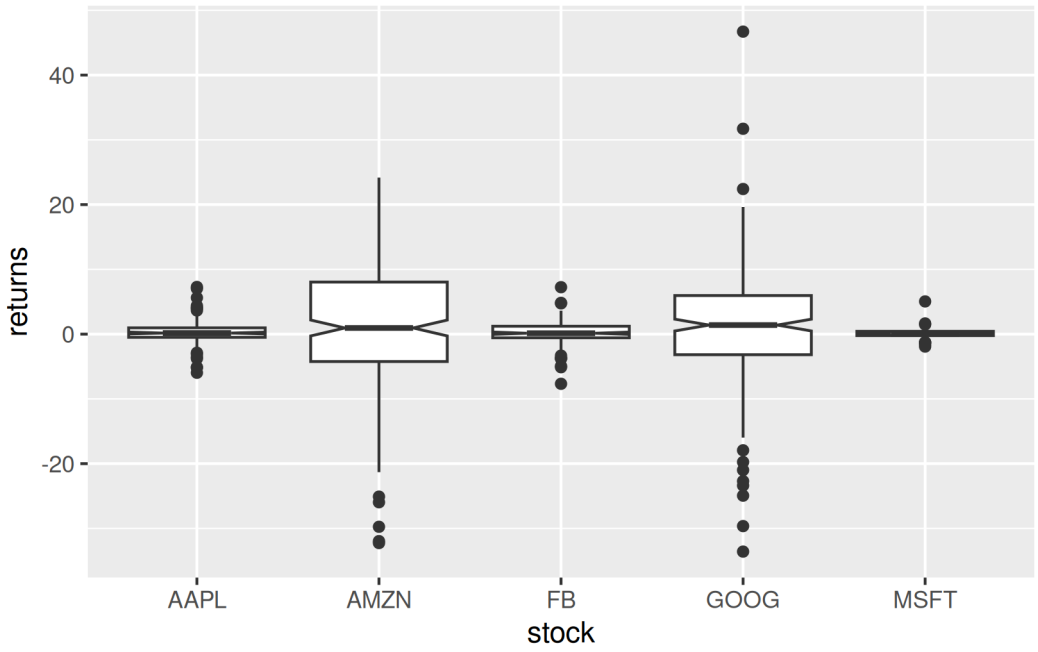
```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns)) +
  coord_flip()
```



9.6.1 Notch

Notches are used to compare medians. You can use the `notch` argument and set it to `TRUE`.

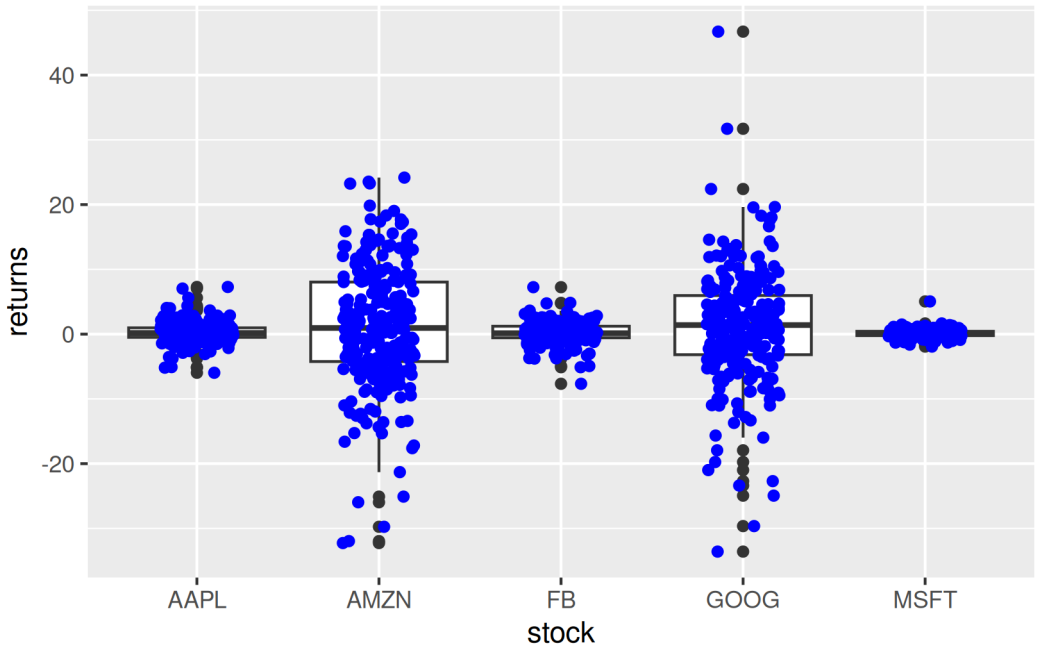
```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns),
    notch = TRUE)
```



9.6.2 Jitter

Just for comparison, let us plot the returns as points on top of the box plot using `geom_jitter()`. We modify the color of the points using the `color` argument and the spread using the `width` argument.

```
ggplot(tidy_returns, aes(x = stock, y = returns)) +
  geom_boxplot() +
  geom_jitter(width = 0.2, color = 'blue')
```



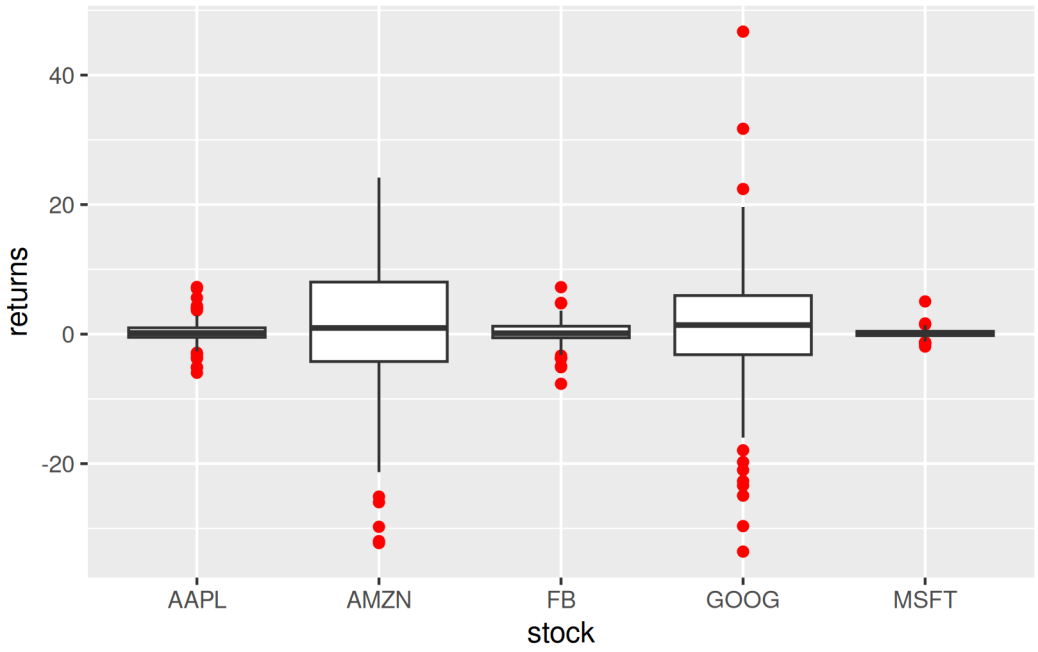
9.7 Outliers

To highlight extreme observations, we can modify the appearance of outliers using the following:

- color
- shape
- size
- alpha

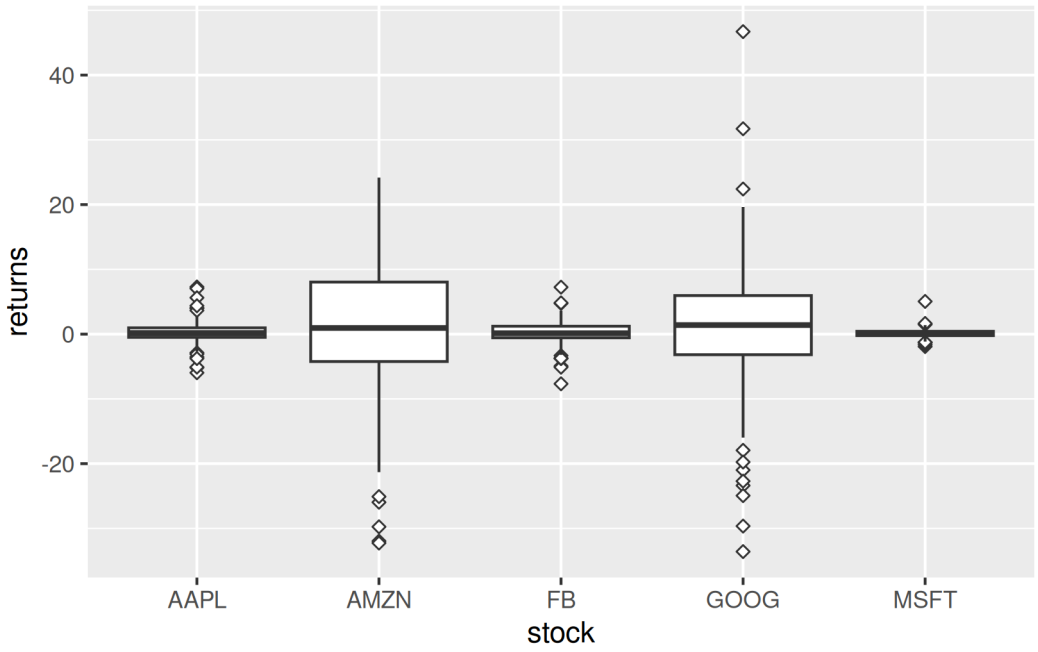
To modify the color of the outliers, use the `outlier.color` argument. The color can be specified either using its name or the associated hex code.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), outlier.color = 'red')
```



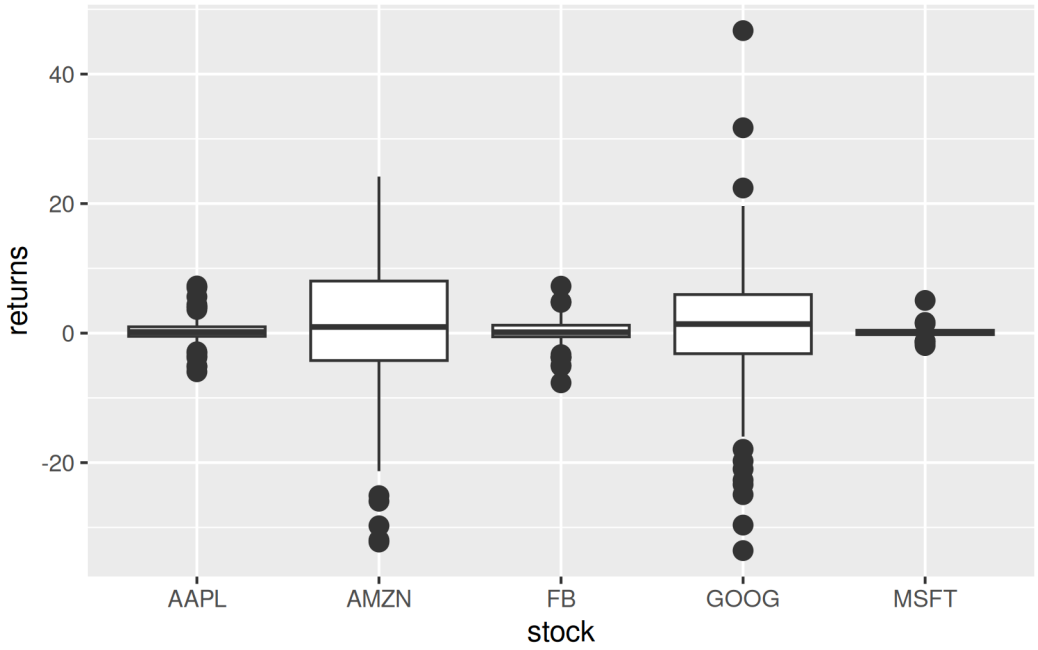
The shape of the outlier can be modified using the `outlier.shape` argument. It can take values between `0` and `25`.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), outlier.shape = 23)
```



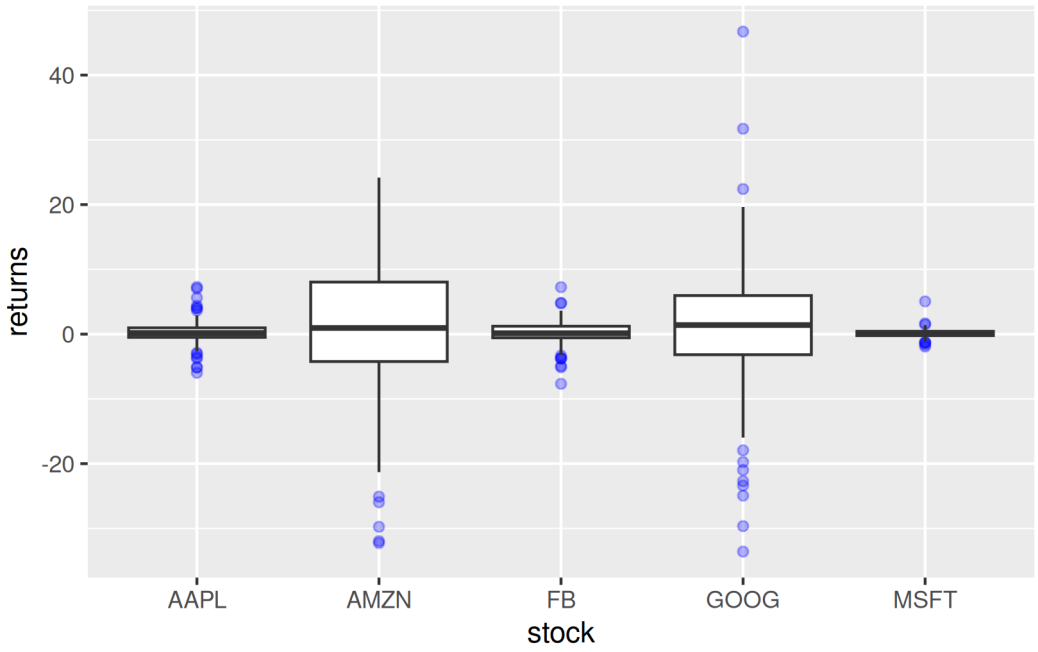
The size of the outlier can be modified using the `outlier.size` argument. It can take any value greater than `0`.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), outlier.size = 3)
```



You can play around with the transparency of the outlier using the `outlier.alpha` argument. It can take values between `0` and `1`.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), outlier.color = 'blue', outlier.alpha = 0.3)
```



9.8 Aesthetics

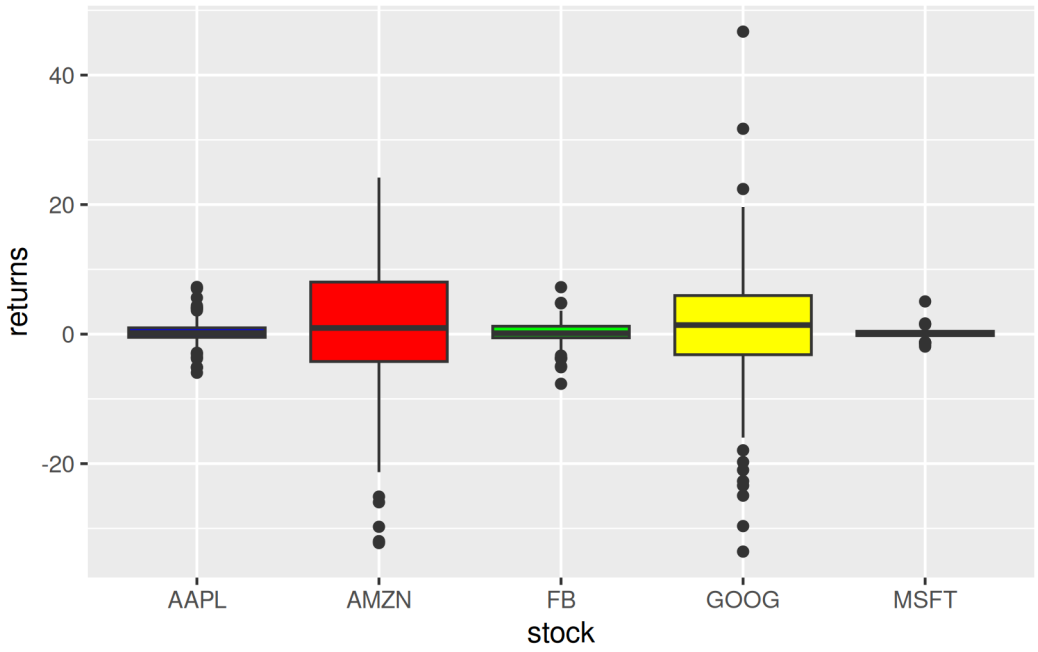
The appearance of the box can be controlled using the following:

- color
- fill
- alpha
- line type
- line width

9.8.1 Specify Values

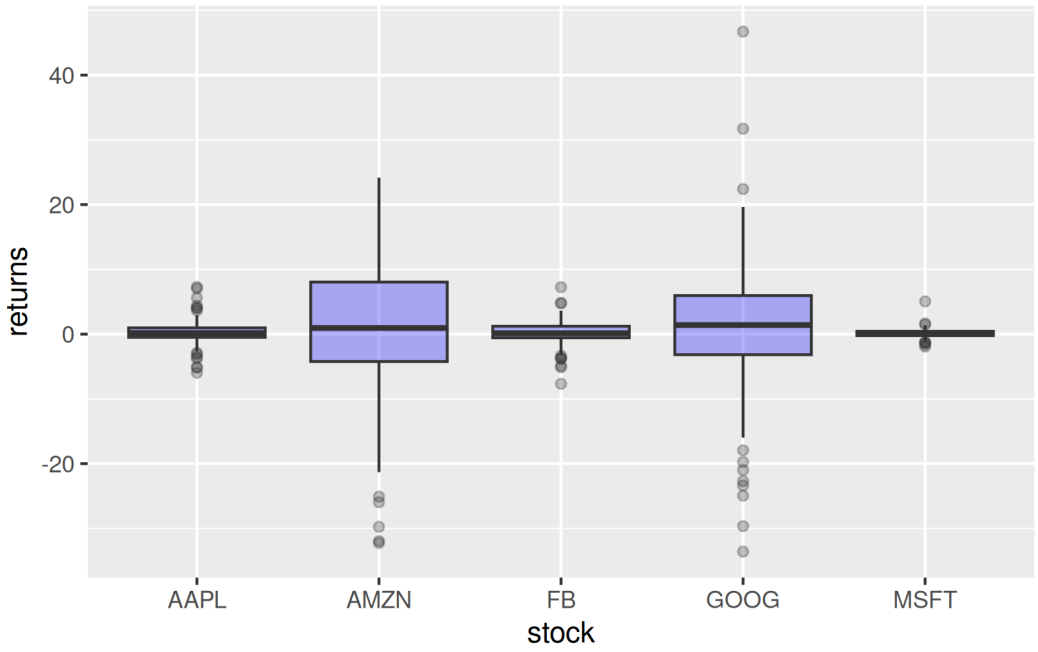
The background color of the box can be modified using the `fill` argument. The color can be specified either using its name or the associated hex code.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), fill = c('blue', 'red', 'green', 'yellow', 'br
```



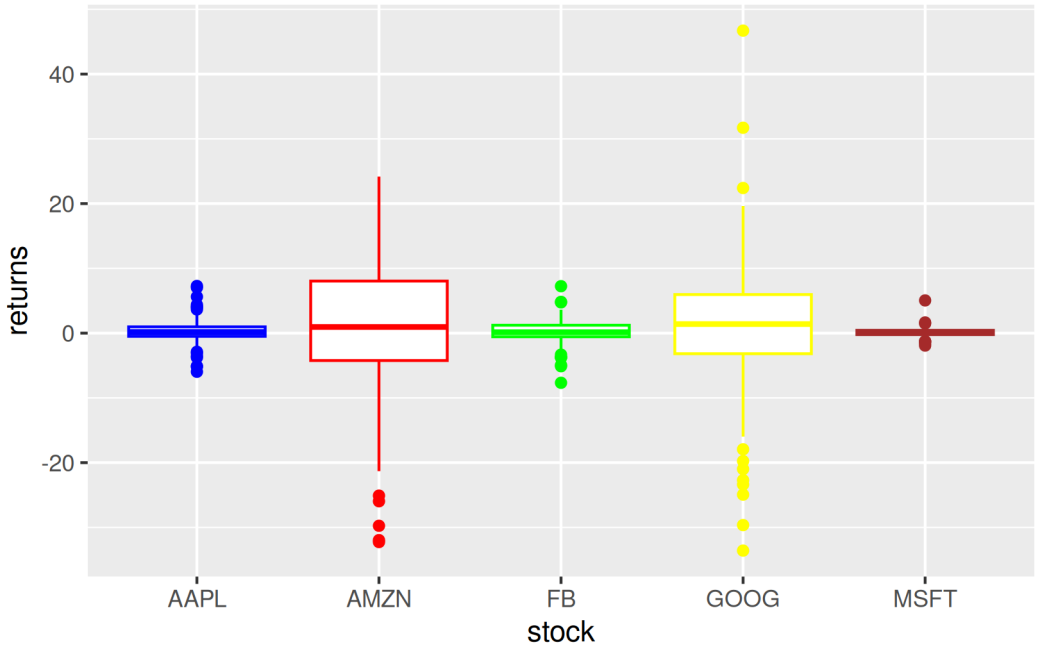
To modify the transparency of the background color, use the `alpha` argument. It can take any value between `0` and `1`.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), fill = 'blue', alpha = 0.3)
```



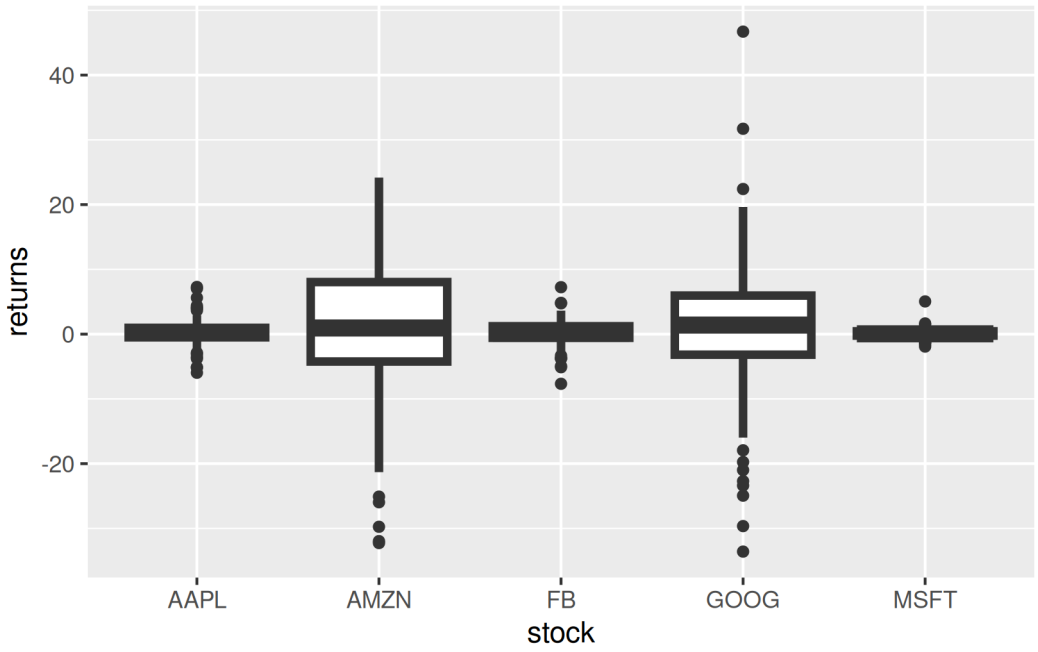
The color of the border can be modified using the `color` argument. The color can be specified either using its name or the associated hex code.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), color = c('blue', 'red', 'green', 'yellow', 'b
```



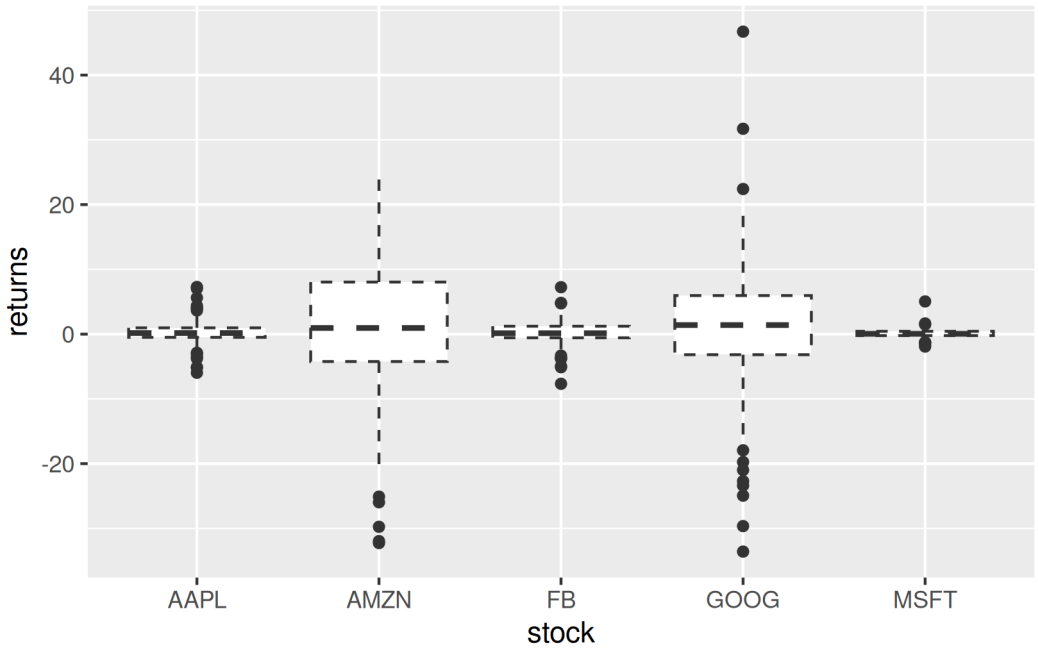
The width of the border can be changed using the `linewidth` argument. It can take any value greater than `0`.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), linewidth = 1.5)
```



To change the line type of the border, use the `linetype` argument. It can take any value between `0` and `6`.

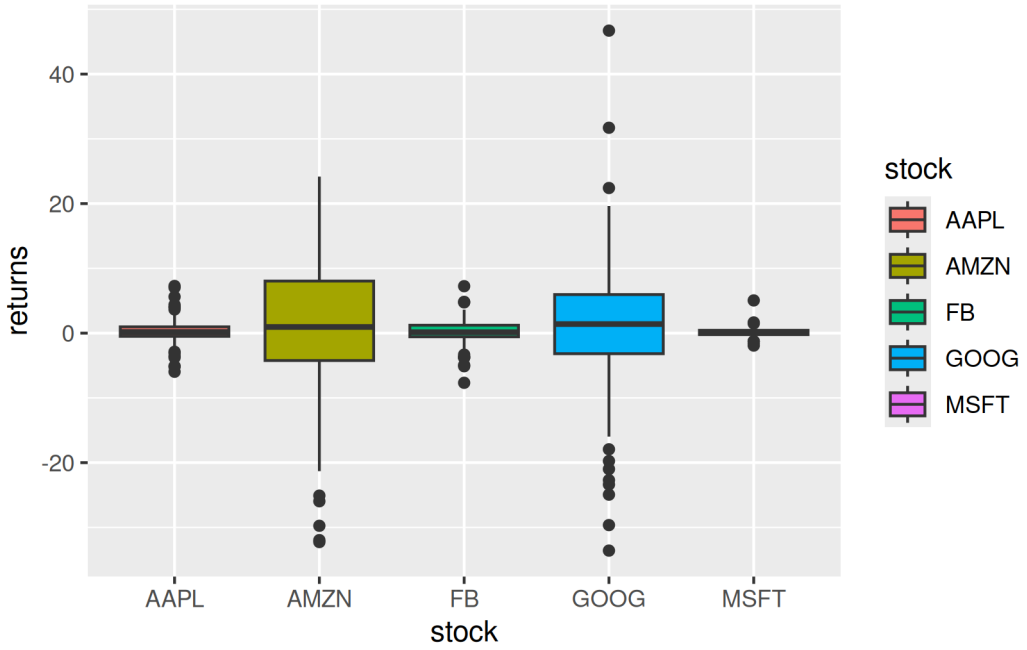
```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns), linetype = 2)
```



9.9 Map Variables

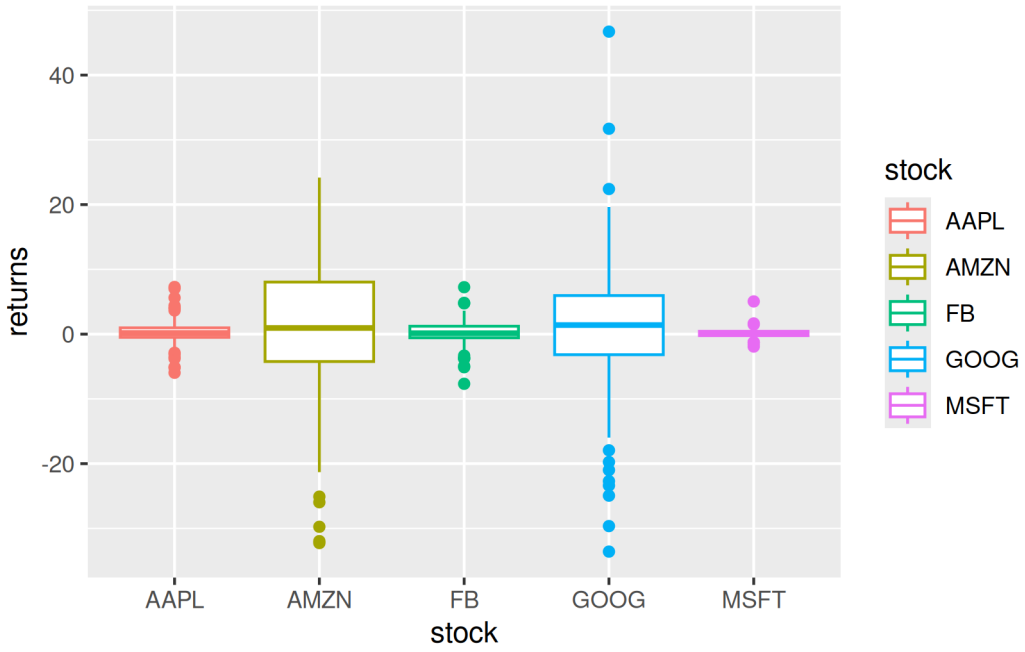
Instead of specifying values, we can map `fill` and `color` to variables as well. In the below example, we map `fill` to the variable stock. It assigns different colors to the different stocks.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns, fill = stock))
```



Let us map `color` to the variable stock. It will assign different colors to the box borders.

```
ggplot(tidy_returns) +
  geom_boxplot(aes(x = stock, y = returns, color = stock))
```



9.10 Exercises

1. Draw mileage by cylinders from `mtcars`.
2. Draw a single-variable boxplot of `mpg` using modern syntax (no `factor(1)`).
3. Rewrite the grouped plot with a message-first title and no redundant legend.

Worked solutions in `solutions/box.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

10 Histograms

10.1 Introduction

> **Why this chart**
>
> Histograms show the shape of one continuous variable: center, spread, skew and gaps.

> **Common pitfall**
>
> Bin width drives the story; try several widths before concluding a distribution is bimodal.

In this chapter, we will learn to

- build histogram
- specify bins
- modify
  - color
  - fill
  - alpha
  - bin width
  - line type
  - line size
- map aesthetics to variables

A histogram is a plot that can be used to examine the shape and spread of continuous data. It looks very similar to a bar graph and can be used to detect outliers and skewness in data. The histogram graphically shows the following:

- center (location) of the data
- spread (dispersion) of the data
- skewness
- outliers
- presence of multiple modes

To construct a histogram, the data is split into intervals called bins. The intervals may or may not be equal sized. For each bin, the number of data points that fall into it are counted (frequency). The Y axis of the histogram represents the frequency and the X axis represents the variable.

10.2 Data

```
ecom <- readr::read_csv('https://raw.githubusercontent.com/rsquaredacademy/datasets/master
ecom
```

```
# A tibble: 1,000 x 11
      id referrer device bouncers n_visit n_pages duration country      purchase
   <dbl> <chr>    <chr>  <lgl>      <dbl>   <dbl>    <dbl> <chr>        <lgl>
 1     1 google   laptop TRUE          10       1      693 Czech Repub~ FALSE
 2     2 yahoo    tablet TRUE           9       1      459 Yemen        FALSE
 3     3 direct   laptop TRUE           0       1      996 Brazil       FALSE
 4     4 bing     tablet FALSE          3      18      468 China        TRUE
 5     5 yahoo    mobile TRUE           9       1      955 Poland       FALSE
 6     6 yahoo    laptop FALSE          5       5      135 South Africa FALSE
 7     7 yahoo    mobile TRUE          10       1       75 Bangladesh   FALSE
 8     8 direct   mobile TRUE          10       1      908 Indonesia    FALSE
 9     9 bing     mobile FALSE          3      19      209 Netherlands  FALSE
10    10 google   mobile TRUE           6       1      208 Czech Repub~ FALSE
# i 990 more rows
# i 2 more variables: order_items <dbl>, order_value <dbl>
```

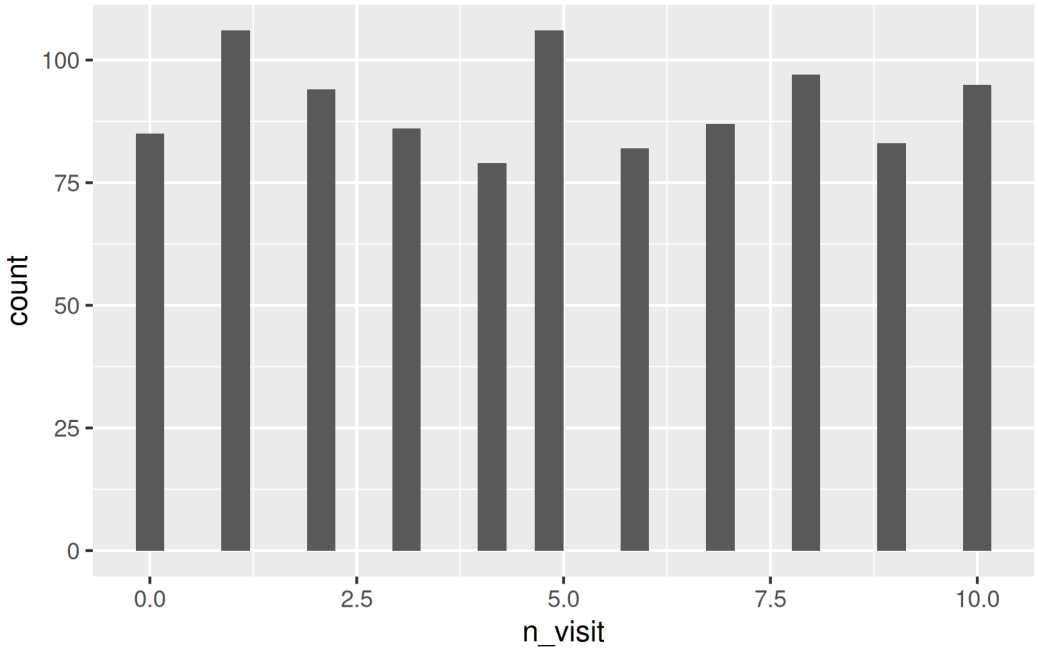
10.2.1 Data Dictionary

- id: row id
- referrer: referrer website/search engine
- device: device used to visit the website
- bouncers: whether the visit bounced (single page viewed)
- n_visit: number of visits
- n_pages: number of pages visited
- duration: time spent on the website (in seconds)
- country: country of origin
- purchase: whether visitor purchased
- order_items: number of items ordered
- order_value: order value of visitor (in dollars)

10.3 Plot

To create a histogram, we will use `geom_histogram()` and specify the variable name within `aes()`. In the below example, we create histogram of the variable `n_visit`.

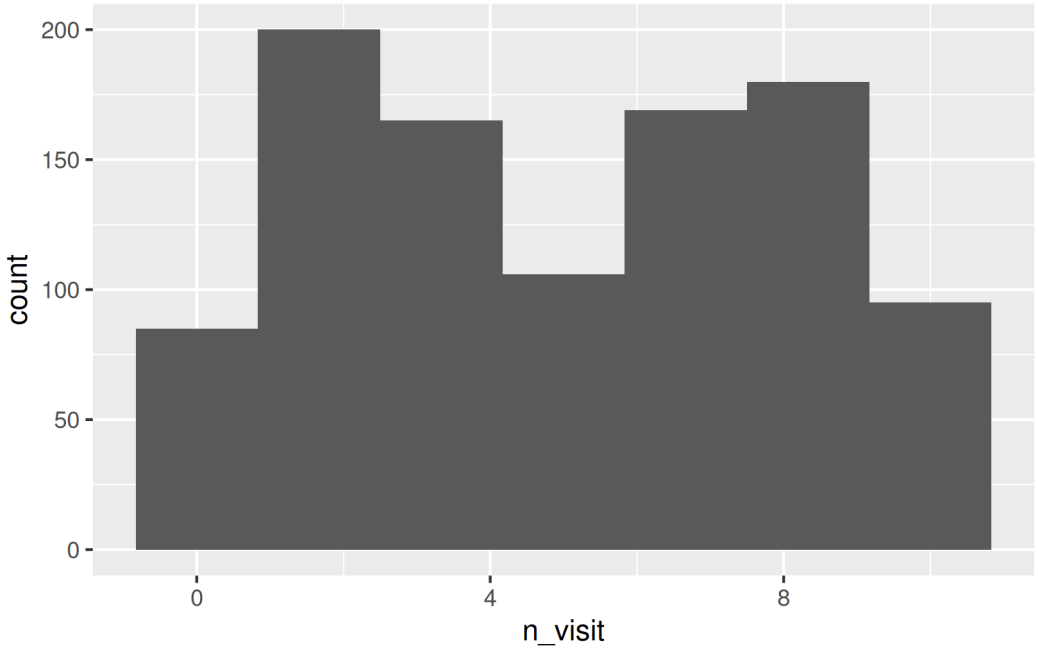
```
ggplot(ecom) +
  geom_histogram(aes(n_visit))
```



10.3.1 Specify Bins

The default number of bins in ggplot2 is 30. You can modify the number of bins using the `bins` argument. In the below example, we create a histogram with 7 bins.

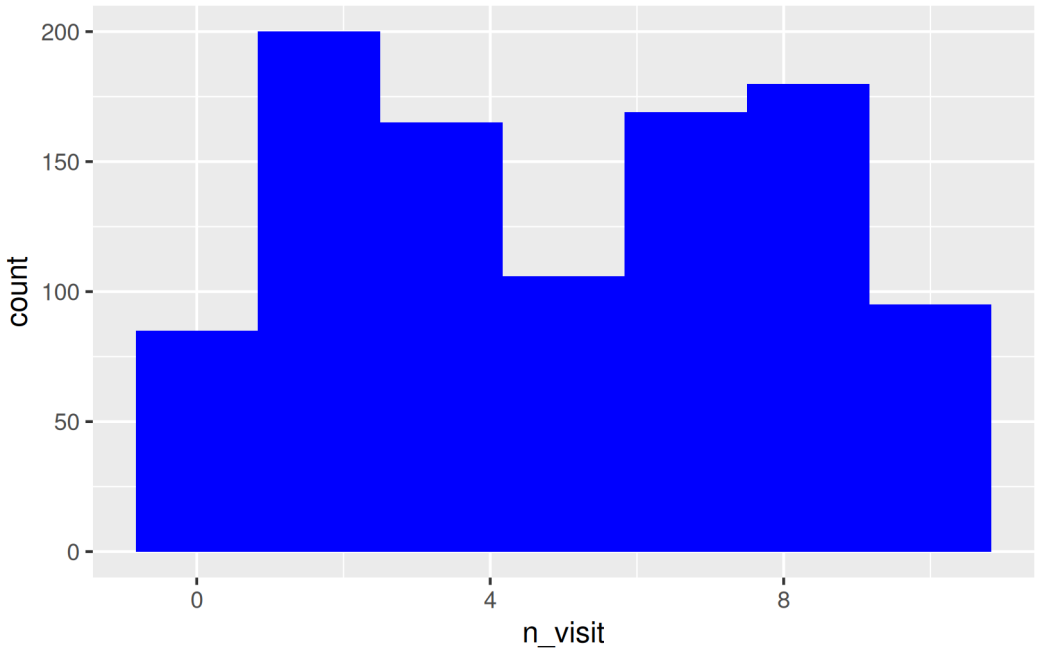
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 7)
```



10.4 Aesthetics

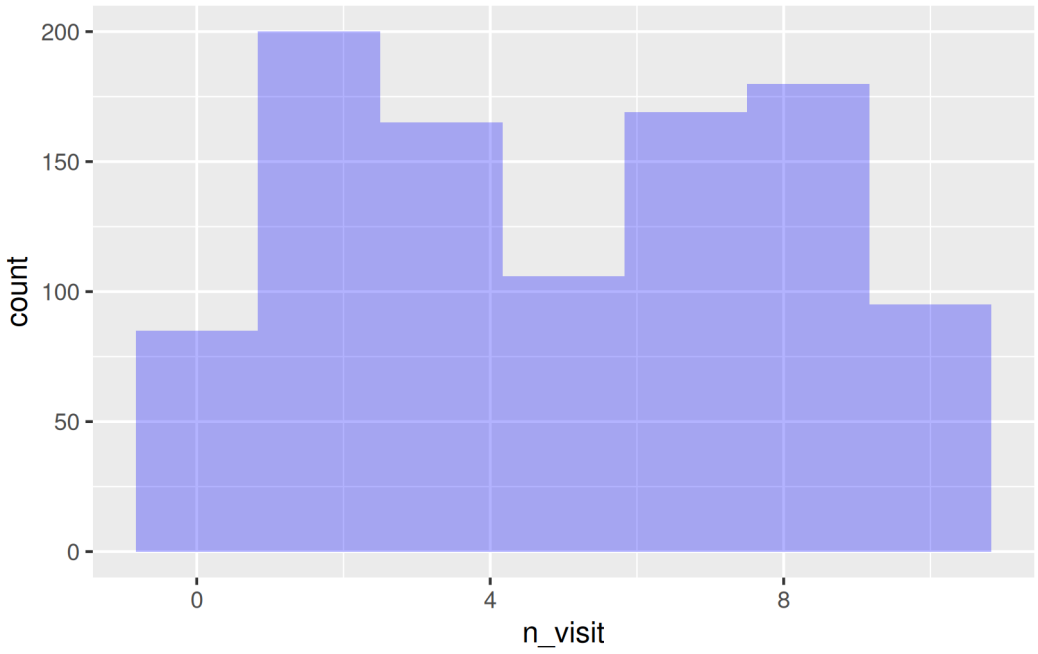
Now that we know how to create a histogram, let us learn to modify its appearance. We will begin with the background color. Use the `fill` argument to modify the background color of the histogram. In the below case, we change the color of the histogram to 'blue'.

```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 7, fill = 'blue')
```



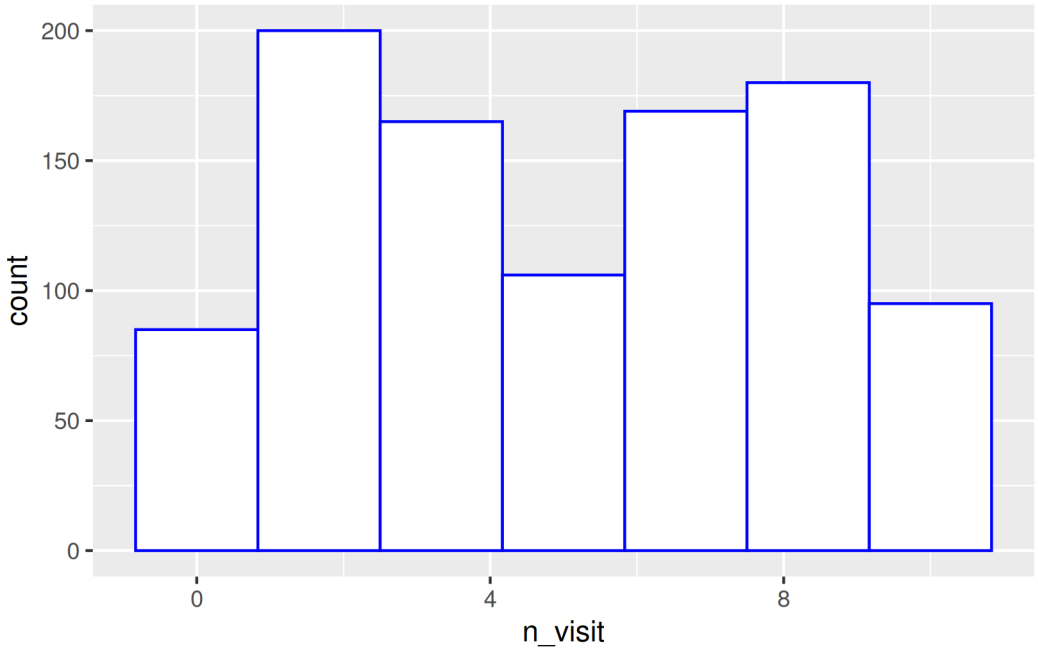
As we have learnt before, the transparency of the background color can be modified using the `alpha` argument. It can take any value between `0` and `1`.

```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 7, fill = 'blue', alpha = 0.3)
```



The color of the histogram border can be modified using the `color` argument. The color can be specified either using its name or the associated hex code.

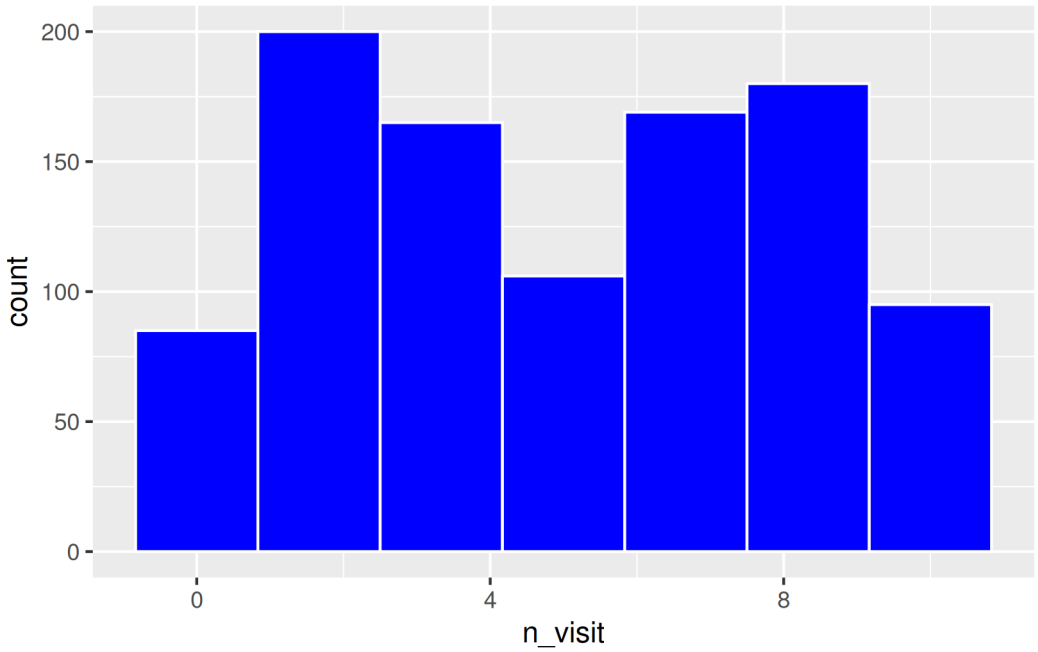
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 7, fill = 'white', color = 'blue')
```



10.5 Putting it all together

Let us modify the bins, the background and border color of the histogram in the below example.

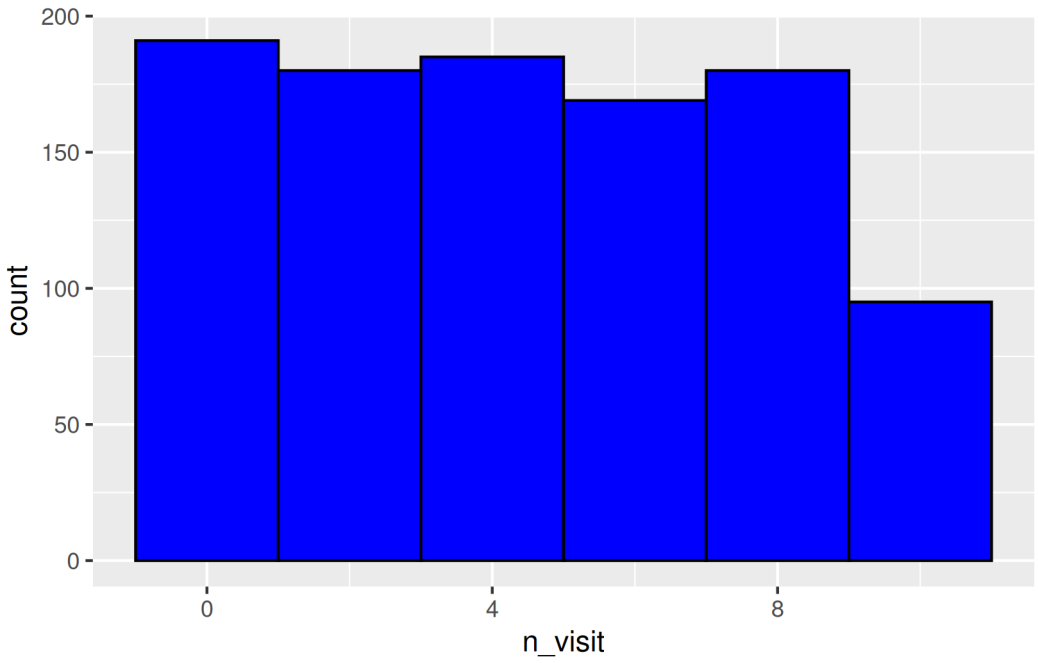
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 7, fill = 'blue', color = 'white')
```



10.6 Bin Width

Another way to control the number of bins in a histogram is by using the `binwidth` argument. In this case, we specify the width of the bins instead of the number of bins. As you can see, in the below example, we do not use the `bins` argument when using the `binwidth` argument. You can use either of them but not both.

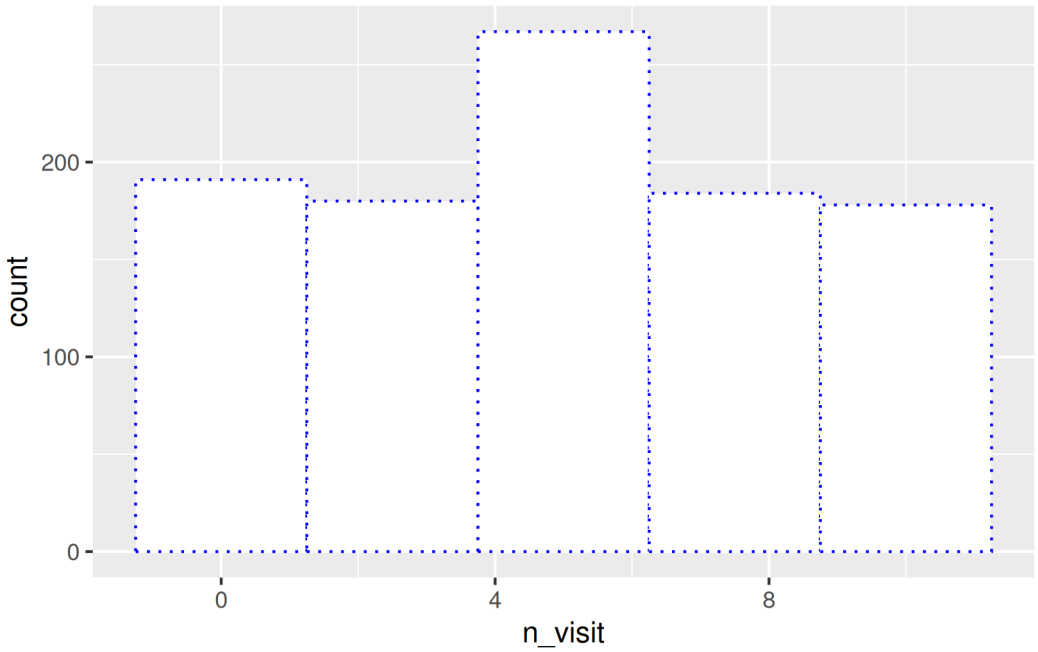
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), binwidth = 2, fill = 'blue', color = 'black')
```



10.7 Line Type

The line type of the histogram border can be modified using the `linetype` argument. It can take any integer value between `0` and `6`.

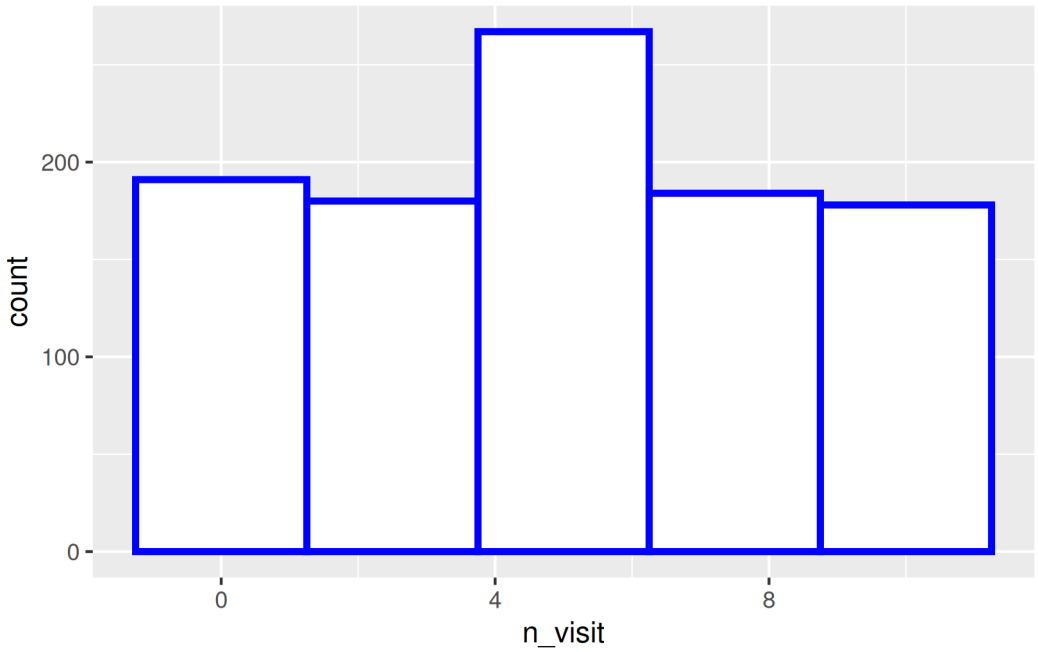
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 5, fill = 'white',
    color = 'blue', linetype = 3)
```



10.8 Line Size

Use the `linewidth` argument to modify the width of the border of the histogram bins. It can take any value greater than `0`.

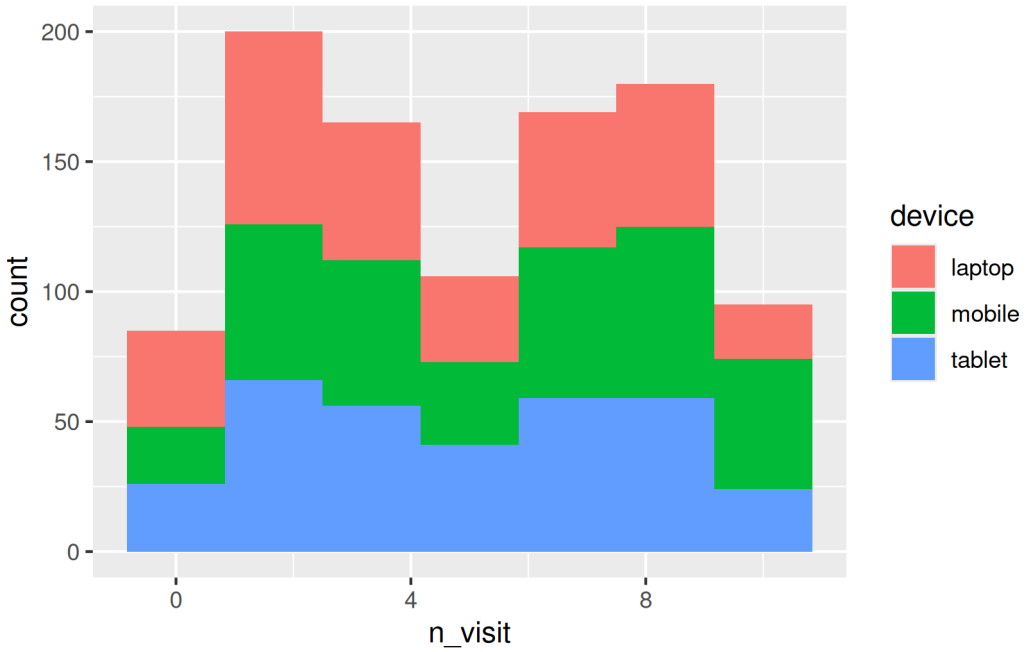
```
ggplot(ecom) +
  geom_histogram(aes(n_visit), bins = 5, fill = 'white',
    color = 'blue', linewidth = 1.25)
```



10.9 Map Variables

You can map the aesthetics to variables as well. In the below example, we map `fill` to the device variable. You can try mapping color, linetype and linewidth to variables as well.

```
ggplot(ecom) +
  geom_histogram(aes(n_visit, fill = device), bins = 7)
```



10.10 Exercises

1. Draw the `mpg` distribution from `mtcars` with 10 bins.
2. Map density instead of counts using `after_stat()`.
3. Rewrite the plot with a message-first title and labeled axes.

Worked solutions in `solutions/histogram.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

11 Modify Axis

In this chapter, we will learn how to modify the X and Y axis using the

> **Why this chart**
>
> Axis scales, breaks and labels decide what readers can read off the chart.

> **Common pitfall**
>
> Fighting default breaks with manual limits can clip data; prefer coord_cartesian() to zoom without dropping.

following functions:

- Continuous Axis
  - `scale_x_continuous()`
  - `scale_y_continuous()`
- Discrete Axis
  - `scale_x_discrete()`
  - `scale_y_discrete()`

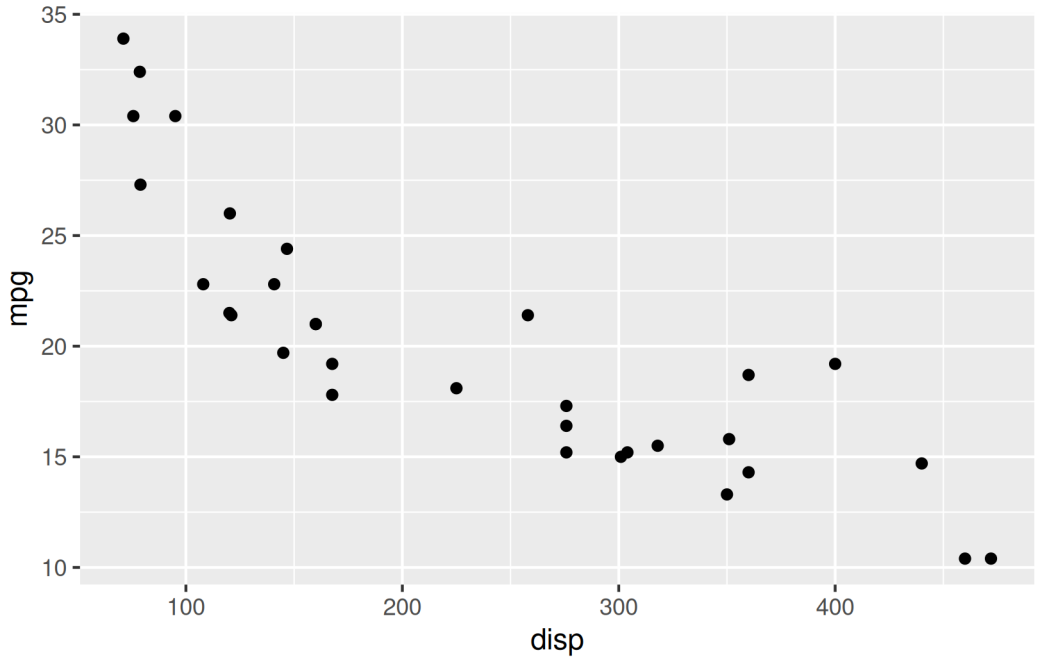
11.1 How to modify a continuous axis in ggplot2

If the X and Y axis represent continuous data, we can use `scale_x_continuous()` and `scale_y_continuous()` to modify the axis. They take the following arguments:

- name
- limits
- breaks
- labels
- position

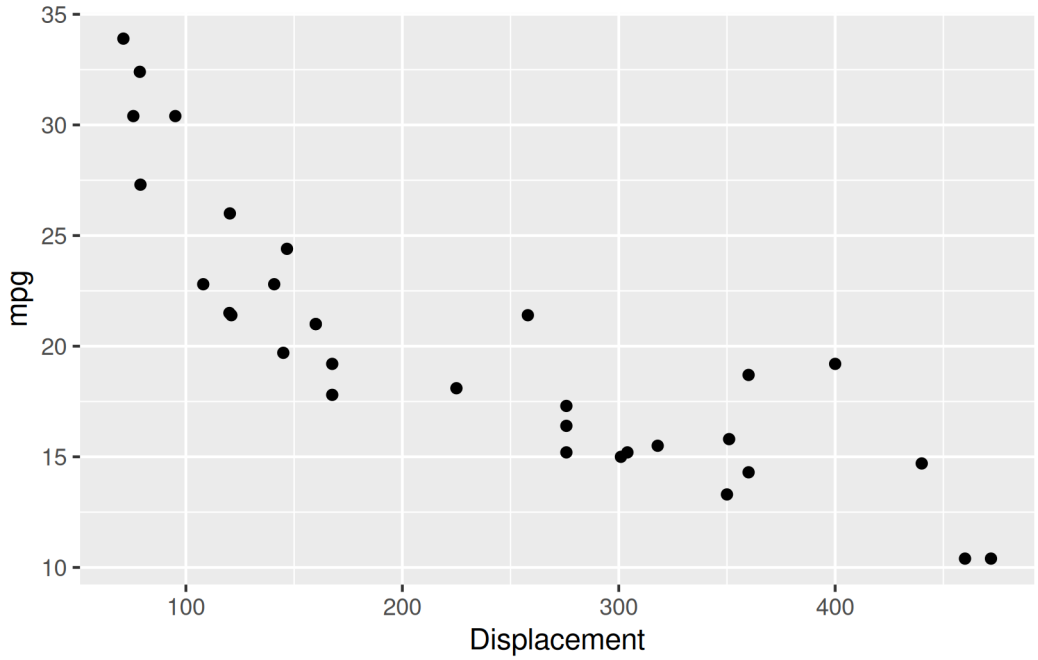
Let us continue with the scatter plot we have used in previous chapter.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg))
```



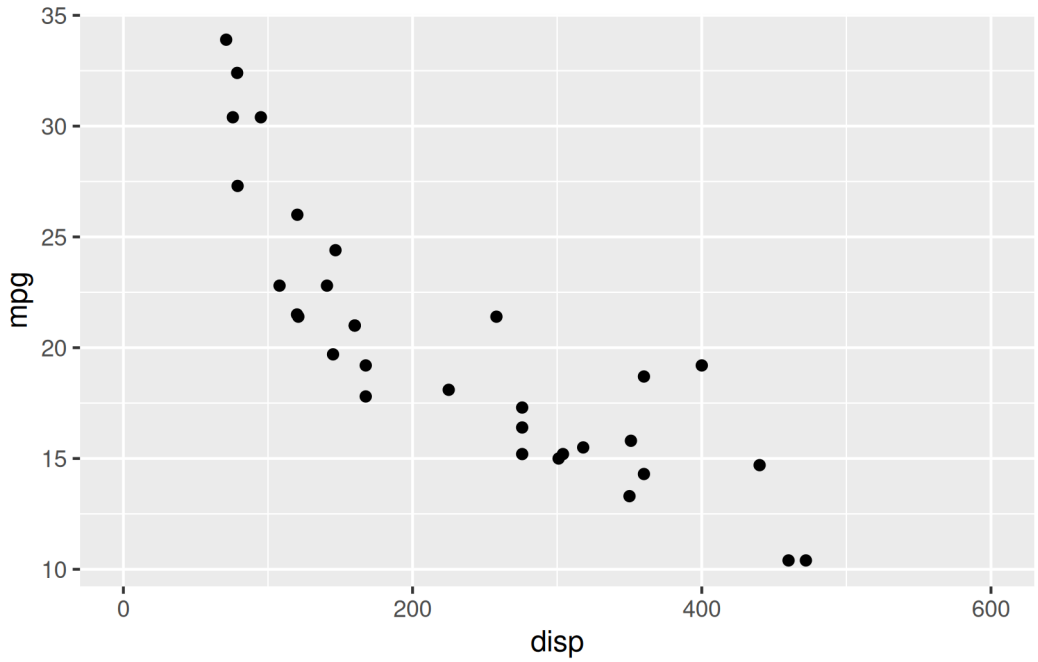
The `name` argument is used to modify the X axis label. In the below example, we change the X axis label to `'Displacement'`. In previous chapters, we have used `xlab()` to work with the X axis label.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  scale_x_continuous(name = "Displacement")
```



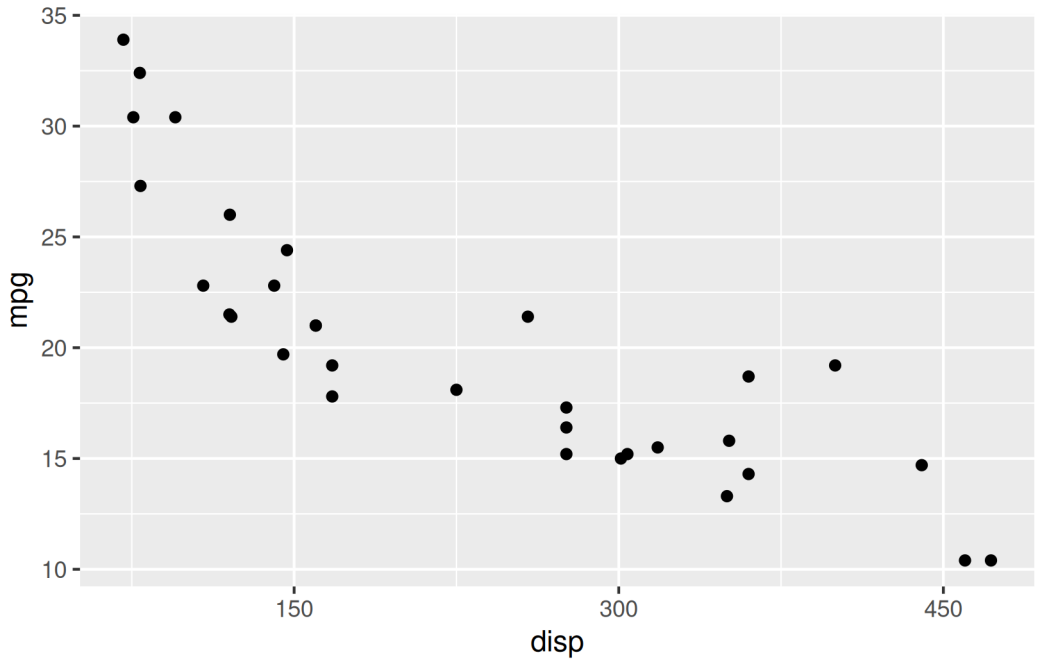
To modify the range, use the `limits` argument. It takes a vector of length 2 i.e. 2 values, the lower and upper limit of the range. It is an alternative for `xlim()`.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  scale_x_continuous(limits = c(0, 600))
```



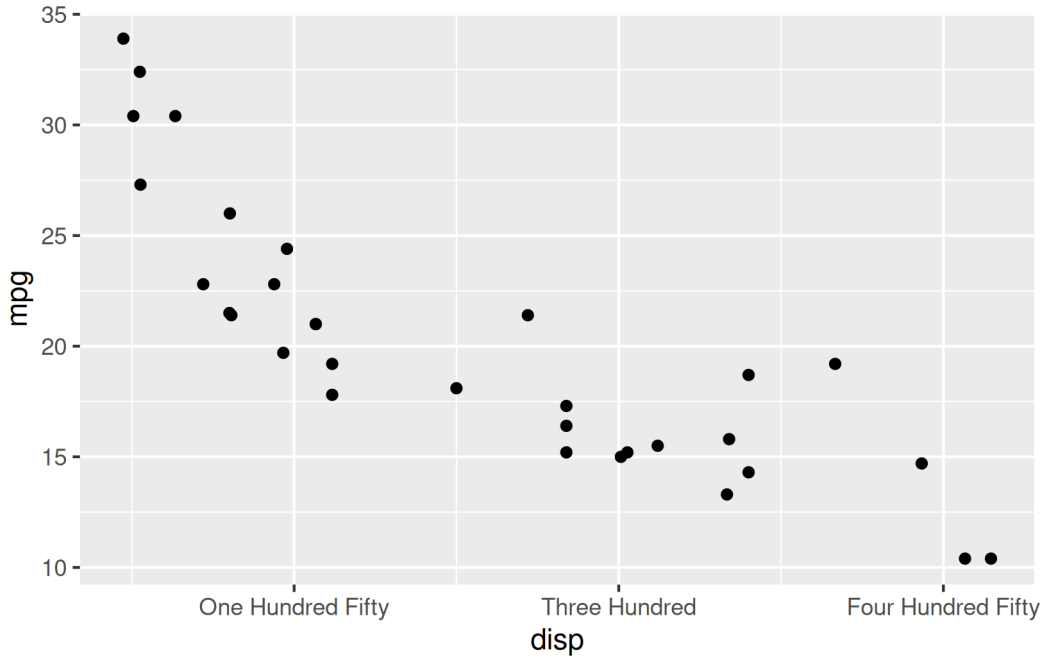
In the above plot, the ticks on the X axis appear at `0`, `200`, `400` and `600`. Let us say we want the ticks to appear more closer i.e. the difference between the tick should be reduced by `50`. The `breaks` argument will allow us to specify where the ticks appear. It takes a numeric vector equal to the length of the number of ticks.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  scale_x_continuous(breaks = c(150, 300, 450))
```



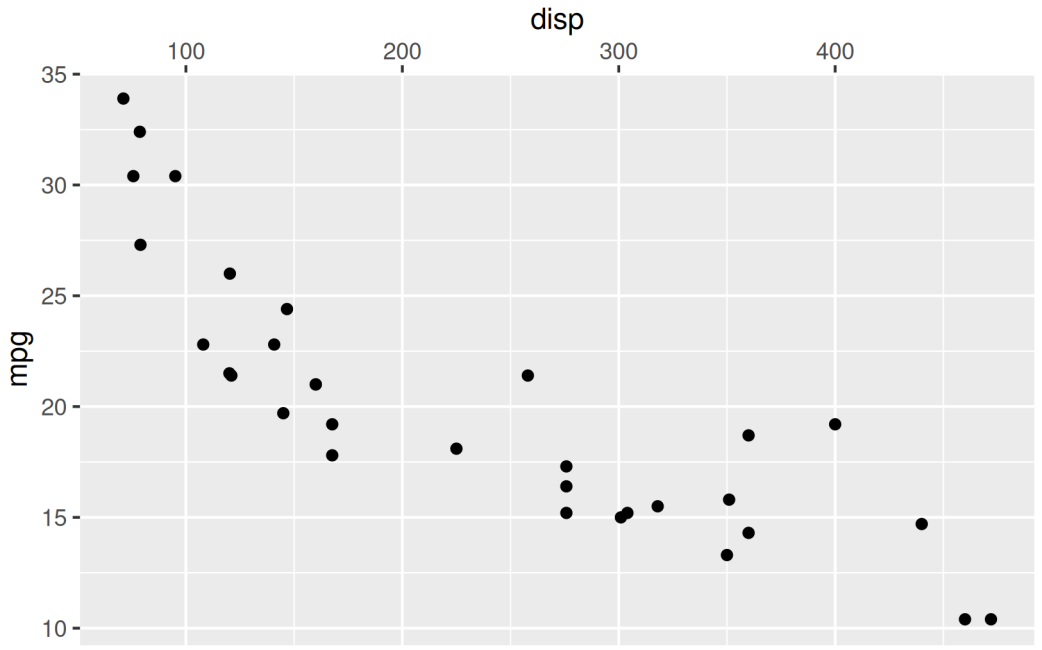
We can change the tick labels using the `labels` argument. In the below example, we use words instead of numbers. When adding labels, we need to ensure that the length of the `breaks` and `labels` are same.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  scale_x_continuous(breaks = c(150, 300, 450),
    labels = c('One Hundred Fifty', 'Three Hundred', 'Four Hundred Fifty'))
```



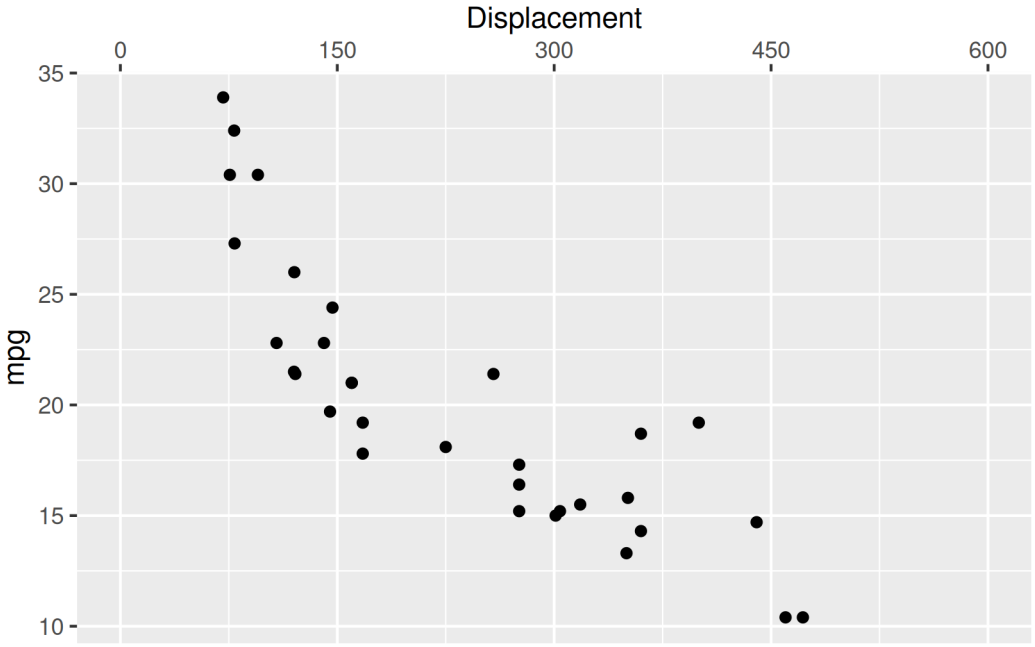
The position of the axes can be changed using the `position` argument. In the below example, we can move the axes to the top of the plot by supplying the value `'top'`.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  scale_x_continuous(position = 'top')
```



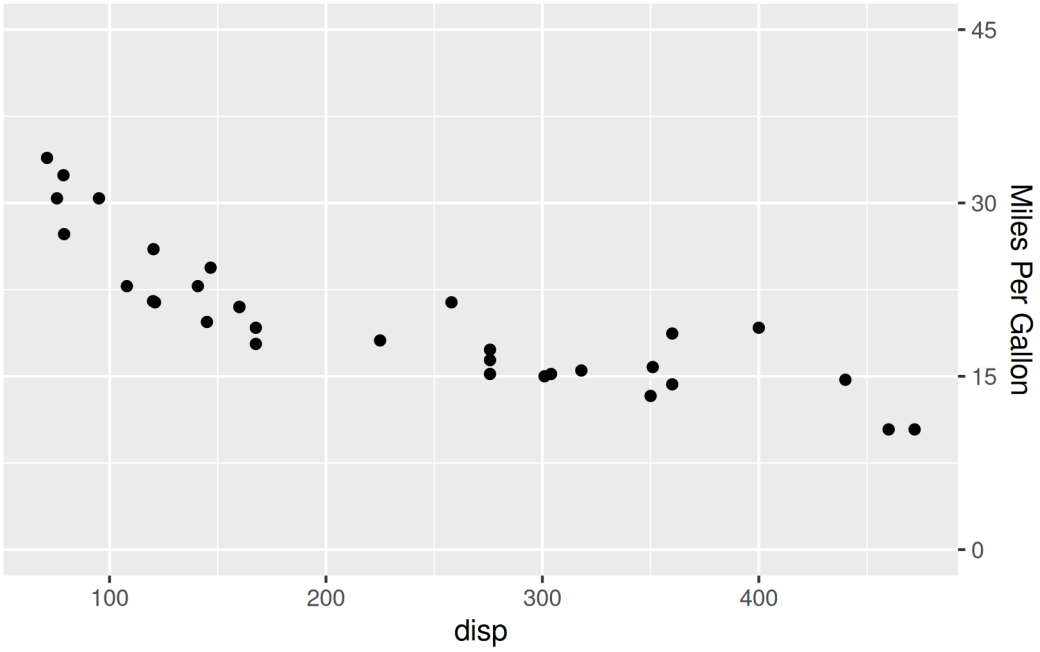
11.1.1 How to combine continuous axis options in ggplot2

```
ggplot(mtcars) + geom_point(aes(disp, mpg)) +
  scale_x_continuous(name = "Displacement", limits = c(0, 600),
                     breaks = c(0, 150, 300, 450, 600), position = 'top',
                     labels = c('0', '150', '300', '450', '600'))
```



11.1.2 How to modify the y-axis in ggplot2

```
ggplot(mtcars) + geom_point(aes(disp, mpg)) +
  scale_y_continuous(name = "Miles Per Gallon", limits = c(0, 45),
                     breaks = c(0, 15, 30, 45), position = 'right',
                     labels = c('0', '15', '30', '45'))
```



11.2 How to modify a discrete axis in ggplot2

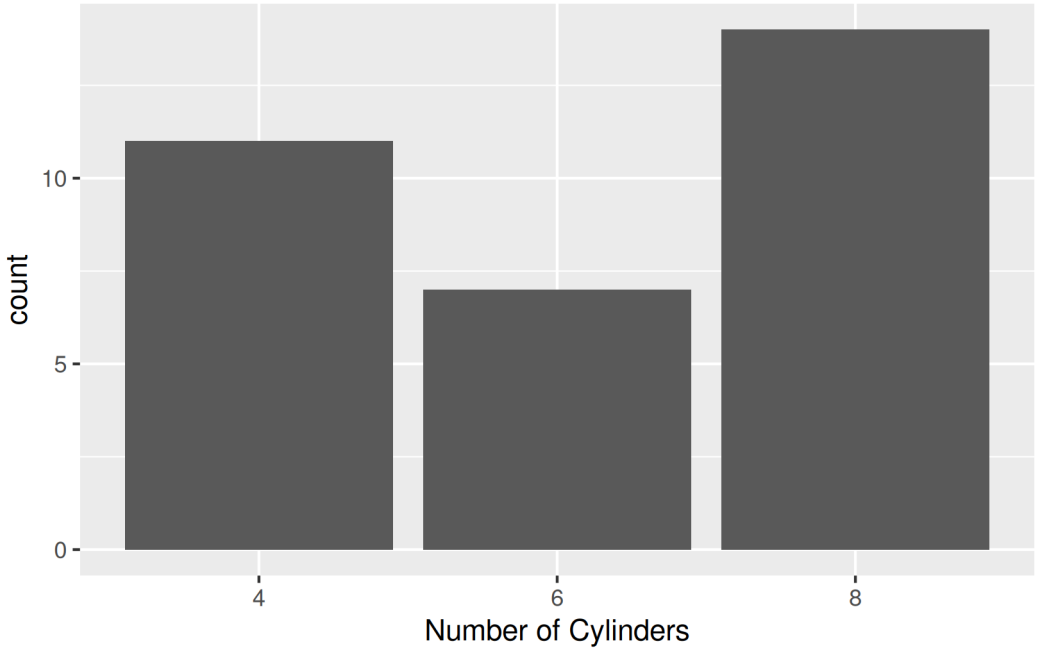
If the X and Y axis represent discrete or categorical data, `scale_x_discrete()` and `scale_y_discrete()` can be used to modify them. They take the following arguments:

- name
- labels
- breaks
- position

The above options serve the same purpose as in the case of continuous scales.

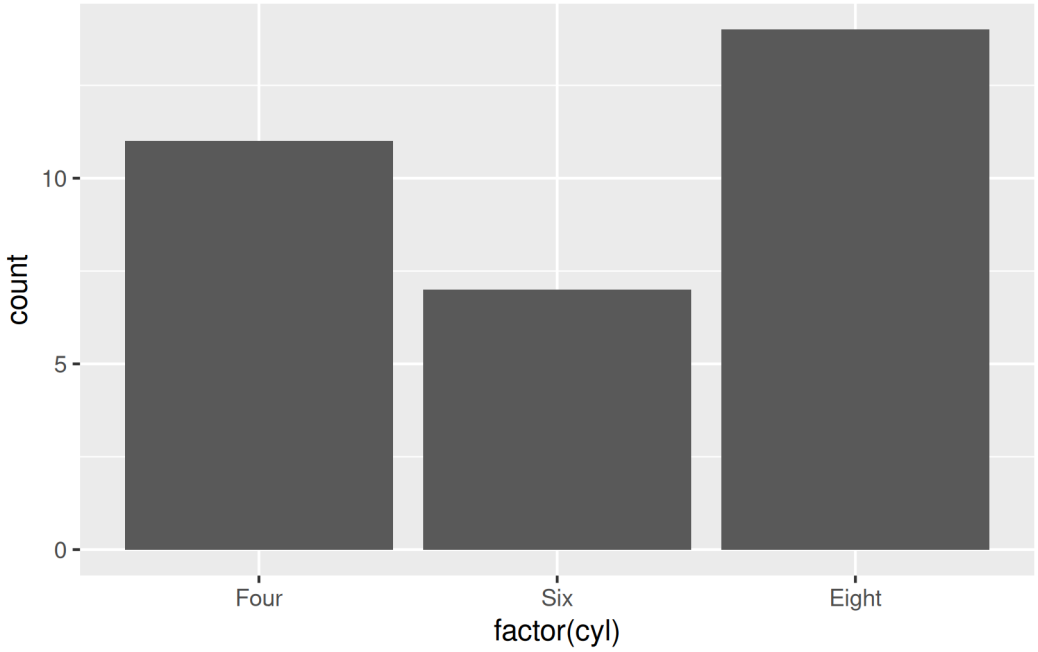
11.2.1 How to change axis labels in ggplot2

```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl))) +
  scale_x_discrete(name = "Number of Cylinders")
```



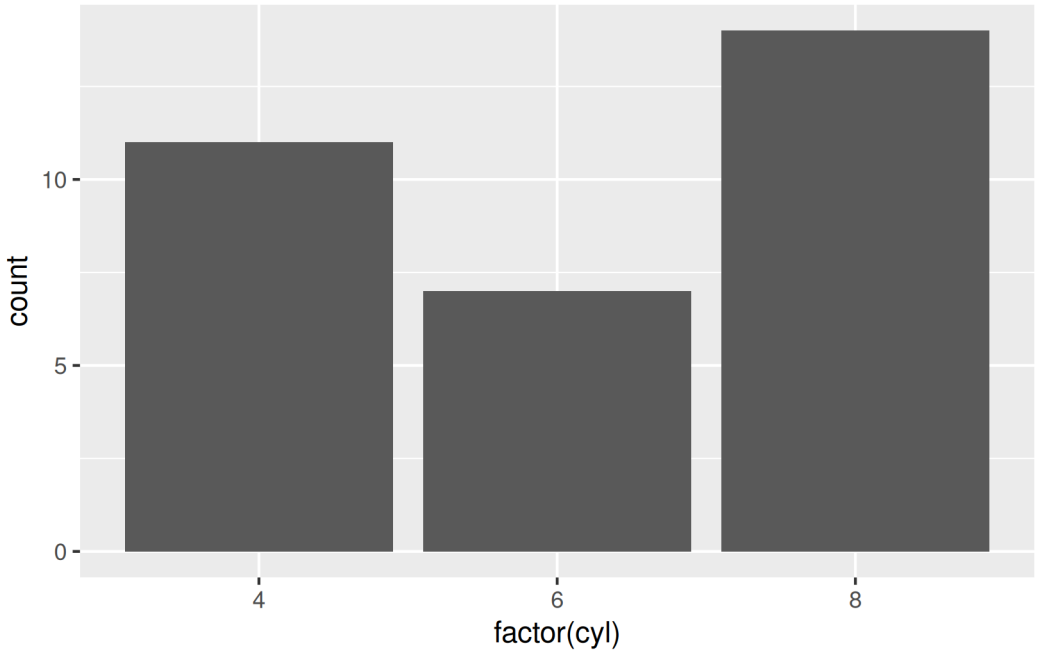
11.2.2 How to change axis tick labels in ggplot2

```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl))) +
  scale_x_discrete(labels = c("4" = "Four", "6" = "Six", "8" = "Eight"))
```



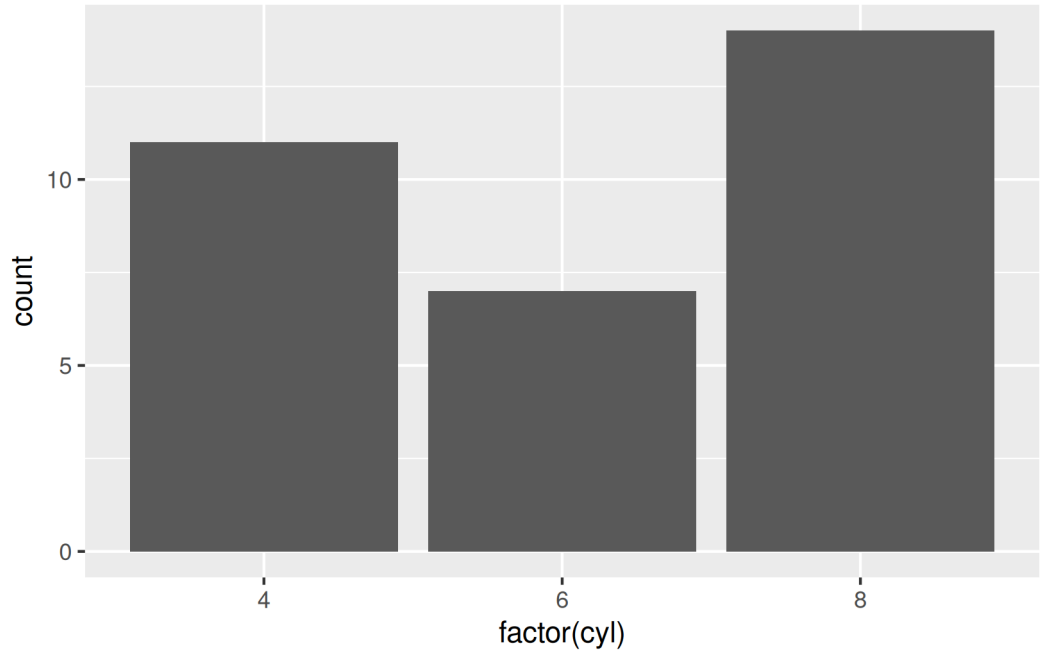
11.2.3 How to change axis breaks in ggplot2

```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl))) +
  scale_x_discrete(breaks = c("4", "6", "8"))
```



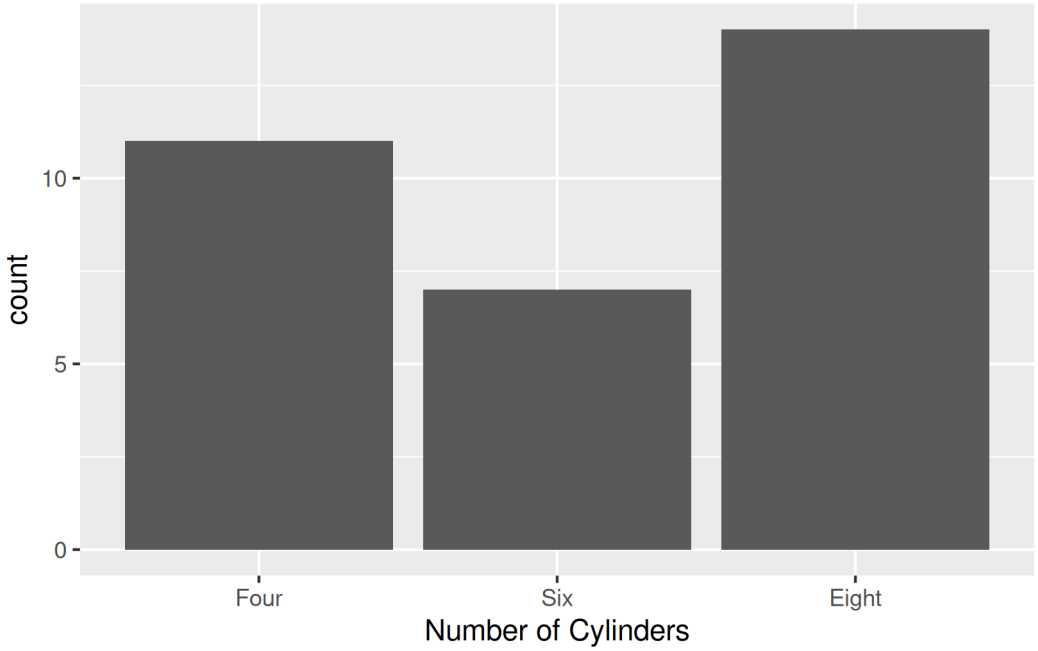
11.2.4 How to move the axis position in ggplot2

```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl))) +
  scale_x_discrete(position = 'bottom')
```



11.3 How to combine discrete axis options in ggplot2

```
ggplot(mtcars) + geom_bar(aes(factor(cyl))) +
  scale_x_discrete(name = "Number of Cylinders",
    labels = c("4" = "Four", "6" = "Six", "8" = "Eight"),
    breaks = c("4", "6", "8"), position = "bottom")
```



11.4 Exercises

1. Set custom breaks on the x axis with `scale_x_continuous()`.
2. Zoom to `mpg > 20` with `coord_cartesian()` without dropping data.
3. Explain when `coord_cartesian()` beats `limits` for zooming.

Worked solutions in `solutions/axis.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

12 Modify Legend

In this chapter, we customize legend guides: titles, labels and colorbars using the guide argument with guide_legend() and guide_colorbar().

> **Why this matters**
>
> Guides control title placement, label layout and colorbar sizing for publication-ready legends.

> **Common pitfall**
>
> Over-customizing direction, rows and justification in one guide hurts readability. Change one element at a time.

12.1 How to customize legends with guides in ggplot2

In this section, we will learn to modify

- title
- label
- and bar

So far, we have learnt to modify the components of a legend using `scale_*` family of functions. Now, we will use the `guide` argument and supply it values using the `guide_legend()` function.

How to change the guide title in ggplot2

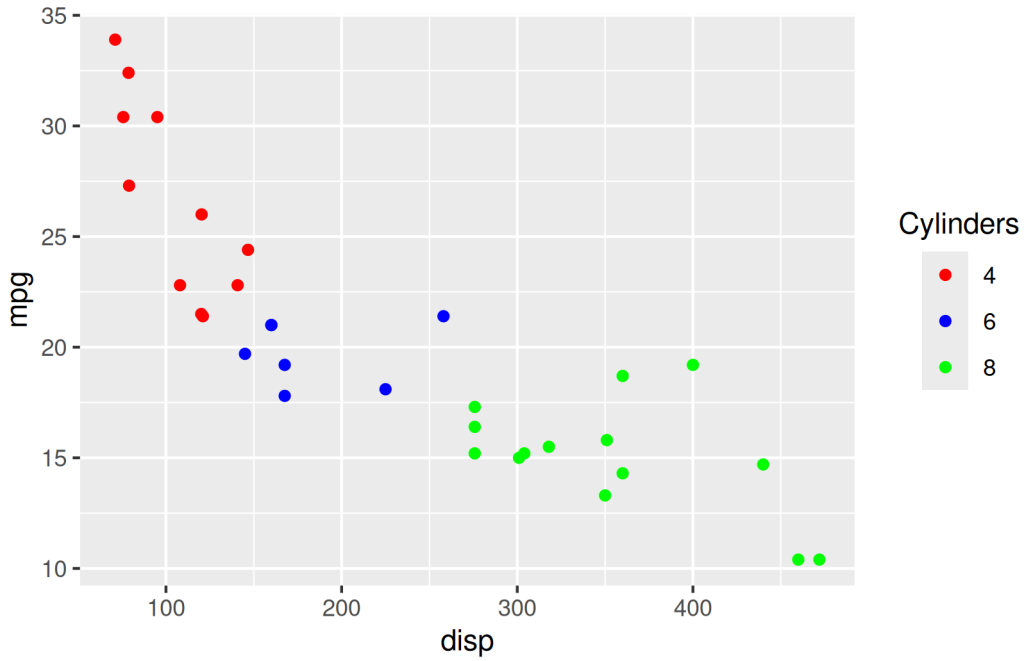
Title Alignment

The horizontal alignment of the title can be managed using the `title.hjust` argument. It can take any value between `0` and `1`.

- 0 (left)
- 1 (right)

In the below example, we align the title to the center by assigning the value `0.5`.

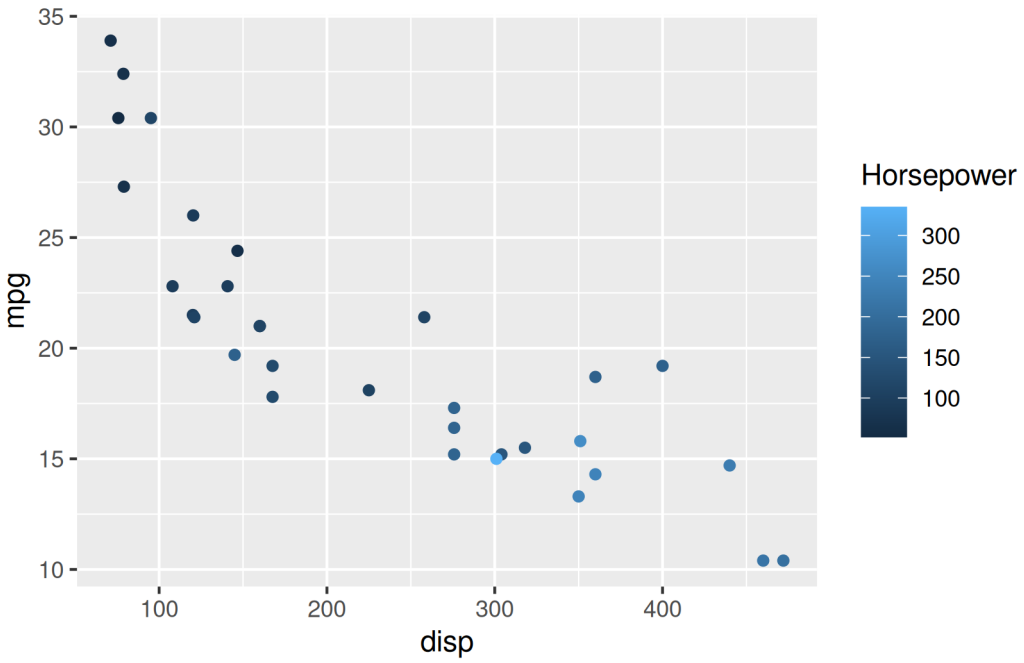
```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(title = "Cylinders", title.hjust = 0.5))
```



Title Alignment (Vertical)

To manage the vertical alignment of the title, use `title.vjust`.

```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    title = "Horsepower", title.position = "top", title.vjust = 1))
```

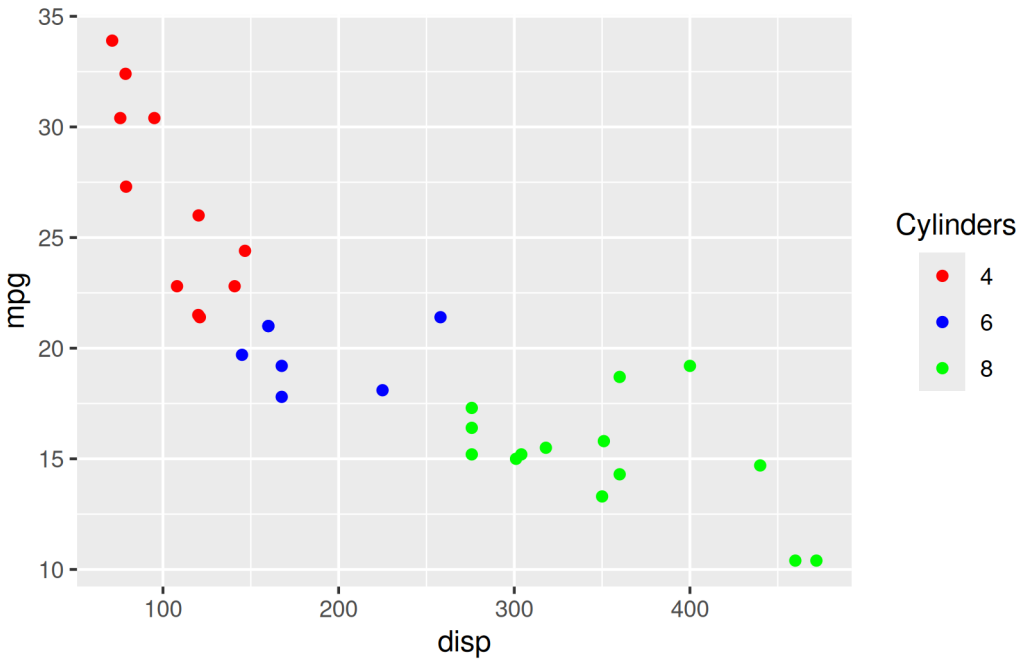


Title Position

The position of the title can be managed using `title.position` argument. It can be positioned at:

- top
- bottom
- left
- right

```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(title = "Cylinders", title.hjust = 0.5,
      title.position = "top"))
```



How to change guide labels in ggplot2

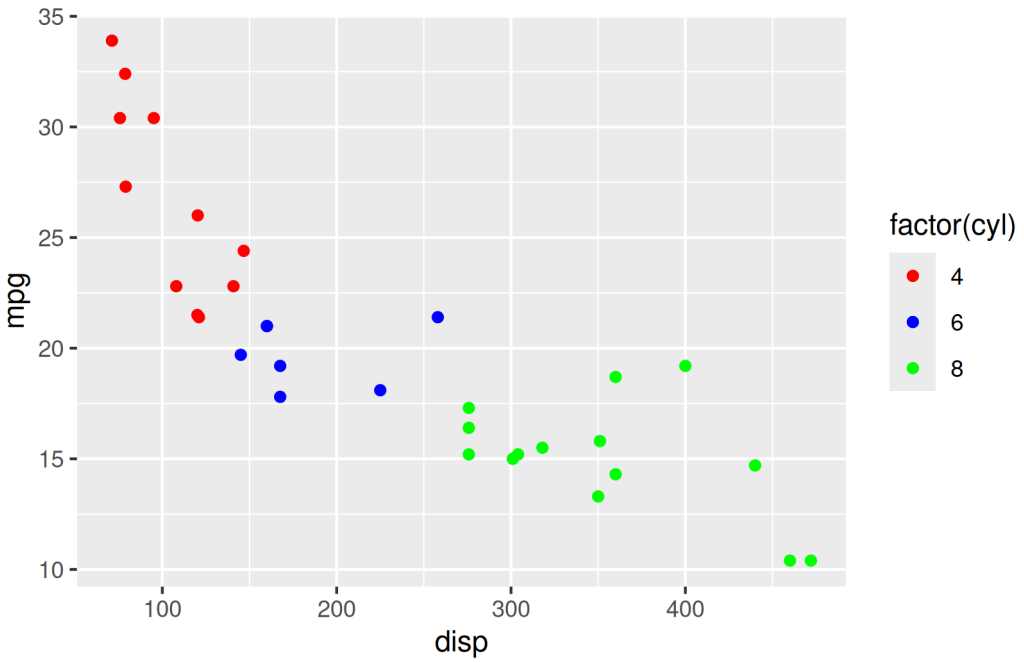
Label Position

The position of the label can be managed using the `label.position` argument. It can be positioned at:

- top
- bottom
- left
- right

In the below example, we position the label at right.

```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(label.position = "right"))
```



Label Alignment

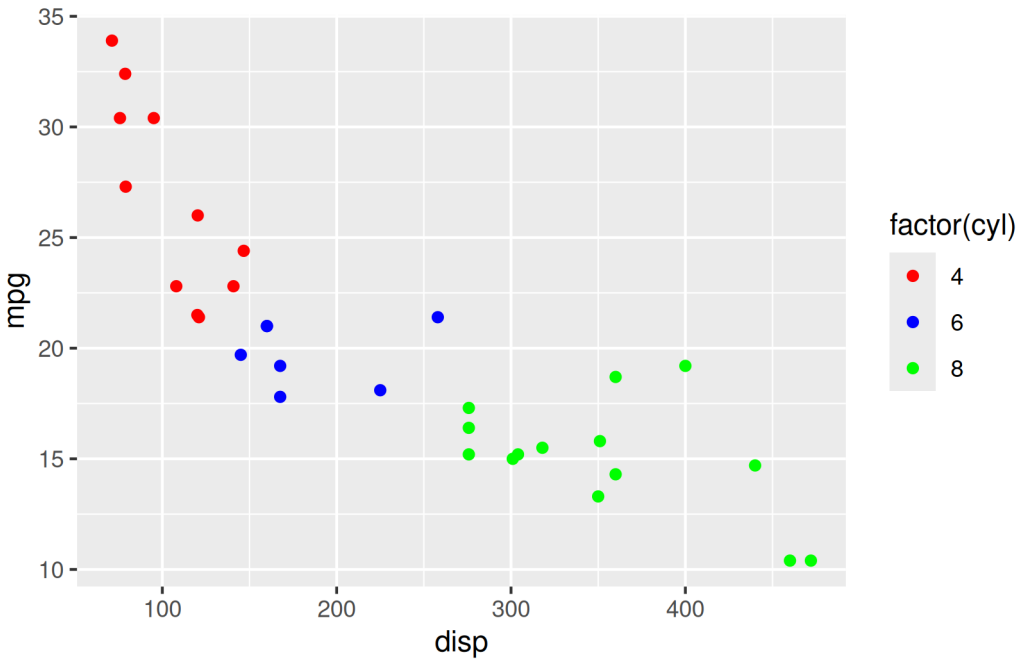
The horizontal alignment of the label can be managed using the `label.hjust` argument. It can take any value between `0` and `1`.

- 0 (left)
- 1 (right)

In the below example, we align the label to the center by assigning the value `0.5`.

- alignment
  - 0 (left)
  - 1 (right)

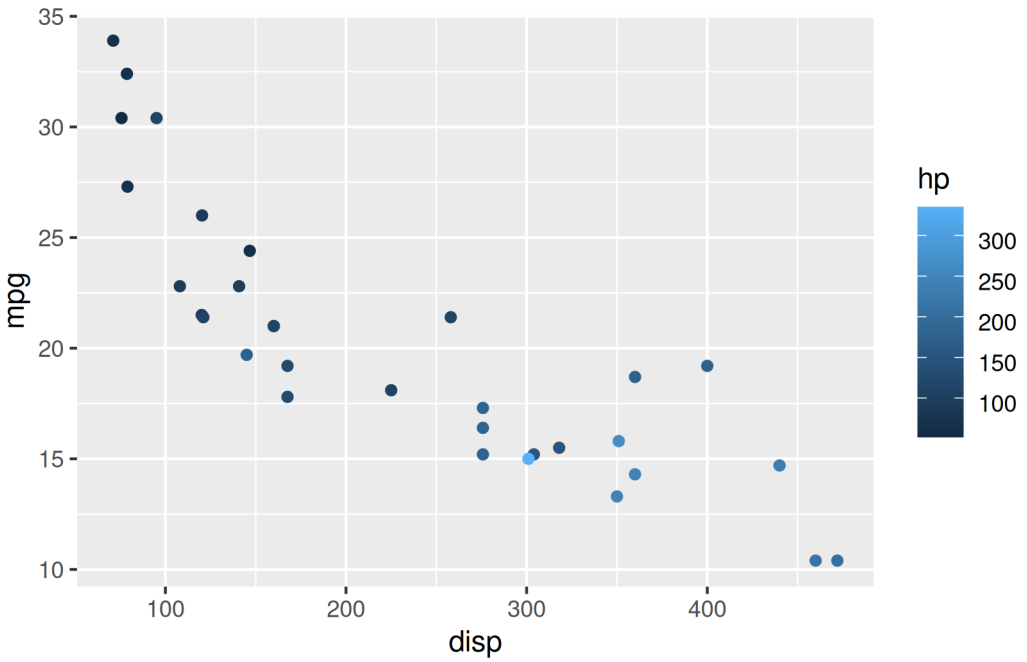
```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(label.hjust = 0.5))
```



Labels Alignment (Vertical)

The vertical alignment of the label can be managed using the `label.vjust` argument.

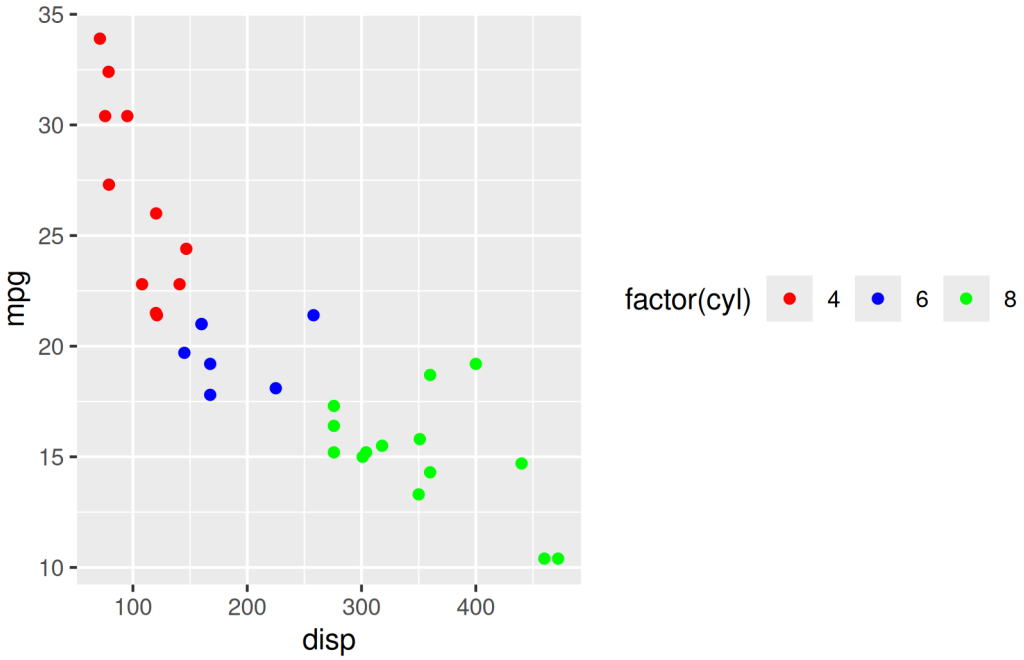
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    label.vjust = 0.8))
```



Direction

The direction of the label can be either horizontal or vertical and it can be set using the `direction` argument.

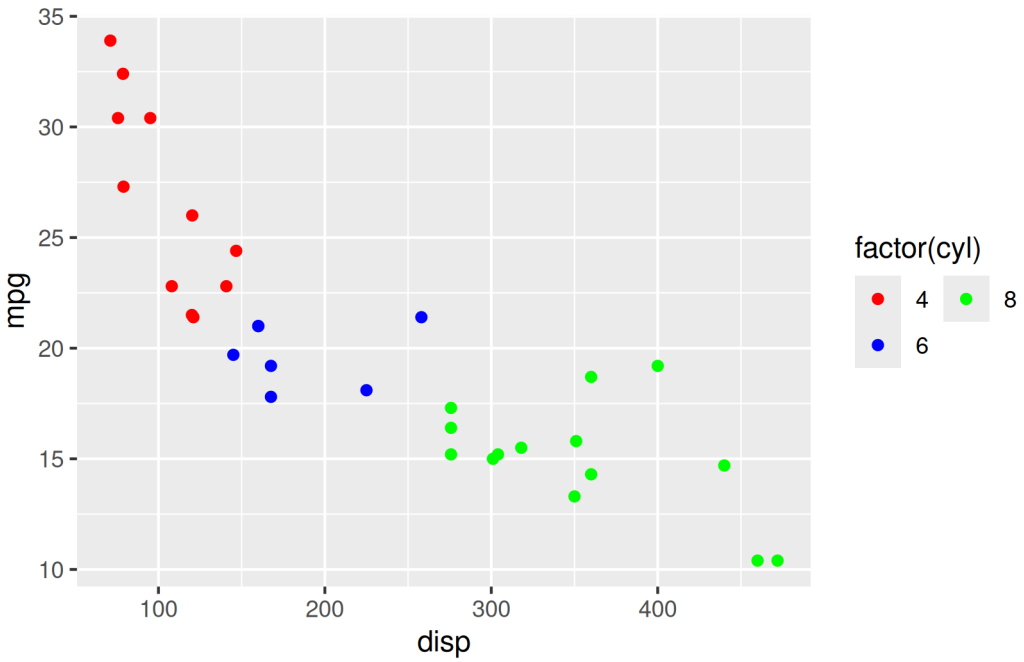
```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(direction = "horizontal"))
```



Rows

The label can be spread across multiple rows using the `nrow` argument. In the below example, the label is spread across 2 rows.

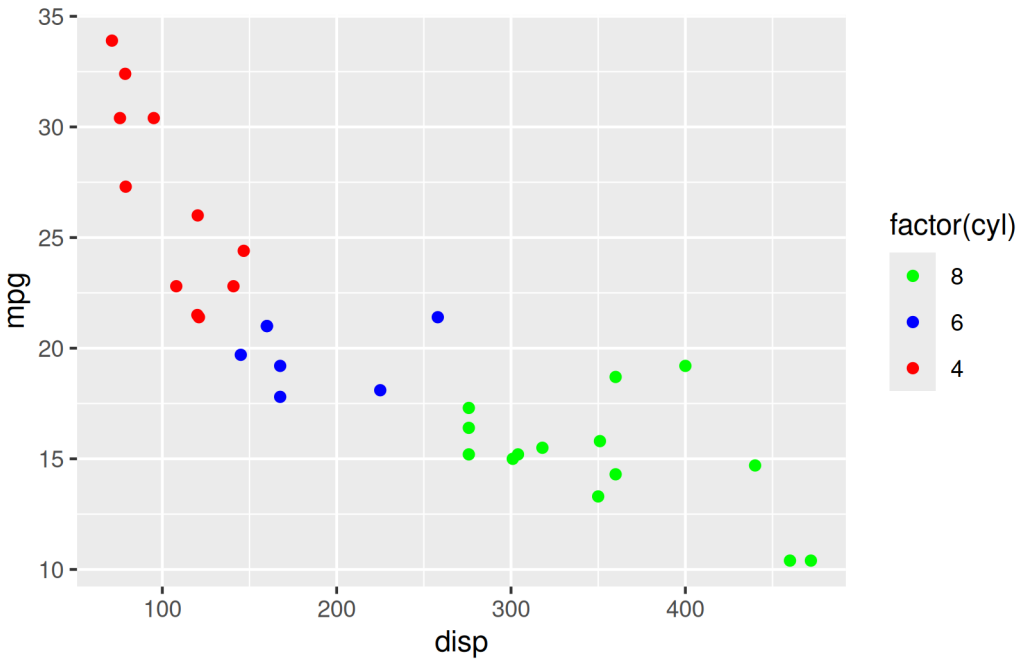
```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(nrow = 2))
```



Reverse

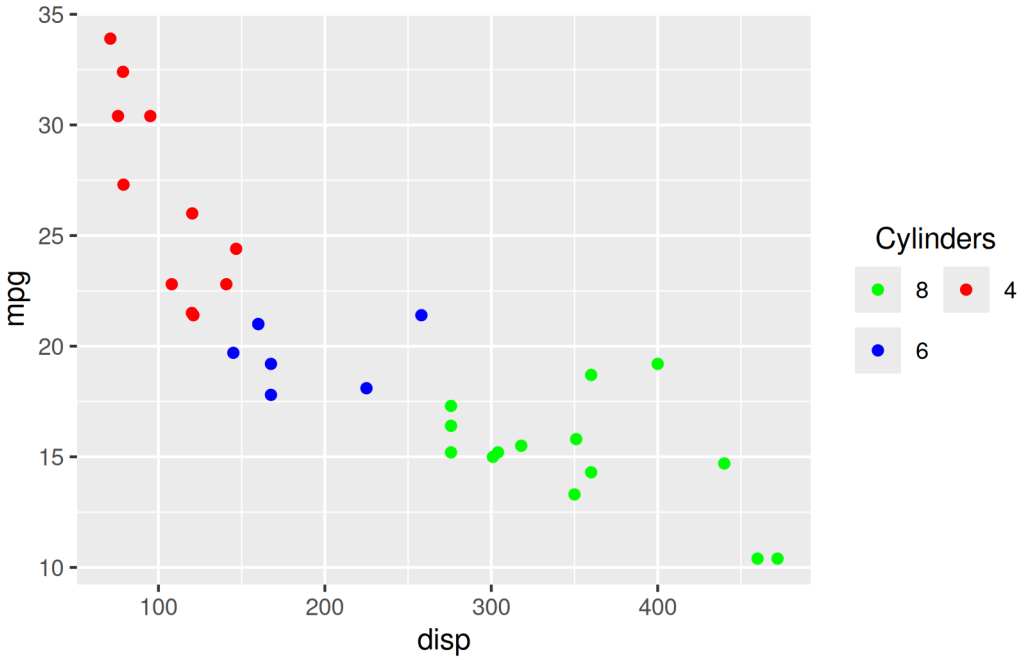
The order of the labels can be reversed using the `reverse` argument. We need to supply logical values i.e. either `TRUE` or `FALSE`. If `TRUE`, the order will be reversed.

```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(reverse = TRUE))
```



Putting it all together

```
ggplot(mtcars) + geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    guide = guide_legend(title = "Cylinders", title.hjust = 0.5,
      title.position = "top", label.position = "right",
      direction = "horizontal", label.hjust = 0.5, nrow = 2, reverse = TRUE)
  )
```



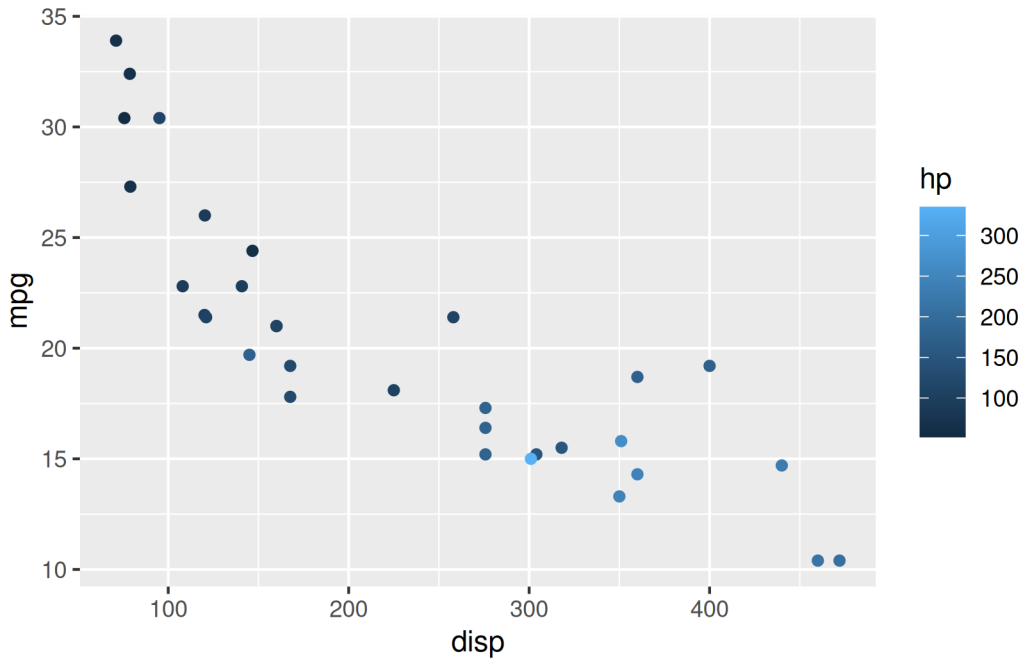
How to change the legend color bar in ggplot2

So far we have looked at modifying components of the legend when it acts as a guide for `color`, `fill` or `shape` i.e. when the aesthetics have been mapped to a categorical variable. In this section, you will learn about `guide_colorbar()` which will allow us to modify the legend when the aesthetics are mapped to a continuous variable.

Plot

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map the color of the points to the `hp` variable.

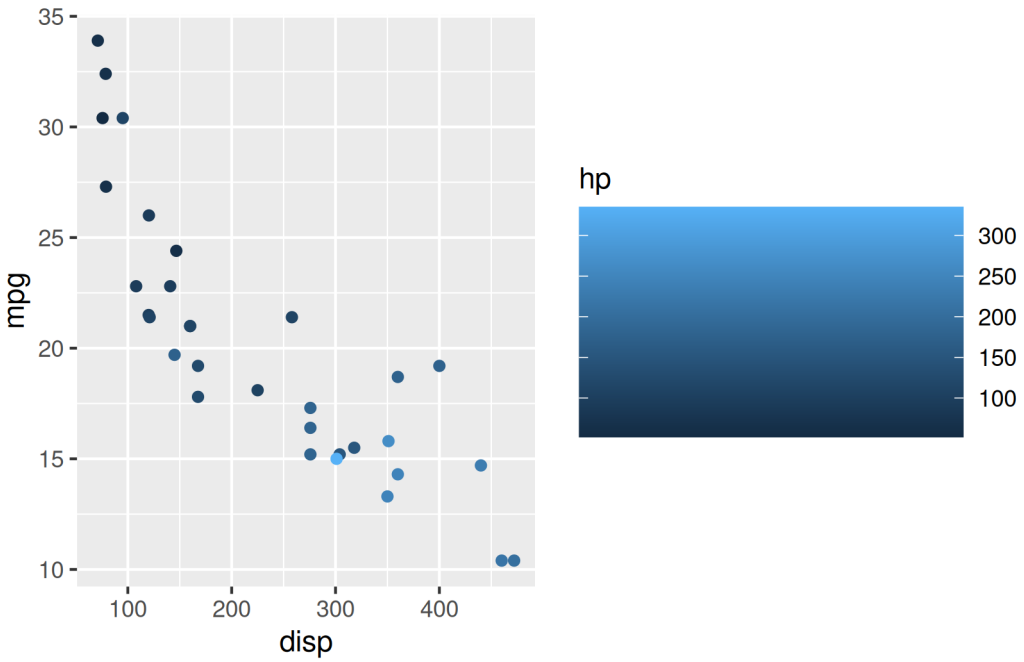
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp))
```



Width

The width of the bar can be modified using the `barwidth` argument. It is used inside the `guide_colorbar()` function which itself is supplied to the `guide` argument of `scale_color_continuous()`.

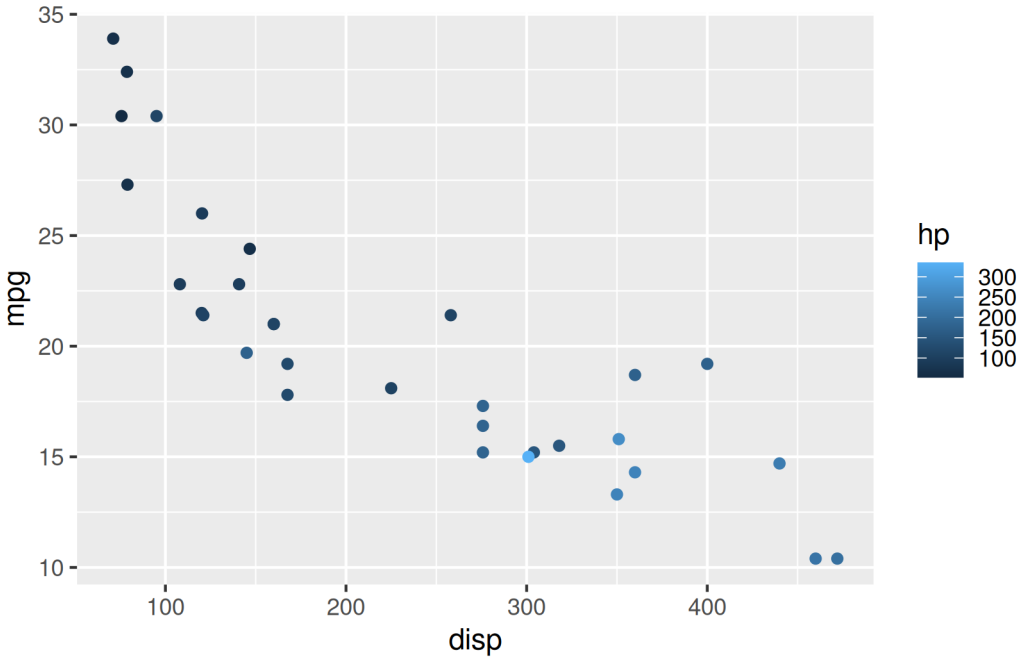
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    barwidth = 10))
```



Height

Similarly, the height of the bar can be modified using the `barheight` argument.

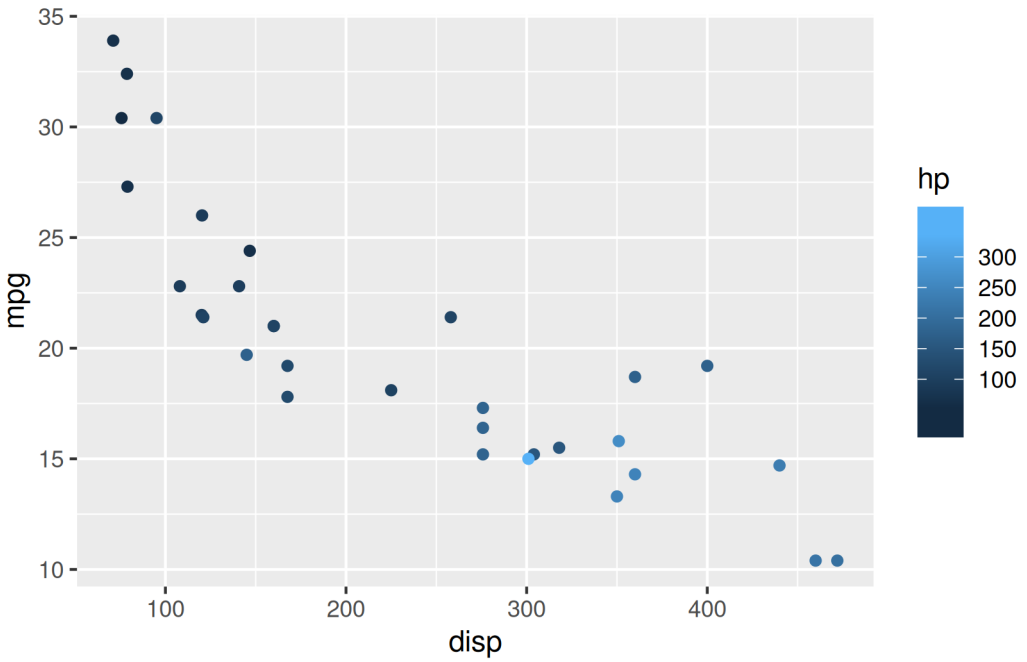
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    barheight = 3))
```



Bins

The `nbin` argument allows us to specify the number of bins in the bar.

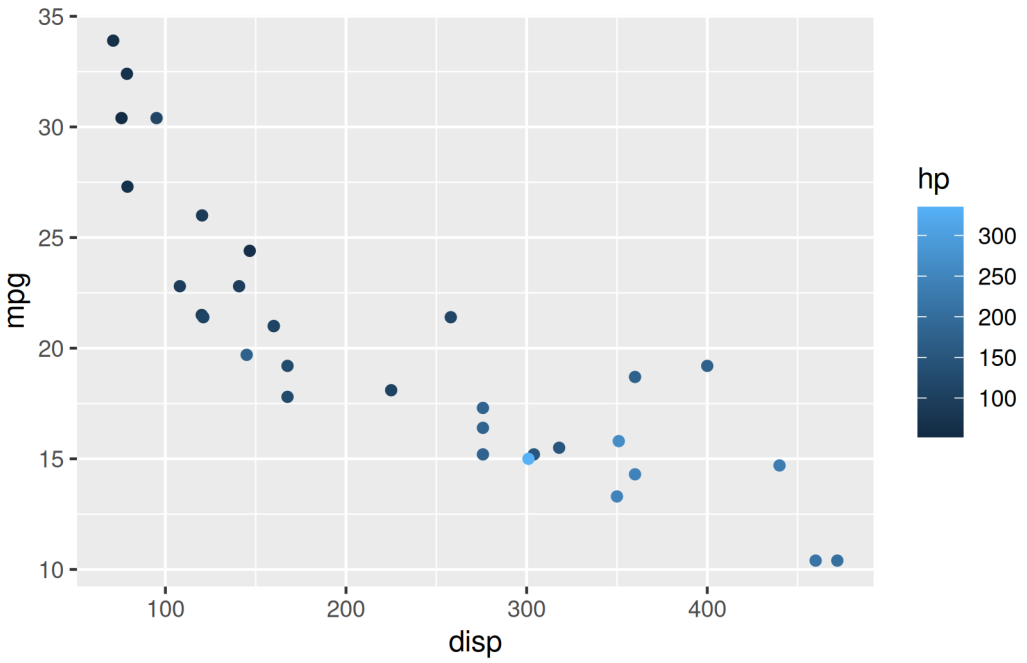
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    nbin = 4))
```



Ticks

The ticks of the bar can be removed using the `ticks` argument and setting it to `FALSE`.

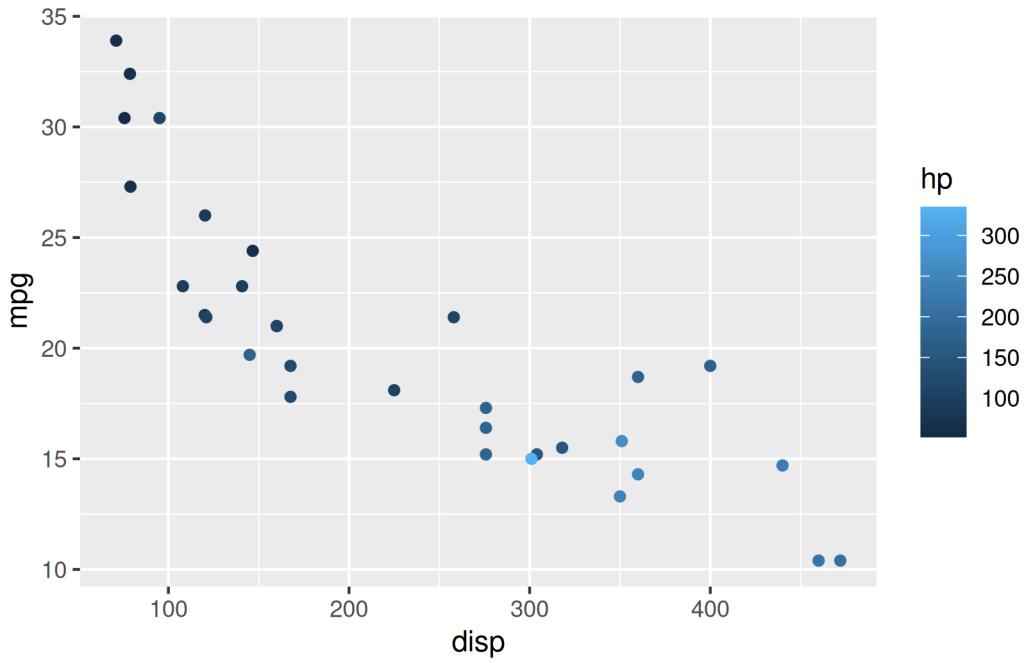
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    ticks = FALSE))
```



Upper/Lower Limits

The upper and lower limits of the bars can be drawn or undrawn using the `draw.ulim` and `draw.llim` arguments. They both accept logical values.

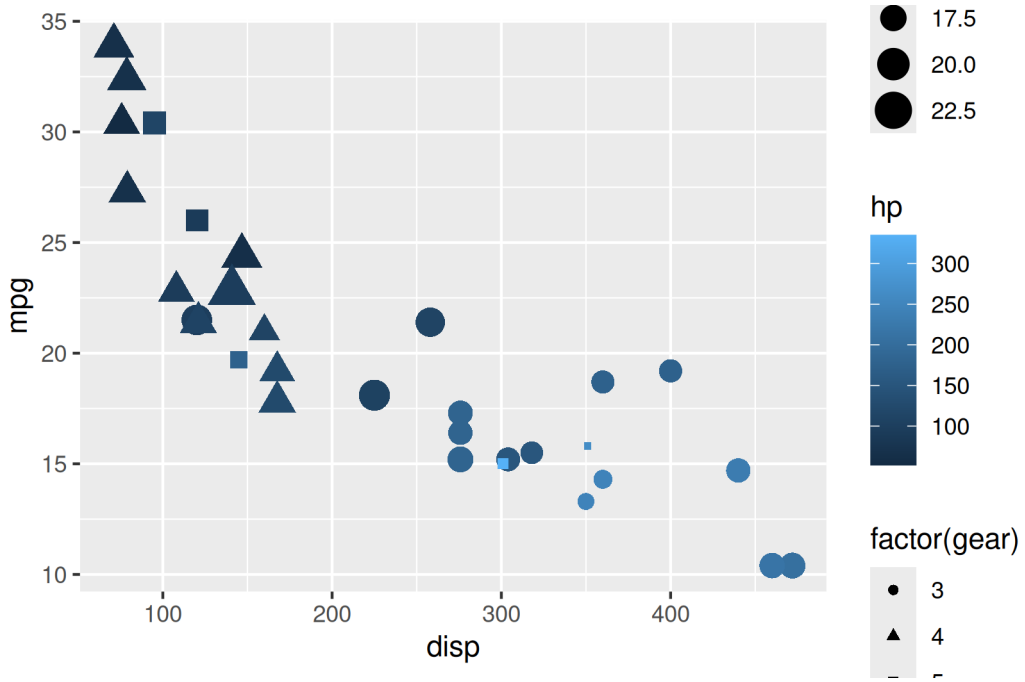
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_continuous(guide = guide_colorbar(
    draw.ulim = TRUE, draw.llim = FALSE))
```



12.1.0.1 Guides: Color, Shape & Size

The `guides()` function can be used to create multiple legends to act as a guide for `color`, `shape`, `size` etc. as shown below. First, we map color, shape and size to different variables. Next, in the `guides()` function, we supply values to each of the above aesthetics to indicate the type of legend.

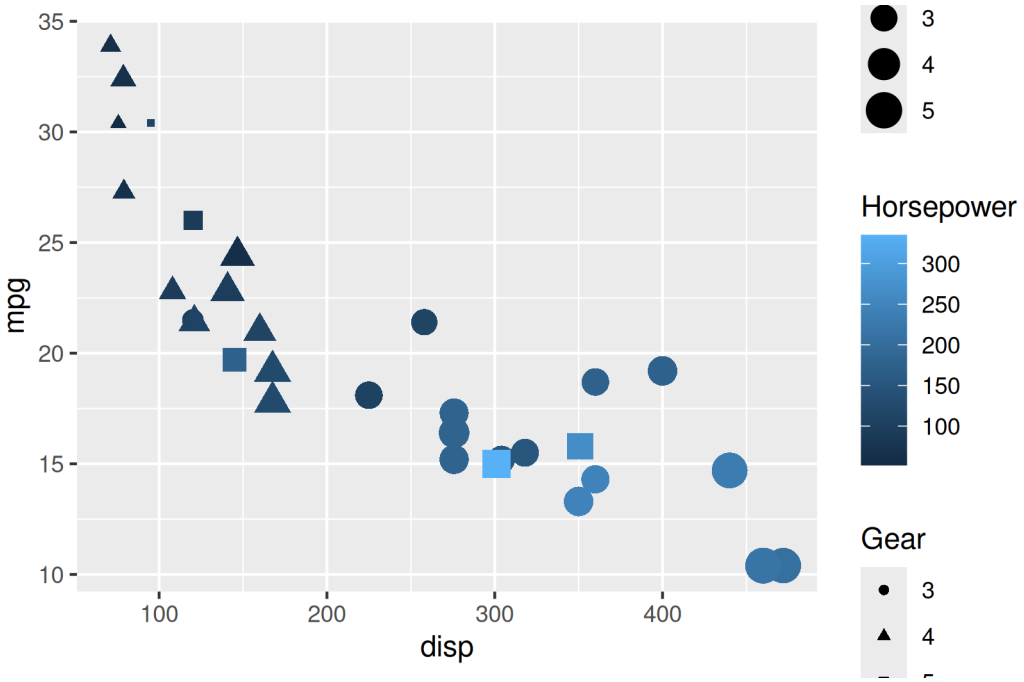
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp,
    size = qsec, shape = factor(gear))) +
  guides(color = "colorbar", shape = "legend", size = "legend")
```



Guides: Title

To modify the components of the different legends, we must use the `guide_*` family of functions. In the below example, we use `guide_colorbar()` for the legend acting as guide for color mapped to a continuous variable and `guide_legend()` for the legends acting as guide for shape/size mapped to categorical variables.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp, size = wt, shape = factor(gear))) +
  guides(color = guide_colorbar(title = "Horsepower"),
    shape = guide_legend(title = "Gear"), size = guide_legend(title = "Weight")
  )
```



12.2 Exercises

1. Center a legend title with `guide_legend(title.hjust = 0.5)`.
2. Move discrete legend labels with `label.position`.
3. Size a colorbar with `barwidth`/`barheight` for a continuous guide.

Worked solutions in `solutions/legend.md` (attempt first).

Where to go next

- ggplot2 book scales and guides chapters

13 Legend: Color and Fill

In this chapter, we modify legend title, breaks, limits, labels and values when color or fill is mapped to categorical variables.

> **Why this matters**
>
> Color and fill legends decode groups; keep titles plain words and show only needed breaks.

> **Common pitfall**
>
> Using limits silently drops data. Filter with dplyr when you mean to subset.

13.1 How to change legend colors in ggplot2

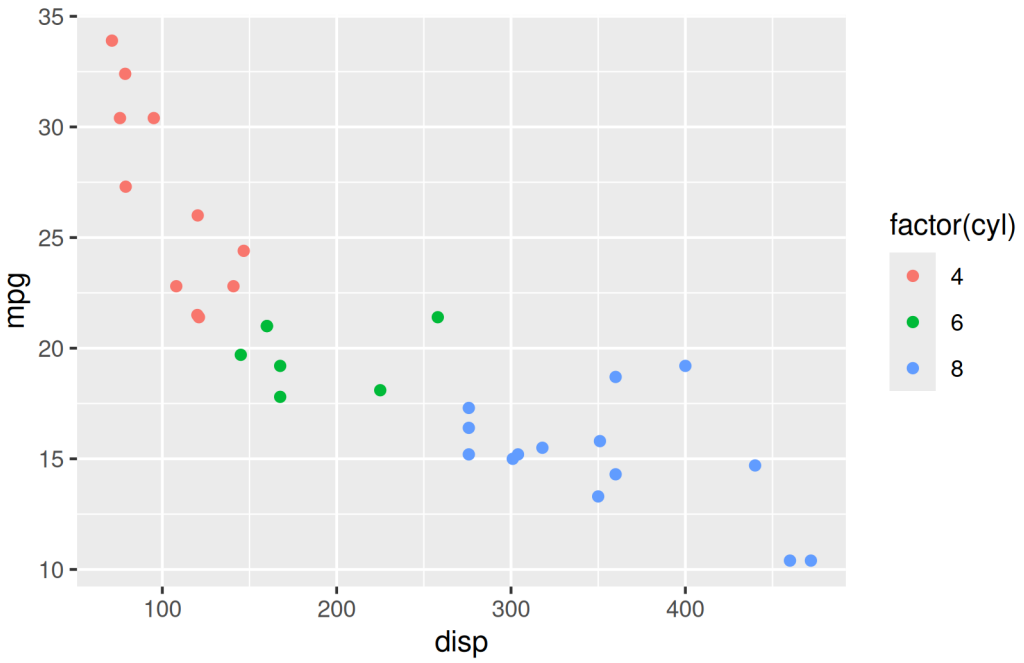
We will learn to modify the following when `color` is mapped to categorical variables:

- title
- breaks
- limits
- labels
- values

How to create a plot with a color legend in ggplot2

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map the color of the points to the `cyl` variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl)))
```

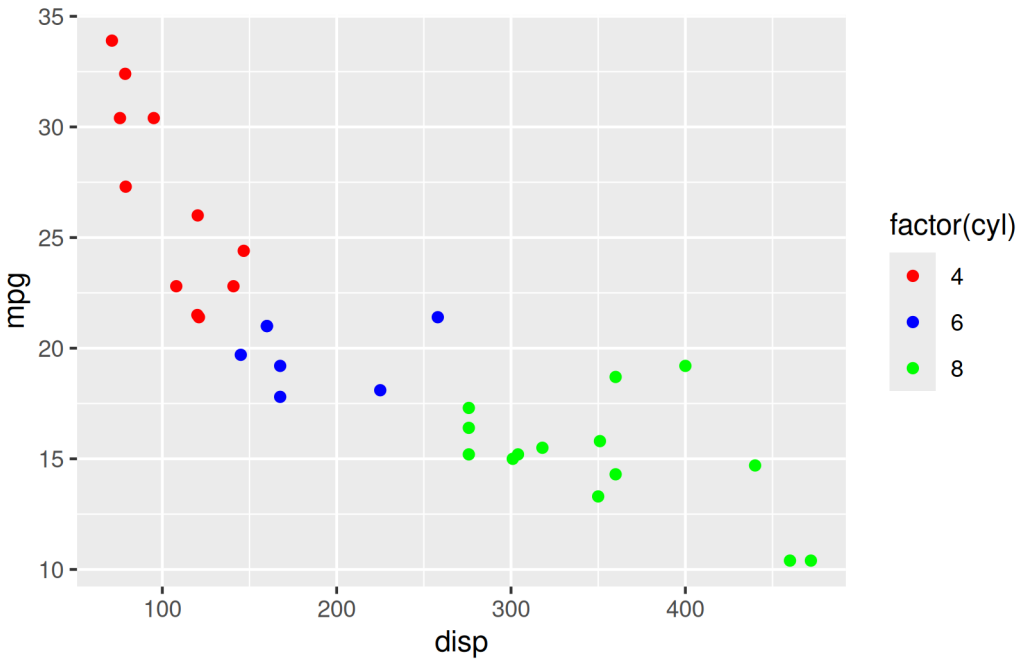


As you can see, the legend acts as a guide for the `color` aesthetic. Now, let us learn to modify the different aspects of the legend.

How to change legend color values in ggplot2

To change the default colors in the legend, use the `values` argument and supply a character vector of color names. The number of colors specified must be equal to the number of levels in the categorical variable mapped. In the below example, `cyl` has 3 levels (4, 6, 8) and hence we have specified 3 colors.

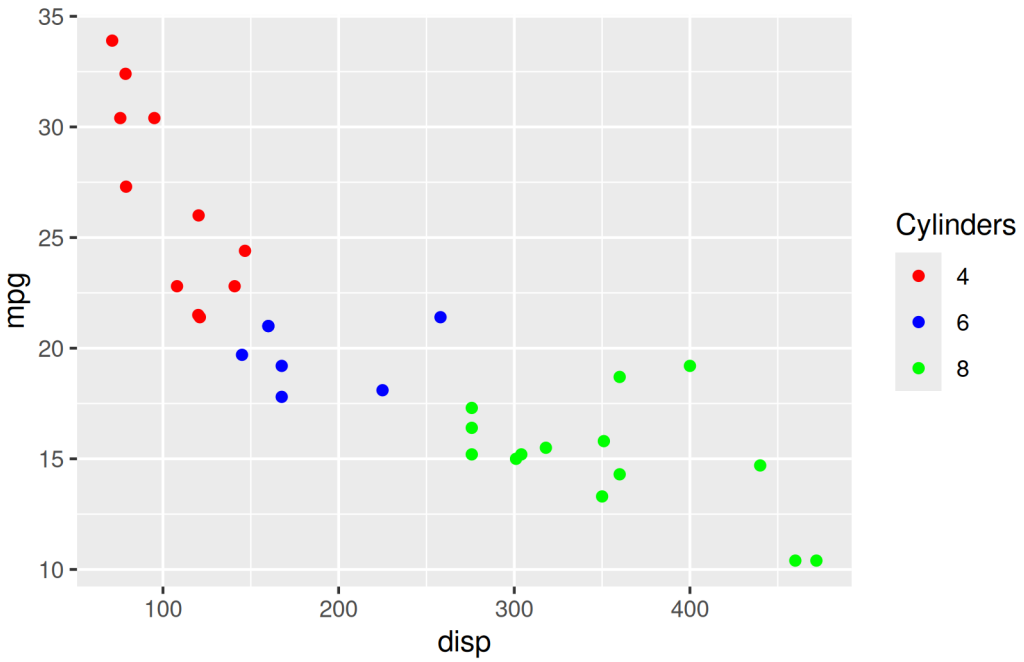
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"))
```



How to change the legend title in ggplot2

In the previous example, the title of the legend (`factor(cyl)`) is not very intuitive. If the user does not know the underlying data, they will not be able to make any sense out of it. Let us change it to `Cylinders` using the `name` argument.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(name = "Cylinders",
    values = c("red", "blue", "green"))
```

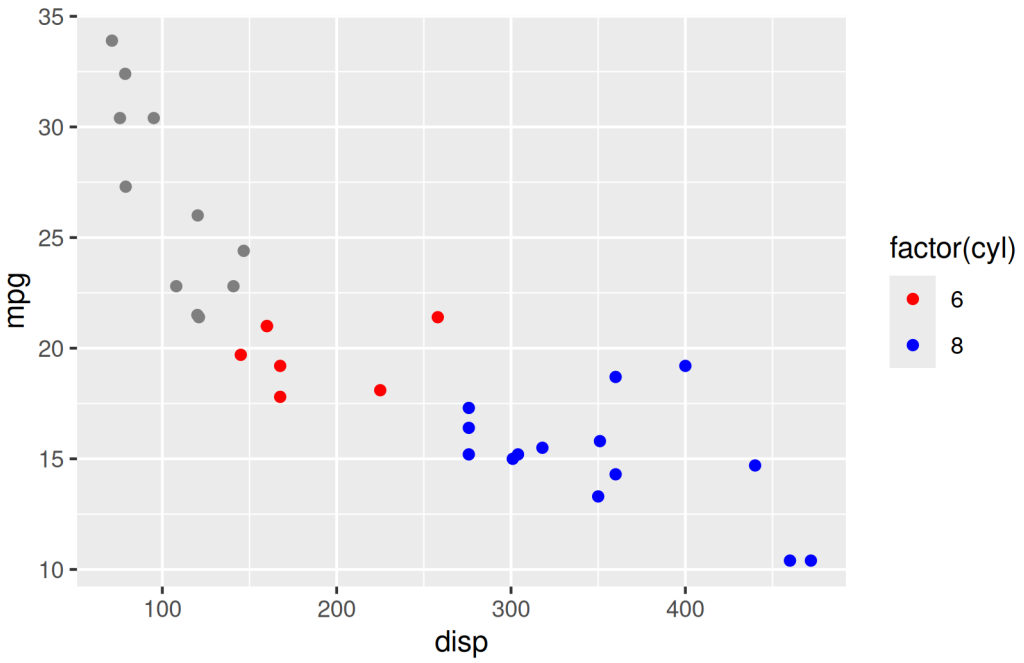


Now, the user will know that the different colors represent number of cylinders in the car.

How to change legend limits in ggplot2

Let us assume that we want to modify the data to be displayed i.e. instead of examining the relationship between mileage and displacement for all cars, we desire to look at only cars with at least 6 cylinders. One way to approach this would be to filter the data using `filter` from dplyr and then visualize it. Instead, we will use the `limits` argument and filter the data for visualization.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"), limits = c(6, 8))
```

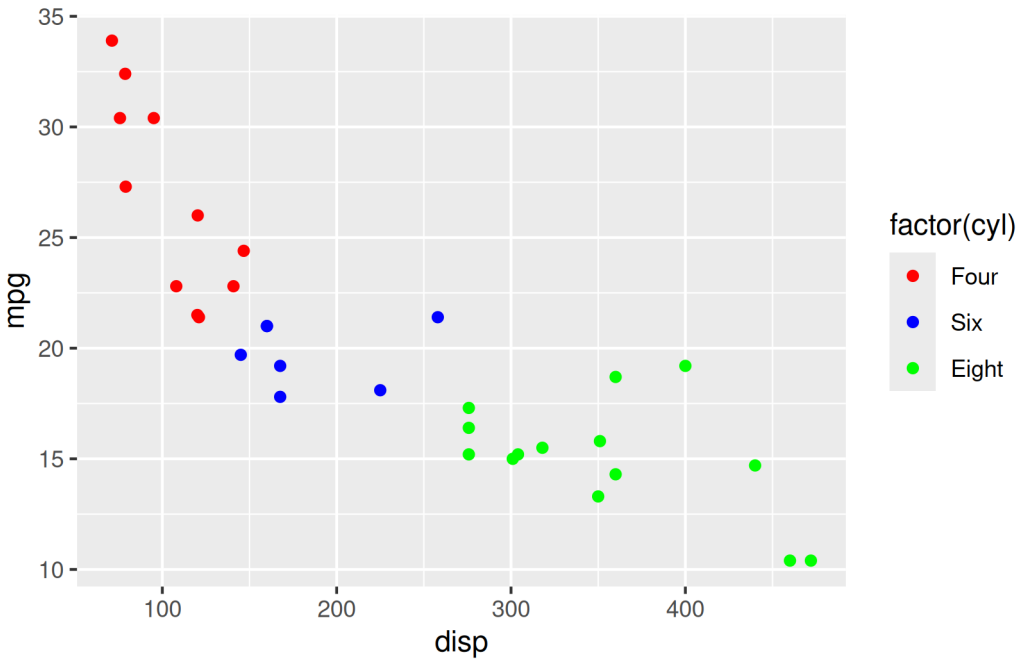


As you can see above, `ggplot2` returns a warning message indicating data related to 4 cylinders has been dropped. If you observe the legend, it now represents only 6 and 8 cylinders.

How to change legend labels in ggplot2

The labels in the legend can be modified using the `labels` argument. Let us change the labels to `Four`, `Six` and `Eight` in the next example. Ensure that the labels are intuitive and easy to interpret for the end user of the plot.

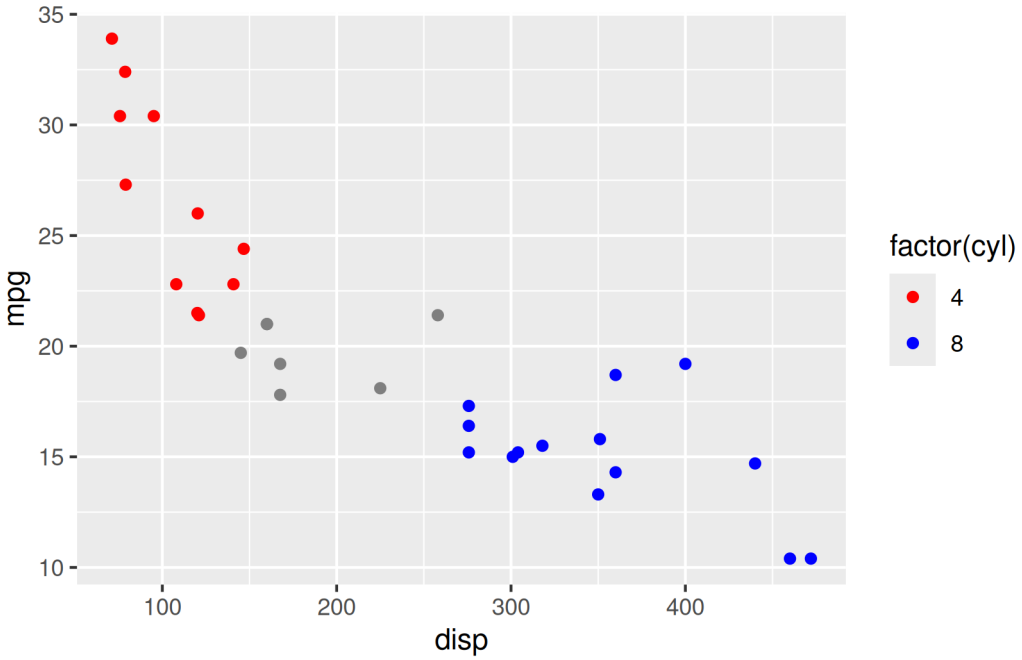
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    labels = c('Four', 'Six', 'Eight'))
```



How to reorder or hide legend items in ggplot2

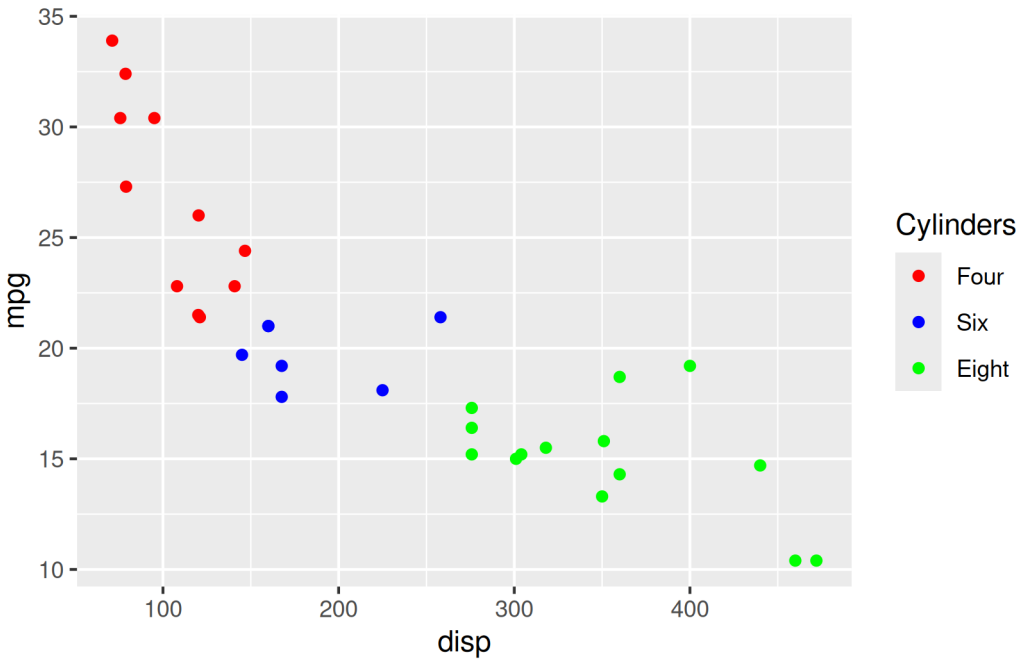
When there are large number of levels in the mapped variable, you may not want the labels in the legend to represent all of them. In such cases, we can use the breaks argument and specify the labels to be used. In the below case, we use the `breaks` argument to ensure that the labels in legend represent two levels (4, 8) of the mapped variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(values = c("red", "blue", "green"),
    breaks = c(4, 8))
```



How to combine color legend options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_manual(name = "Cylinders", values = c("red", "blue", "green"),
    labels = c('Four', 'Six', 'Eight'), limits = c(4, 6, 8), breaks = c(4, 6, 8))
```



13.2 How to change legend fill colors in ggplot2

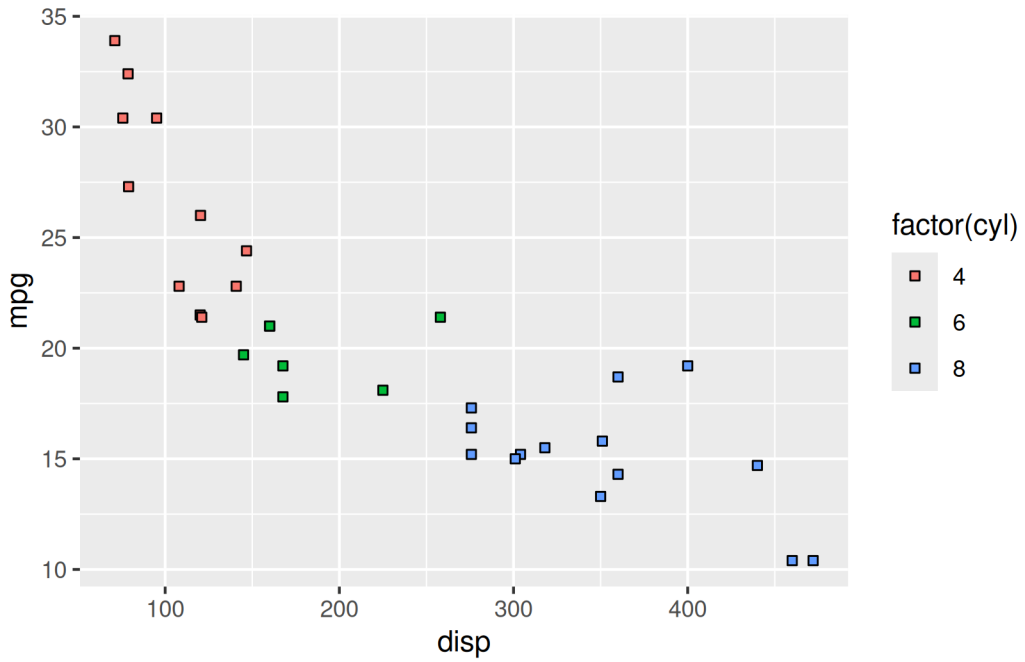
we will learn to modify the following using `scale_fill_manual()` when `fill` is mapped to categorical variables:

- title
- breaks
- limits
- labels
- values

How to create a plot with a fill legend in ggplot2

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map `fill` to the `cyl` variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22)
```

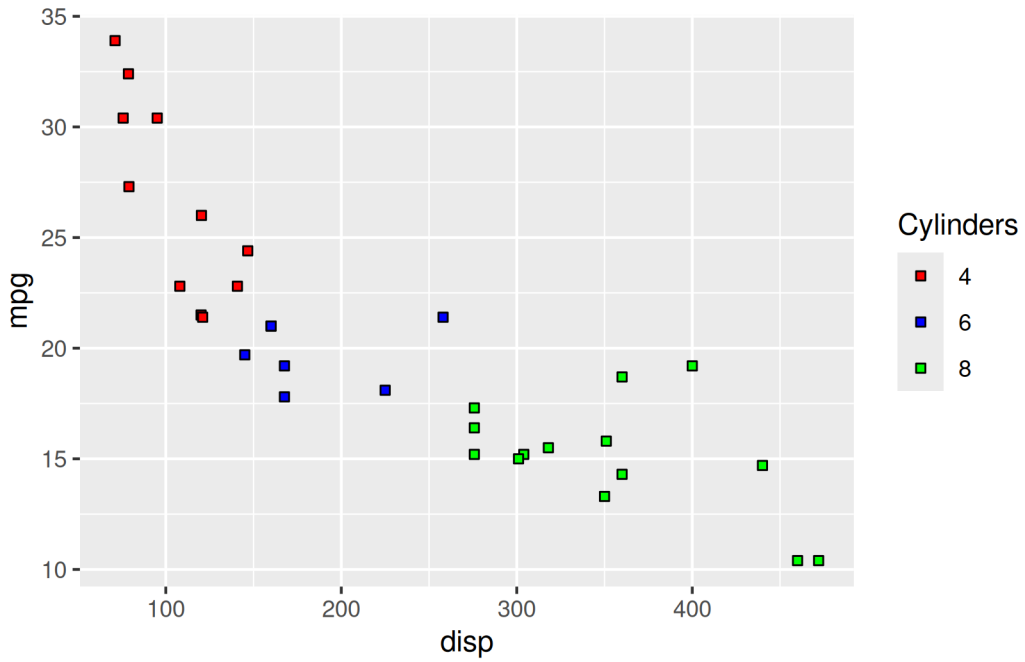


As you can see, the legend acts as a guide for the `color` aesthetic. Now, let us learn to modify the different aspects of the legend.

How to change the fill legend title in ggplot2

The title of the legend (`factor(cyl)`) is not very intuitive. If the user does not know the underlying data, they will not be able to make any sense out of it. Let us change it to `Cylinders` using the `name` argument.

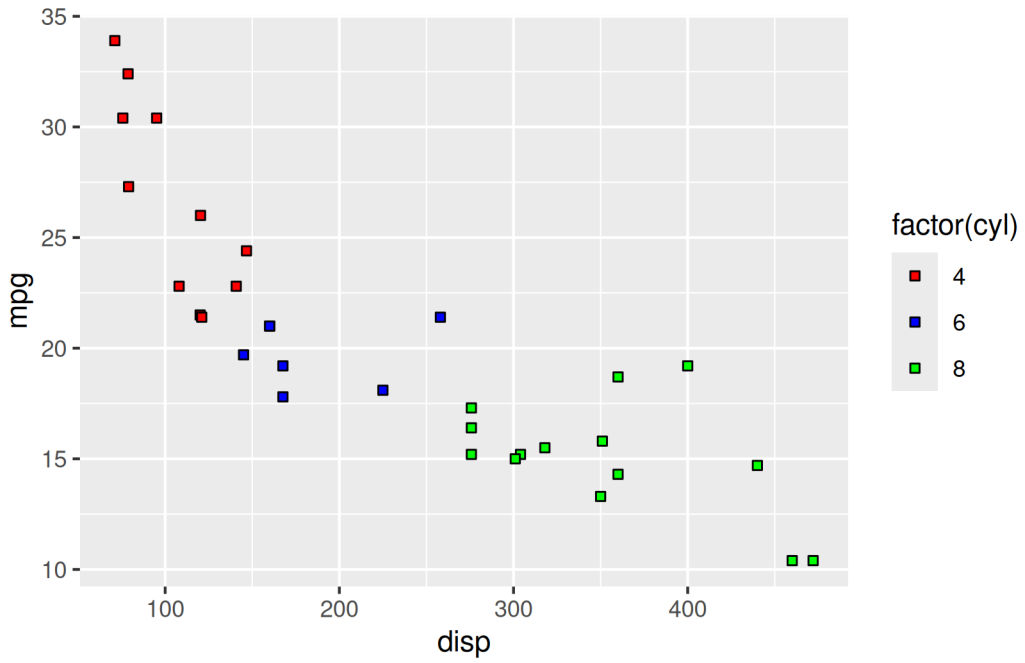
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(name = "Cylinders",
    values = c("red", "blue", "green"))
```



How to change legend fill values in ggplot2

To change the default colors in the legend, use the `values` argument and supply a character vector of color names. The number of colors specified must be equal to the number of levels in the categorical variable mapped. In the below example, `cyl` has 3 levels (4, 6, 8) and hence we have specified 3 colors.

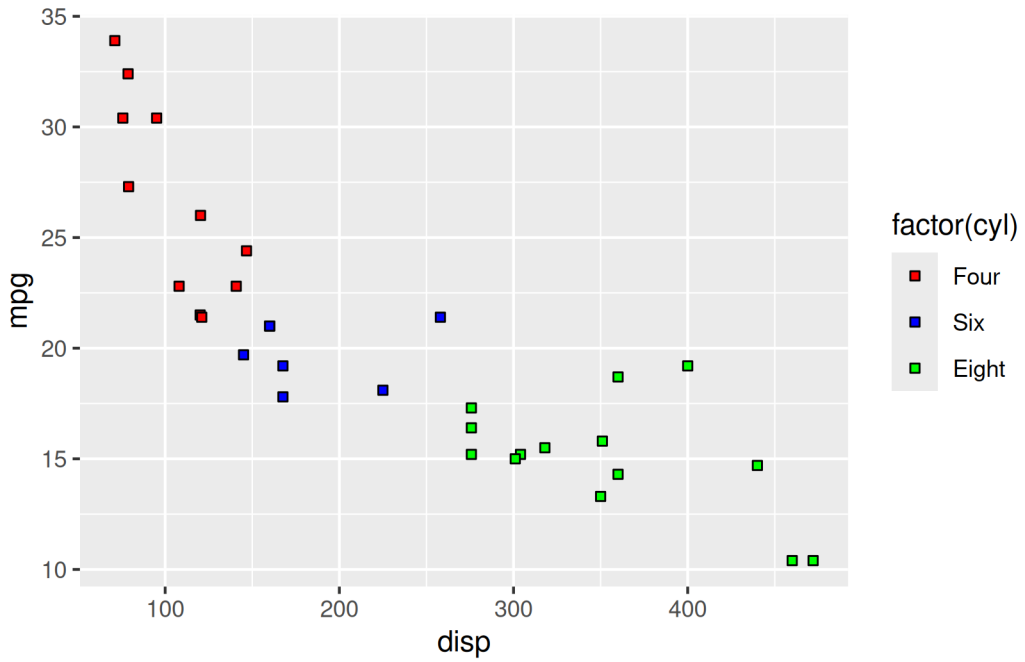
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(values = c("red", "blue", "green"))
```



How to change fill legend labels in ggplot2

The labels in the legend can be modified using the `labels` argument. Let us change the labels to `Four`, `Six` and `Eight` in the next example. Ensure that the labels are intuitive and easy to interpret for the end user of the plot.

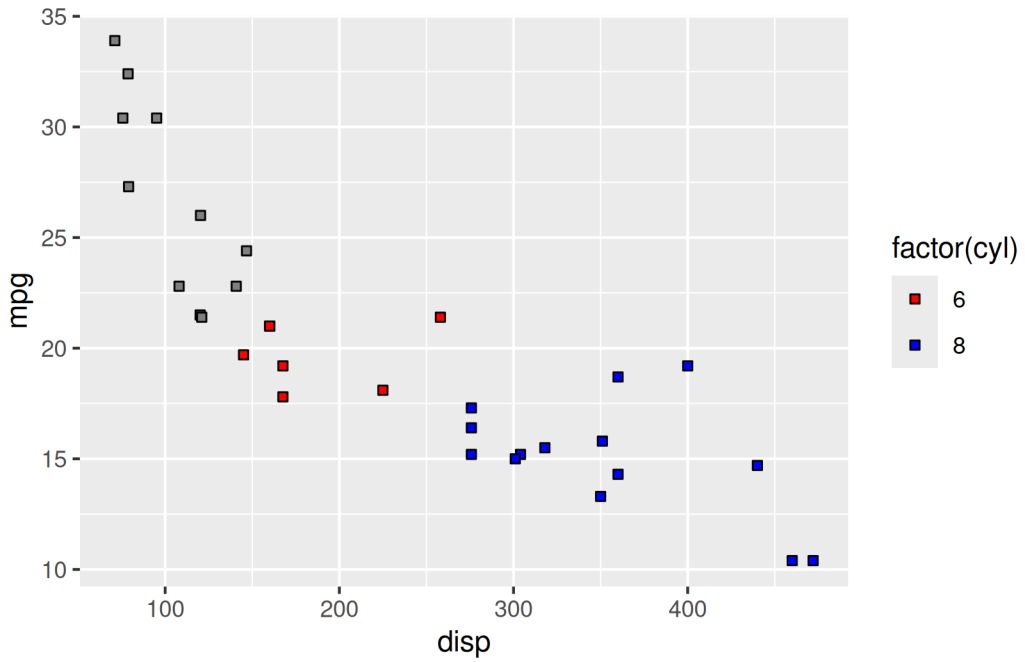
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(values = c("red", "blue", "green"),
    labels = c('Four', 'Six', 'Eight'))
```



How to change fill legend limits in ggplot2

Let us assume that we want to modify the data to be displayed i.e. instead of examining the relationship between mileage and displacement for all cars, we desire to look at only cars with at least 6 cylinders. One way to approach this would be to filter the data using `filter` from dplyr and then visualize it. Instead, we will use the `limits` argument and filter the data for visualization.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(values = c("red", "blue", "green"),
    limits = c(6, 8))
```

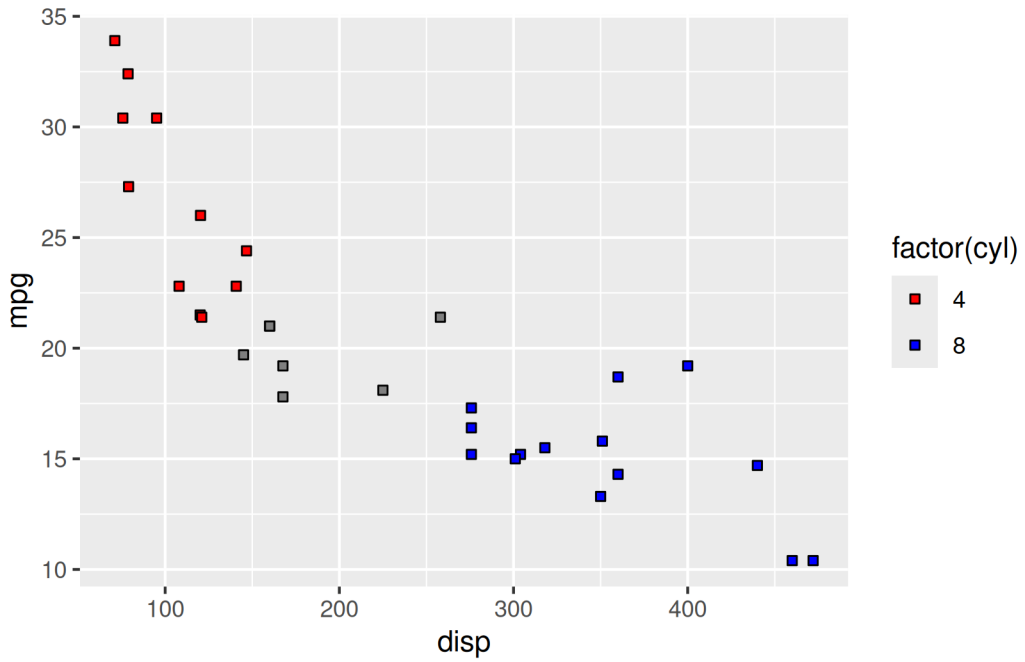


As you can see above, `ggplot2` returns a warning message indicating data related to 4 cylinders has been dropped. If you observe the legend, it now represents only 6 and 8 cylinders.

How to reorder or hide fill legend items in ggplot2

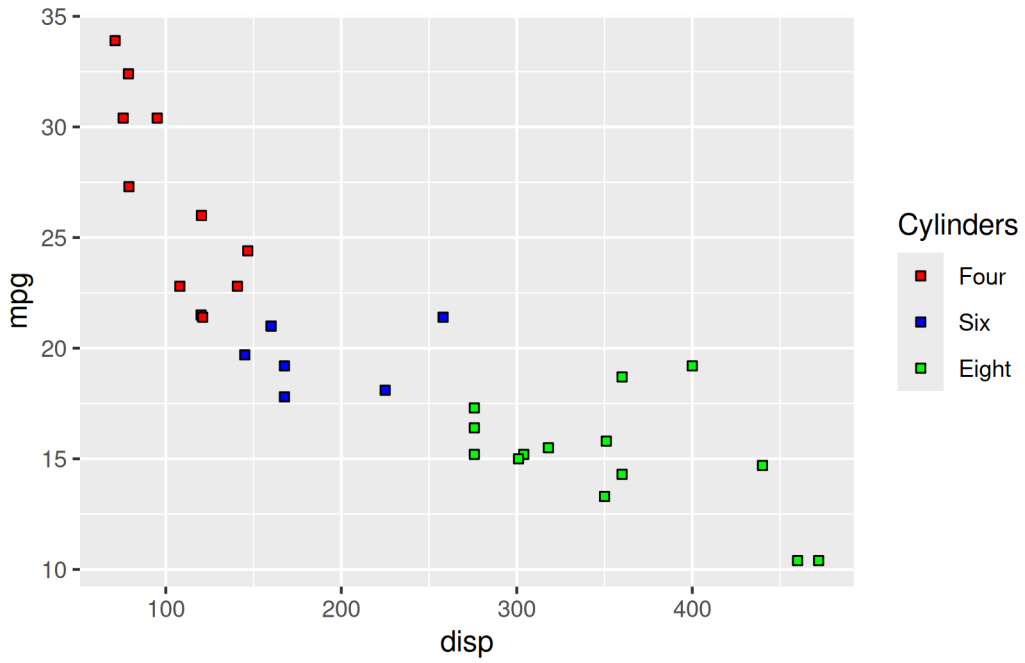
When there are large number of levels in the mapped variable, you may not want the labels in the legend to represent all of them. In such cases, we can use the breaks argument and specify the labels to be used. In the below case, we use the `breaks` argument to ensure that the labels in legend represent two levels (4, 8) of the mapped variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(values = c("red", "blue", "green"),
    breaks = c(4, 8))
```



How to combine fill legend options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, fill = factor(cyl)), shape = 22) +
  scale_fill_manual(name = "Cylinders", values = c("red", "blue", "green"),
    labels = c('Four', 'Six', 'Eight'), limits = c(4, 6, 8), breaks = c(4, 6, 8))
```



13.3 Exercises

1. Recolor cylinder groups with `scale_color_manual(values = ...)`.
2. Rename legend entries with the `labels` argument.
3. Show only 4- and 8-cylinder levels with `breaks` and justify vs `limits`.

Worked solutions in `solutions/legend-color-fill.md` (attempt first).

Where to go next

- ggplot2 book scales and guides chapters

14 Legend: Shape Size and Alpha

In this chapter, we modify legends when shape, size or alpha is mapped to variables.

> **Why this matters**
>
> Shape, size and alpha encode extra variables without new panels; map size and alpha to continuous variables only.

> **Common pitfall**
>
> Mapping size or alpha to a discrete variable produces a misleading legend.

14.1 How to change legend point shapes in ggplot2

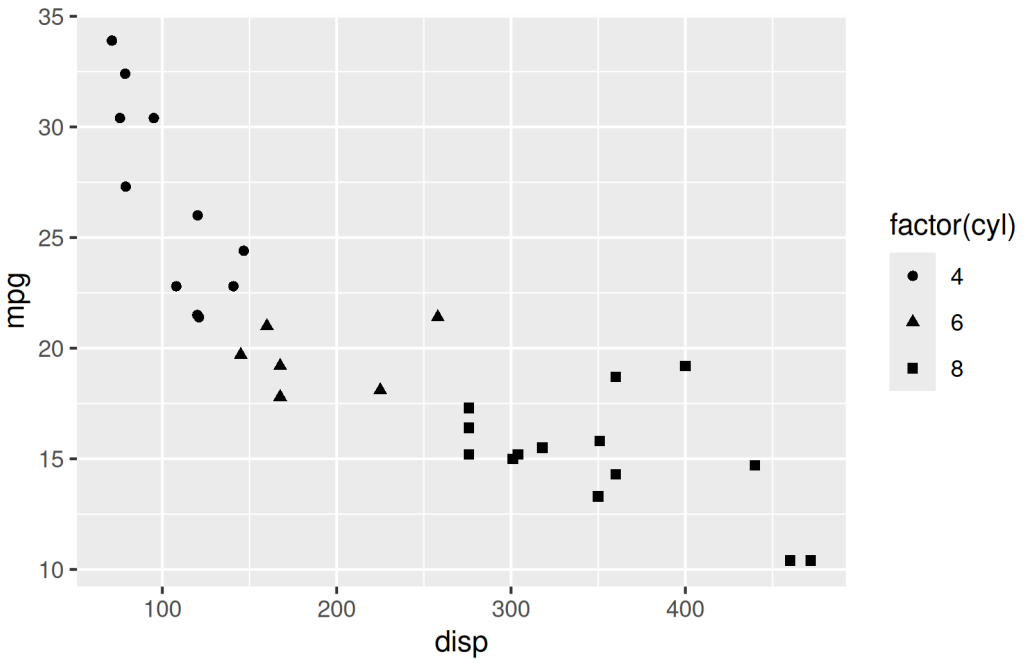
We will learn to modify the following using `scale_shape_manual` when `shape` is mapped to categorical variables:

- title
- breaks
- limits
- labels
- values

How to create a plot with a shape legend in ggplot2

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map the shape of the points to the `cyl` variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl)))
```

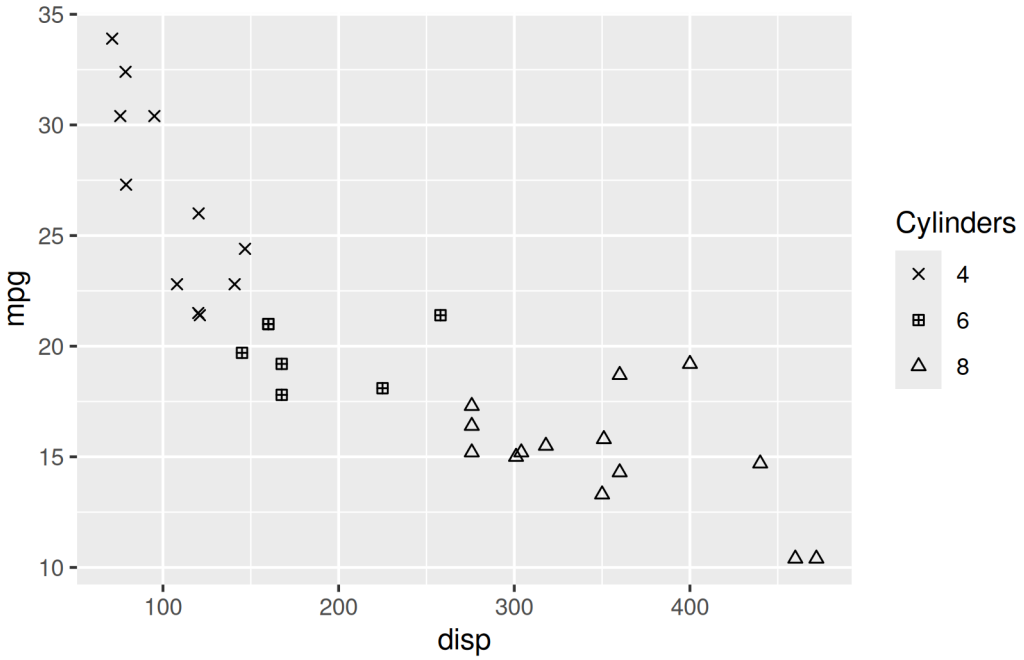


As you can see, the legend acts as a guide for the `shape` aesthetic. Now, let us learn to modify the different aspects of the legend.

How to change the shape legend title in ggplot2

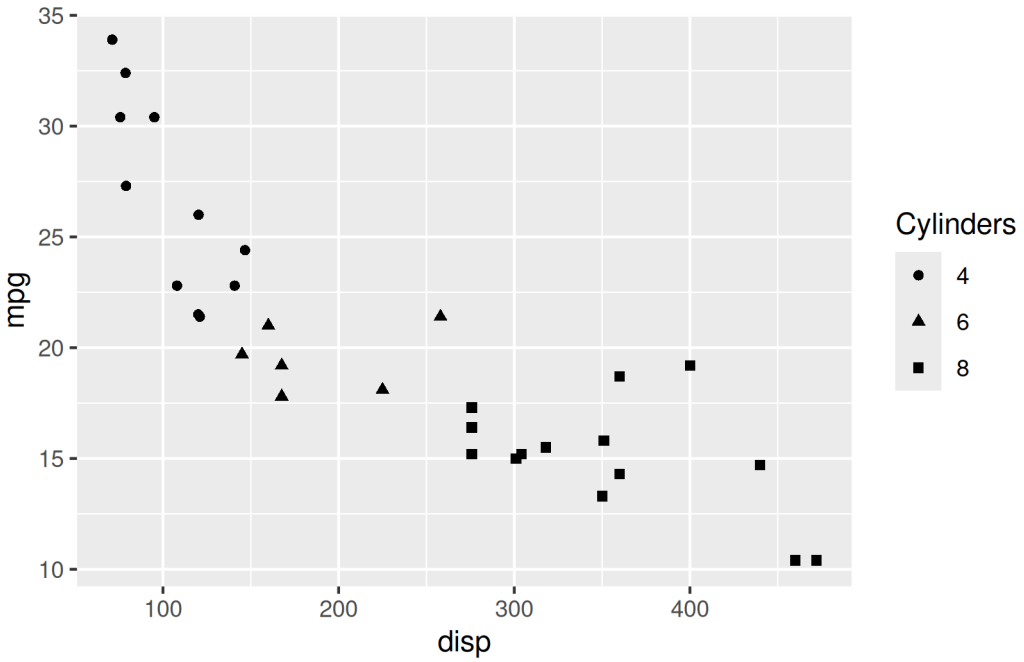
The title of the legend (`factor(cyl)`) is not very intuitive. If the user does not know the underlying data, they will not be able to make any sense out of it. Let us change it to `Cylinders` using the `name` argument.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(name = "Cylinders", values = c(4, 12, 24))
```



If you have mapped shape/size to a discrete variable which has less than six categories, you can use `scale_shape()`.

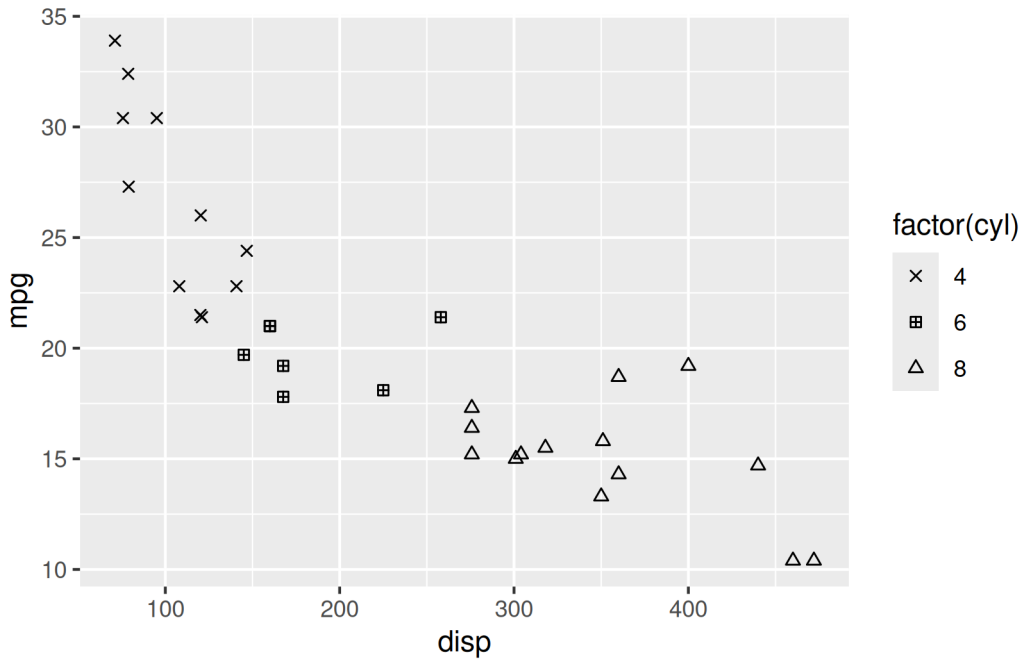
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape(name = 'Cylinders')
```



How to change legend shape values in ggplot2

To change the default shapes in the legend, use the `values` argument and supply a numeric vector of shapes. The number of shapes specified must be equal to the number of levels in the categorical variable mapped. In the below example, `cyl` has 3 levels (4, 6, 8) and hence we have specified 3 different shapes.

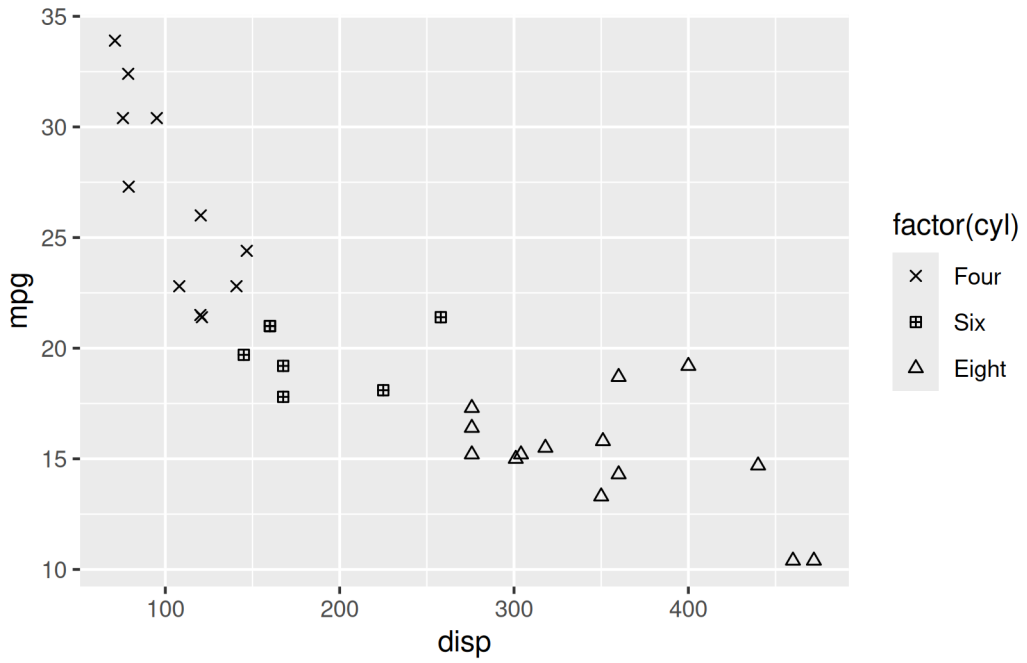
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(values = c(4, 12, 24))
```



How to change shape legend labels in ggplot2

The labels in the legend can be modified using the `labels` argument. Let us change the labels to `Four`, `Six` and `Eight` in the next example. Ensure that the labels are intuitive and easy to interpret for the end user of the plot.

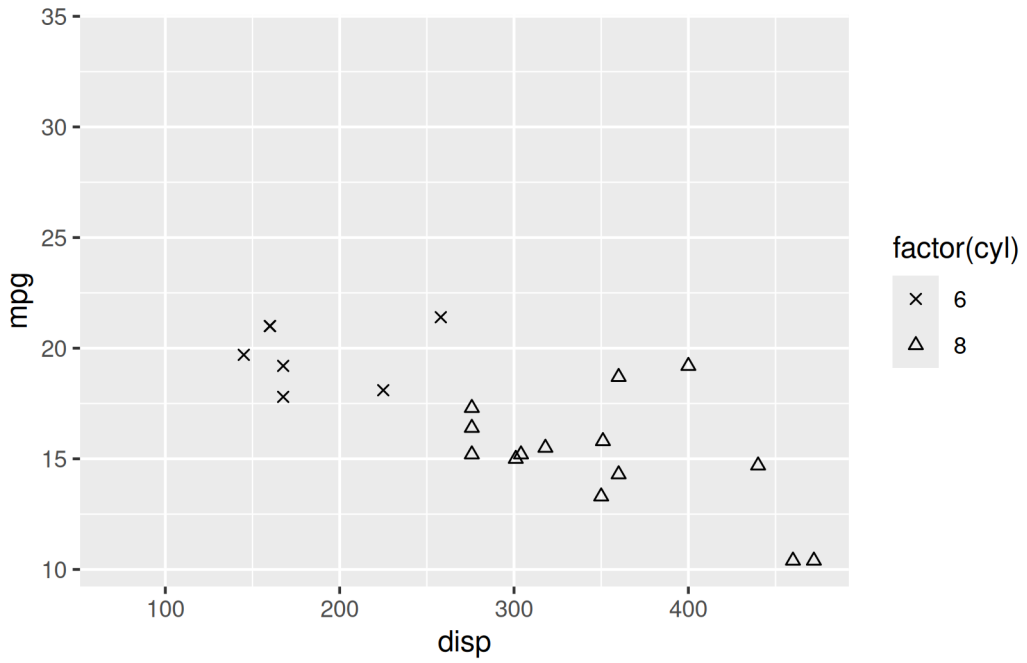
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(values = c(4, 12, 24), labels = c('Four', 'Six', 'Eight'))
```



How to change shape legend limits in ggplot2

Let us assume that we want to modify the data to be displayed i.e. instead of examining the relationship between mileage and displacement for all cars, we desire to look at only cars with at least 6 cylinders. One way to approach this would be to filter the data using `filter` from dplyr and then visualize it. Instead, we will use the `limits` argument and filter the data for visualization.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(values = c(4, 24), limits = c(6, 8))
```

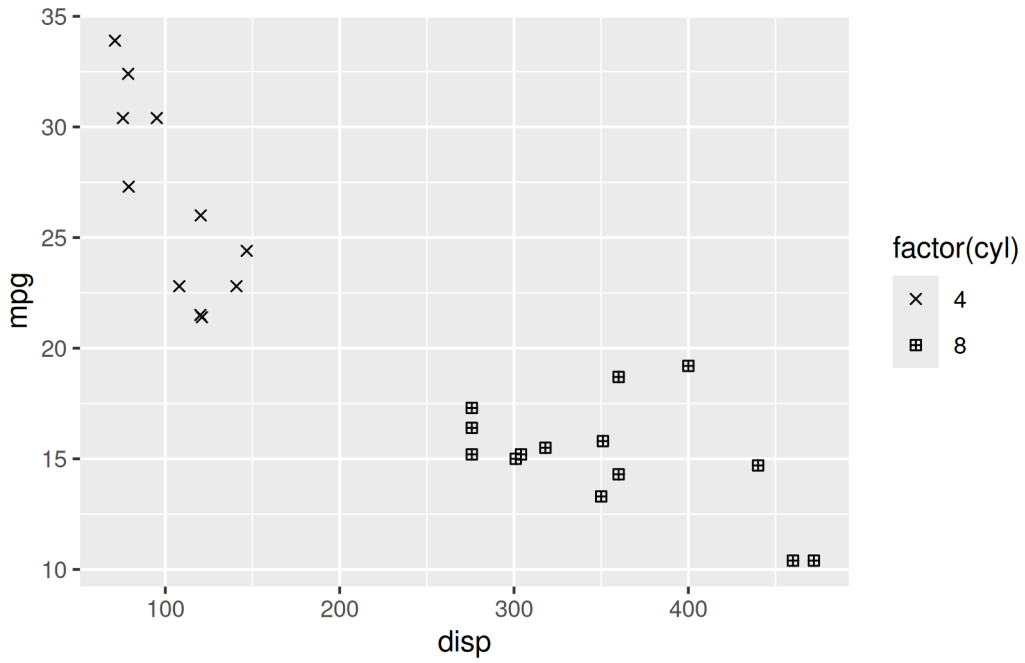


As you can see above, `ggplot2` returns a warning message indicating data related to 4 cylinders has been dropped. If you observe the legend, it now represents only 6 and 8 cylinders.

How to reorder or hide shape legend items in ggplot2

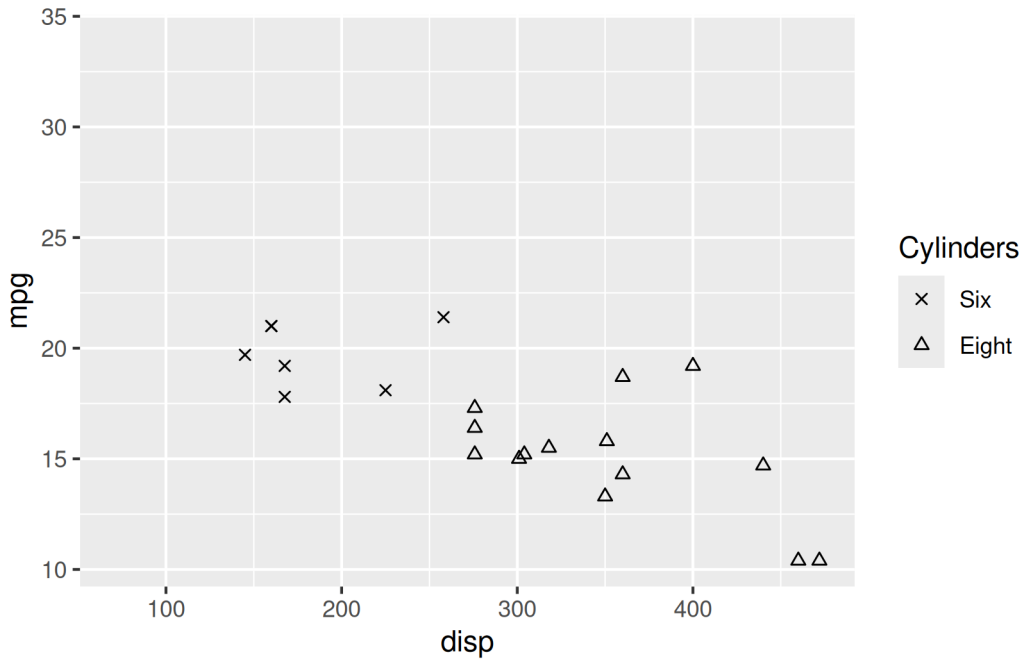
When there are large number of levels in the mapped variable, you may not want the labels in the legend to represent all of them. In such cases, we can use the breaks argument and specify the labels to be used. In the below case, we use the `breaks` argument to ensure that the labels in legend represent two levels (4, 8) of the mapped variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(values = c(4, 12, 24), breaks = c(4, 8))
```



How to combine shape legend options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, shape = factor(cyl))) +
  scale_shape_manual(name = "Cylinders", labels = c('Six', 'Eight'),
     values = c(4, 24), limits = c(6, 8), breaks = c(6, 8))
```



14.2 How to change legend point sizes in ggplot2

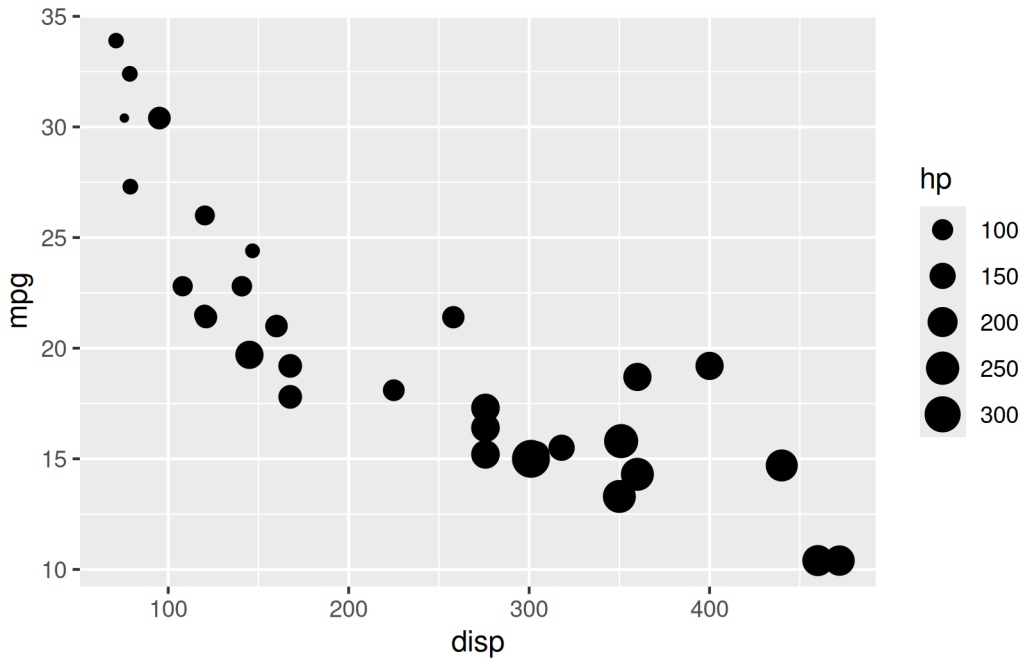
We will learn to modify the following using `scale_size_continuous` when `size` aesthetic is mapped to variables:

- title
- breaks
- limits
- range
- labels
- values

How to create a plot with a size legend in ggplot2

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map the size of the points to the `hp` variable. Remember, `size` must always be mapped to a continuous variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp))
```

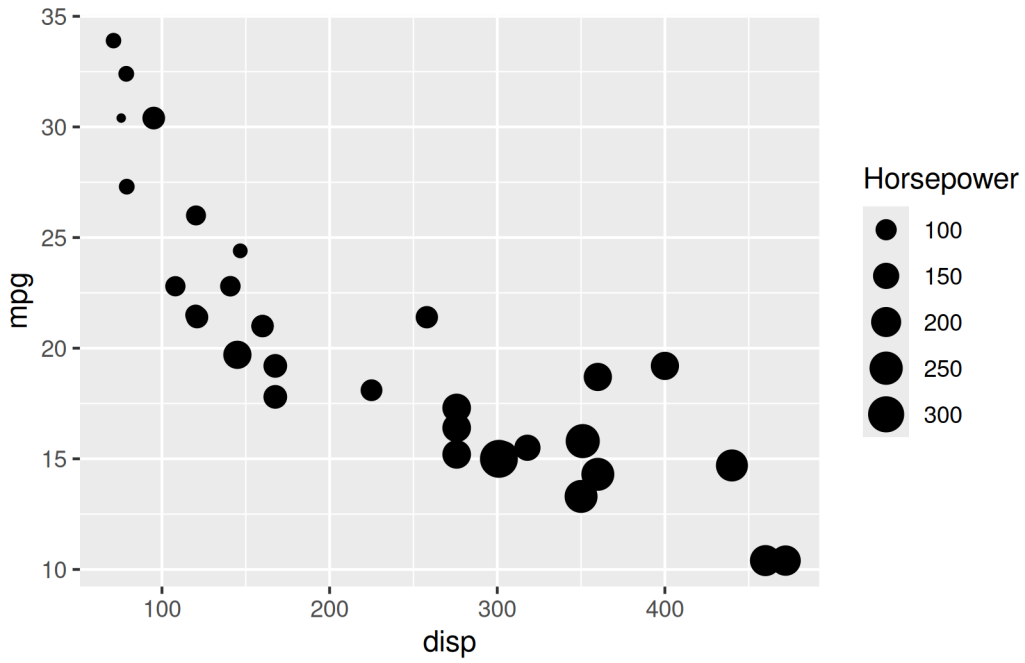


As you can see, the legend acts as a guide for the `size` aesthetic. Now, let us learn to modify the different aspects of the legend.

How to change the size legend title in ggplot2

The title of the legend (`hp`) is not very intuitive. If the user does not know the underlying data, they will not be able to make any sense out of it. Let us change it to `Horsepower` using the `name` argument.

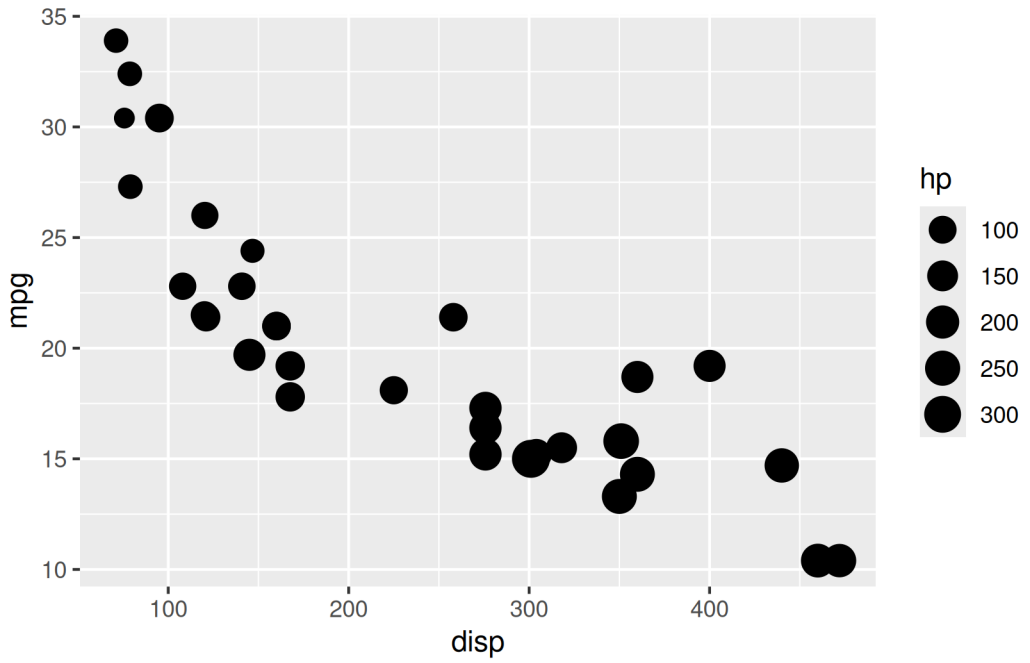
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(name = "Horsepower")
```



How to change the legend size range in ggplot2

The range of the size of points can be modified using the `range` argument. We need to specify a lower and upper range using a numeric vector. In the below example, we use `range` and supply the lower and upper limits as `3` and `6`. The size of the points will now lie between `3` and `6` only.

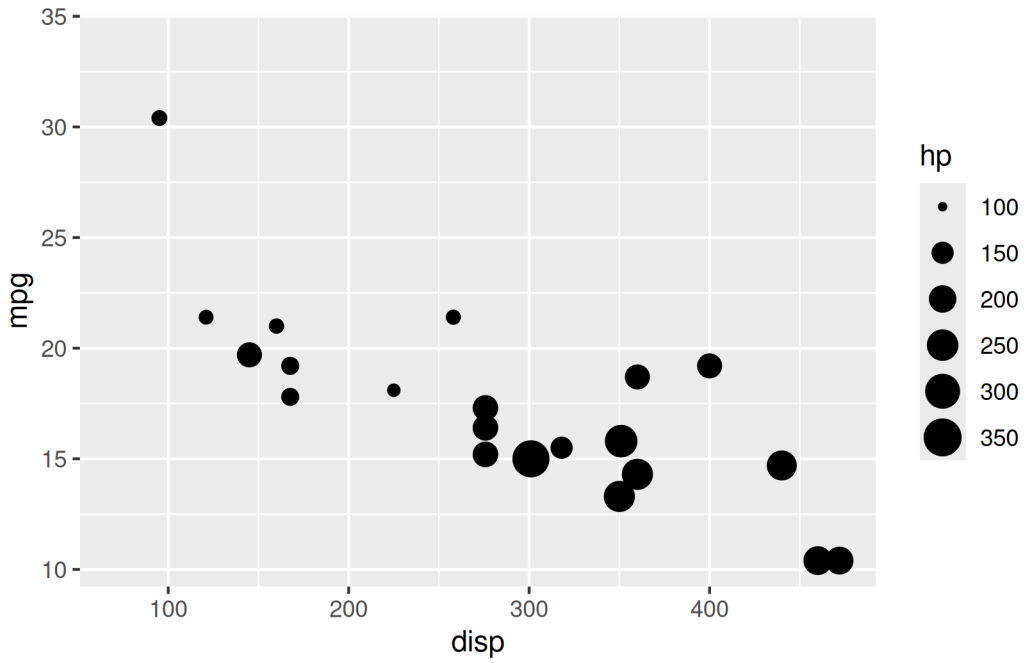
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(range = c(3, 6))
```



How to change size legend limits in ggplot2

Let us assume that we want to modify the data to be displayed i.e. instead of examining the relationship between mileage and displacement for all cars, we desire to look at only cars whose horsepower is between `100` and `350`. One way to approach this would be to filter the data using `filter` from dplyr and then visualize it. Instead, we will use the `limits` argument and filter the data for visualization.

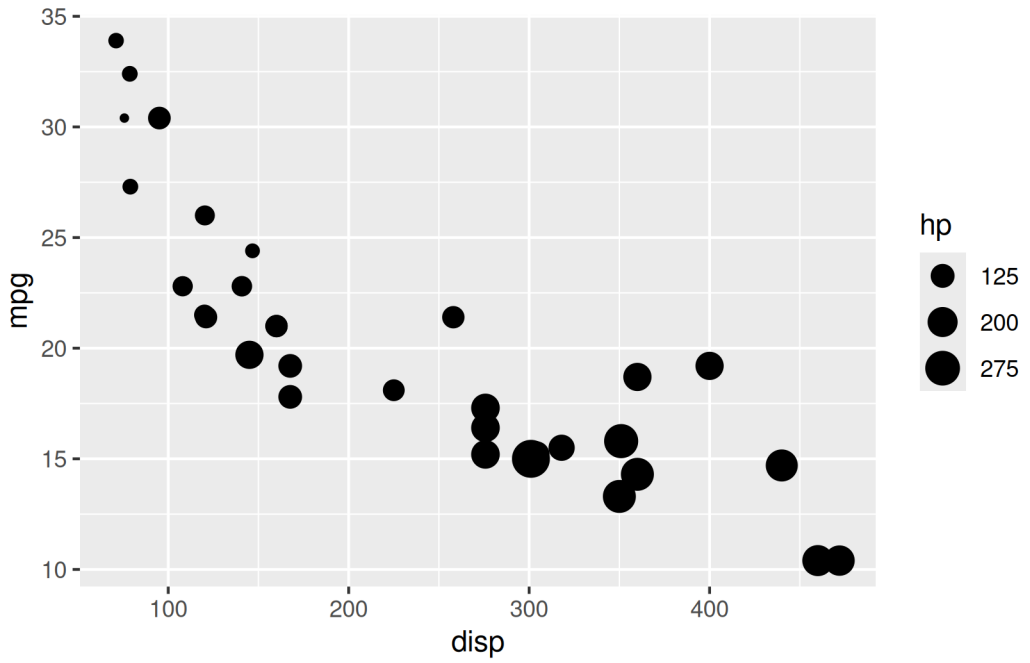
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(limits = c(100, 350))
```



How to reorder or hide size legend items in ggplot2

When the range of the variable mapped to size is large, you may not want the labels in the legend to represent all of them. In such cases, we can use the breaks argument and specify the labels to be used. In the below case, we use the `breaks` argument to ensure that the labels in legend represent certain midpoints (125, 200, 275) of the mapped variable.

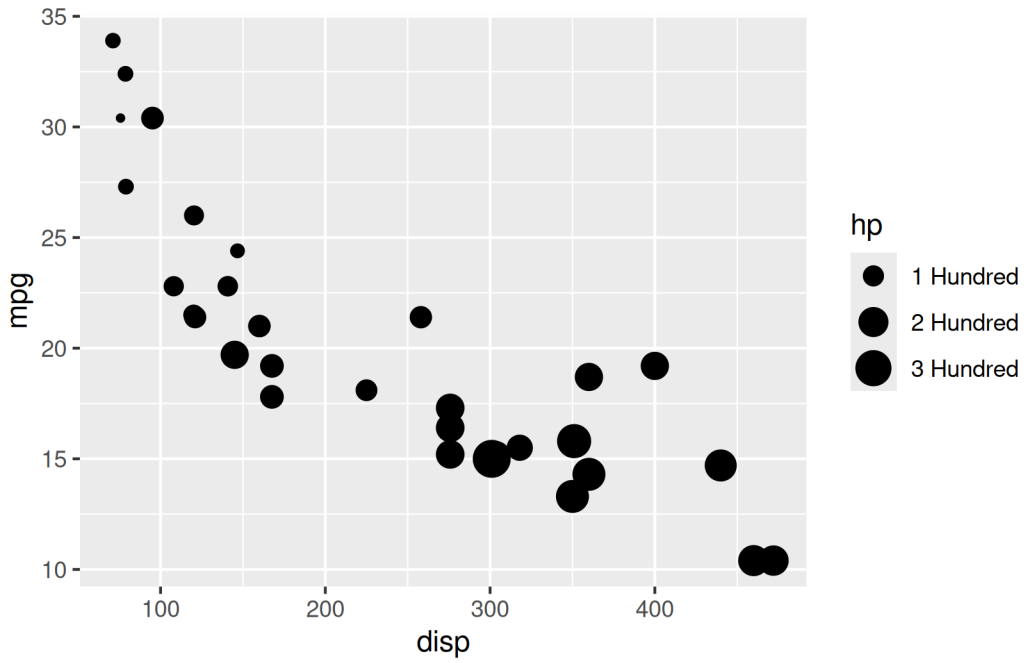
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(breaks = c(125, 200, 275))
```



How to change size legend labels in ggplot2

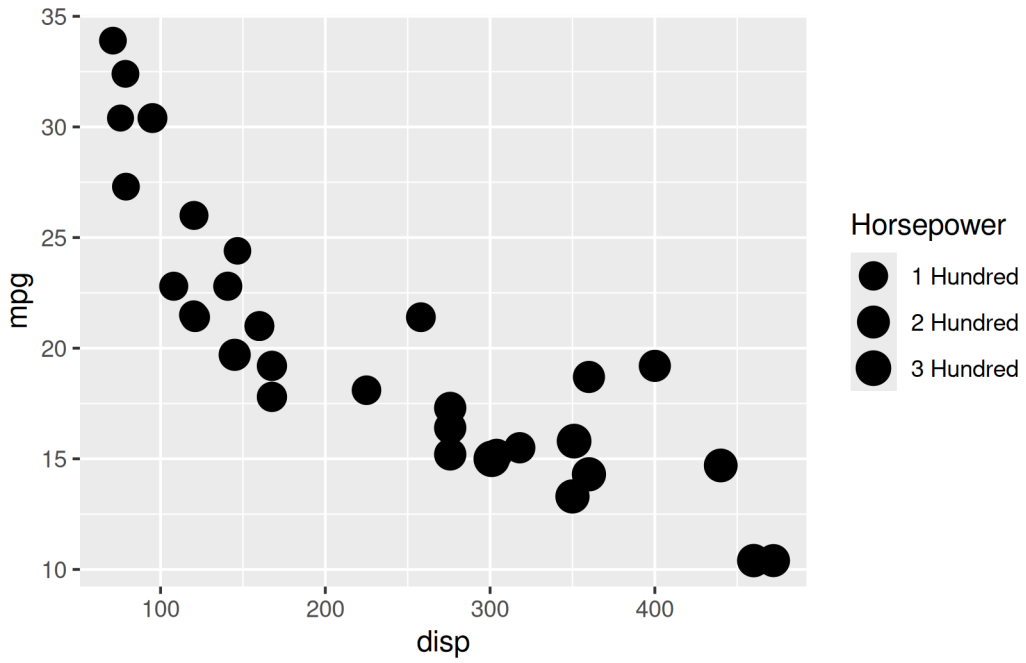
The labels in the legend can be modified using the `labels` argument. Let us change the labels to "1 Hundred", "2 Hundred" and "3 Hundred" in the next example. Ensure that the labels are intuitive and easy to interpret for the end user of the plot.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(breaks = c(100, 200, 300),
    labels = c("1 Hundred", "2 Hundred", "3 Hundred"))
```



How to combine size legend options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, size = hp)) +
  scale_size_continuous(name = "Horsepower", range = c(3, 6),
    limits = c(0, 400), breaks = c(100, 200, 300),
    labels = c("1 Hundred", "2 Hundred", "3 Hundred"))
```



14.3 How to change legend transparency in ggplot2

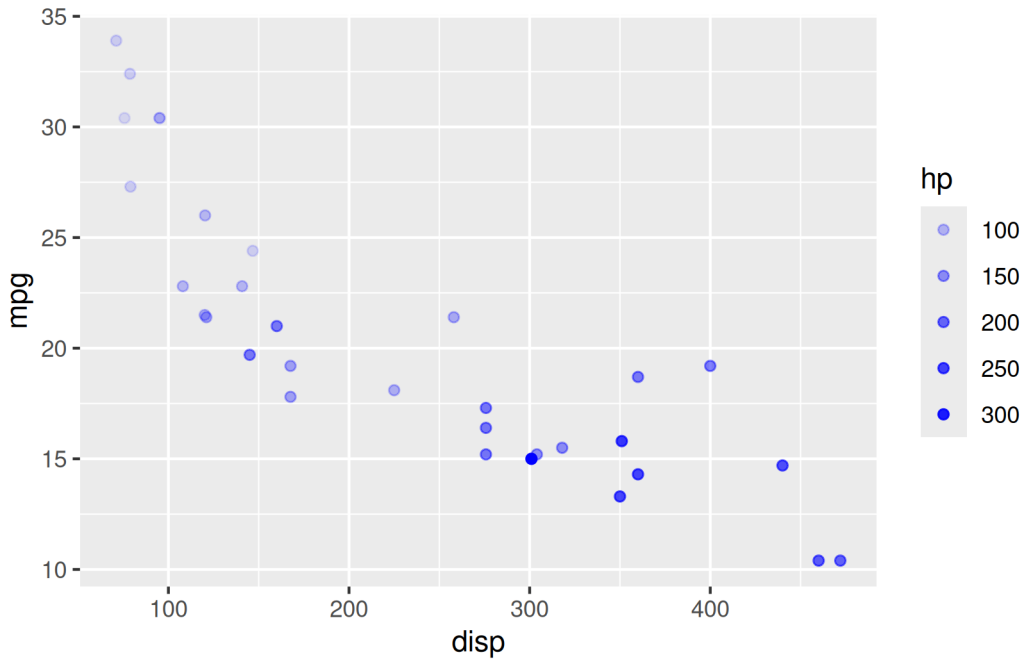
We will learn to modify the following using `scale_alpha_continuous()` when `alpha` or transparency is mapped to variables:

- title
- breaks
- limits
- range
- labels
- values

How to create a plot with an alpha legend in ggplot2

Let us start with a scatter plot examining the relationship between displacement and miles per gallon from the mtcars data set. We will map the transparency of the points to the `hp` variable. Remember, `alpha` must always be mapped to a continuous variable.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue')
```

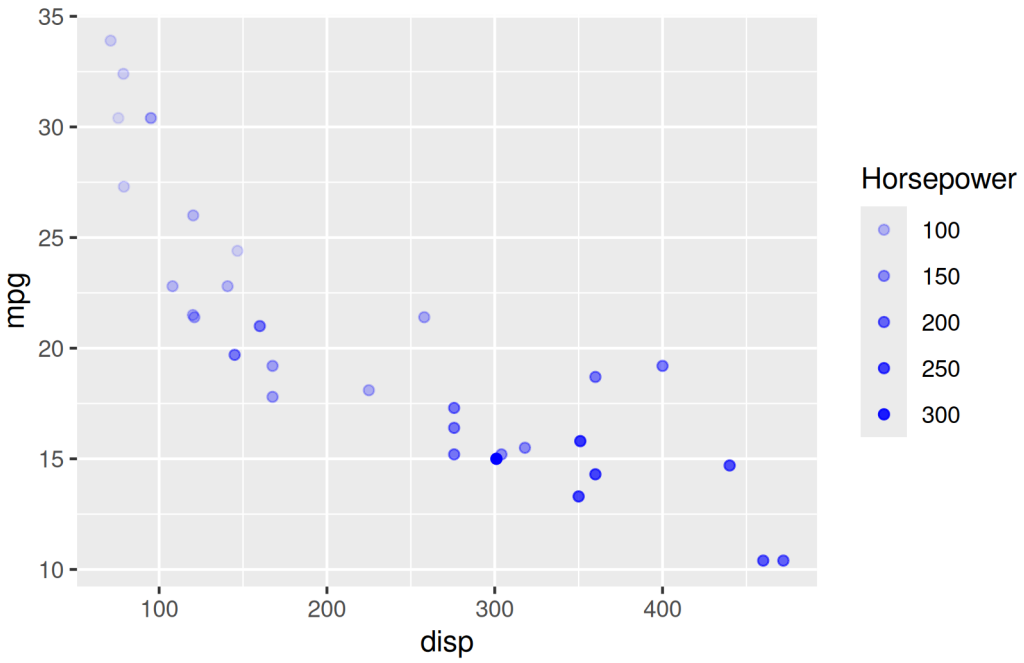


As you can see, the legend acts as a guide for the `alpha` aesthetic. Now, let us learn to modify the different aspects of the legend.

How to change the transparency legend title in ggplot2

The title of the legend (`hp`) is not very intuitive. If the user does not know the underlying data, they will not be able to make any sense out of it. Let us change it to `Horsepower` using the `name` argument.

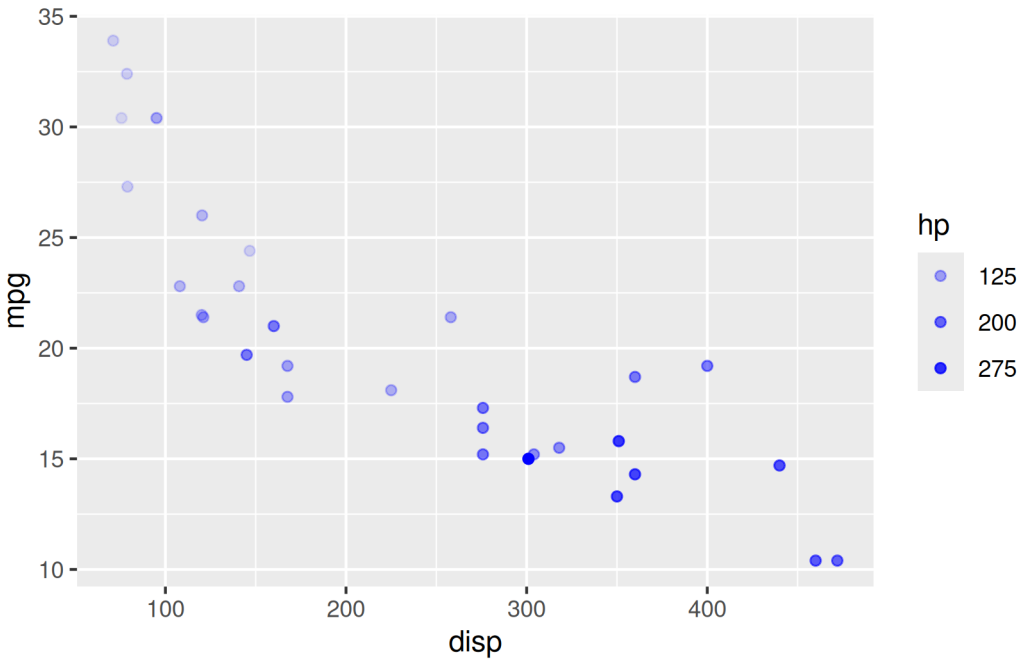
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous("Horsepower")
```



How to reorder or hide alpha legend items in ggplot2

When the range of the variable mapped to size is large, you may not want the labels in the legend to represent all of them. In such cases, we can use the breaks argument and specify the labels to be used. In the below case, we use the `breaks` argument to ensure that the labels in legend represent certain midpoints (125, 200, 275) of the mapped variable.

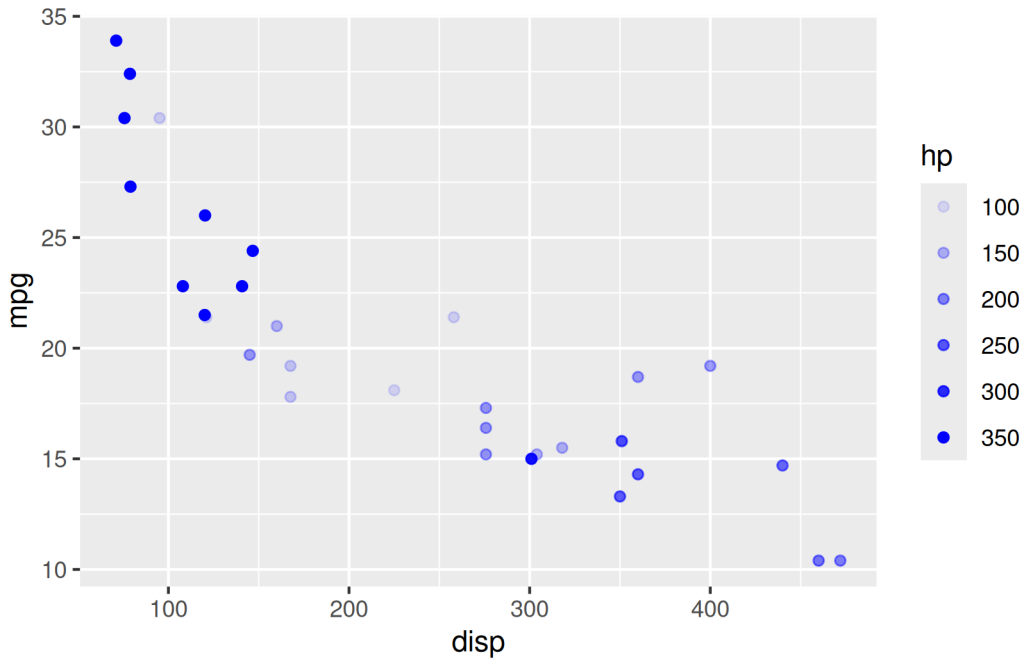
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous(breaks = c(125, 200, 275))
```



How to change transparency legend limits in ggplot2

Let us assume that we want to modify the data to be displayed i.e. instead of examining the relationship between mileage and displacement for all cars, we desire to look at only cars whose horsepower is between 100 and 350. One way to approach this would be to filter the data using `filter` from dplyr and then visualize it. Instead, we will use the `limits` argument and filter the data for visualization.

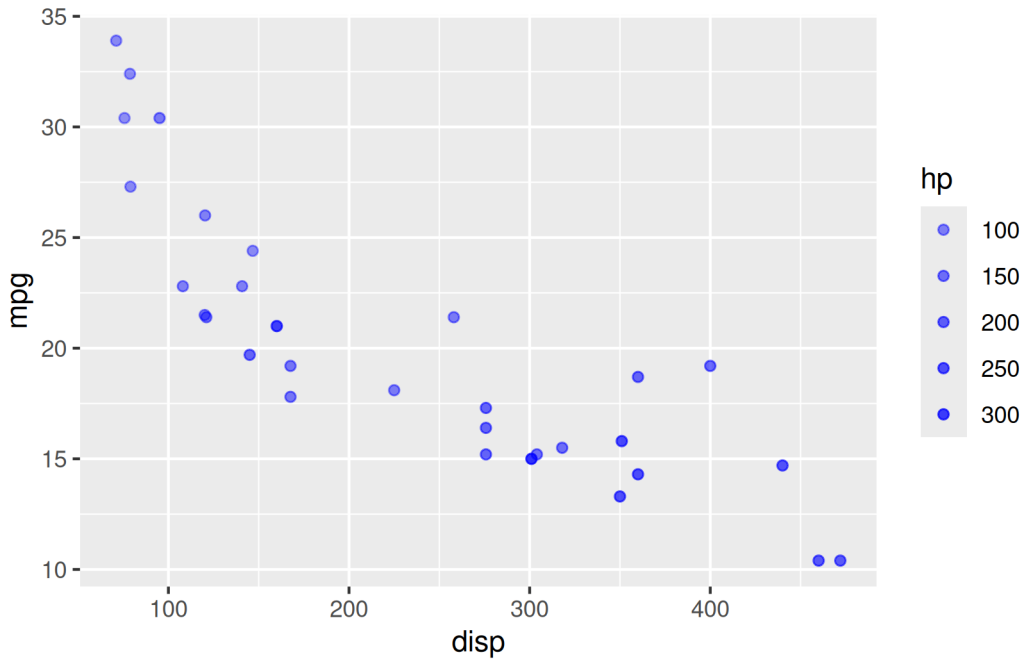
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous(limits = c(100, 350))
```



How to change the legend transparency range in ggplot2

The range of the transparency of points can be modified using the `range` argument. We need to specify a lower and upper range using a numeric vector. In the below example, we use `range` and supply the lower and upper limits as `0.4` and `0.8`. The transparency of the points will now lie between `0.4` and `0.8` only.

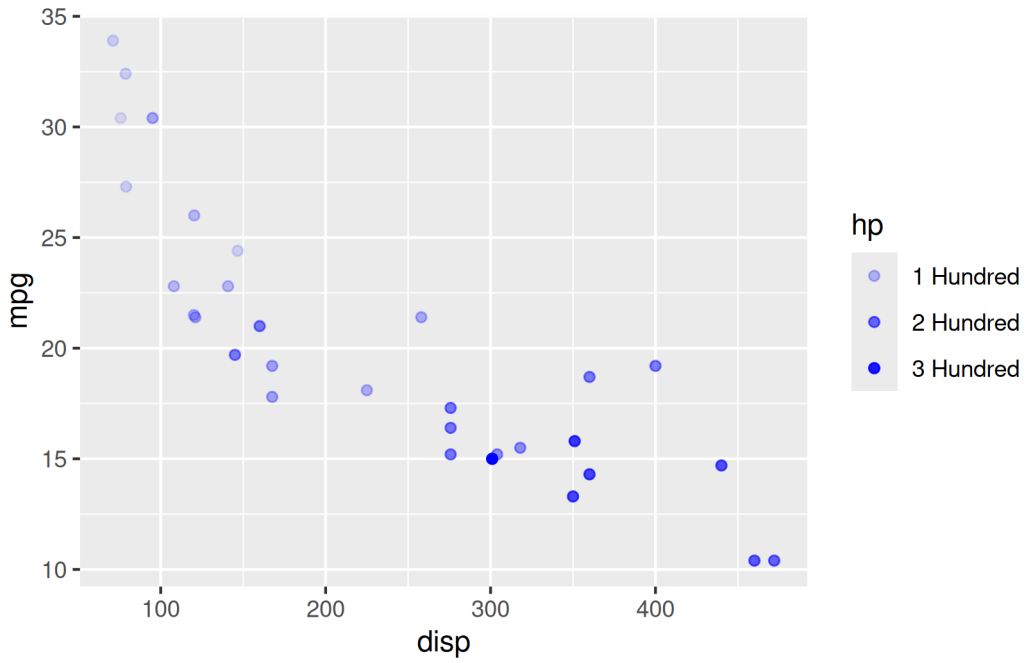
```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous(range = c(0.4, 0.8))
```



How to change transparency legend labels in ggplot2

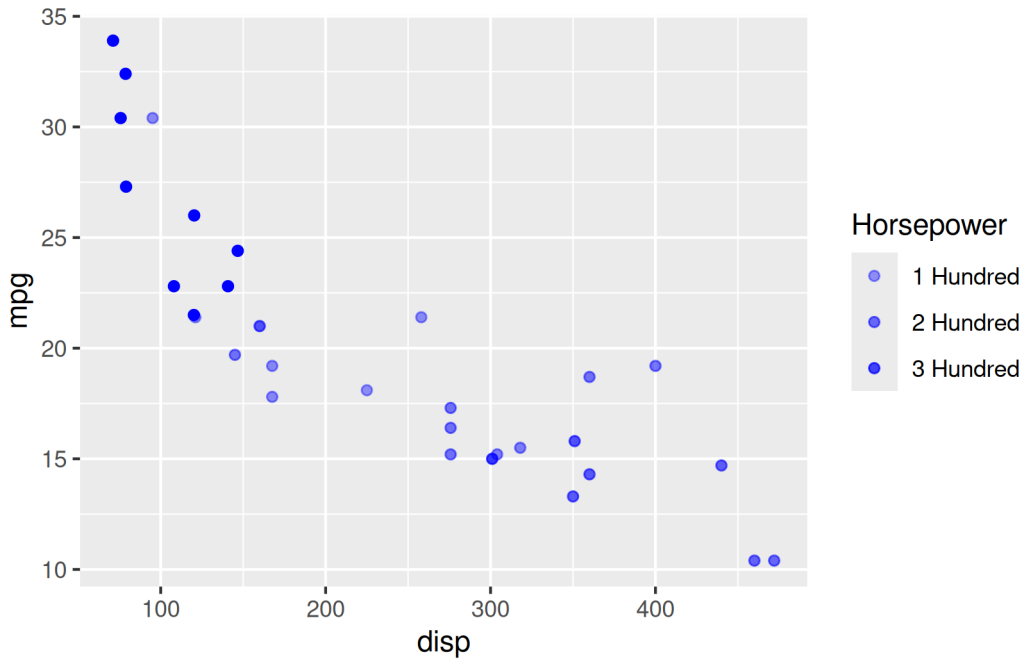
The labels in the legend can be modified using the `labels` argument. Let us change the labels to "1 Hundred", "2 Hundred" and "3 Hundred" in the next example. Ensure that the labels are intuitive and easy to interpret for the end user of the plot.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous(breaks = c(100, 200, 300),
    labels = c("1 Hundred", "2 Hundred",
      "3 Hundred"))
```



How to combine transparency legend options in ggplot2

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, alpha = hp), color = 'blue') +
  scale_alpha_continuous("Horsepower", breaks = c(100, 200, 300),
    limits = c(100, 350), range = c(0.4, 0.8),
    labels = c("1 Hundred", "2 Hundred", "3 Hundred"))
```



14.4 Exercises

1. Map `shape` to cylinders with `scale_shape_manual(values = ...)`.
2. Control point spread with `scale_size_continuous(range = ...)`.
3. Explain why `size`/`alpha` must map to continuous variables.

Worked solutions in `solutions/legend-shape-size-alpha.md` (attempt first).

Where to go next

- ggplot2 book scales and guides chapters

15 Faceting

15.1 Introduction

> **Why this chart**
>
> Facets split one plot into small multiples sharing scales, so group differences are comparable.

> **Common pitfall**
>
> Too many facet levels shrink each panel past readability; filter or wrap sensibly.

In this chapter, we will learn about faceting i.e. combining plots.

Let us continue with the scatter plot examining the relationship between displacement and miles per gallon but let us make one additional change. We now want 3 sub plots for each type of cylinder. How can we do this? We can split or group the data by cylinder type and plot the subset of data which means dealing with 3 different data sets, plotting 3 plots and arranging them for comparison. ggplot2 offers the following 2 functions which allow us to plot subset of data with a simple formula based interface:

- `facet_grid()`
- `facet_wrap()`

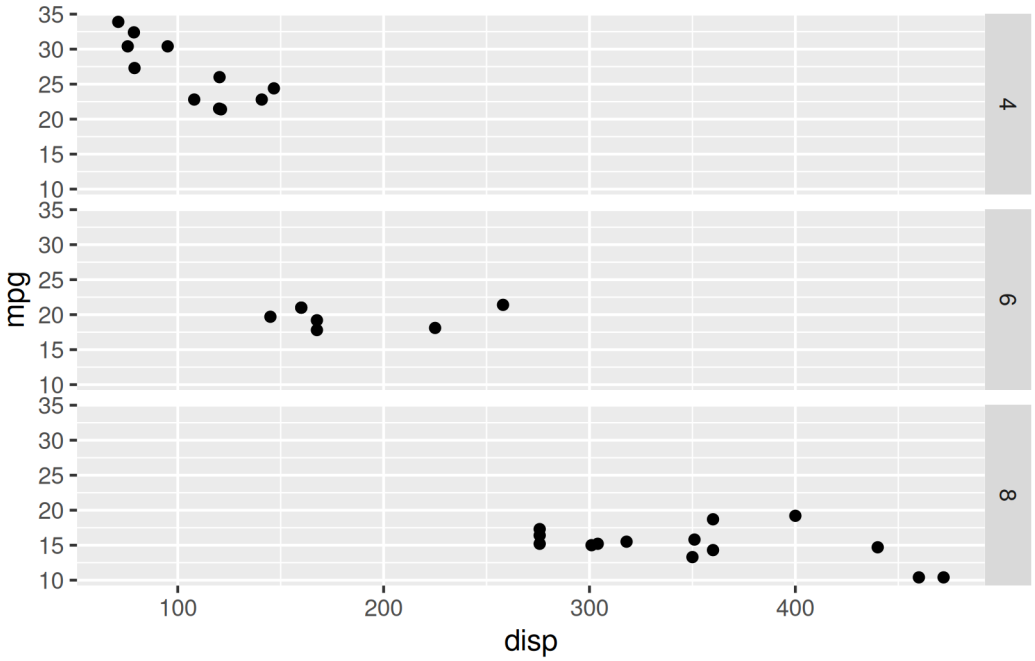
Faceting allows us to create multiple sub plots. It partitions a plot into a matrix of panels with each panel showing a different subset of data.

15.2 Arrange facets in a grid

15.2.1 Stack facets vertically

`facet_grid()` allows us to split up the data by one or two discrete variables and create sub plots. The sub plots can be arranged horizontally or vertically using a formula of the form `vertical ~ horizontal`. In the below example, 3 sub plots are created, one each for the levels of the `cyl` variable and the sub plots are arranged vertically

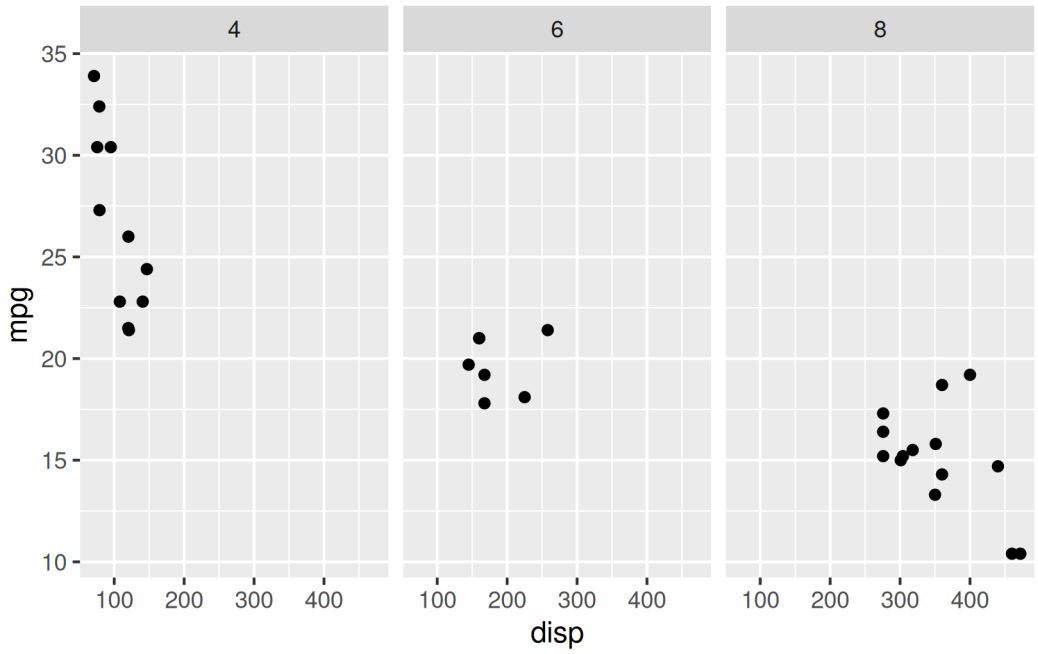
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_grid(cyl ~ .)
```



15.2.2 Arrange facets horizontally

Below we reproduce the previous example but arrange the sub plots horizontally.

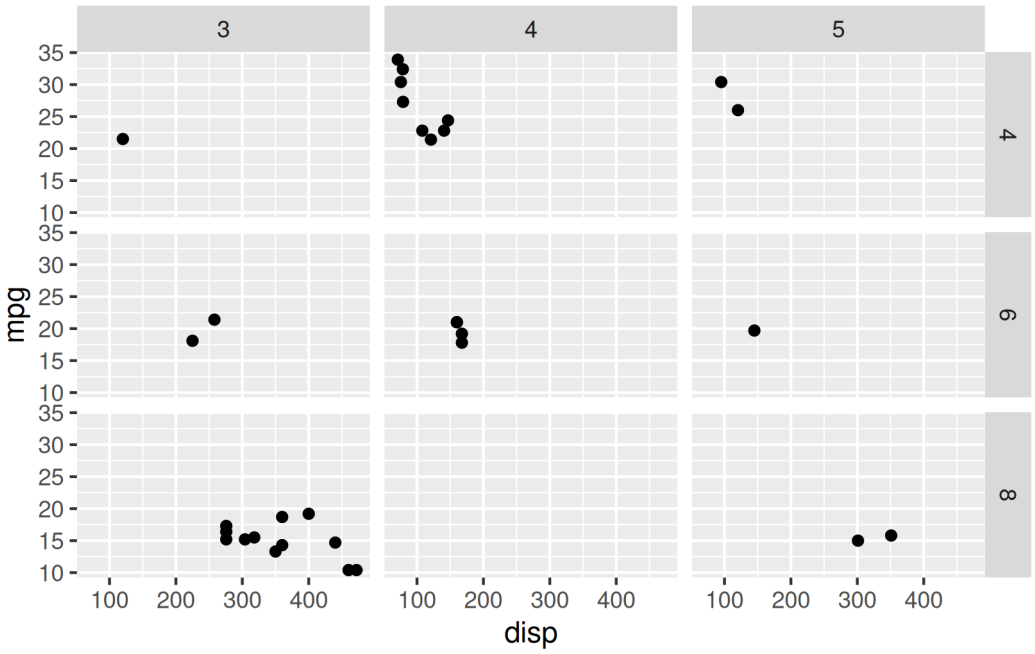
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_grid(. ~ cyl)
```



15.2.3 Facet rows and columns together

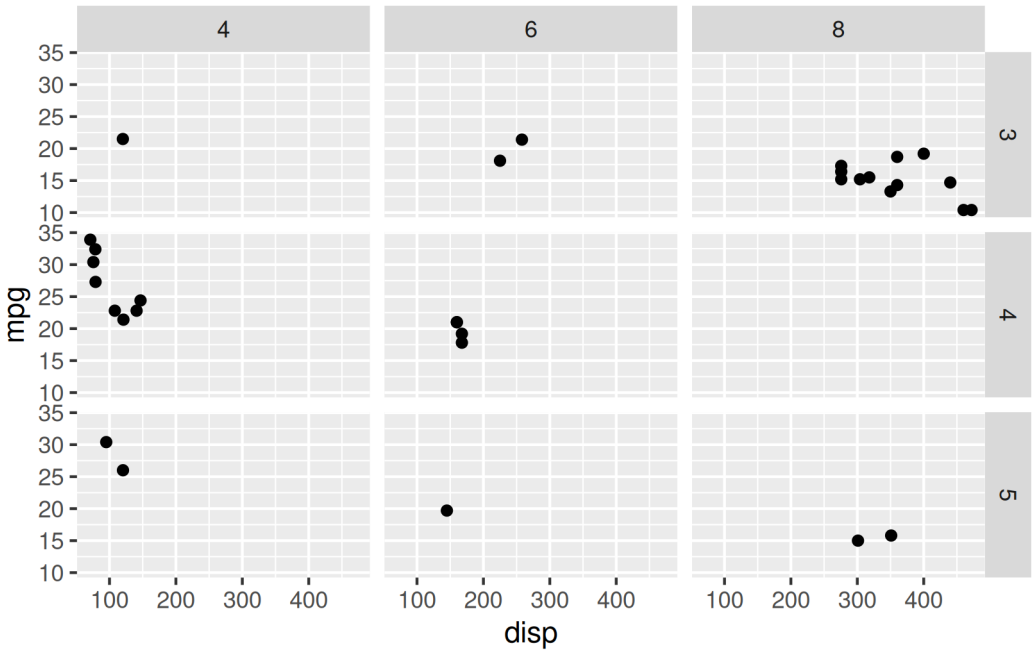
In certain cases, we might want different discrete variables to represent the horizontal and vertical direction. In the below example, we examine the relationship between displacement and miles per gallon for different combinations of `cyl` and `gear` variables.

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_grid(cyl ~ gear)
```



Below, we switch the variables representing the vertical and horizontal directions.

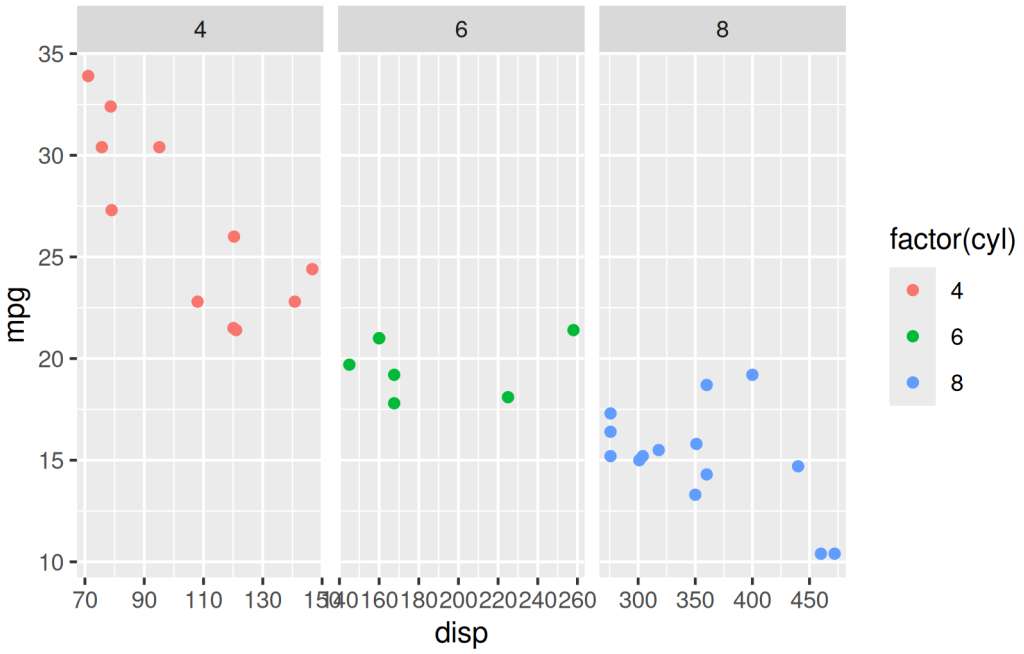
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_grid(gear ~ cyl)
```



15.2.4 Use free scales in a facet grid

If you carefully observe the second example, the range of `X` axis is same for all the 3 sub plots i.e. it is a fixed range. You can allow each of the sub plots to have different range using the `scales` argument and supplying it the value `'free'`.

```
ggplot(mtcars, aes(disp, mpg, color = factor(cyl))) +
  geom_point() +
  facet_grid(. ~ cyl, scales = "free")
```

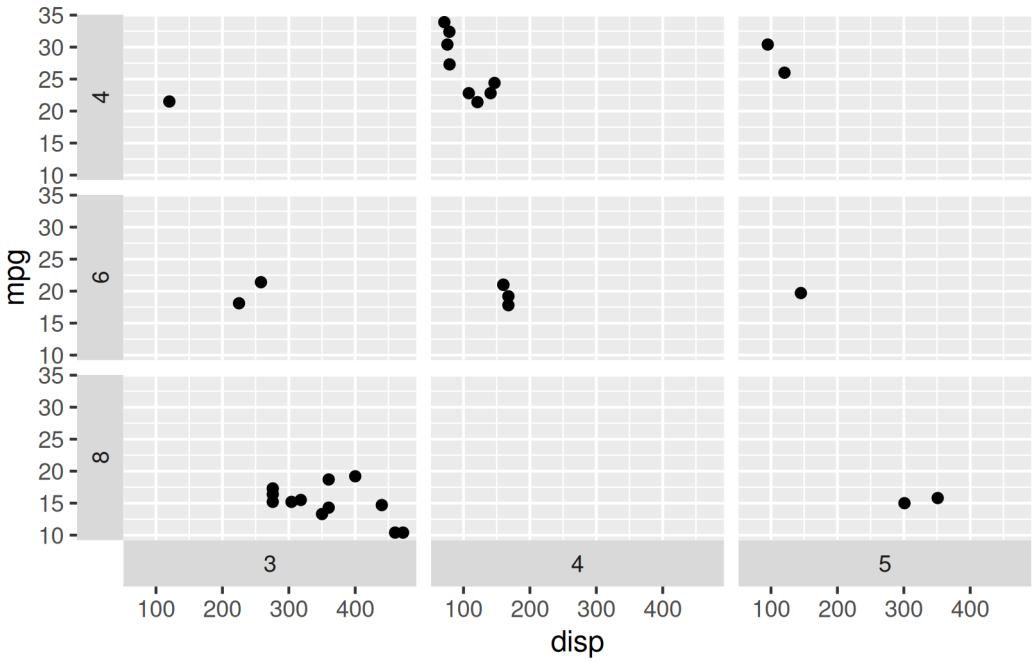


Now, each of the sub plot has a different range.

15.2.5 Move facet labels with switch

In the third example, the labels are displayed at the bottom for `X` axis and at the right for the `Y` axis. It can be changed using the `switch` argument and supplying the value `'both'`. The labels will now be displayed at the top for the `X` axis and at left for the `Y` axis. If you just want to change the labels for a particular axis, use the values `x` and `y` for the `X` and `Y` axis respectively.

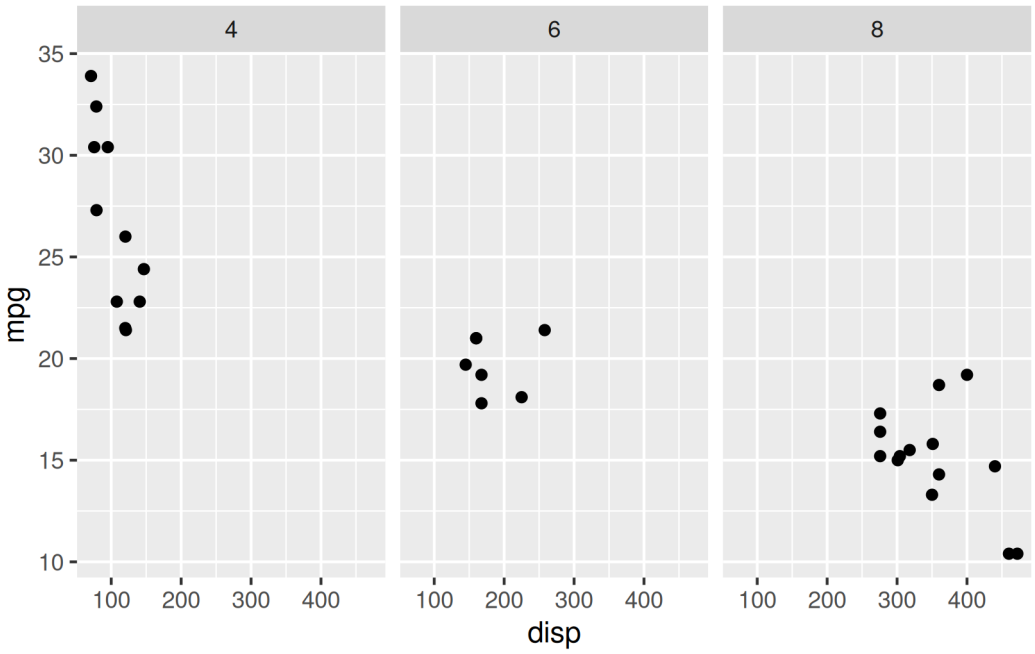
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_grid(cyl ~ gear, switch = "both")
```



15.3 Wrap facets into panels

`facet_wrap()` allows us to arrange sub plots in a certain number of rows and columns. In the below example, we will use `facet_wrap()` to arrange the sub plots in a single row.

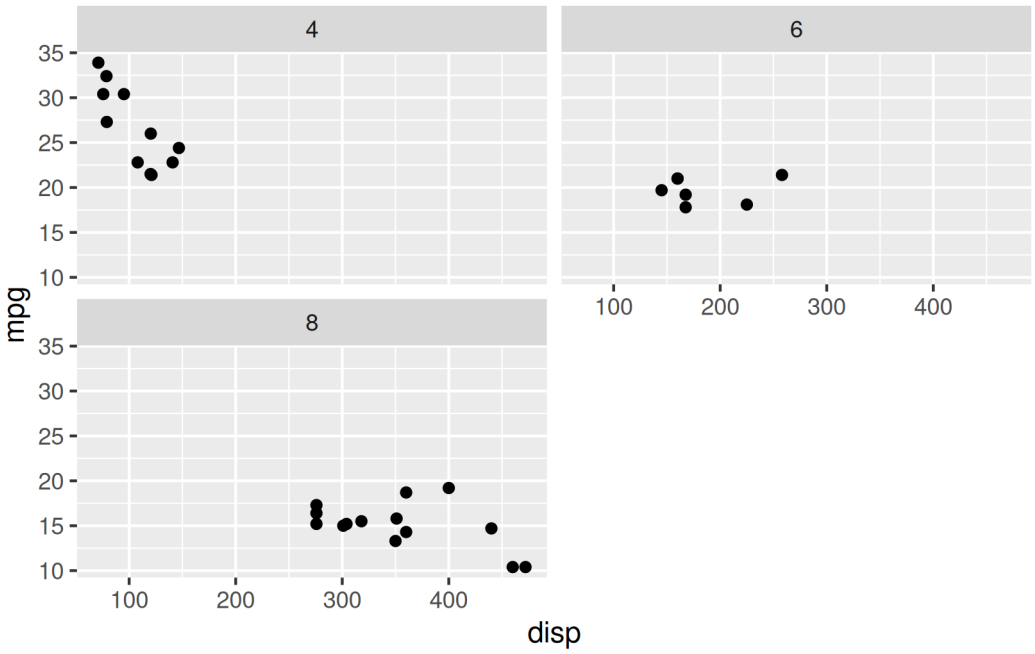
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(~cyl)
```



15.3.1 Set facet rows

To arrange the sub plots in a specific number of rows, use the `nrow` argument. In the below example, we arrange the sub plots in 2 rows.

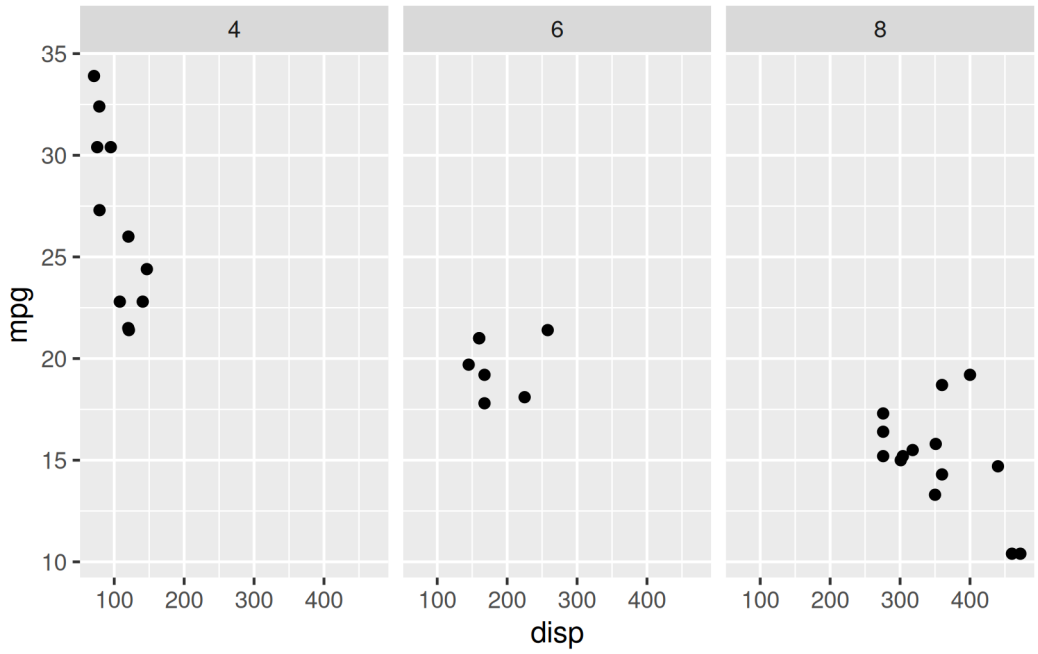
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(~cyl, nrow = 2)
```



15.3.2 Set facet columns

Here, we arrange the sub plots in 3 columns instead of rows using the `ncol` argument.

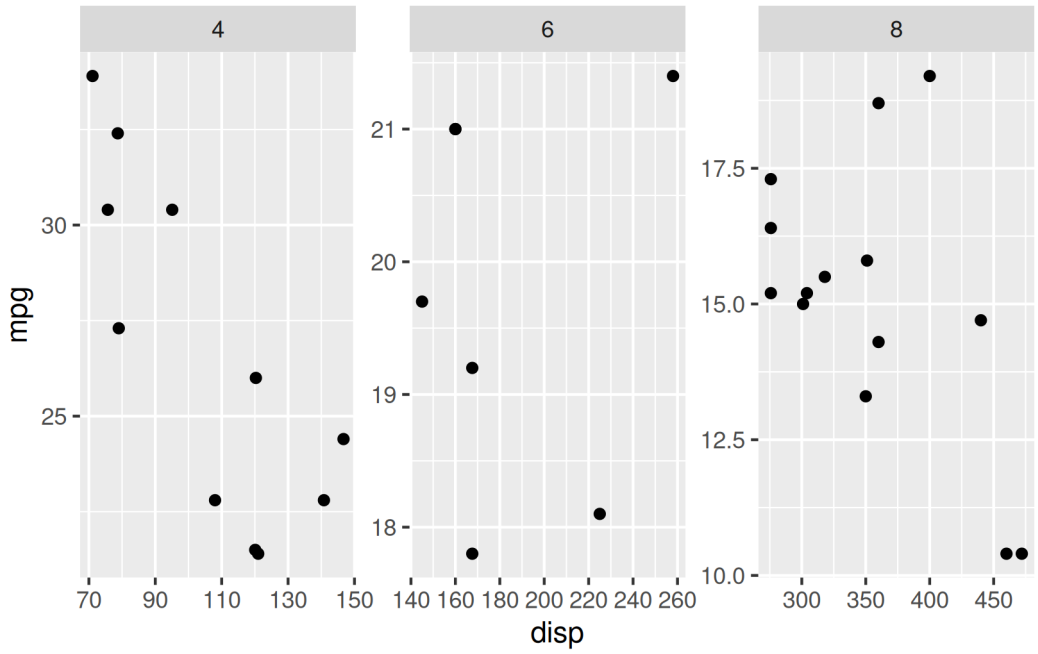
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(~cyl, ncol = 3)
```



15.3.3 Use free scales in wrapped facets

You can allow each of the sub plots to have different range using the `scales` argument and supplying it the value `'free'`.

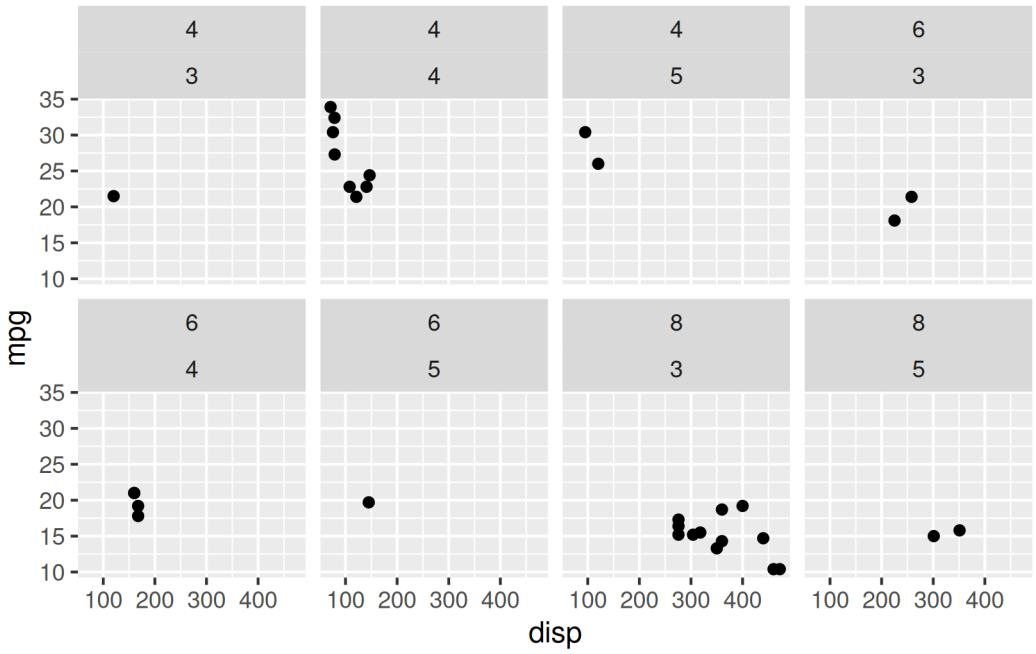
```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(~cyl, scales = "free")
```



15.3.4 Set facet rows and columns

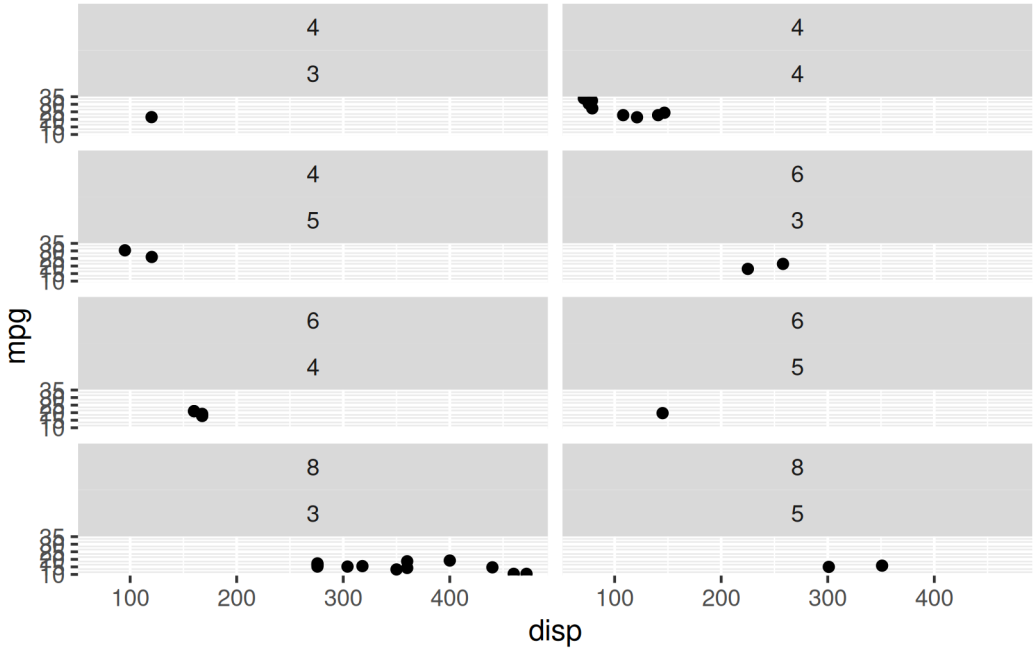
If 2 discrete variables are used to create the sub plots, we can either use the formula interface to specify the variables as shown below

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(~cyl + gear, nrow = 2)
```



or use a character vector of variable names.

```
ggplot(mtcars, aes(disp, mpg)) +
  geom_point() +
  facet_wrap(c("cyl", "gear"), ncol = 2)
```



15.4 Exercises

1. Facet `disp` vs `mpg` by cylinders with `facet_wrap()`.
2. Grid `gear` rows against `carb` columns with `facet_grid()`.
3. Decide when facets beat color grouping and justify it.

Worked solutions in `solutions/facets.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

16 Themes

16.1 Introduction

> **Why this chart**
>
> Themes take a correct plot to publication-ready: fonts, gridlines and background tuned once, reused everywhere.

> **Common pitfall**
>
> Styling before the data mapping is right wastes effort; get geoms and scales correct first.

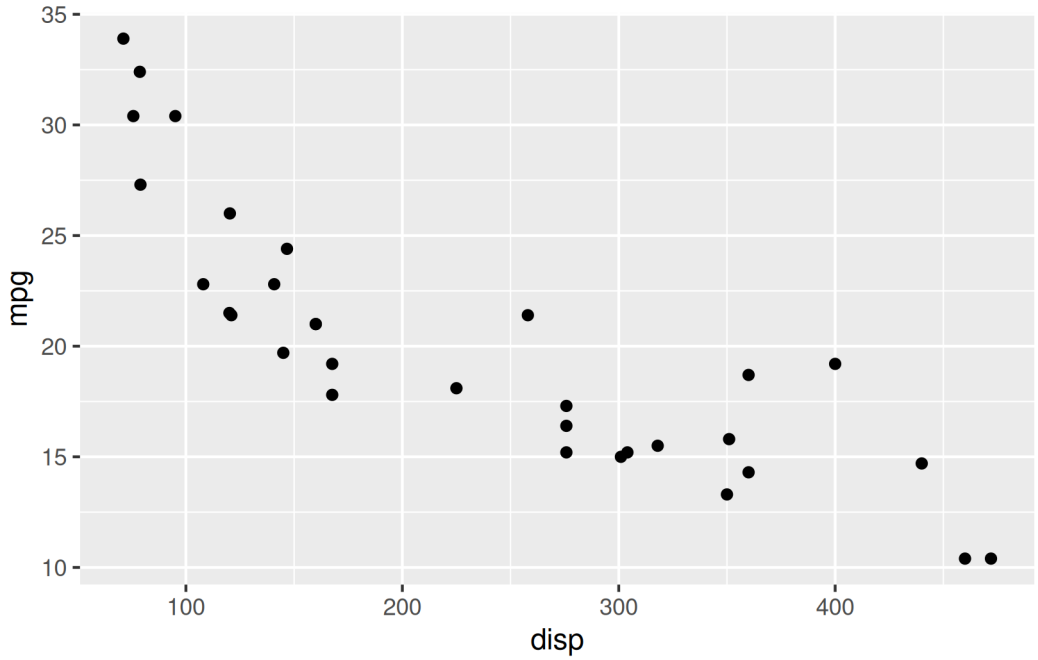
In this final chapter, we will learn to modify the appearance of all non data components of the plot such as:

- axis
- legend
- panel
- plot area
- background
- margin
- facets

16.2 Basic Plot

We will continue with the scatter plot examining the relationship between displacement and miles per gallon from the the mtcars data set.

```
p <- ggplot(mtcars) +
  geom_point(aes(disp, mpg))
p
```



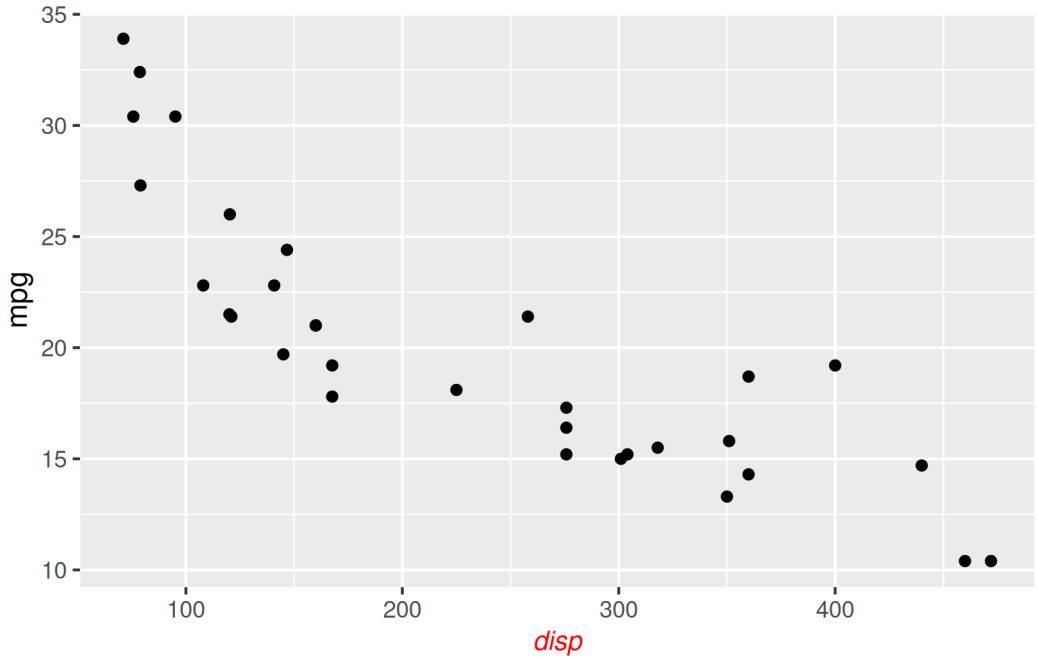
16.3 Style plot axes

16.3.1 Style axis text

The `axis.title.x` argument can be used to modify the appearance of the `X` axis. In the below example, we modify the color and size of the title using the `element_text()` function. Remember, whenever you are trying to modify the appearance of a theme element which is a text, you must use `element_text()`.

You can use `axis.title.y` to modify the `Y` axis title and to modify the title of both the axis together, use `axis.title`.

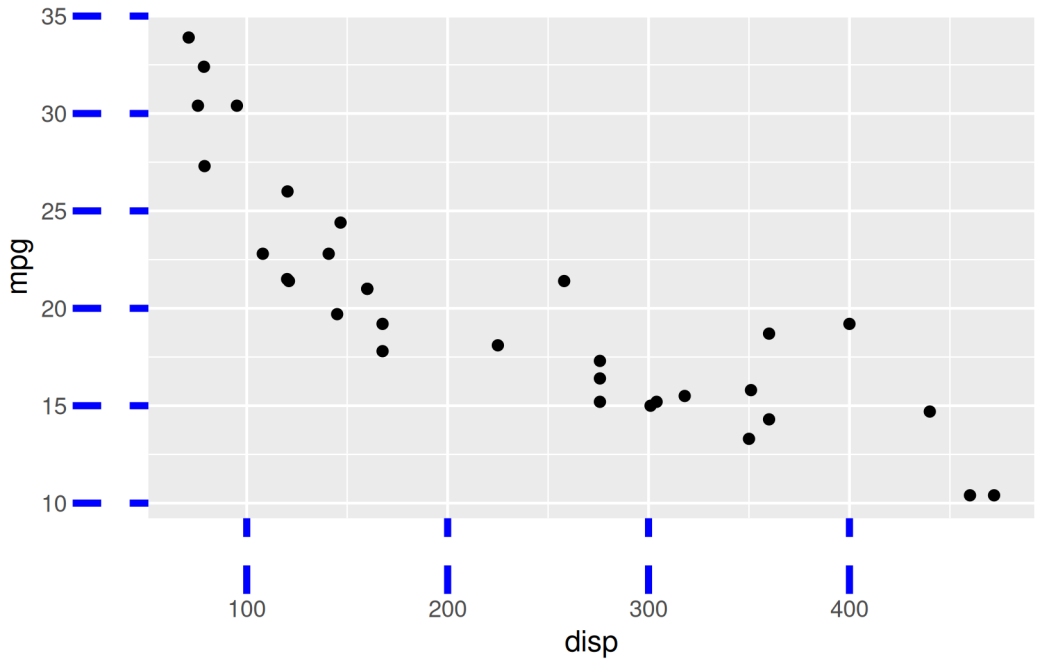
```
p + theme(axis.title.x = element_text(color = "red", size = 10, face = "italic"))
```



16.3.2 Style axis ticks

To modify the appearance of the axis ticks, use the `axis.ticks` argument. You can change the color, linewidth, linetype and length of the ticks using the `element_line()` function as shown below.

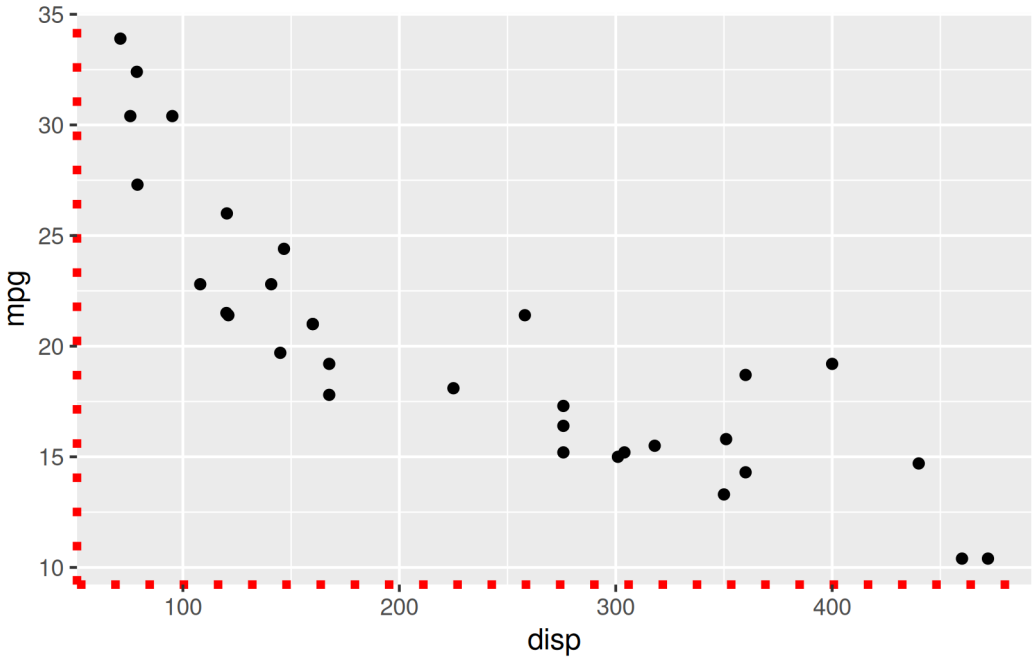
```
p + theme(axis.ticks = element_line(color = 'blue', linewidth = 1.25, linetype = 2),
          axis.ticks.length = unit(1, "cm"))
```



16.3.3 Style axis lines

The `axis.line` argument should be used to modify the appearance of the axis lines. You can change the color, linewidth and linetype of the line using the `element_line()` function.

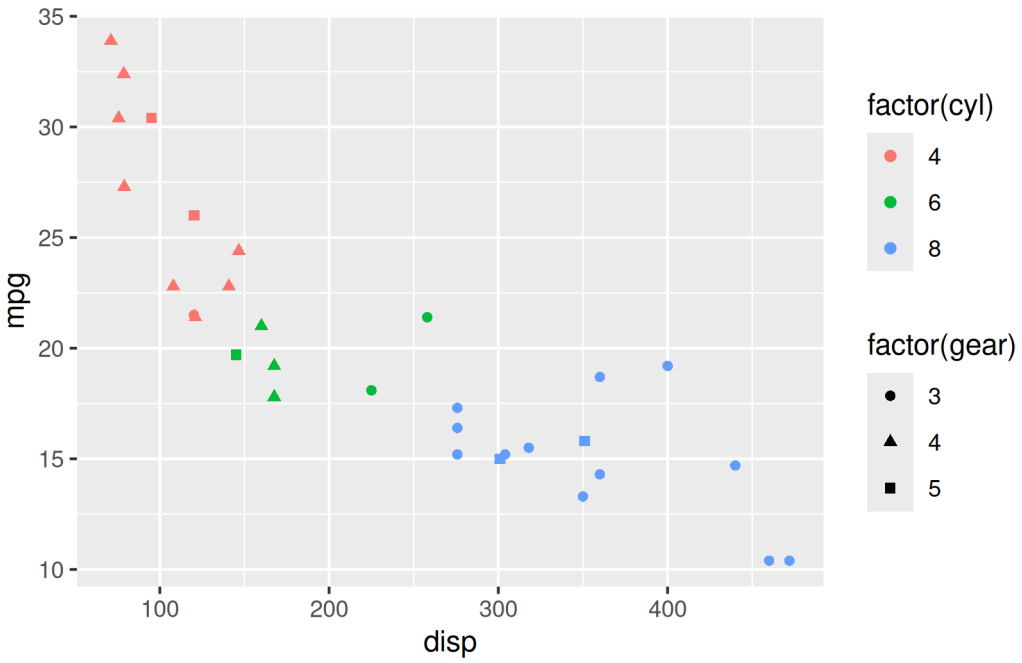
```
p + theme(axis.line = element_line(color = 'red', linewidth = 1.5, linetype = 3))
```



16.4 Style the legend

Now, let us look at modifying the non-data components of a legend.

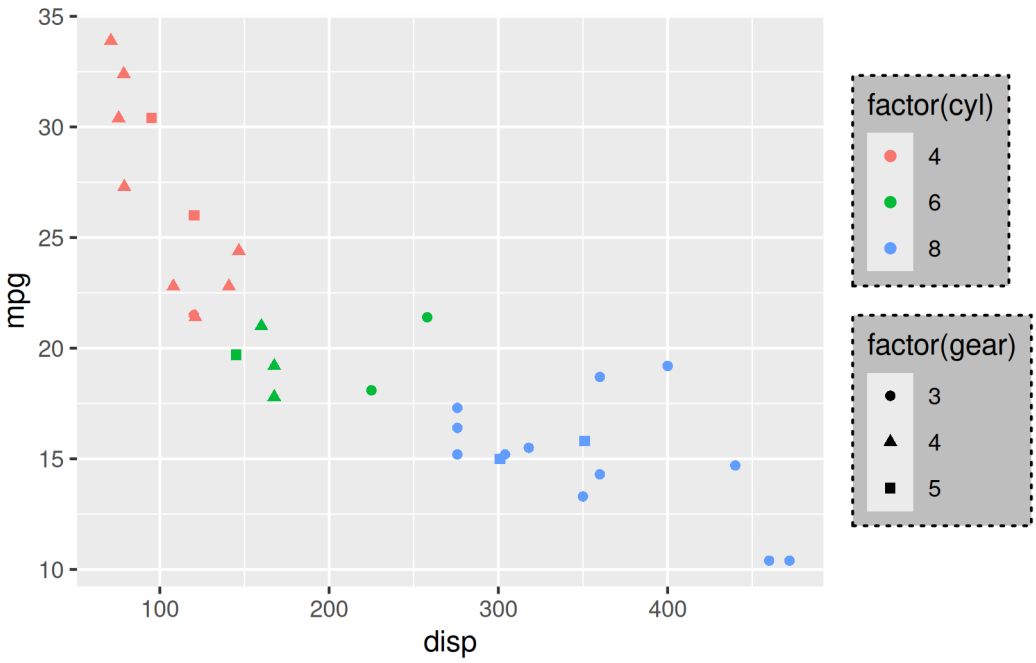
```
p <- ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl), shape = factor(gear)))
p
```



16.4.1 Style the legend background

The background of the legend can be modified using the `legend.background` argument. You can change the background color, the border color and line type using `element_rect()`.

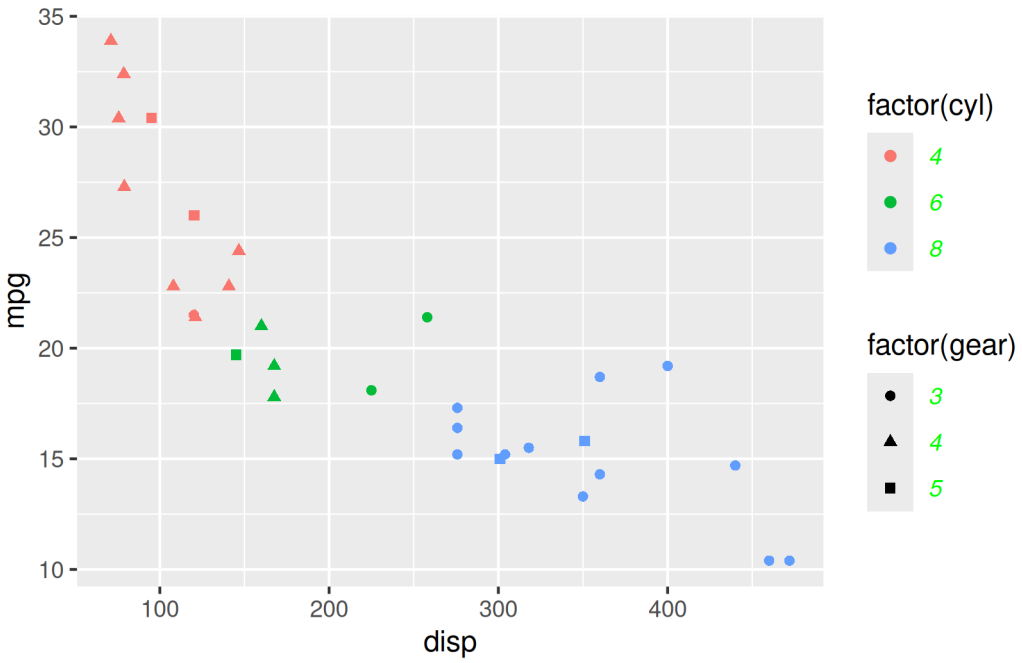
```
p + theme(legend.background = element_rect(fill = 'gray', linetype = 3,
          color = "black"))
```



16.4.2 Style legend text

The appearance of the text can be modified using the `legend.text` argument. You can change the color, size and font using the `element_text()` function.

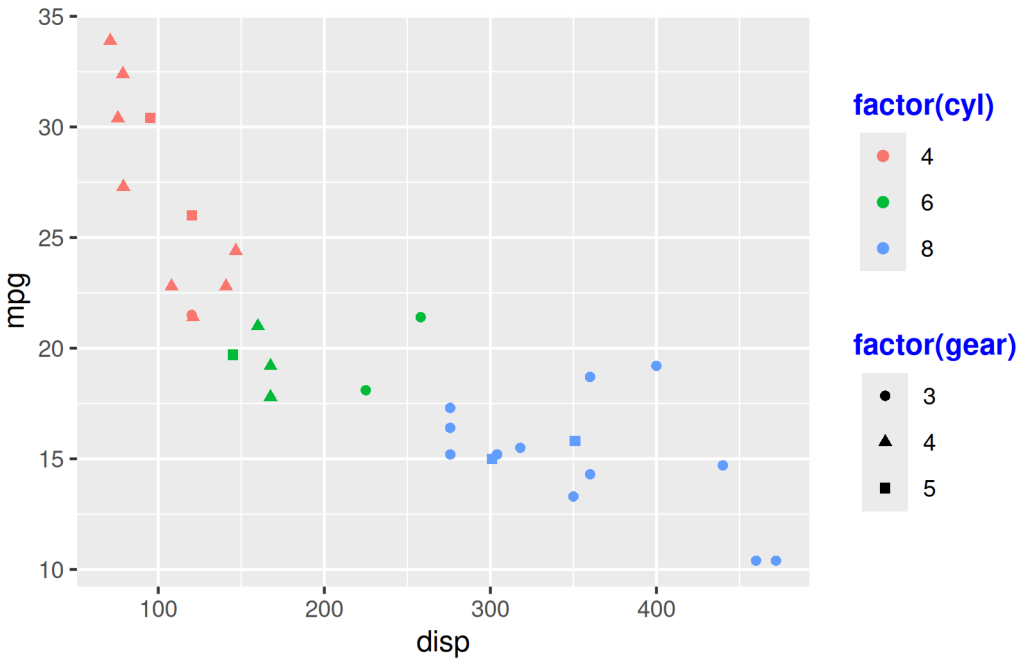
```
p + theme(legend.text = element_text(color = 'green', face = 'italic'))
```



16.4.3 Style the legend title

The appearance of the title of the legend can be modified using the `legend.title` argument. You can change the color, size, font and alignment using `element_text()`.

```
p + theme(legend.title = element_text(color = 'blue', face = 'bold', hjust = 0.1))
```

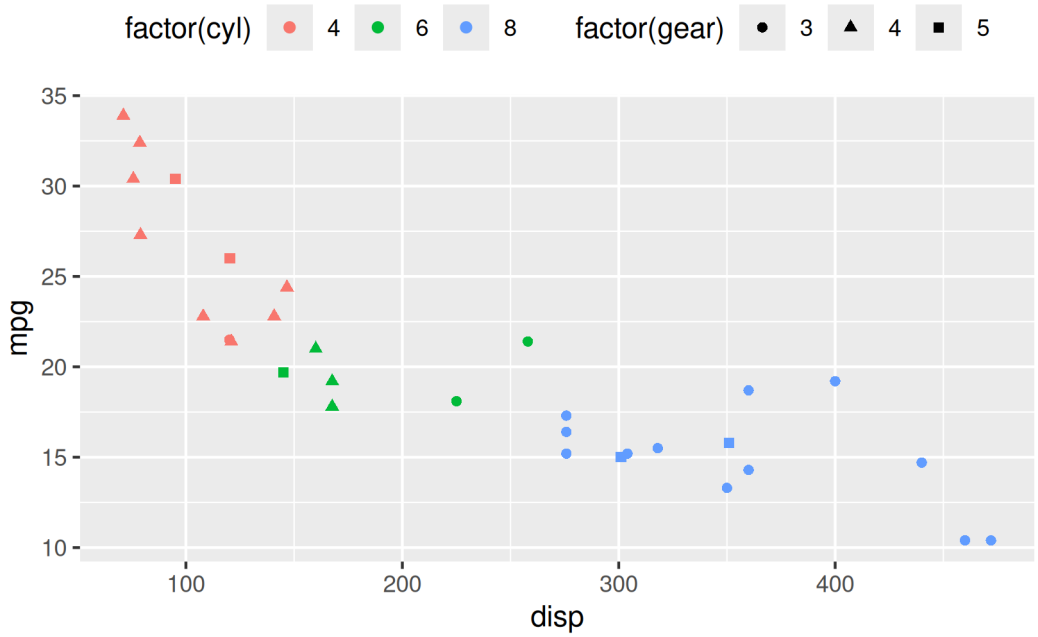


16.4.4 Move the legend position

The position and direction of the legend can be changed using:

- `legend.position`
- and `legend.direction`

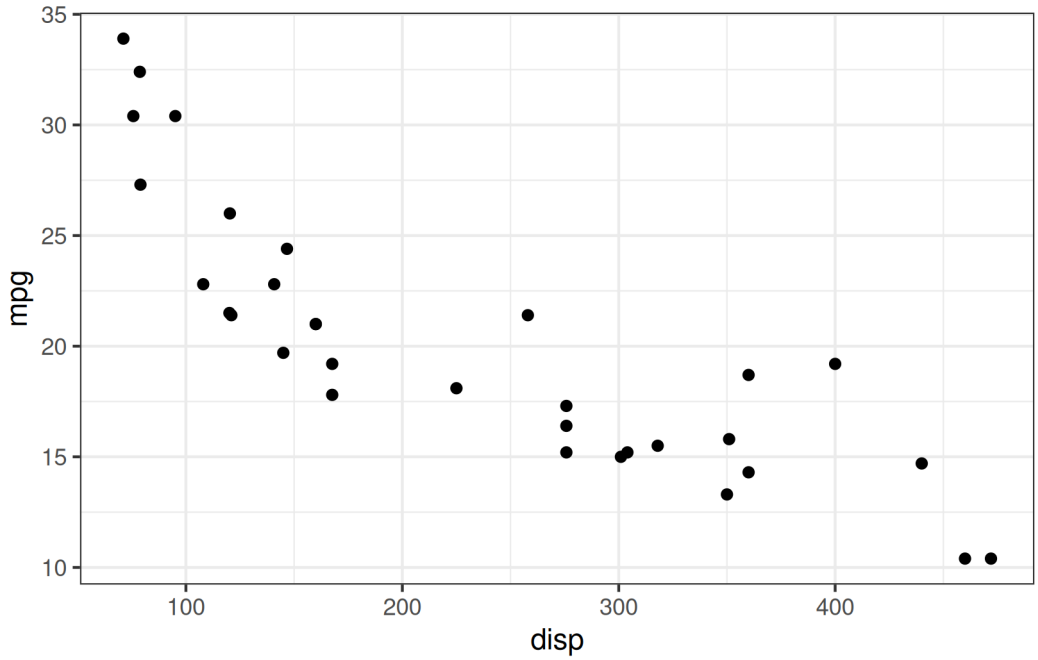
```
p + theme(legend.position = "top", legend.direction = "horizontal")
```



16.5 Apply a complete theme

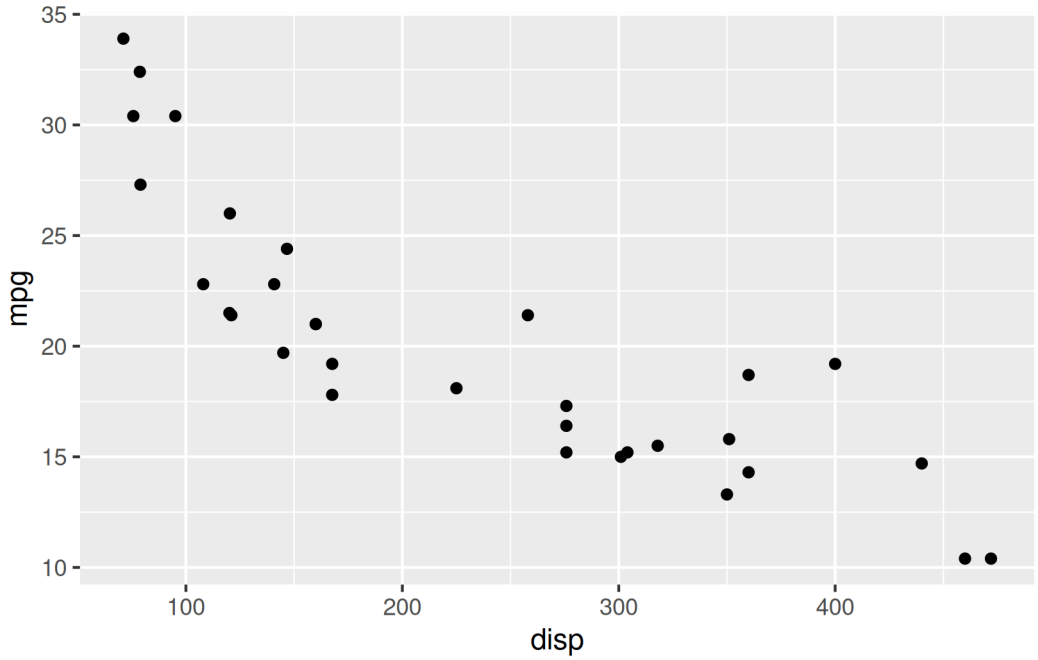
16.5.1 Classic Dark on Light

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_bw()
```



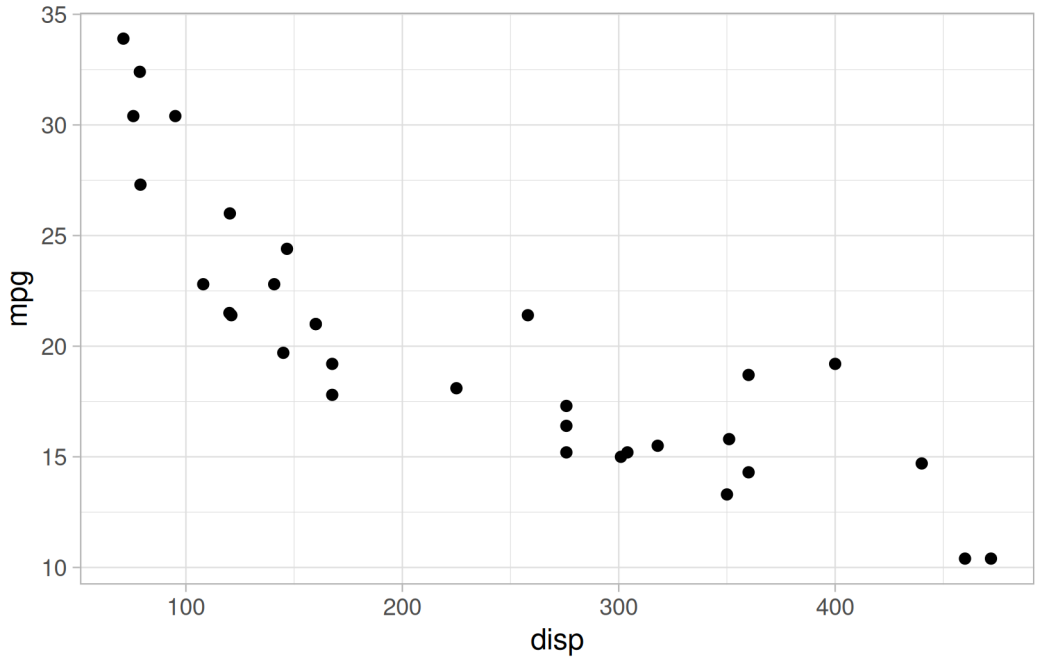
16.5.2 Default Gray

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_gray()
```



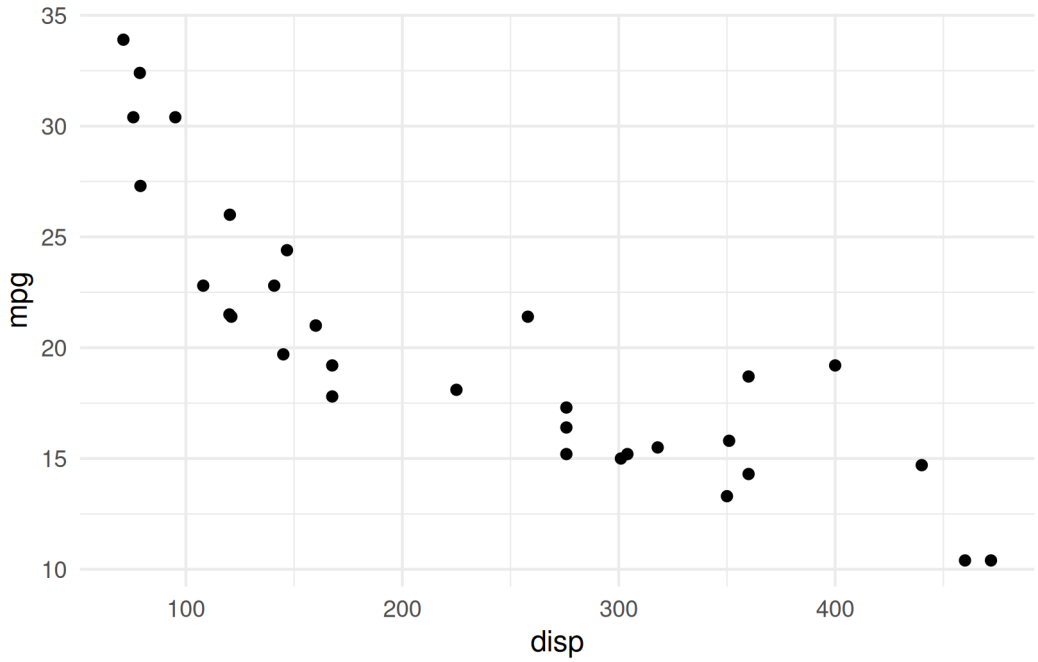
16.5.3 Light

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_light()
```



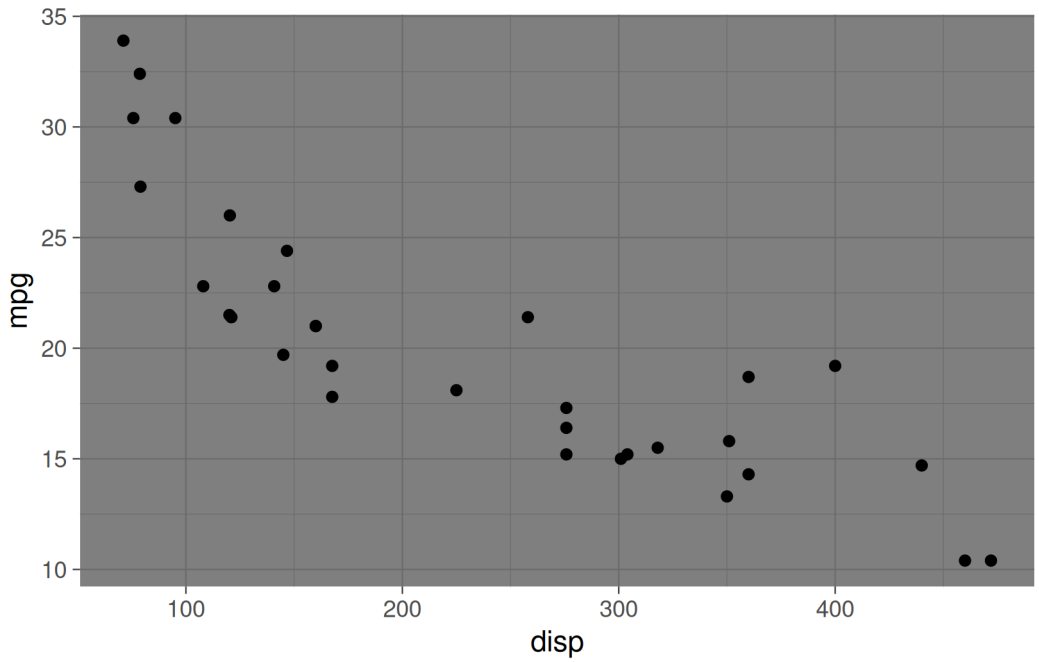
16.5.4 Minimal

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_minimal()
```



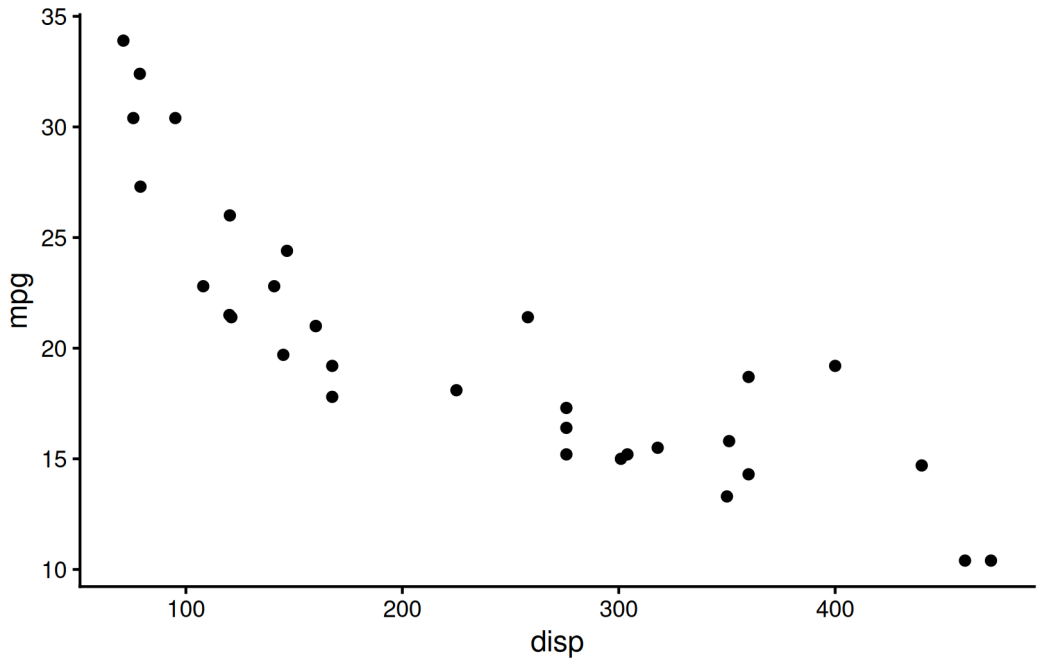
16.5.5 Dark

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_dark()
```



16.5.6 Classic

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_classic()
```



16.5.7 Void (Empty)

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg)) +
  theme_void()
```



16.6 Exercises

1. Apply `theme_minimal()` to a scatter plot.
2. Enlarge all text with `theme(text = element_text(size = 14))`.
3. Build a reusable custom theme and explain what it standardizes.

Worked solutions in `solutions/themes.md` (attempt first).

Where to go next

- ggplot2 book (ggplot2-book.org) for theory behind this layer
- Claus Wilke, Fundamentals of Data Visualization, for perception guidance

17 Position & Stats Tuning

In this chapter, we will fine-tune how ggplot2 arranges and transforms marks: parametric position adjustments, summary statistics and modern stat mapping syntax.

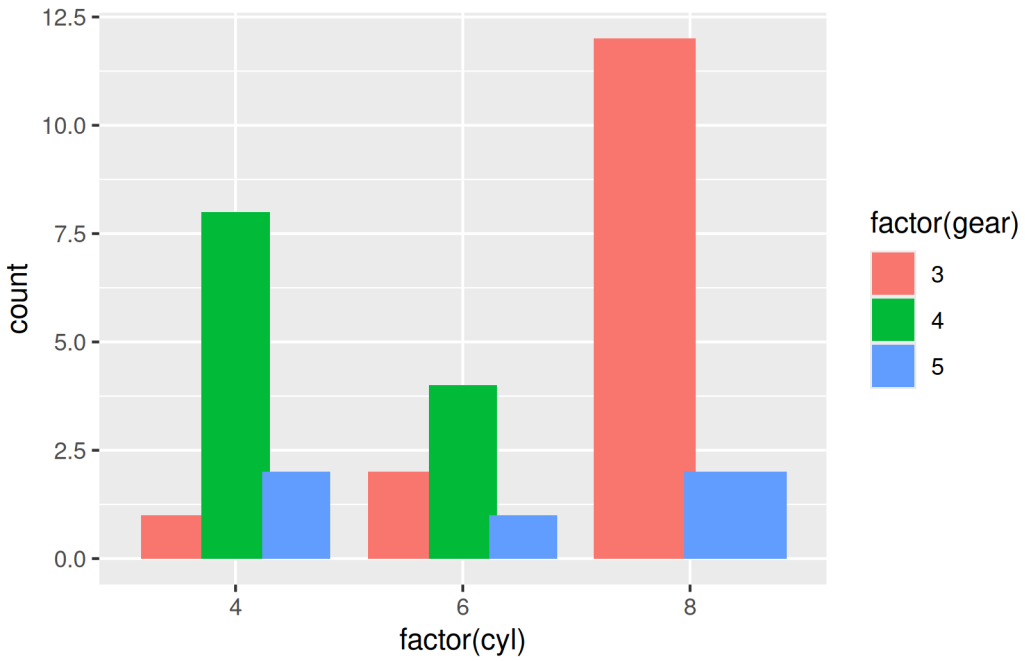
> **Why this matters**
>
> Defaults get you 80% there. Tuning `position_dodge()` widths, `position_jitter()` spreads and `stat_summary()` aggregations takes a plot from roughly right to exactly right.

> **Common pitfall**
>
> String shorthands like `position = "dodge"` hide their parameters. Use the `position_*()` function form whenever you need to tune width, height or stacking behavior.

17.1 Position Dodge

Grouped bar plots dodge side by side. Control the gap with `width` — smaller values widen the gap between groups.

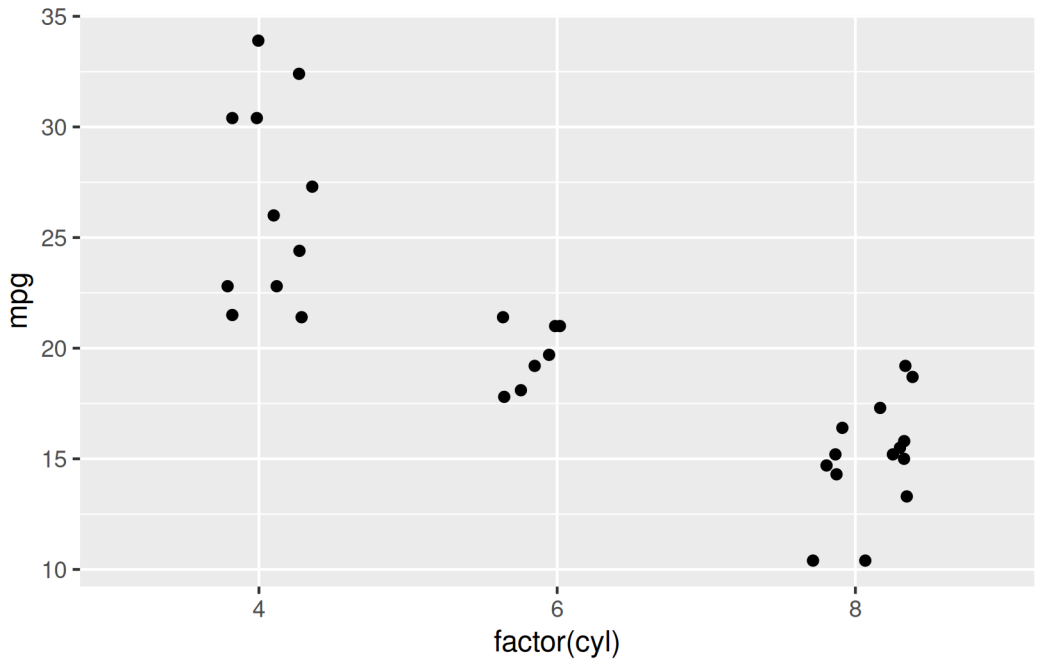
```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl), fill = factor(gear)), position = position_dodge(width = 0.8))
```



17.2 Position Jitter

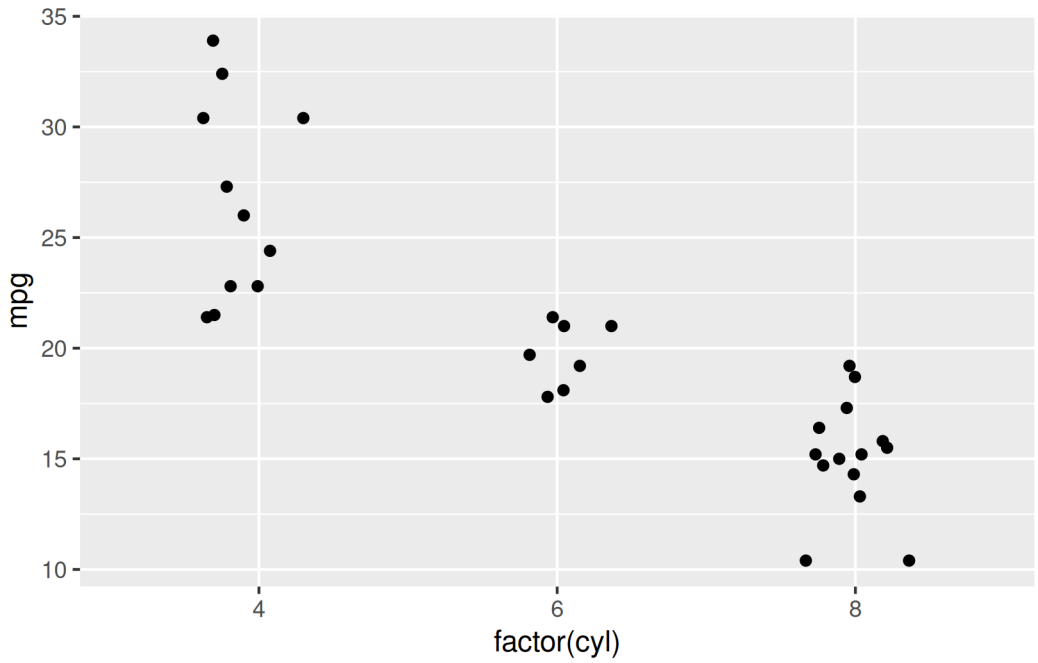
Jitter spreads overlapping points. Set `width` to spread horizontally and `height = 0` when the y axis is exact and must not be distorted.

```
ggplot(mtcars) +
  geom_point(aes(factor(cyl), mpg), position = position_jitter(width = 0.2, height = 0))
```



`geom_jitter()` is a shortcut with the same arguments:

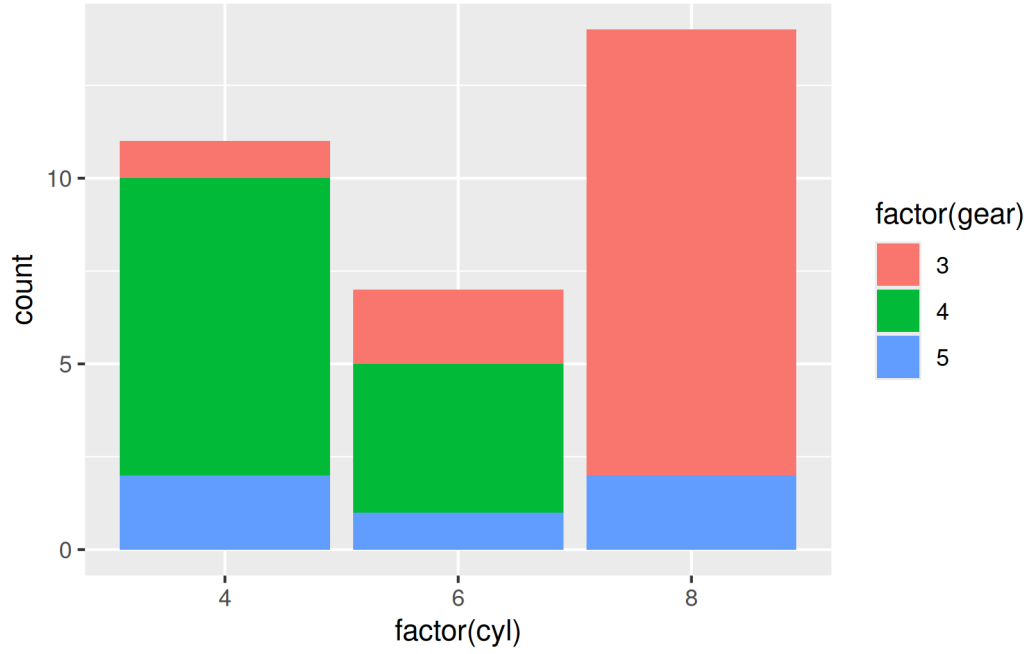
```
ggplot(mtcars) +
  geom_jitter(aes(factor(cyl), mpg), width = 0.2, height = 0)
```



17.3 Position Stack

`position_stack()` piles bars (the default for `geom_bar()`). Use it explicitly when combining stacked bars with labels:

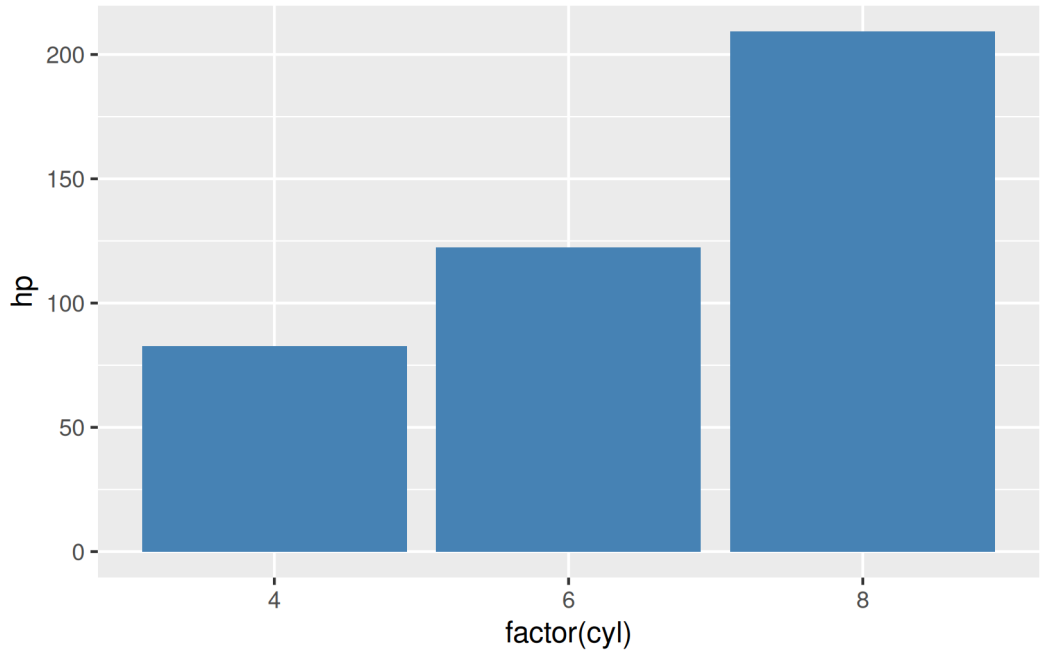
```
ggplot(mtcars) +
  geom_bar(aes(factor(cyl), fill = factor(gear)), position = position_stack())
```



17.4 Stat Summary

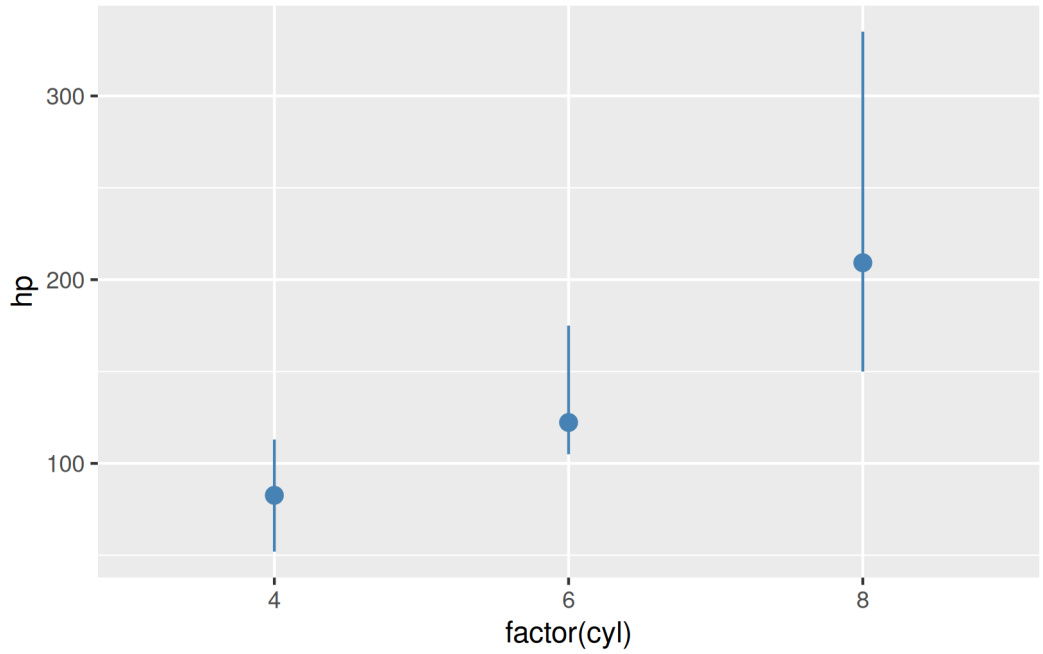
`stat_summary()` aggregates y values per x group on the fly. Here we plot mean horsepower per cylinder count:

```
ggplot(mtcars) +
  stat_summary(aes(factor(cyl), hp), fun = mean, geom = "bar", fill = "steelblue")
```



Add spread with `fun.min` and `fun.max`:

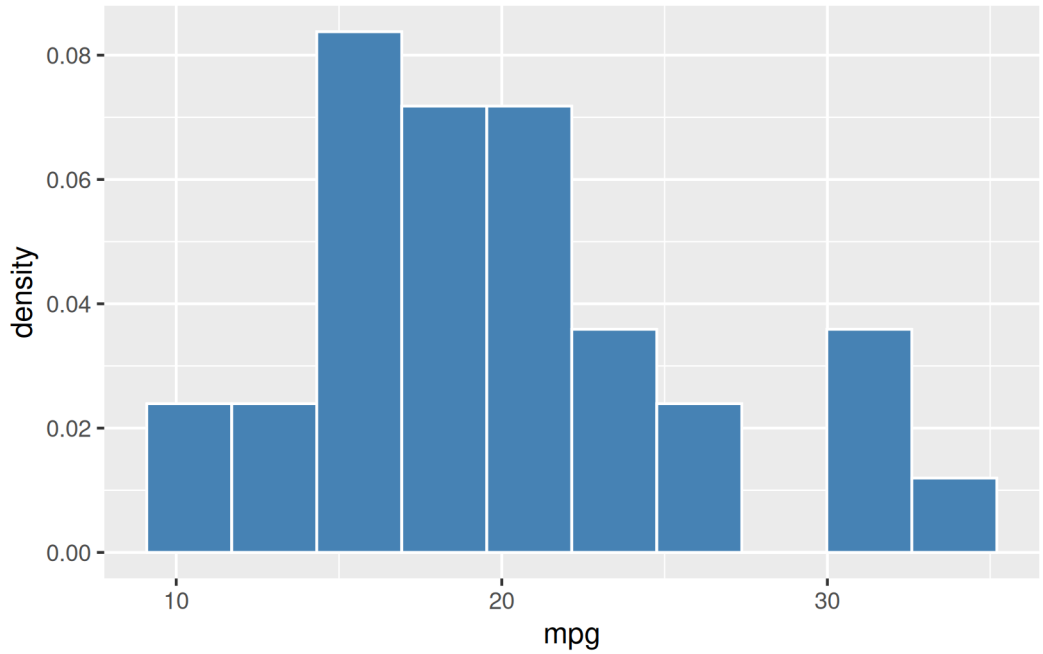
```
ggplot(mtcars) +
  stat_summary(aes(factor(cyl), hp), fun = mean, fun.min = min, fun.max = max,
    geom = "pointrange", color = "steelblue")
```



17.5 After Stat

Computed statistics must be mapped with `after_stat()`. The old `..count..` syntax is deprecated. Here density is mapped on a histogram:

```
ggplot(mtcars) +
  geom_histogram(aes(mpg, y = after_stat(density)), bins = 10,
    fill = "steelblue", color = "white", linewidth = 0.5)
```



Where to go next

- ggplot2 book (ggplot2-book.org): position and statistical summaries chapters
- Claus Wilke, Fundamentals of Data Visualization: visualizing distributions

18 Scales, Formatting & Color Accessibility

In this chapter, we will format axis labels with the `scales` package, use colorblind-safe viridis palettes and check contrast for accessibility.

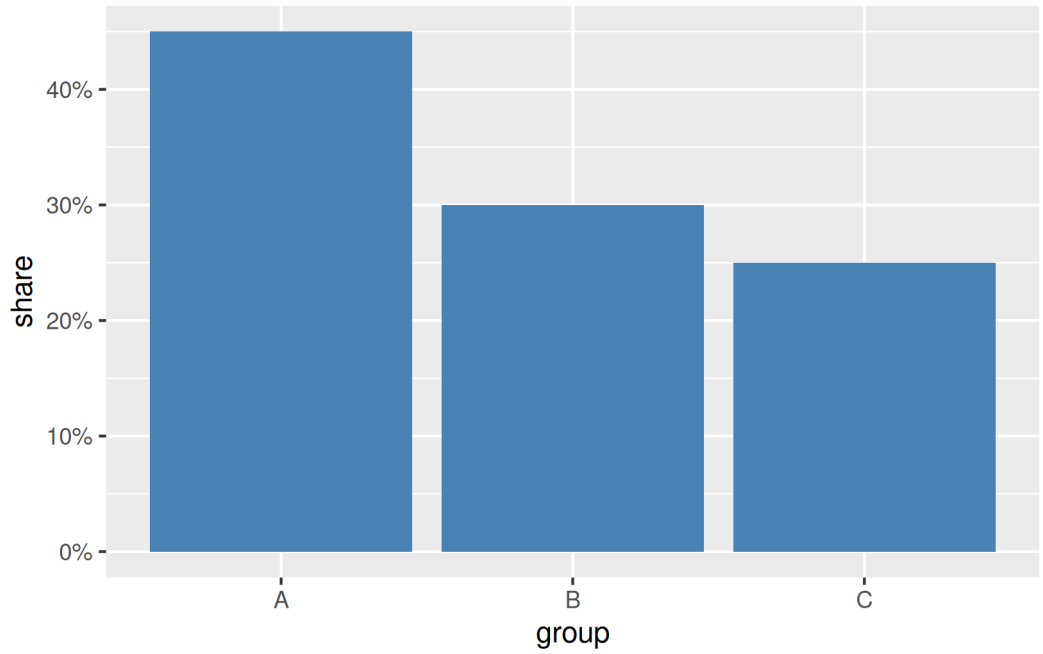
> **Why this matters**
>
> Raw numbers like `150000` slow readers down. Formatted labels (`$150k`, `45%`) and palettes readable by colorblind viewers make charts instantly legible to everyone.

> **Common pitfall**
>
> Red-green contrasts are invisible to the most common form of colorblindness. Never encode critical groups with a red-versus-green scale alone.

18.1 Percent Labels

`label_percent()` turns 0–1 fractions into readable percentages:

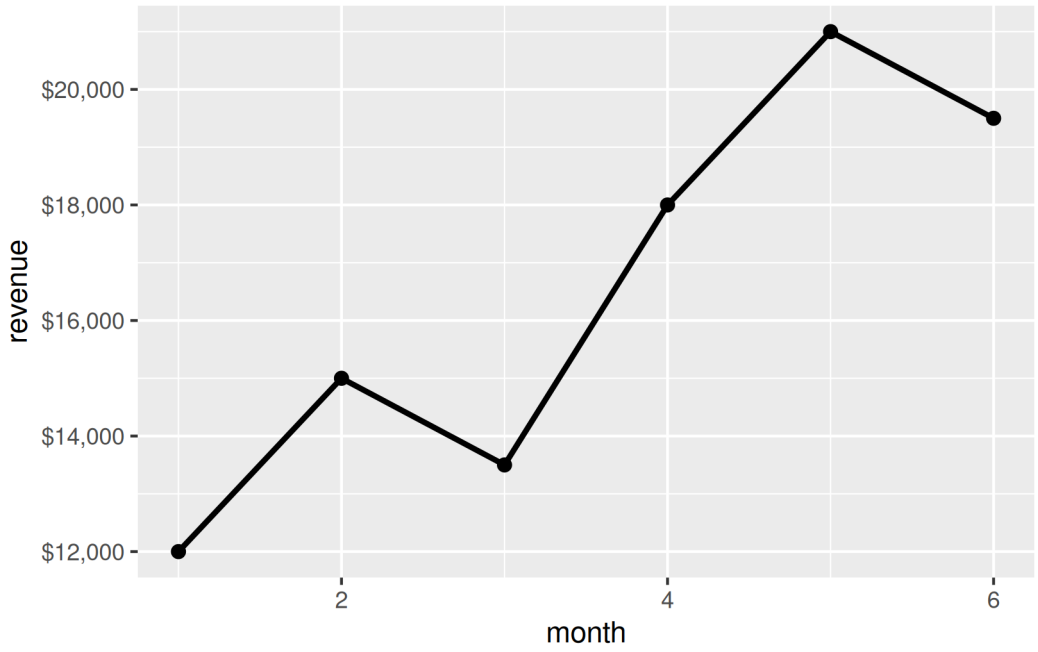
```
df <- data.frame(group = c("A", "B", "C"), share = c(0.45, 0.30, 0.25))
ggplot(df) +
  geom_bar(aes(group, share), stat = "identity", fill = "steelblue") +
  scale_y_continuous(labels = label_percent())
```



18.2 Dollar Labels

`label_dollar()` formats currency axes:

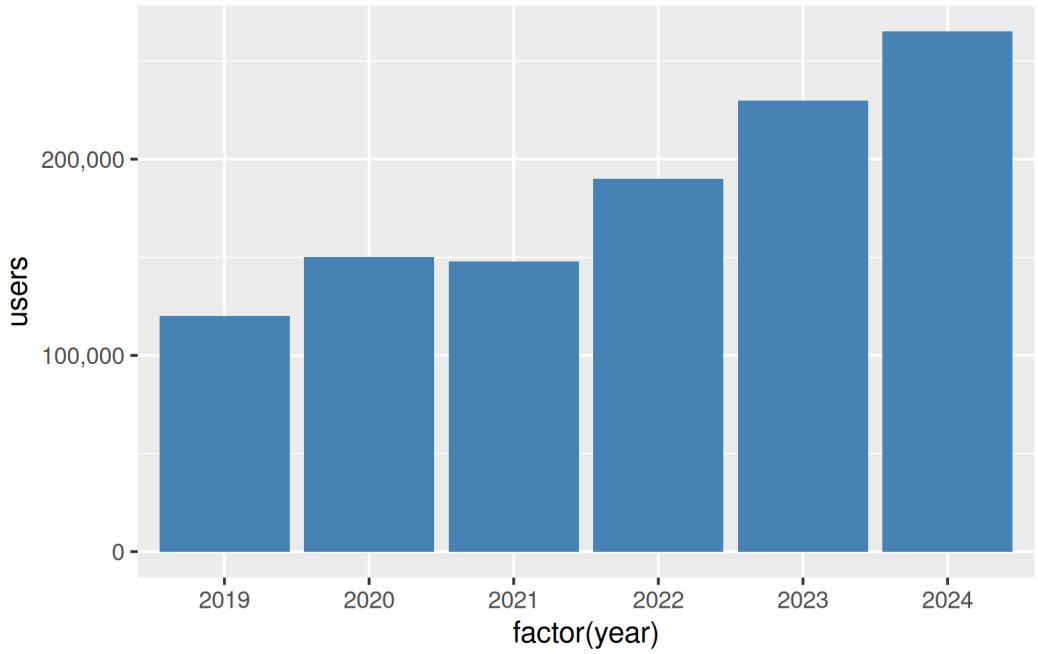
```
sales <- data.frame(month = 1:6, revenue = c(12000, 15000, 13500, 18000, 21000, 19500))
ggplot(sales) +
  geom_line(aes(month, revenue), linewidth = 1) +
  geom_point(aes(month, revenue), size = 2) +
  scale_y_continuous(labels = label_dollar())
```



18.3 Comma Labels

`label_comma()` keeps large counts readable:

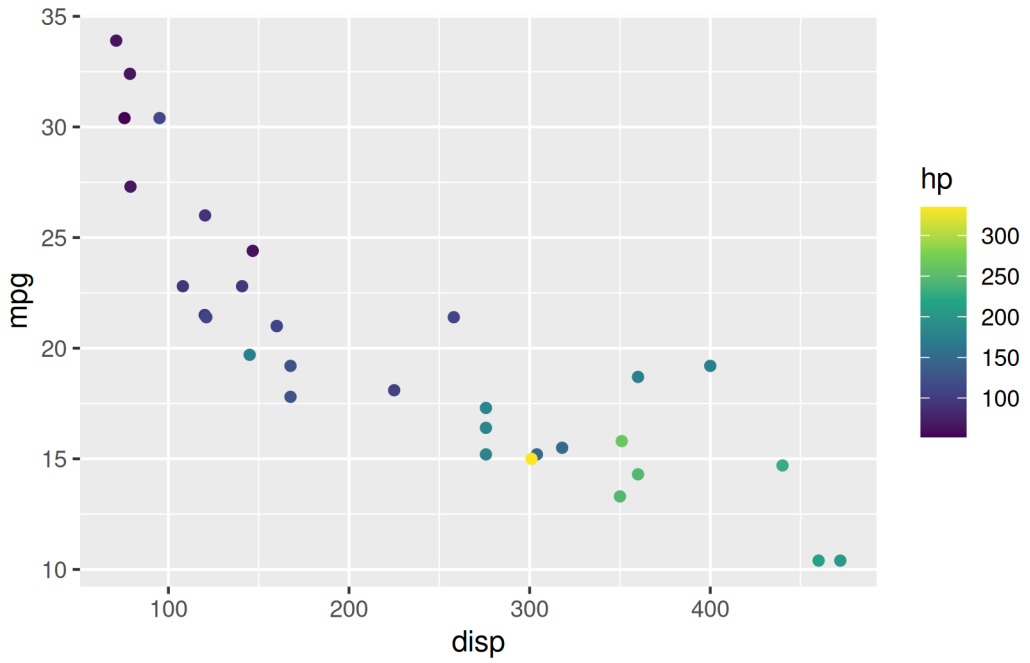
```
counts <- data.frame(year = 2019:2024, users = c(120000, 150000, 148000, 190000, 230000, 2
ggplot(counts) +
  geom_bar(aes(factor(year), users), stat = "identity", fill = "steelblue") +
  scale_y_continuous(labels = label_comma())
```



18.4 Viridis Palettes

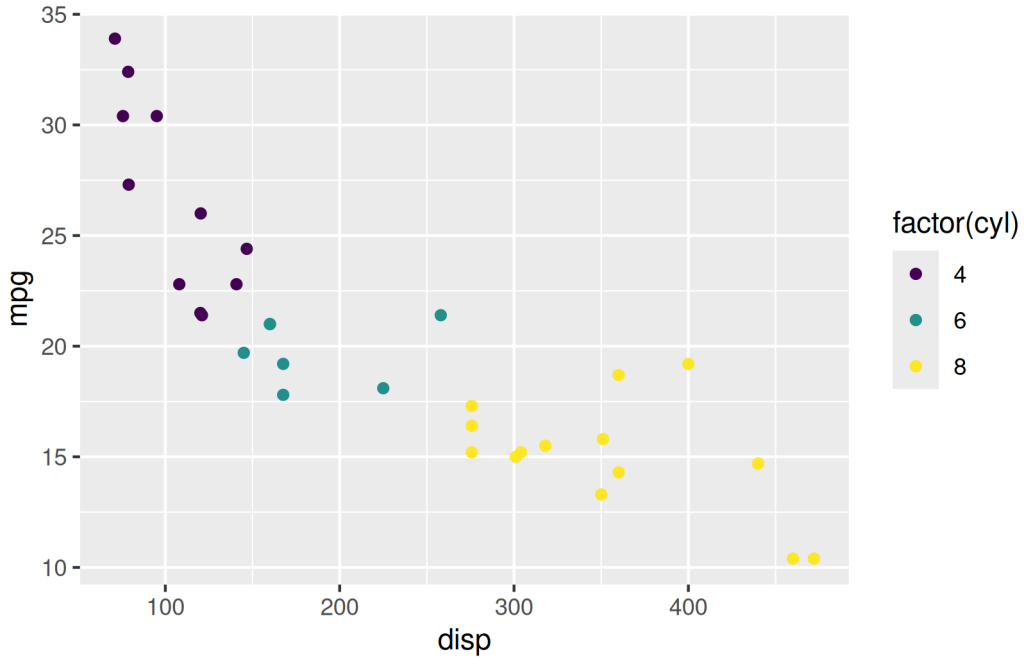
Viridis palettes are perceptually uniform and readable by most colorblind viewers. Use them for continuous fills:

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = hp)) +
  scale_color_viridis_c()
```



And for discrete groups:

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  scale_color_viridis_d()
```



18.5 Contrast Check

Before publishing, verify your figure in grayscale: print it or desaturate it. If groups are still distinguishable by luminance, the contrast passes. The `scale_*_viridis_*()` palettes pass this test by design; hand-picked pastels often fail it.

Where to go next

- scales package reference: formatting functions gallery
- Claus Wilke, Fundamentals of Data Visualization: color chapters
- Colorblind simulator (e.g. Coblis) to check exported figures

19 Data Reshaping for ggplot2

In this chapter, we will learn the one idea that unlocks faceting, grouped aesthetics and multi-series line graphs: long versus wide data.

> **Why this matters**
>
> ggplot2 maps one column to one aesthetic. Wide tables (one column per series) must be reshaped to long tables (one key column plus one value column) before color, fill or facets can separate the series.

> **Common pitfall**
>
> `gather()` appears in older answers but is retired. Always use `tidyr::pivot_longer()` in new code.

19.1 Wide Data

Wide data keeps each series in its own column. Here quarterly revenue for two regions sits in `north` and `south`:

```
wide <- data.frame(
  quarter = c("Q1", "Q2", "Q3", "Q4"),
  north = c(120, 150, 135, 180),
  south = c(100, 130, 160, 175)
)
wide
```

```
  quarter north south
1      Q1   120   100
2      Q2   150   130
3      Q3   135   160
4      Q4   180   175
```

Plotting this directly forces one layer per series — repetitive and unscalable.

19.2 Long Data

`pivot_longer()` collapses the series columns into a `region` key column and a `revenue` value column:

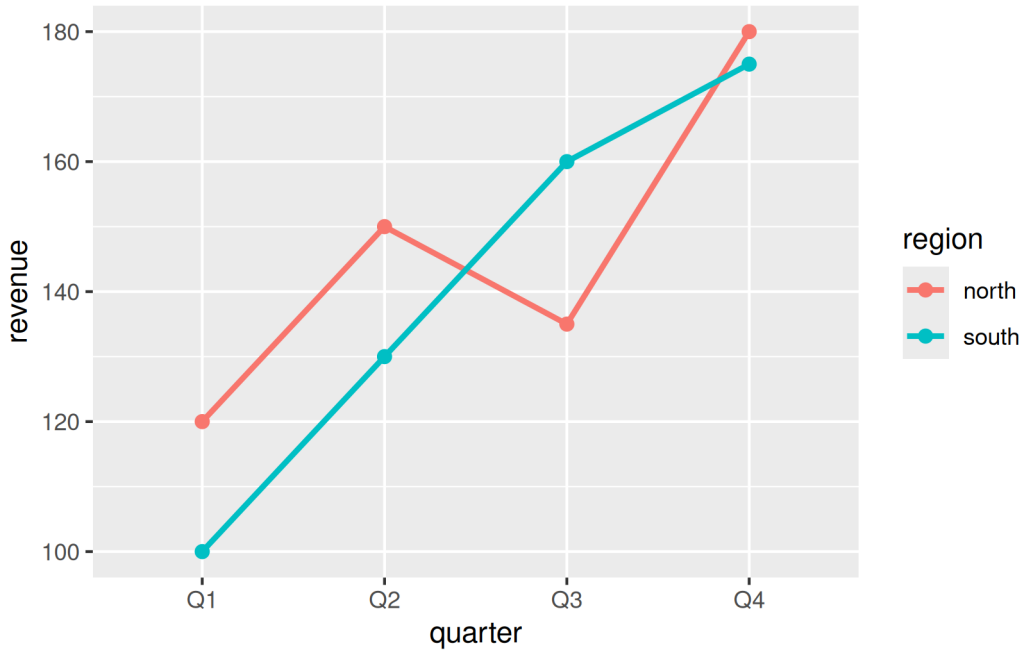
```
long <- tidyr::pivot_longer(wide, cols = c(north, south),
  names_to = "region", values_to = "revenue")
long
```

```
# A tibble: 8 x 3
  quarter region revenue
  <chr>   <chr>    <dbl>
1 Q1      north      120
2 Q1      south      100
3 Q2      north      150
4 Q2      south      130
5 Q3      north      135
6 Q3      south      160
7 Q4      north      180
8 Q4      south      175
```

19.3 Plotting Long Data

One mapping now draws both series, distinguished by color:

```
ggplot(long) +
  geom_line(aes(quarter, revenue, color = region, group = region), linewidth = 1) +
  geom_point(aes(quarter, revenue, color = region), size = 2)
```

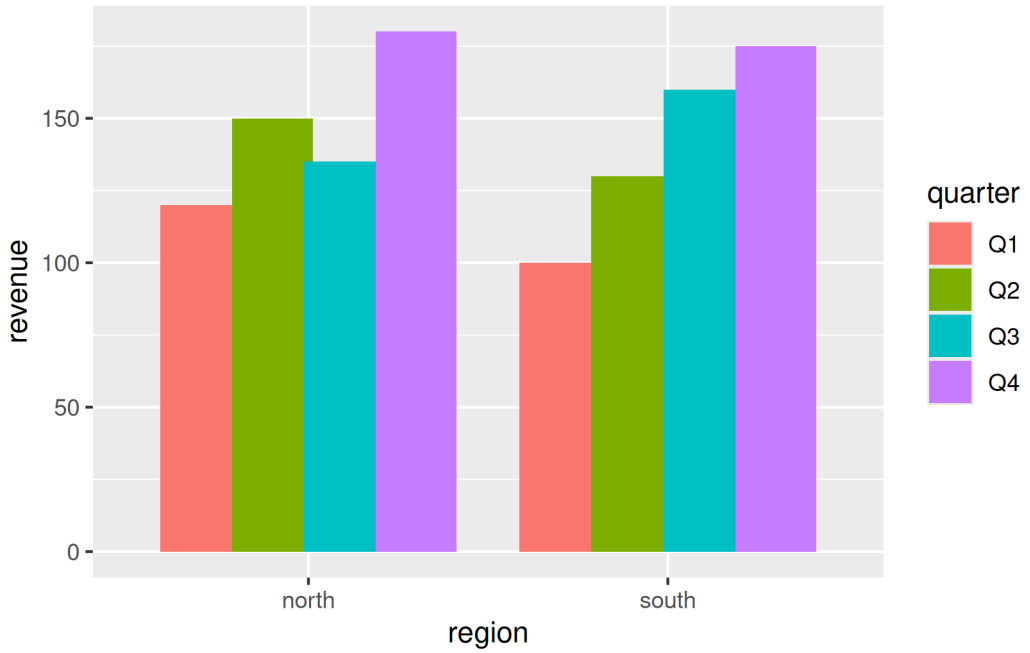


The same long shape powers facets and dodged bars with no extra reshaping.

19.4 Modern Pipe

Chain reshaping into plotting with the native pipe `|>`:

```
wide |>
  tidyr::pivot_longer(cols = c(north, south), names_to = "region", values_to = "revenue") |>
  ggplot() +
  geom_bar(aes(region, revenue, fill = quarter), stat = "identity",
    position = position_dodge(width = 0.8))
```



Where to go next

- tidyr `pivot_longer()` reference for multi-column specs
- Line Graphs chapter for multi-series line recipes
- Faceting chapter for small multiples of long data

20 Label Polishing & Wrapping

In this chapter, we will polish text that overflows: wrapping long axis labels and legend entries with `stringr` and `scales`.

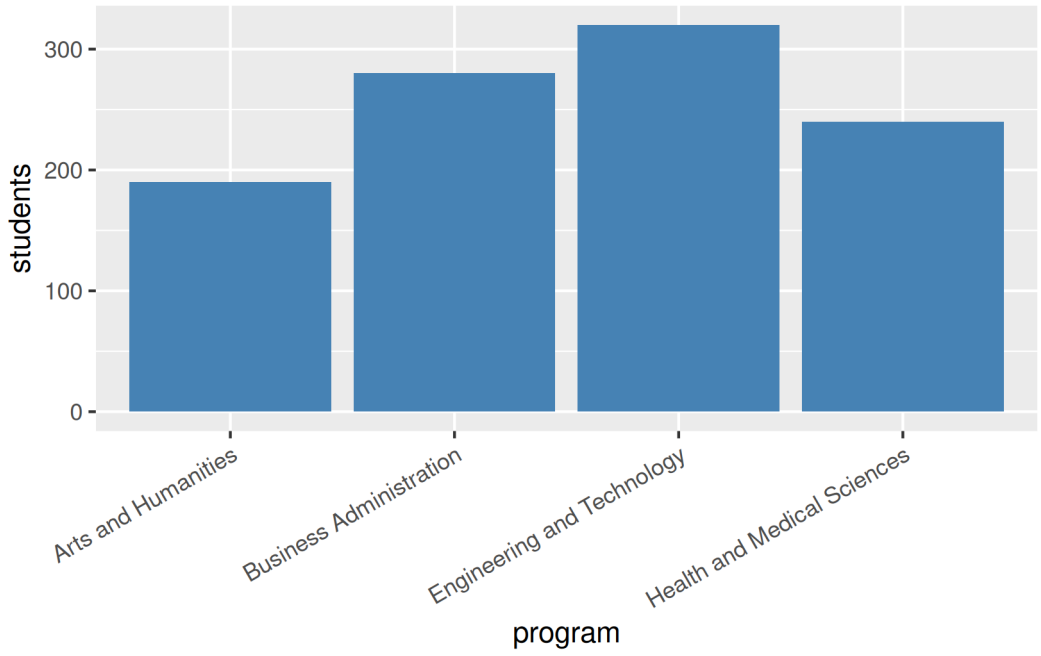
> **Why this matters**
>
> Long category names overlap and wreck otherwise clean charts. Wrapped labels keep every category readable without shrinking fonts to microscopic sizes.

> **Common pitfall**
>
> Angling text 90 degrees feels like a fix but forces readers to tilt their heads. Wrap first; rotate only when wrapping cannot save the layout.

20.1 The Overflow Problem

Long labels collide on crowded axes:

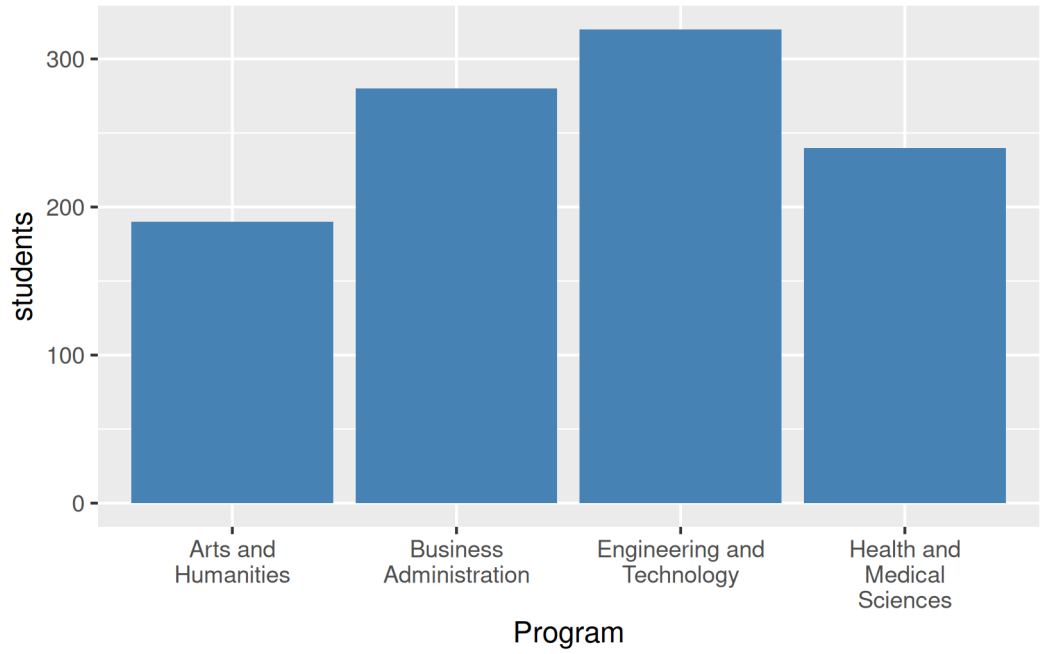
```
df <- data.frame(
  program = c("Engineering and Technology", "Business Administration",
    "Health and Medical Sciences", "Arts and Humanities"),
  students = c(320, 280, 240, 190)
)
ggplot(df) +
  geom_bar(aes(program, students), stat = "identity", fill = "steelblue") +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



20.2 Wrapping with stringr

`str_wrap()` inserts line breaks at a fixed width:

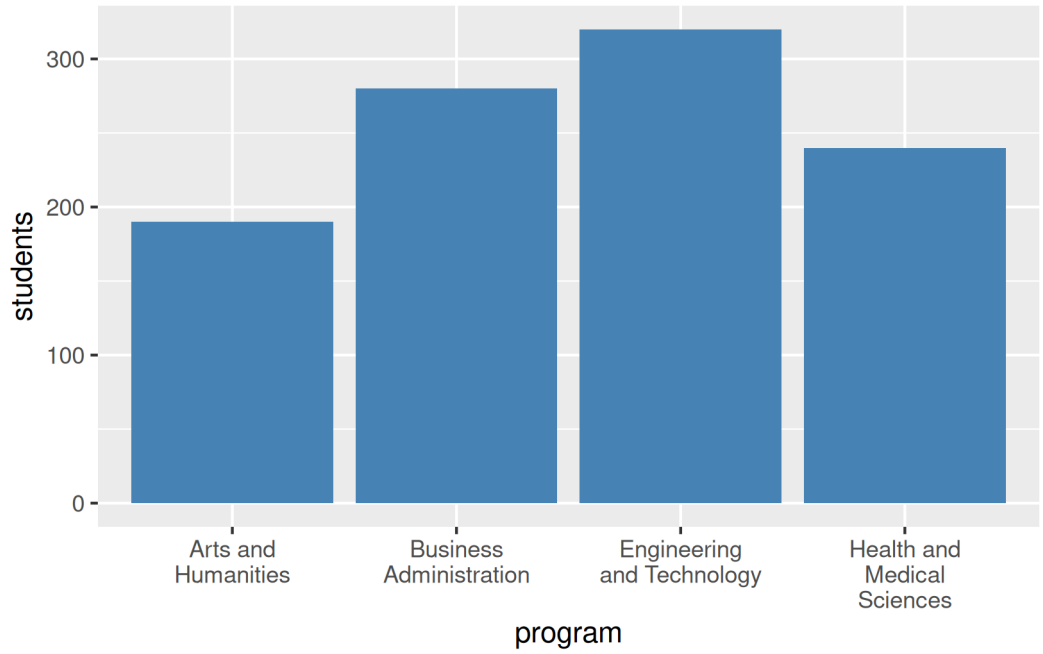
```
ggplot(df) +
  geom_bar(aes(stringr::str_wrap(program, width = 15), students),
    stat = "identity", fill = "steelblue") +
  labs(x = "Program")
```



20.3 Wrapping Inside Scales

`label_wrap()` wraps at the scale level, keeping the underlying data untouched:

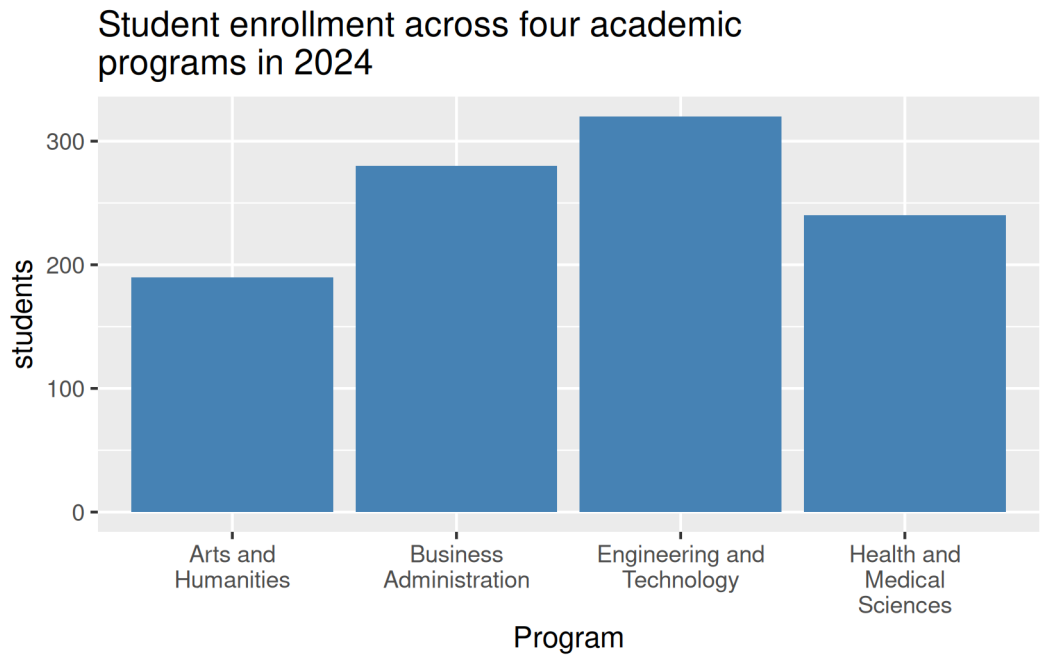
```
ggplot(df) +
  geom_bar(aes(program, students), stat = "identity", fill = "steelblue") +
  scale_x_discrete(labels = scales::label_wrap(15))
```



20.4 Wrapping Long Titles

The same helper tidies wrapped plot titles and subtitles:

```
ggplot(df) +
  geom_bar(aes(stringr::str_wrap(program, width = 15), students),
    stat = "identity", fill = "steelblue") +
  labs(x = "Program",
    title = stringr::str_wrap("Student enrollment across four academic programs in 2024",
```



Where to go next

- stringr reference for wider text manipulation
- Themes chapter for font sizes that pair with wrapped labels

21 Finale: Publication Polish with patchwork, ggrepel & ggtext

In this chapter, we will combine multi-plot layouts (`patchwork`), collision-free labels (`ggrepel`) and rich text (`ggtext`) into three polished before/after case studies.

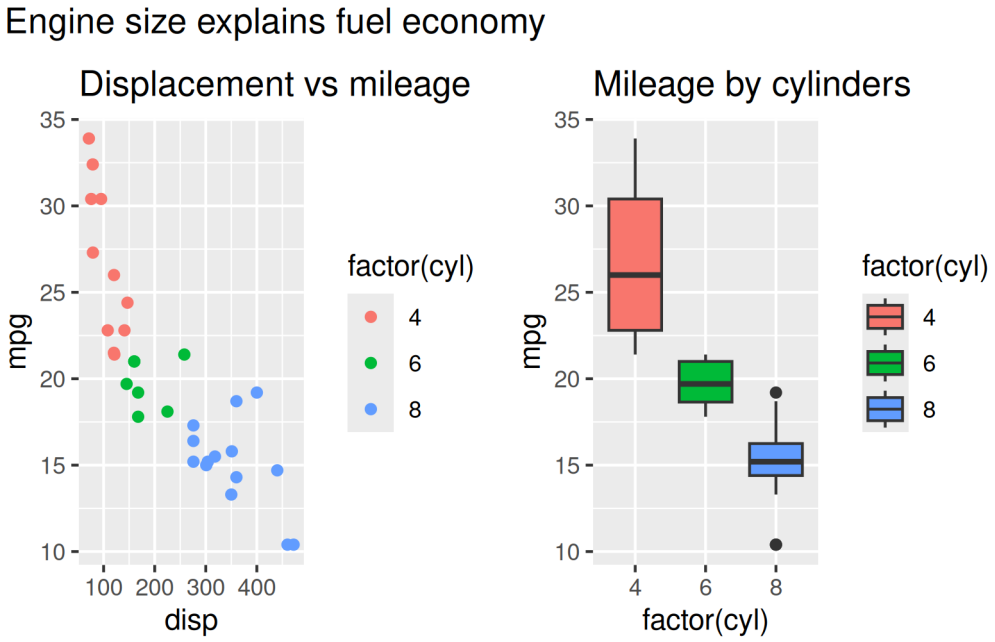
> **Why this matters**
>
> Individual plots answer single questions. Combining them into one composed figure with clean labels is what separates an exploratory chart from a published one.

> **Common pitfall**
>
> Each extension adds load time and new syntax. Master core ggplot2 first; reach for these only for final figures.

21.1 Case 1: Multi-Plot Layout

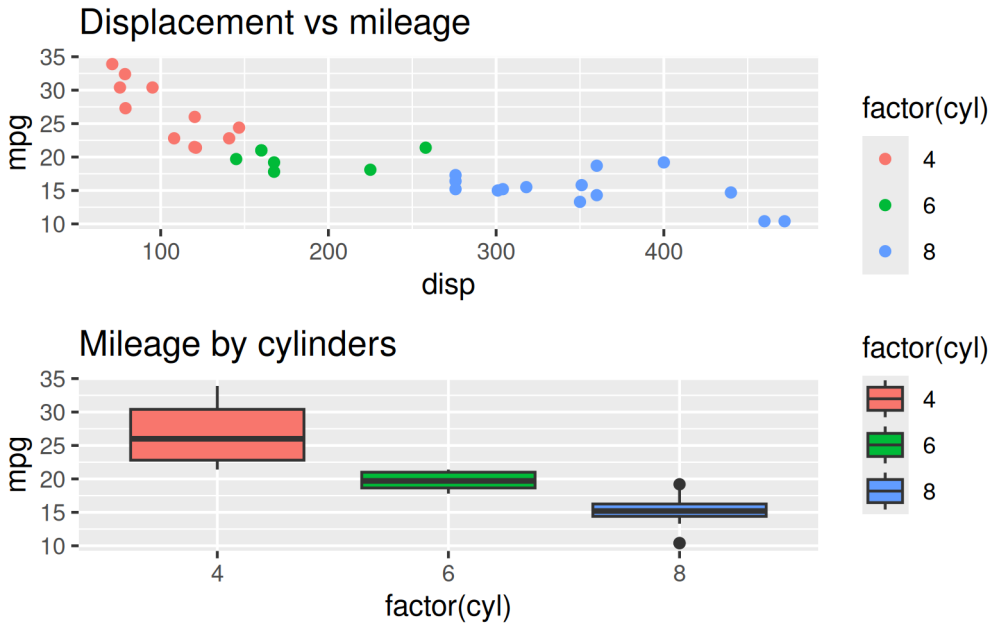
Before: two related plots shown separately. After: one composed figure with a shared title.

```
p1 <- ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  labs(title = "Displacement vs mileage")
p2 <- ggplot(mtcars) +
  geom_boxplot(aes(factor(cyl), mpg, fill = factor(cyl))) +
  labs(title = "Mileage by cylinders")
p1 + p2 + plot_annotation(title = "Engine size explains fuel economy")
```



Stack vertically with `/` and collect legends with `plot_layout(guides = "collect")`:

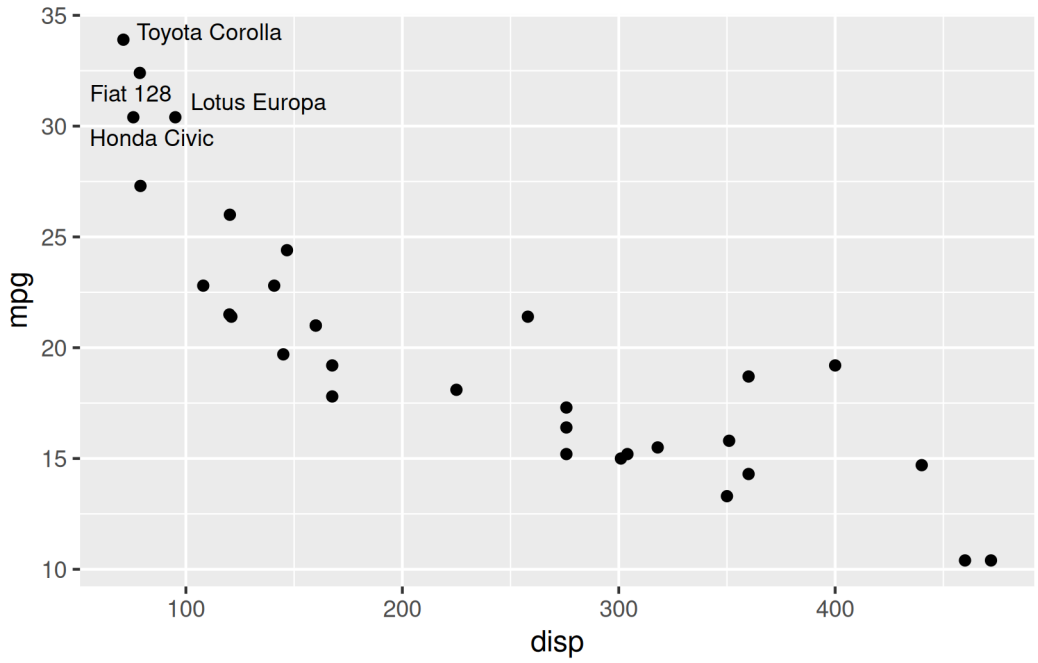
```
p1 / p2 + plot_layout(guides = "collect")
```



21.2 Case 2: Collision-Free Labels

Before: `geom_text()` labels pile onto each other. After: `geom_text_repel()` pushes them apart with thin leader lines.

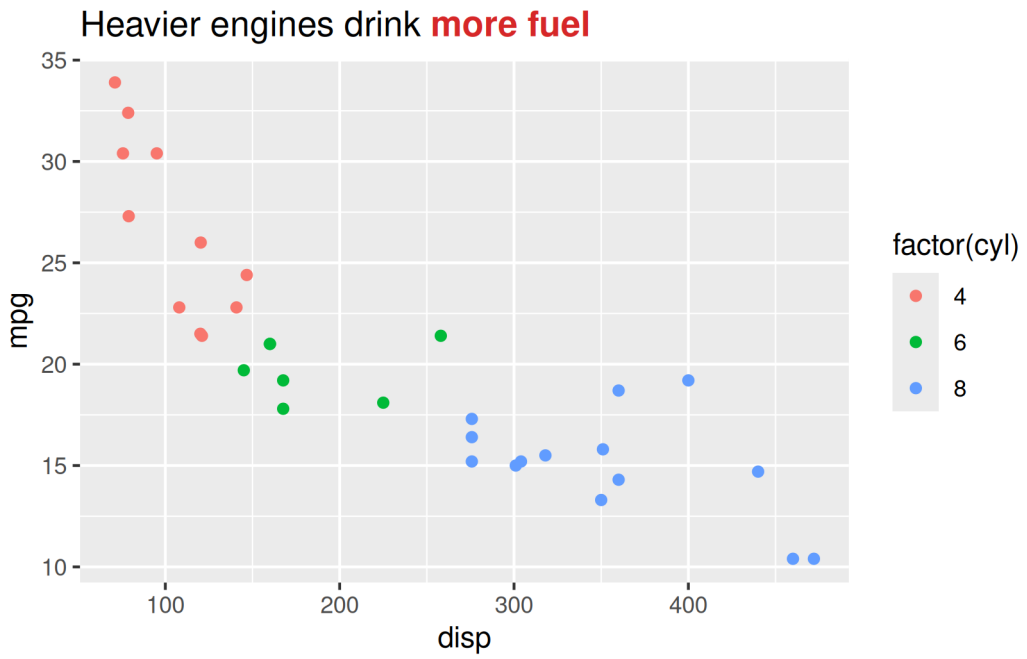
```
cars <- mtcars
cars$model <- rownames(mtcars)
efficient <- cars[cars$mpg > 30, ]
ggplot(cars) +
  geom_point(aes(disp, mpg)) +
  ggrepel::geom_text_repel(data = efficient, aes(disp, mpg, label = model), size = 3)
```



21.3 Case 3: Rich Text Titles

Before: plain titles. After: `element_markdown()` renders color and emphasis inline.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  labs(title = "Heavier engines drink <b style='color:#d62728'>more fuel</b>") +
  theme(plot.title = ggtext::element_markdown())
```



Where to go next

- patchwork reference for nested and tagged layouts
- ggrepel and ggtext references for label-heavy figures
- Export chapter to save the composed figure at print resolution

22 Export & Communicate

In this chapter, we will save figures at print resolution with `ggsave()` and run a storytelling checklist before sharing.

> **Why this matters**
>
> A great on-screen plot that exports blurry or unlabeled fails at the finish line. Export settings and a final message check are what your audience actually sees.

> **Common pitfall**
>
> Screenshots of the RStudio plot pane are low resolution and carry UI chrome. Always export with `ggsave()` for anything you share.

22.1 Saving with ggsave

`ggsave()` writes the last plot. Set width, height and dpi explicitly for print-crisp output:

```
p <- ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl))) +
  labs(title = "Displacement vs mileage",
    x = "Displacement (cu. in.)", y = "Miles per gallon",
    color = "Cylinders")

ggplot2::ggsave("displacement-vs-mileage.png", plot = p,
  width = 8, height = 6, dpi = 300)
```

Match dimensions to the destination: 8x6 inches suits slides, while narrower widths (e.g. 6x4) fit single-column manuscripts.

22.2 Vector Formats

For manuscripts and posters, export PDF or SVG so lines stay sharp at any zoom:

```
ggplot2::ggsave("displacement-vs-mileage.pdf", plot = p, width = 8, height = 6)
ggplot2::ggsave("displacement-vs-mileage.svg", plot = p, width = 8, height = 6)
```

22.3 Storytelling Checklist

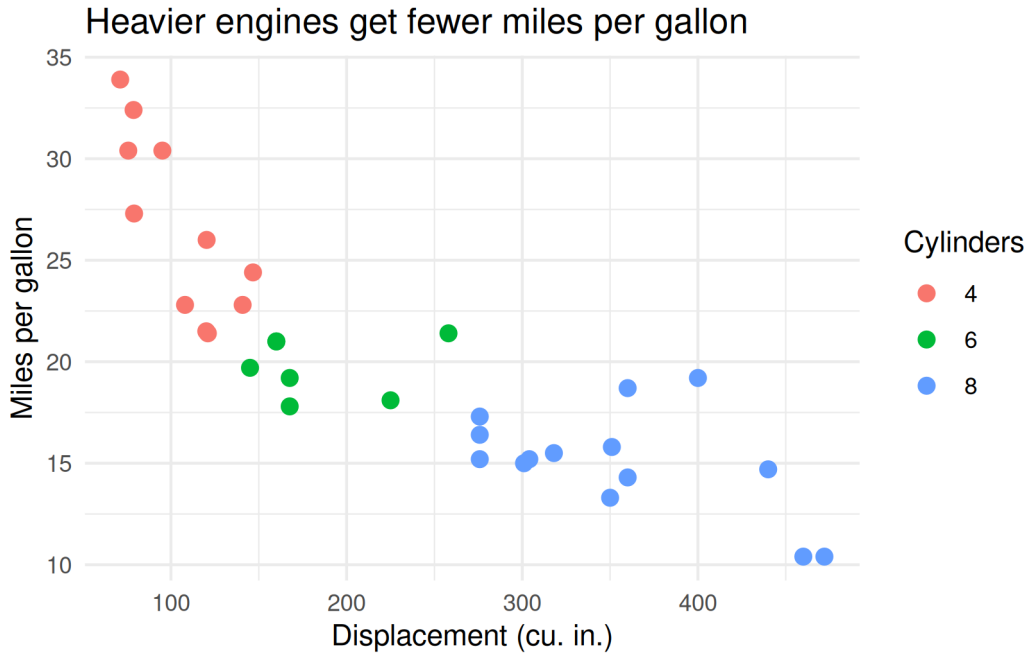
Before sharing any figure, run this Knaflic-inspired check:

- **Context:** does the title state the conclusion, not just the topic?
- **Clutter:** are gridlines, borders and decimals reduced to the minimum?
- **Focus:** is the key series emphasized with color while the rest recede?
- **Labels:** are axes, units and sources stated so the figure stands alone?
- **Honesty:** does the axis start at zero for bars, and are missing data disclosed?

22.4 Before and After

Apply the checklist to a default scatter plot: add a message-first title, direct axis units and a restrained palette.

```
ggplot(mtcars) +
  geom_point(aes(disp, mpg, color = factor(cyl)), size = 2.5) +
  labs(title = "Heavier engines get fewer miles per gallon",
    x = "Displacement (cu. in.)", y = "Miles per gallon", color = "Cylinders") +
  theme_minimal()
```



Where to go next

- Storytelling with Data (Knaflic) for the full narrative framework
- Claus Wilke, Fundamentals of Data Visualization, for design principles